

Лаборатория (начало 01.12.2021)

Software Engineering Practices

IT fundamentals and principles

ООП, принципы, SOLID

Суть ООП заключается в том, чтобы представить программу в виде объектов, которые каким-то образом взаимодействуют друг с другом.

Все, что угодно, можно представить в виде объекта: человека, воздушный шарик, сообщение в мессенджере. У объекта могут быть свойства, например, цвет – красный, размер – большой. Также у объекта могут быть методы для совершения операций. Например, если объект телевизор, вызываем метод «включить», и телевизор включается.

Объект — это экземпляр какого-то класса. Класс — это шаблон, в котором описаны все свойства будущего объекта и его методы. При этом если класс воздушного шарика определяет свойство цвет, то сам класс никакого значения цвета не имеет. Но экземпляры этого класса, которых, к слову, можно создавать сколько угодно, уже будут раскрашены в любые цвета.

Классы могут выстраиваться в хитрые витиеватые структуры. Чем структура хитрее, тем программа гибче, легче поддается изменениям и внедрениям нового функционала, но не обязательно. Такие слова как наследование, полиморфизм, инкапсуляция позволяют создавать структуры объектов еще витиеватее, при этом избавляют код от дублирования и делают его интуитивно понятным, но не всегда.

Суть мастерства ООП в умении конструировать многоуровневые структуры из классов, при этом оставляя код читаемым, надежным и гибким.

1. Инкапсуляция – сокрытие поведения объекта внутри него. Объекту «водитель» не нужно знать, что происходит в объекте «машина», чтобы она ехала. Это ключевой принцип ООП.
2. Наследование. Есть объекты «человек» и «водитель». У них есть явно что-то общее. Наследование позволяет выделить это общее в один объект (в данном случае более общим будет человек), а водителя — определить как человека, но с дополнительными свойствами и/или поведением. Например, у водителя есть водительские права, а у человека их может не быть.
3. Полиморфизм – это переопределение поведения. Можно снова рассмотреть «человека» и «водителя», но теперь добавить

«пешехода». Человек умеет как-то передвигаться, но как именно, зависит от того, водитель он или пешеход. То есть у пешехода и водителя схожее поведение, но реализованное по-разному: один перемещается ногами, другой – на машине.

ООП позволяет упростить сложные объекты, составляя их из более маленьких и простых, поэтому над программой могут работать сотни разработчиков, каждый из которых занят своим блоком.

Абстракция (принцип №0)

Абстракция — это придание объекту характеристик, которые отличают его от всех объектов, четко определяя его концептуальные границы. Основная идея состоит в том, чтобы отделить способ использования составных объектов данных от деталей их реализации в виде более простых объектов, подобно тому, как функциональная абстракция разделяет способ использования функции и деталей её реализации в терминах более примитивных функций, таким образом, данные обрабатываются функцией высокого уровня с помощью вызова функций низкого уровня.

Это позволяет работать с объектами, не вдаваясь в особенности их реализации. В каждом конкретном случае применяется тот или иной подход: инкапсуляция, полиморфизм или наследование. Например, при необходимости обратиться к скрытым данным объекта, следует воспользоваться инкапсуляцией, создав, так называемую, функцию доступа или свойство.

Фундаментальная идея состоит в разделении несущественных деталей реализации подпрограммы и характеристик существенных для корректного ее использования. Такое разделение может быть выражено через специальный «интерфейс», сосредотачивающий описание всех возможных применений программы

Инкапсуляция (принцип №1)

Инкапсуляция — свойство программирования, позволяющее пользователю не задумываться о сложности реализации используемого программного компонента (что у него внутри?), а взаимодействовать с ним посредством предоставляемого интерфейса (публичных методов и членов), а также объединить и защитить жизненно важные для компонента данные. При этом пользователю предоставляется только спецификация (интерфейс) объекта.

Пользователь может взаимодействовать с объектом только через этот интерфейс. Реализуется с помощью ключевого слова: `public`.

Пользователь не может использовать закрытые данные и методы. Реализуется с помощью ключевых слов: `private`, `protected`, `internal`

INTERNAL (видимость внутри модуля)

Как правило при разработке проекта мы делим его на независимые модули. Каждый модуль состоит из файлов, компилируемых вместе. Так вот модификатор `internal` позволяет сделать данные видимыми для всего модуля.

Internal полезен только в том случае, если в проекте есть более одного модуля. Иначе используется модификатор `public`.

Модули

Модификатор доступа `internal` означает, что этот член видно в рамках его модуля. Модуль - это набор скомпилированных вместе **Kotlin** файлов:

- модуль в IntelliJ IDEA;
- Maven или Gradle проект;
- набор скомпилированных вместе файлов с одним способом вызова `<kotlinc>` задачи в Ant.

Наследование (принцип №2)

Наследование — один из четырёх важнейших механизмов объектно-ориентированного программирования (наряду с инкапсуляцией, полиморфизмом и абстракцией), позволяющий описать **новый класс на основе уже существующего** (родительского), при этом свойства и функциональность родительского класса заимствуются новым классом.

Другими словами, класс-наследник реализует спецификацию уже существующего класса (базовый класс). Это позволяет обращаться с объектами класса-наследника точно так же, как с объектами базового класса.

В некоторых языках используются абстрактные классы. Абстрактный класс — это класс, содержащий хотя бы один абстрактный метод, он описан в программе, имеет поля, методы и **не может использоваться для непосредственного создания объекта**. То есть **от абстрактного класса можно только наследовать**. Объекты создаются только на основе производных классов, наследованных от абстрактного.

- `::` создает [ссылку на элемент](#) или [ссылку на класс](#)

Полиморфизм (принцип № 3)

Полиморфизм — возможность объектов с одинаковой спецификацией иметь различную реализацию.

Язык программирования поддерживает полиморфизм, если классы с одинаковой спецификацией могут иметь различную реализацию —

например, реализация класса может быть изменена в процессе наследования.

Кратко смысл полиморфизма можно выразить фразой: «Один интерфейс, множество реализаций».

Полиморфизм позволяет писать более абстрактные программы и повысить коэффициент повторного использования кода. Общие свойства объектов объединяются в систему, которую могут называть по-разному — интерфейс, класс. Общность имеет внешнее и внутреннее выражение:

- внешняя общность проявляется как одинаковый набор методов с одинаковыми именами и сигнатурами (именем методов и типами аргументов, и их количеством);
- внутренняя общность — одинаковая функциональность методов. Её можно описать интуитивно или выразить в виде строгих законов, правил, которым должны подчиняться методы. Возможность приписывать разную функциональность одному методу (функции, операции) называется перегрузкой метода (перегрузкой функций, перегрузкой операций).

SOLID (single responsibility, open–closed, Liskov substitution, interface segregation и dependency inversion)

<https://medium.com/webbdev/solid-4ffc018077da> - source

Избавиться от «признаков плохого проекта»^[4] помогают следующие 5 принципов SOLID:

Инициал	Представляет ^[1]	Название ^[4] , понятие
S	SRP ^[5]	Принцип единственной ответственности (single responsibility principle) Для каждого класса должно быть определено единственное назначение. Все ресурсы, необходимые для его осуществления, должны быть инкапсулированы в этот класс и подчинены только этой задаче.
O	OCP ^[6]	Принцип открытости/закрытости (open–closed principle) «программные сущности ... должны быть открыты для расширения, но закрыты для модификации».
L	LSP ^[7]	Принцип подстановки Лисков (Liskov substitution principle) «объекты в программе должны быть заменяемыми на экземпляры их подтипов без изменения правильности выполнения программы». См. также контрактное программирование. Производный класс должен быть взаимозаменяем с родительским классом.
I	ISP ^[8]	Принцип разделения интерфейса (interface segregation principle) «много интерфейсов, специально предназначенных для клиентов, лучше, чем один интерфейс общего назначения» ^[9] .
D	DIP ^[10]	Принцип инверсии зависимостей (dependency inversion principle) «Зависимость на Абстрациях. Нет зависимости на что-то конкретное» ^[9] .

- **S: Single Responsibility Principle** (Принцип единственной ответственности).

Класс должен быть ответственен лишь за что-то одно. Если класс отвечает за решение нескольких задач, его подсистемы, реализующие решение этих задач, оказываются связанными друг с другом. Изменения в одной такой подсистеме ведут к изменениям в другой.

- **O: Open-Closed Principle** (Принцип открытости-закрытости).

Программные сущности (классы, модули, функции) должны быть открыты для расширения, но не для модификации.

- **L: Liskov Substitution Principle** (Принцип подстановки Барбары Лисков)

Необходимо, чтобы подклассы могли бы служить заменой для своих суперклассов.

плохо

```

//...
class Pigeon extends Animal {

}

const animals[]: Array<Animal> = [
  //...,
  new Pigeon();
]

function AnimalLegCount(a: Array<Animal>) {
  for(int i = 0; i <= a.length; i++) {
    if(typeof a[i] == Lion)
      return LionLegCount(a[i]);
    if(typeof a[i] == Mouse)
      return MouseLegCount(a[i]);
    if(typeof a[i] == Snake)
      return SnakeLegCount(a[i]);
    if(typeof a[i] == Pigeon)
      return PigeonLegCount(a[i]);
  }
}

AnimalLegCount(animals);
```

хорошо

```

function AnimalLegCount(a: Array<Animal>) {
  for(let i = 0; i <= a.length; i++) {
    a[i].LegCount();
  }
}

AnimalLegCount(animals);
```

- **I: Interface Segregation Principle** (Принцип разделения интерфейса)

Создавайте узкоспециализированные интерфейсы, предназначенные для конкретного клиента. Клиенты не должны зависеть от интерфейсов, которые они не используют.

Этот принцип направлен на устранение недостатков, связанных с реализацией больших интерфейсов

плохо

```
interface Shape {  
    drawCircle();  
    drawSquare();  
    drawRectangle();  
    drawTriangle();  
}
```

хорошо

```

interface Shape {
    draw();
}

interface ICircle {
    drawCircle();
}

interface ISquare {
    drawSquare();
}

interface IRectangle {
    drawRectangle();
}

interface ITriangle {
    drawTriangle();
}

class Circle implements ICircle {
    drawCircle() {
        //...
    }
}

```

- **D: Dependency Inversion Principle** (Принцип инверсии зависимостей)

Объектом зависимости должна быть абстракция, а не что-то конкретное.

1. Модули верхних уровней не должны зависеть от модулей нижних уровней. Оба типа модулей должны зависеть от абстракций.
2. Абстракции не должны зависеть от деталей. Детали должны зависеть от абстракций.

В процессе разработки программного обеспечения существует момент, когда функционал приложения перестаёт помещаться в рамках одного модуля. Когда это происходит, нам приходится решать проблему зависимостей модулей. В результате, например, может оказаться так, что высокоуровневые компоненты зависят от низкоуровневых компонентов.

плохо

```

class XMLHttpService extends XMLHttpRequestService {}

class Http {
  constructor(private xmlhttpService: XMLHttpService) { }
  get(url: string , options: any) {
    this.xmlhttpService.request(url, 'GET');
  }
  post() {
    this.xmlhttpService.request(url, 'POST');
  }
  //...
}

```

Высокоуровневый HTTP имеет в конструкторе XMLHttpService хорошо

Класс Http не должен знать о том, что именно используется для организации сетевого соединения. Поэтому мы создадим интерфейс Connection:

```

interface Connection {
  request(url: string, opts: any);
}

```

Интерфейс Connection содержит описание метода request и мы передаём классу Http аргумент типа Connection:

```

class Http {
  constructor(private httpConnection: Connection) { }
  get(url: string , options: any) {
    this.httpConnection.request(url, 'GET');
  }
  post() {
    this.httpConnection.request(url, 'POST');
  }
  //...
}

```

Теперь, вне зависимости от того, что именно используется для организации взаимодействия с сетью, класс Http может пользоваться тем, что ему передали, не заботясь о том, что скрывается за интерфейсом Connection.

Перепишем класс XMLHttpService таким образом, чтобы он реализовывал этот интерфейс:

```

class XMLHttpRequestService implements Connection {
    const xhr = new XMLHttpRequest();
    //...
    request(url: string, opts: any) {
        xhr.open();
        xhr.send();
    }
}

```

В результате мы можем создать множество классов, реализующих интерфейс Connection и подходящих для использования в классе Http для организации обмена данными по сети

1. *Software Engineering Practices*

2. *Version Control System*

VCS, Git vs SVN, основные команды, Git flow

Система управления версиями (также используется определение «система контроля версий», *Version Control System*, VCS или *Revision Control System*) — **программное обеспечение** для облегчения работы с изменяющейся информацией. Система управления версиями позволяет хранить несколько версий одного и того же документа, при необходимости возвращаться к более ранним версиям, определять, кто и когда сделал то или иное изменение, и многое другое.

1. GIT распределяется, а SVN - нет. Другими словами, если есть несколько разработчиков работающих с репозиторием у каждого на локальной машине будет ПОЛНАЯ копия этого репозитория. Разумеется есть и где-то и центральная машина, с которой можно клонировать репозиторий. Это напоминает SVN. Основной плюс в том, что если вдруг у вас нет доступа к интернету, сохраняется возможность работать с репозиторием. Потом только один раз сделать синхронизацию и все остальные разработчики получат полную историю.
2. GIT сохраняет метаданные изменений, а SVN целые файлы. Это экономит место и время.
3. Система создания branches, versions и прочее в GIT и SVN отличаются значительно. В GIT проще переключаться с ветки на ветку, делать merge между ними.

SVN, конечно же, поддерживает ветки, но при сложных ветвлениях (ветки мерджатся между собой) случаются неожиданные конфликты и т.п. Хотя, если у вас классическая схема - фича-ветка от мастера (транка) и обратный мердж в мастер (транк) - разница почти незаметна.

Ключевое отличие заключается в том, что он децентрализован. Представьте, что вы разработчик в дороге, вы разрабатываете на своем ноутбуке и хотите иметь контроль над версиями, чтобы вернуться назад на 3 часа.

С Subversion у вас есть проблема: репозиторий SVN может находиться в месте, до которого вы не можете добраться (в вашей компании, и у вас сейчас нет интернета), вы не можете совершить коммит. Если вы хотите сделать копию своего кода, Вы должны буквально скопировать/вставить его.

С Git у вас нет этой проблемы. Ваша локальная копия является репозиторием, и вы можете зафиксировать ее и получить все преимущества системы управления версиями. Когда вы восстановите подключение к основному хранилищу, вы сможете совершить коммит против него.

git stash временная сдача наработок в архив к примеру с целью перейти в другую ветвь без коммита, что бы продолжить работу позже

git add

Команда git add добавляет содержимое рабочего каталога в индекс

git status

Команда git status показывает состояния файлов в рабочем каталоге и индексе

git commit

Команда git commit берёт все данные, добавленные в индекс с помощью git add, и сохраняет их слепок во внутренней базе данных, а затем сдвигает указатель текущей ветки на этот слепок.

git reset

Команда git reset, как можно догадаться из названия, используется в основном для отмены изменений. Она изменяет указатель HEAD и, опционально, состояние индекса. Также эта команда может изменить файлы в рабочем каталоге при использовании параметра --hard, что может привести к потере наработок при неправильном использовании, так что убедитесь в серьёзности своих намерений прежде чем использовать его.

git branch

Команда git branch — это своего рода "менеджер веток". Она умеет перечислять ваши ветки, создавать новые, удалять и переименовывать их.

git checkout

Команда git checkout используется для переключения веток и выгрузки их содержимого в рабочий каталог.

git merge

Команда git merge используется для слияния одной или нескольких веток в текущую. Затем она устанавливает указатель текущей ветки на результирующий коммит.

git fetch

Команда git fetch связывается с удалённым репозиторием и забирает из него все изменения, которых у вас пока нет и сохраняет их локально.

git pull

Команда git pull работает как комбинация команд git fetch и git merge, т. е. Git вначале забирает изменения из указанного удалённого репозитория, а затем пытается слить их с текущей веткой.

git remote

Команда git remote служит для управления списком удалённых репозиториях. Она позволяет сохранять длинные URL репозиториях в виде понятных коротких строк, например «origin», так что вам не придётся забивать голову всякой ерундой и набирать её каждый раз для связи с сервером. Вы можете использовать несколько удалённых репозиториях для работы и git remote поможет добавлять, изменять и удалять их.

git show

Команда git show отображает объект в простом и человекопонятном виде. Обычно она используется для просмотра информации о метке или коммите.

git cherry-pick

Команда git cherry-pick берёт изменения, вносимые одним коммитом, и пытается повторно применить их в виде нового коммита в текущей ветке. Эта возможность полезна в ситуации, когда нужно забрать парочку коммитов из другой ветки, а не сливать ветку целиком со всеми внесенными в нее изменениями.

git rebase

git rebase — это «автоматизированный» cherry-pick. Он выполняет ту же работу, но для цепочки коммитов, тем самым как бы перенося ветку на новое место.

git revert

Команда `git revert` — полная противоположность `git cherry-pick`. Она создаёт новый коммит, который вносит изменения, противоположные указанному коммиту, по существу отменяя его.

git svn

Команда `git svn` используется для работы с сервером Subversion. Это означает, что вы можете использовать Git в качестве SVN клиента, забирать изменения и отправлять свои собственные на сервер Subversion.

Что такое магистральная разработка? ЭТО НЕ ГИТ-ФЛОУ

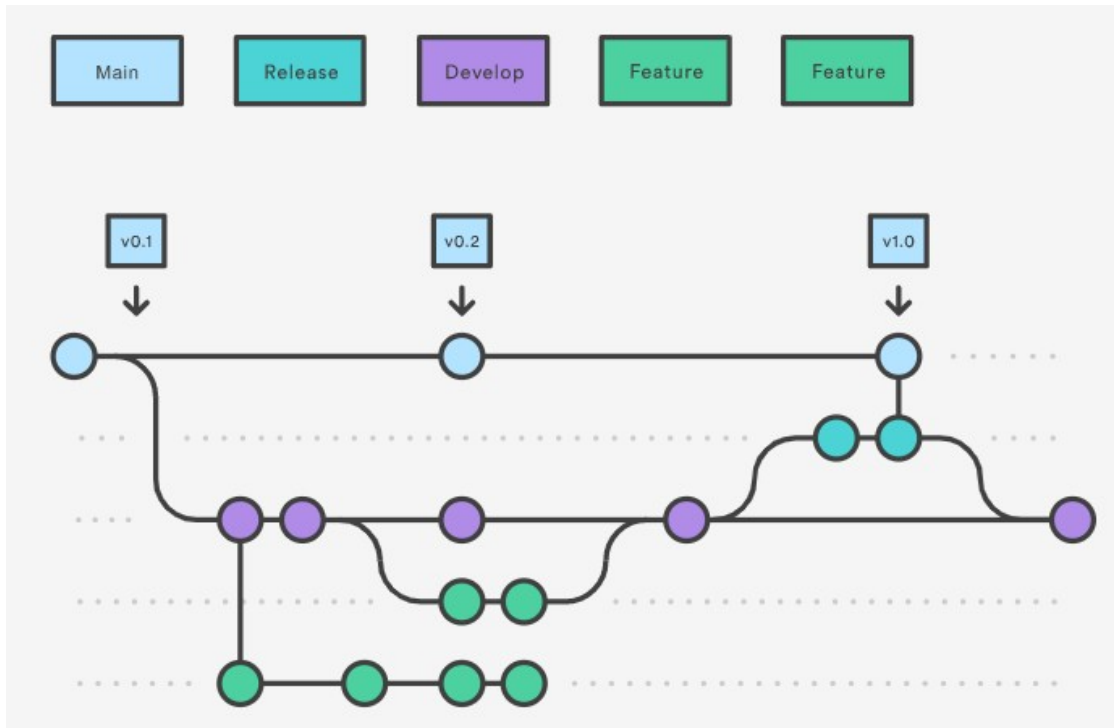
Модель ветвления, в которой разработчики совместно работают над кодом в одной ветви, называемой “стволом”. При этом другие ветви имеют короткий срок жизни благодаря использованию документированных методов.

Другими словами... команда занимается разработкой **без использования веток**. Вернее, они есть, но их цикл жизни короткий

Магистральная разработка требует наличия сильного набора тестов и качественного мониторинга, чтобы как можно быстрее выявлять ошибки. Чем быстрее цикл обратной связи, тем лучше

Git-flow — альтернативная модель ветвления Git, в которой используются функциональные ветки и несколько основных веток.

Собственно, это стандартная модель к которой я привык с ветками фичами и отдельной главной.



Когда в ветке develop оказывается достаточно функций для выпуска (или приближается назначенная дата релиза), от ветки develop создается ветка release. Создание этой ветки запускает следующий цикл релиза, и с этого момента новые функции добавить больше нельзя — допускается лишь исправление багов, создание документации и решение других задач, связанных с релизом. Когда подготовка к поставке завершается, ветка release сливается с main и ей присваивается номер версии. Кроме того, нужно выполнить ее слияние с веткой develop, в которой с момента создания ветки релиза могли возникнуть изменения.

Если ваша компания придерживается месячного или квартального цикла выпуска ПО, а команда параллельно работает над несколькими релизами, то Git-flow *может* стать неплохим выбором.

Для стартапа, сайта или веб-приложения со множеством релизов каждый день Git-flow не подходит. Если команда разработчиков невелика (менее 10 человек), то Git-flow вносит в ее работу слишком много церемоний и лишних движений.

2. JAVA

1. BASICS

equals() vs *==*, *enum*, **методы** *Object*, *hashCode()*,

final, *static*, *mutable* vs *immutable*, *finally*, *finalize()*,

generics, *boxing*, *exceptions*, *inner classes*

1 Метод equals() сравнивает "value" внутри экземпляров строк (в куче) независимо от того, ссылаются ли две ссылки на объекты на один и тот же экземпляр строки или нет.

С другой стороны, **оператор "=="** сравнивает значение **двух ссылок на объекты**, чтобы увидеть, ссылаются ли они на один и тот же **экземпляр строки**.

2 enum или перечисление. Перечисления представляют набор логически связанных констант

enum хороши когда возможность хранить значения ограничена реальными константами,

т.е. Если enum Month то в нем будут January, February ...

если WeekDay то Sunday, Monday ...

хорошо это тем что нельзя внутрь перечисления поместить значение которое там не ожидается например день или месяц «Яблоко»

3 методы класса Object

1 notify() - возобновляет исполнение потока, ожидающего вызывающего объекта. (Многопоточность)

2 notifyAll() - возобновляет исполнение всех потоков, ожидающих вызывающего объекта. (Многопоточность)

3 wait() - ожидает другого потока исполнения. (Многопоточность)

4 getClass() - получить класс объекта во время выполнения. В основном используется для рефлексии.

5 clone() - получить точную копию объекта. Не рекомендуется использовать. Чаще советуют использовать конструктор копирования.

6 equals() - сравнивает два объекта.

7 hashCode() - числовое представление объекта. Значение по-умолчанию - целочисленный адрес в памяти.

8 toString() - возвращает представление объекта в виде строки. По-умолчанию возвращает имя_класса@hashCode в 16-ричной системе. Если hashCode не переопределен, то вернут значение по-умолчанию.

Иногда для коллекций требуется переопределить equals и hashCode.

Правила, которые следует учесть при переопределении equals:

- Рефлексивность: `x.equals(x) == true`
- Симметричность: если `x.equals(y)`, тогда `y.equals(x) == true`

- Переносимость: если `x.equals(y)` и `y.equals(z)`, тогда `x.equals(z) == true`
- Консистентность: если `x.equals(y)`, тогда `x.equals(y) == true`
- Null проверка: `x.equals(null) == false`

почему нужно переопределить `hashCode`? Дело в том, что значение по умолчанию, как писалось выше, это адрес в памяти. То есть может случиться такая ситуация, что вы переопределили `equals` и объекты равным по этому `equals`, но они находятся на разных адресах памяти и это нарушает соглашение между `equals` и `hashCode`:

Если объекты равны по `equals` - они должны быть равны и по `hashCode`. Если объекты не равны по `equals`, то их `hashCode` может быть равен, а может и нет.

Однако, равенство `hashCode` при неравенстве по `equals` — это коллизия и ее стоит избегать при помощи правильно переопределенного `hashCode`.

9 `protected void finalize()` - вызывается перед удалением неиспользуемого объекта. Он вызовется всего однажды, и его не рекомендуется переопределять. Он устаревший.

`native` - Приложение на языке Java может вызывать методы, написанные на языке C++. Такие методы объявляются в языке Java с ключевым словом `native`, которое сообщает компилятору, что метод реализован в другом месте.

4 `hashCode ()`

Одинаковые объекты — это объекты одного класса с одинаковым содержимым полей и одинаковым `hashCode` !!!

если хеш-коды равны, то входные объекты не всегда равны (коллизия)

если хеш-коды разные, то и объекты гарантированно разные

Реализация по умолчанию `hashCode()` возвращает значение, которое называется **идентификационный хеш (identity hash code)**.

Для сведения: даже если класс переопределяет `hashCode()`, вы всегда можете получить идентификационный хеш объекта `o` с помощью вызова `System.identityHashCode(o)`.

Как же нужно произвести сравнение двух объектов в Java, чтобы оно выполнялось качественно и эффективно?

- 1) При сравнении двух объектов, стоит начать с проверки ссылок. Ведь если мы сравниваем объект с самим собой, зачем нам выполнять лишние проверки и сравнивать поля объекта.
- 2) Прежде чем приступить к глубокому сравнению, нужно выполнить еще 2 проверки, которые так же могут отсеять объекты, которые точно не равны. Это поможет ускорить работу метода equals(). Это проверки помогают понять, не является ли объект null и сравниваем ли мы объекты одинаковых классов. Если мы изначально сравниваем два null объекта, то первая проверка вернёт нам true, а значит ко второму шагу объект точно не должен быть null.
- 3) Теперь, когда мы уверены что оба объекта относятся к одинаковому классу и не равны null, можно выполнить приведение объекта к нужному типу и приступить к сравнению полей.

5 final

Для класса это означает, что класс не сможет иметь подклассов, т.е. запрещено наследование. Это полезно при создании immutable (неизменяемых) объектов, например, класс String объявлен, как final.

Следует также отметить, что к абстрактным классам (с ключевым словом abstract), нельзя применить модификатор final, т.к. это взаимоисключающие понятия.

Для метода final означает, что он не может быть переопределен в подклассах. Это полезно, когда мы хотим, чтобы исходную реализацию нельзя было переопределить.

Для переменных примитивного типа это означает, что однажды присвоенное значение не может быть изменено.

Для ссылочных переменных это означает, что после присвоения объекта, нельзя изменить ссылку на данный объект.

Это важно! Ссылку изменить нельзя, но состояние объекта изменять можно!!!

6 Static

Static — модификатор, применяемый к полю, блоку, методу или внутреннему классу. Данный модификатор указывает на привязку субъекта к текущему классу. Используется без создания объекта класса

Вы НЕ можете получить доступ к НЕ статическим членам класса, внутри статического контекста, как вариант, метода или блока и наоборот

В отличие от локальных переменных, статические поля и методы НЕ потокобезопасны (Thread-safe) в Java

В котлин аналог статика это Компаньон, к его методам можно обращаться Foo.a() но из Java так Foo.Companion.a();

Что бы в джава тоже коротко обращаться как к реальной статической надо аннотировать метод @JvmStatic

Internal – видимость внутри модуля (модуль это папка app например), т.е. если у тебя один модуль как всегда бывает в обычных проектах,

то internal = public, а если модулей много то публик видно ото всюду, интернал внутри одного модуля

7 mutable and immutable

Immutable (неизменным) типом в ООП называется такой тип, объект которого после создания становится неизменным, так сказать read-only

1) Неизменяемые объекты можно реализовать значительно проще, чем изменяемые.

2) Неизменяемые объекты можно свободно использовать одновременно из разных нитей. (ПОТОКОБЕЗОПАСНЫ)

у неизменяемого скрыты все сеттеры, есть только конструктор

Обычно immutable классы содержат различные методы, которые «как бы» меняют объект, но вместо изменения самого объекта эти методы просто создают новый объект и возвращают его.

- Отпадает необходимость дополнительных проверок, синхронизации записи, поддержания инварианта класса после изменения полей.
- Отпадает необходимость копирования разделяемых данных.

Кроме того

- Создается возможность для кеширования экземпляров, как явного, так и по усмотрению компилятора.
- Становится возможной более агрессивная оптимизация.

8 finally

при обработке исключений с применением блоков try-catch имеет смысл дополнительно предусмотреть блок finally. Этот блок выполняется всегда, независимо от того, произошло исключение в процессе работы блока try или нет.

Наилучшее применение для блока finally - освобождение ресурсов, заказанных программой перед возникновением исключений. Хотя система сборки мусора автоматически освобождает ненужную более оперативную память, открытые потоки, например, следует закрывать явным образом при помощи метода close.

9 finalize()

Вызывается сборщиком мусора на объекте, когда GC определяет, что больше нет ссылок на объект.

Этот метод вызывается Java-машиной у объекта перед тем, как объект будет уничтожен. Фактически этот метод – противоположность конструктору. В нем можно освобождать ресурсы, используемые объектом.

Если объект взаимодействует с какими-то ресурсами, например открывает поток вывода и читает из него, то такой поток необходимо закрыть перед удалением объекта из памяти. Для этого в языке Java достаточно переопределить метод `finalize()`, который вызывается в исполняющей среде Java непосредственно перед удалением объекта данного класса. В теле метода `finalize()` нужно указать те действия, которые должны быть выполнены перед уничтожением объекта. Метод `finalize()` вызывается лишь непосредственно перед сборкой "мусора".

Метод `finalize()` не вызывается при выходе объекта из области действия. Заранее неизвестно, когда будет (и будет ли вообще) выполняться метод `finalize()`. И самое главное - начиная с Java 9 этот метод не рекомендуется к использованию.

10 generics

Обобщения или generics (обобщенные типы и методы) позволяют нам уйти от жесткого определения используемых типов.

С помощью буквы **T** в определении класса `class SomeClass<T>` мы указываем, что данный тип **T** будет использоваться этим классом. Параметр **T** в угловых скобках называется **универсальным параметром**, так как вместо него можно подставить любой тип. При этом пока мы не знаем, какой именно это будет тип: `String`, `int` или какой-то другой. Причем буква **T** выбрана условно, это может и любая другая буква или набор символов.

Интерфейсы, как и классы, также могут быть обобщенными. `interface Accountable<T>` / `class Account<T> implements Accountable<T>`

Кроме обобщенных типов можно также создавать обобщенные методы `public <T> void print(T[] items)`

Мы можем также задать сразу несколько универсальных параметров `class Account<T, S>`

Мы можем также задать сразу несколько универсальных параметров `<T>Account(T id, int sum)`

11 boxing

Autoboxing и unboxing в Java

речь пойдет об автоупаковке (autoboxing) и распаковке (unboxing)

Autoboxing происходит:

1. При присвоении значения примитивного типа переменной соответствующего класса-обёртки.
2. При передаче примитивного типа в параметр метода, ожидающего соответствующий ему класс-обёртку.

БЕЗ АВТОУПАКОВКИ

```
public class Main {  
    public static void main(String[] args) {  
        Integer iOb = new Integer(7);  
        Double dOb = new Double(7.0);  
        Character cOb = new Character('a');  
        Boolean bOb = new Boolean(true);  
  
        method(new Integer(7));  
    }  
  
    public static void method(Integer iOb) {  
        System.out.println("Integer");  
    }  
}
```

С АВТОУПАКОВКОЙ

```
public class Main {  
    public static void main(String[] args) {  
        Integer iOb = 7;  
        Double dOb = 7.0;  
        Character cOb = 'a';  
        Boolean bOb = true;  
  
        method(7);  
    }  
  
    public static void method(Integer iOb) {  
        System.out.println("Integer");  
    }  
}
```

Автораспаковка

Это преобразование класса-обёртки в соответствующий ему примитивный тип. Если при распаковке класс-обёртка был равен **null**, произойдет исключение **java.lang.NullPointerException**.

Unboxing происходит:

1. При присвоении экземпляра класса-обёртки переменной соответствующего примитивного типа.
2. В выражениях, в которых один или оба аргумента являются экземплярами классов-обёрток (кроме операции == и !=).
3. При передаче объекта класса-обёртки в метод, ожидающий соответствующий примитивный тип.

До JDK 5

```
int i = iOb.intValue();
double d = dOb.doubleValue();
char c = cOb.charValue();
boolean b = bOb.booleanValue();
```

Начиная с JDK 5

```
int i = iOb;
double d = dOb;
char c = cOb;
boolean b = bOb;
```

12 exception

возникновение ошибок и непредвиденных ситуаций при выполнении программы называют **исключением**

В программе исключения могут возникать в результате неправильных действий пользователя, отсутствии необходимого ресурса на диске, или потери соединения с сервером по сети.

Кратко о ключевых словах try, catch, finally, throws

Обработка исключений в Java основана на использовании в программе следующих ключевых слов:

- **try** – определяет блок кода, в котором может произойти исключение;
- **catch** – определяет блок кода, в котором происходит обработка исключения;
- **finally** – определяет блок кода, который является необязательным, но при его наличии выполняется в любом случае независимо от результатов выполнения блока try.

Эти ключевые слова используются для создания в программном коде специальных обрабатывающих конструкций: try{}catch, try{}catch{}finally, try{}finally{}.

- **throw** – используется для возбуждения исключения;
- **throws** – используется в сигнатуре методов для предупреждения, о том что метод может выбросить исключение.

Исключительные ситуации, возникающие в программе, можно разделить на две группы:

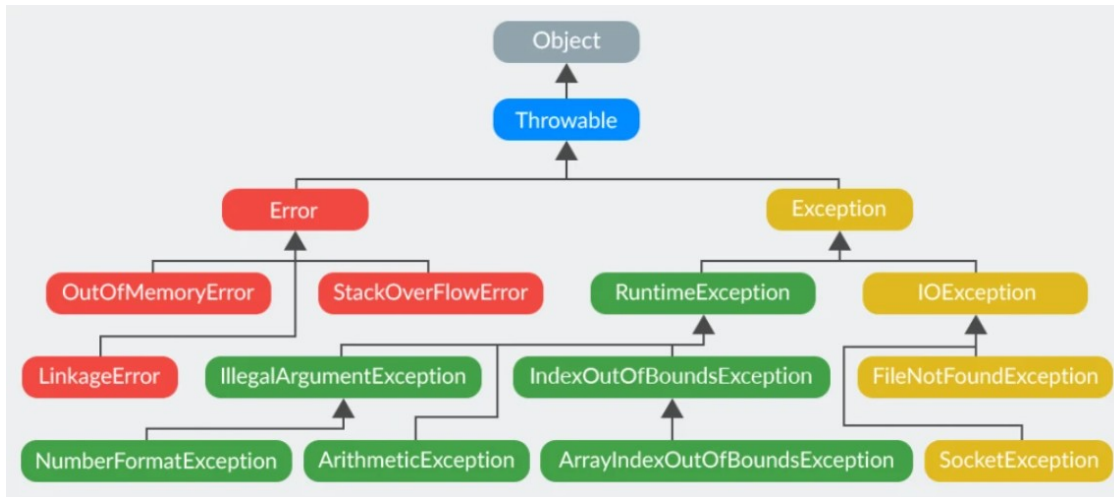
1. Ситуации, при которых восстановление дальнейшей нормальной работы программы невозможно
2. Восстановление возможно.

К первой группе относят ситуации, когда возникают исключения, унаследованные из класса Error.

Это ошибки, возникающие при выполнении программы в результате сбоя работы JVM, переполнения памяти или сбоя системы. Обычно они свидетельствуют о серьезных проблемах, устранить которые программными средствами невозможно. Такой вид исключений в Java относится к неконтролируемым (unchecked) на стадии компиляции.

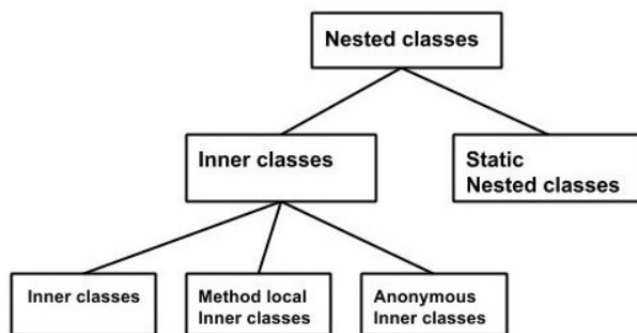
К этой группе также относят RuntimeException – исключения, наследники класса Exception, генерируемые JVM во время выполнения программы. Часто причиной возникновения их являются ошибки программирования. Эти исключения также являются неконтролируемыми (unchecked) на стадии компиляции, поэтому написание кода по их обработке не является обязательным.

Ко второй группе относят исключительные ситуации, предвидимые еще на стадии написания программы, и для которых должен быть написан код обработки. Такие исключения являются контролируемыми (checked). Основная часть работы разработчика на Java при работе с исключениями – обработка таких ситуаций.



13 inner class java

- Static Nested classes(статические вложенные классы)
- Nested Inner classes (вложенные внутренние классы)
- Method Local Inner classes (внутренние классы в локальном методе)
- Anonymous Inner classes (анонимные классы)



Для чего они нужны?

Вложенными классами описывают типы, которые будут использоваться только в одном классе или вообще будут использованы только один раз (анонимные классы). С помощью этих классов реализуется необходимость разделить данные и методы их обработки в разные типы (сущности). Сохранив, при этом, инкапсуляцию.

Static Nested class объявляется, как и любой класс, НО внутри тела класса с ключевым словом **static**. Имена вложенных классов не имеют особых правил.

Static Nested class не имеет привязки к объекту внешнего класса (не хранит в себе ссылку на конкретный экземпляр внешнего класса).

Вложенный класс (Static Nested) имеет доступ к статическим членам своего внешнего класса, в том числе и к *закрытым (private)*, даже без упоминания имени класса.

К нестатическим членам внешнего класса Nested(static) класс не имеет прямого доступа. Но такой доступ возможен через объект (экземпляр) внешнего класса, в том числе к закрытым (private) членам внешнего класса.

Вложенный класс (InnerClass)

```
public class OuterClass {  
    public class InnerClass{  
    }  
}
```

Из него видны:

- все (даже private) свойства и методы OuterClassa обычные и статические.
- public и protected свойства и методы родителя OuterClassa обычные и статические. То есть те, которые видны в OuterClassе.

Его видно:

- согласно модификатору доступа.

Может наследовать:

- обычные классы.
- такие же внутренние классы в OuterClassе и его предках.

Может быть наследован:

- таким же внутренним классом в OuterClassе и его наследниках.

Может имплементировать интерфейс

Может содержать:

- только обычные свойства и методы (не статические).

Экземпляр этого класса создаётся так:

```
OuterClass outerClass = new OuterClass();
```

```
OuterClass.InnerClass innerClass = outerClass.new InnerClass();
```

Статический вложенный класс (StaticInnerClass)

```
public class OuterClass {  
    public static class StaticInnerClass{  
    }  
}
```

Из него (самого класса) видны:

- статические свойства и методы OuterClassa (даже private).
- статические свойства и методы родителя OuterClassa public и protected. То есть те, которые видны в OuterClassе.

Из его экземпляра видны:

- все (даже private) свойства и методы OuterClassa обычные и статические.
- public и protected свойства и методы родителя OuterClassa обычные и статические. То есть те, которые видны в OuterClassе.

Его видно:

- согласно модификатору доступа.

Может наследовать:

- обычные классы.
- такие же статические внутренние классы в OuterClassе и его предках.

Может быть наследован:

- любым классом:
- вложенным
- не вложенным
- статическим
- не статическим!

Может имплементировать интерфейс**Может содержать:**

- статические свойства и методы.
- не статические свойства и методы.

Экземпляр этого класса создаётся так:

```
OuterClass.StaticInnerClass staticInnerClass = new OuterClass.StaticInnerClass();
```

Локальный класс (LocalClass)

```
public class OuterClass {  
    public void someMethod(){  
        class LocalClass{  
        }  
    }  
}
```

Из него видны:

- все (даже private) свойства и методы OuterClassa обычные и статические.
- public и protected свойства и методы родителя OuterClassa обычные и статические. То есть те, которые видны в OuterClasse.

Его видно:

- только в том методе где он определён.

Может наследовать:

- обычные классы.
- внутренние классы в OuterClasse и его предках.
- такие же локальные классы определённые в том же методе.

Может быть наследован:

- таким же локальным классом определённым в том же методе.

Может имплементировать интерфейс**Может содержать:**

- только обычные свойства и методы (не статические).

Анонимный класс (имени нет)

Локальный класс без имени. Наследует какой-то класс, или имплементирует какой-то интерфейс.

```
public class OuterClass {  
    public void someMethod(){  
        Callable callable = new Callable() {  
            @Override  
            public Object call() throws Exception {  
                return null;  
            }  
        };  
    }  
}
```

Из него видны:

- все (даже private) свойства и методы OuterClassa обычные и статические.
- public и protected свойства и методы родителя OuterClassa обычные и статические. То есть те, которые видны в OuterClasse.

Его видно:

— только в том методе, где он определён.

Не может быть наследован

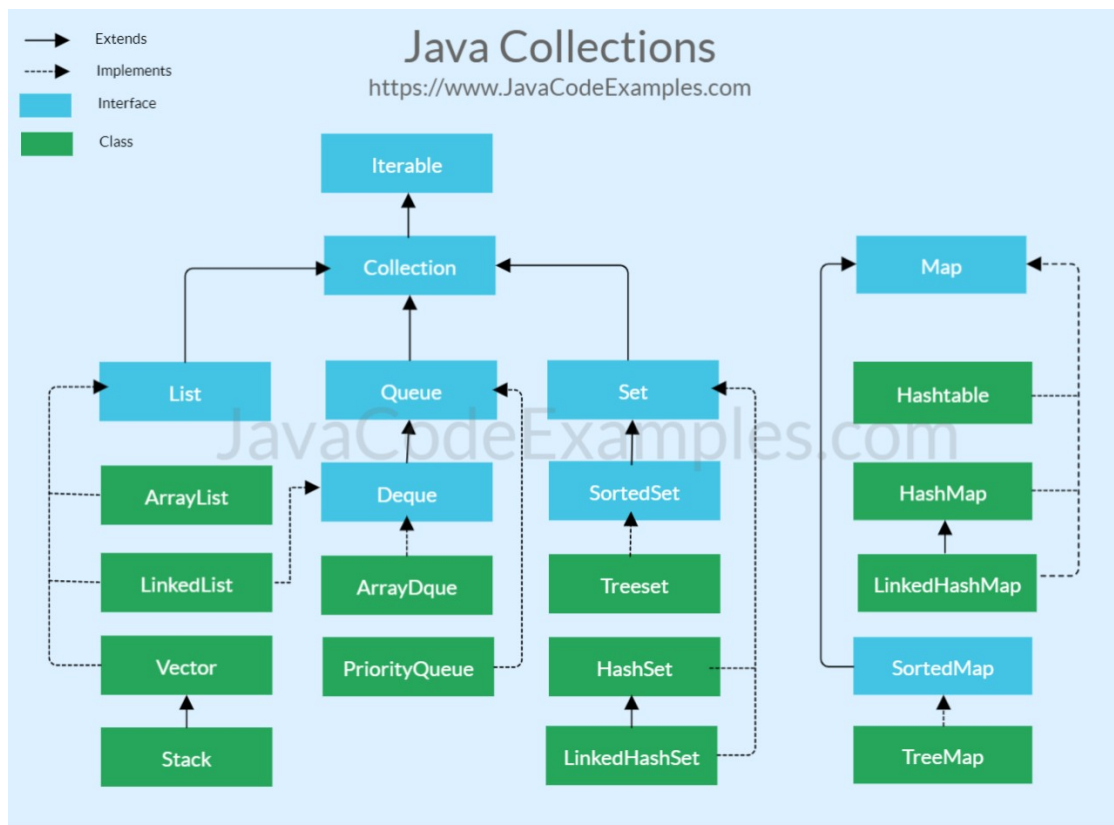
Может содержать:

— только обычные свойства и методы (не статические).

2 JAVA

2 Collections

внутреннее устройство основных коллекций, Collection vs Collections



Iterable (Базовый интерфейс реализация которого дает возможность юзать for-each)

реализуя интерфейс Iterable объект сможет быть использован внутри цикла for-each.

Чтобы класс можно было использовать в цикле for-each придется определить метод Iterator. Под капотом компилятор вызывает `list.iterator()` и выполняет итерацию, предоставляя вам `i` внутри цикла `for`.

Collection Interface [Интерфейс коллекции]

[Интерфейс коллекции является корневым интерфейсом и предоставляет общие методы, add, remove, clear, contains, equals, hashCode, and iterator.

List interface [Интерфейс списка]

[Интерфейс List расширяет интерфейс коллекций.]

[Список представляет собой упорядоченную коллекцию объектов на основе индекса.]

[Элементы, содержащиеся в Списке, упорядочены, и их можно вставлять, просматривать или искать на основе их индекса.]

[Список может содержать повторяющиеся элементы.]

[Основными классами, реализующими интерфейс List, являются ArrayList и LinkedList.]

Set interface [Установить интерфейс]

[Интерфейс Set расширяет интерфейс Collection и представляет собой коллекцию, которая не содержит никаких повторяющихся элементов (она также может иметь только один нулевой элемент).]

[Основными классами, реализующими интерфейс Set, являются TreeSet, HashSet и LinkedHashSet.]

Queue Interface [Интерфейс очереди]

[Интерфейс Queue расширяет интерфейс Collection и представляет коллекцию, которая обычно упорядочивается по FIFO (первым пришел - первым ушел).]

Deque наследует Queue || PriorityQueue реализует Queue

LinkedList, ArrayDeque реализуют Deque

Map Interface [Интерфейс карты]

[Интерфейс карты является корневым интерфейсом и позволяет хранить пары ключ-значение.]

[Карта не допускает дублирования ключей.]

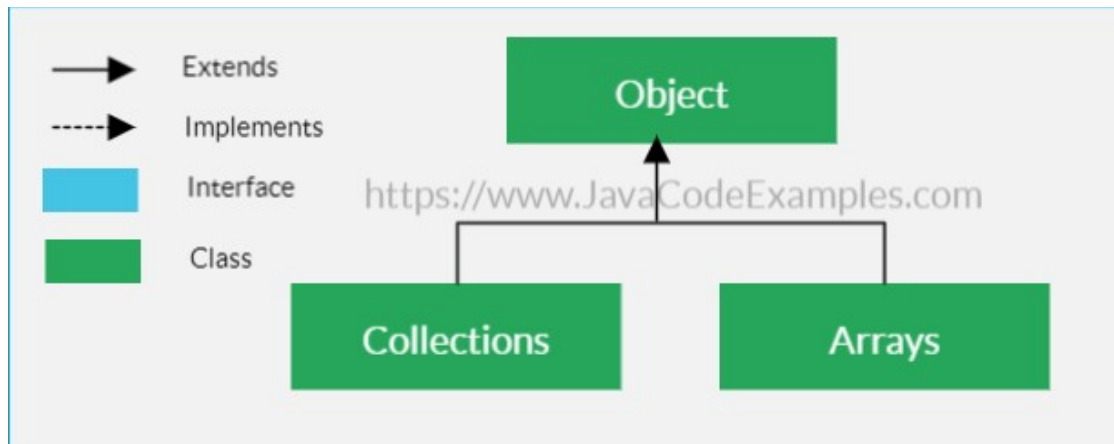
[Интерфейс карты не гарантирует порядок элементов, однако некоторые реализации, такие как TreeMap, делают.]

[Основными классами, реализующими интерфейс Map, являются Hashtable, HashMap, TreeMap и LinkedHashMap.]

Utility Classes [Полезные классы] Collections and Arrays Arrays Classes

In addition to the above-mentioned main interfaces and classes, there are two utility classes that are part of the Java collection framework.

[В дополнение к вышеупомянутым основным интерфейсам и классам есть два служебных класса, которые являются частью структуры коллекции Java.]



Java Collections and Arrays Utility classes

1. [1.] Collections class [Класс коллекций]

[Класс Collections содержит статические служебные методы, которые либо принимают, либо возвращают коллекцию.]

[Класс коллекции предоставляет множество полезных методов для перетасовки, обращения, сортировки и поиска объектов коллекции.]

2. [2.] Arrays Class [Класс массивов]

[Подобно классу Collections, класс Arrays содержит статические служебные методы для управления массивами.]

[Класс Arrays предоставляет множество полезных методов для сортировки, поиска, копирования и заполнения массивов.]

ArrayList

динамический массив, по умолчанию размер 10, но увеличивается в случае добавления элементов сверх размера

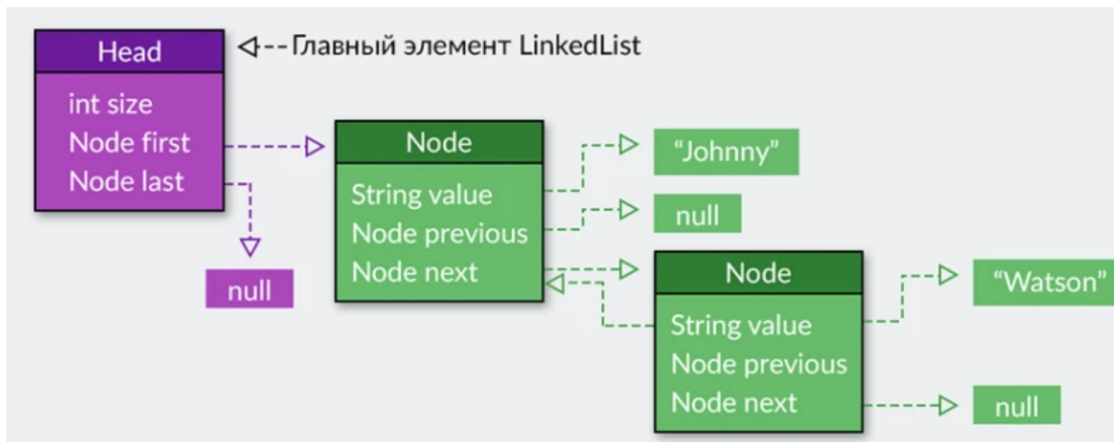
лучше заранее указать размер если известно сколько элементов будет в нем храниться

- Так же, как и статический массив, индексируется с 0;
- Вставка в конец и доступ по индексу очень быстрые — $O(1)$;

- Чтобы вставить элемент в начало или середину, понадобится скопировать все элементы на одну ячейку вправо, а затем вставить новый элемент на необходимую позицию;
- Доступ по значению зависит от количества элементов — $O(n)$;
- В отличие от классического массива, может хранить null;

LinkedList

Внутри LinkedList у нас есть главный объект — Head, хранит информацию о количестве элементов, и ссылку на первый и последний элементы



Подводя итог рассказу о LinkedList, можно вывести несколько правил его работы:

Так же, как и массив, индексируется с 0;

Доступ к первому и последнему элементу не зависят от количества элементов — $O(1)$;

Получение элемента по индексу, вставка или удаление из середины списка зависят от количества элементов — $O(n)$;

Можно использовать механизм итератора: тогда вставка и удаление будут происходить за константное время;

В отличие от классического массива, может хранить null.

HashSet

Название Hash... происходит от понятия хэш-функция. Хэш-функция — это функция, сужающая множество значений объекта до некоторого подмножества целых чисел. Класс **Object** имеет метод **hashCode()**, который используется классом **HashSet** для эффективного размещения объектов, заносимых в коллекцию. В классах объектов, заносимых в **HashSet**, этот метод должен быть переопределен (override).

метод `add(Object o)` добавляет объект в множество только в том случае, если его там нет

HashSet использует хэширование для ускорения выборки. Если вам нужно, чтобы результат был отсортирован, то пользуйтесь **TreeSet**.

LinkedHashSet

Класс **LinkedHashSet** расширяет класс **HashSet**, не добавляя никаких новых методов. Класс поддерживает связный список элементов набора в том порядке, в котором они вставлялись. Это позволяет организовать упорядоченную итерацию вставки в набор.

TreeSet реализует SortedSet

Если сортировка для вас важна, то используйте **TreeSet**.

SortedSet

В примере с **TreeSet** использовался интерфейс **SortedSet**, который позволяет сортировать элементы множества. По умолчанию сортировка производится привычным способом, но можно изменить это поведение через интерфейс **Comparable**.

Интерфейс **Map<K, V>** представляет отображение или иначе говоря словарь, где каждый элемент представляет пару "ключ-значение". При этом все ключи уникальные в рамках объекта **Map**. Такие коллекции облегчают поиск элемента, если нам известен ключ - уникальный идентификатор объекта.

Следует отметить, что в отличие от других интерфейсов, которые представляют коллекции, интерфейс **Map** НЕ расширяет интерфейс **Collection**.

Конструктор с используемыми по умолчанию значениями начальной емкости (16) и коэффициентом загрузки (0.75)

2. Java

3. Concurrency

Thread vs Runnable

Процесс — это экземпляр программы, который запускается независимо от остальных. Например, когда вы запускаете программу на Java, ОС создает новый процесс, который работает параллельно другим. Внутри процессов мы можем использовать потоки, тем самым выжав из процессора максимум возможностей.

Потоки (*threads*) в Java поддерживаются начиная с JDK 1.0. Прежде чем запустить поток, ему надо предоставить участок кода, который обычно называется «задачей» (*task*). Это делается через реализацию интерфейса `Runnable`, у которого есть только один метод без аргументов, возвращающий `void` — `run()`. Вот пример того, как это работает:

```
1 Runnable task = () -> {  
2     String threadName = Thread.currentThread().getName();  
3     System.out.println("Hello " + threadName);  
4 };  
5  
6 task.run();  
7  
8 Thread thread = new Thread(task);  
9 thread.start();  
10  
11 System.out.println("Done!");
```

Исполнители не создают потоки, выполняют задачи асинхронно

Исполнитель нужно остановить явно `shutdown()` / `shutdownnow()`

Исполнители

Concurrency API вводит понятие сервиса-исполнителя (*ExecutorService*) — высокоуровневую замену работе с потоками напрямую. Исполнители выполняют задачи асинхронно и обычно используют пул потоков, так что нам не надо создавать их вручную. Все потоки из пула будут использованы повторно после выполнения задачи, а значит, мы можем создать в приложении столько задач, сколько хотим, используя один исполнитель.

Вот как будет выглядеть наш первый пример с использованием исполнителя:

```
1 | ExecutorService executor = Executors.newSingleThreadExecutor();
2 | executor.submit(() -> {
3 |     String threadName = Thread.currentThread().getName();
4 |     System.out.println("Hello " + threadName);
5 | });
6 |
7 | // => Hello pool-1-thread-1
```

Класс `Executors` предоставляет удобные методы-фабрики для создания различных сервисов исполнителей. В данном случае мы использовали исполнитель с одним потоком.

Результат выглядит так же, как в прошлый раз. Но у этого кода есть важное отличие — он никогда не остановится. Работу исполнителей надо завершать явно. Для этого в интерфейсе `ExecutorService` есть два метода: `shutdown()`, который ждет завершения запущенных задач, и `shutdownNow()`, который останавливает исполнитель немедленно.

Callable и Future

Кроме `Runnable`, исполнители могут принимать другой вид задач, который называется `Callable`. `Callable` — это также функциональный интерфейс, но, в отличие от `Runnable`, он может возвращать значение.

Давайте напишем задачу, которая возвращает целое число после секундной паузы:

```
1 | Callable task = () -> {
2 |     try {
3 |         TimeUnit.SECONDS.sleep(1);
4 |         return 123;
5 |     }
6 |     catch (InterruptedException e) {
7 |         throw new IllegalStateException("task interrupted", e);
8 |     }
9 | };
```

Callable-задачи также могут быть переданы исполнителям. Но как тогда получить результат, который они возвращают? Поскольку метод `submit()` не ждет завершения задачи, исполнитель не может вернуть результат задачи напрямую. Вместо этого исполнитель возвращает специальный объект `Future`, у которого мы сможем запросить результат задачи.

```
1 | ExecutorService executor = Executors.newFixedThreadPool(1);
2 | Future<Integer> future = executor.submit(task);
3 |
4 | System.out.println("future done? " + future.isDone());
5 |
6 | Integer result = future.get();
7 |
8 | System.out.println("future done? " + future.isDone());
9 | System.out.print("result: " + result);
```

После отправки задачи исполнителю мы сначала проверяем, завершено ли ее выполнение, с помощью метода `isDone()`. Поскольку задача имеет задержку в одну секунду, прежде чем вернуть число, я более чем уверен, что она еще не завершена.

Вызов метода `get()` блокирует поток и ждет завершения задачи, а затем возвращает результат ее выполнения. Теперь `future.isDone()` вернет `true`, и мы увидим на консоли следующее:

```
1 | future done? false
2 | future done? true
3 | result: 123
```

Задачи жестко связаны с сервисом исполнителей, и, если вы его остановите, попытка получить результат задачи выбросит исключение:

```
1 | executor.shutdownNow();
2 | future.get();
```

Вы, возможно, заметили, что на этот раз мы создаем сервис немного по-другому: с помощью метода `newFixedThreadPool(1)`, который вернет исполнителя с пулом в один поток. Это эквивалентно вызову метода `newSingleThreadExecutor()`, однако мы можем изменить количество потоков в пуле.

Таймауты

Любой вызов метода `future.get()` блокирует поток до тех пор, пока задача не будет завершена. В наихудшем случае выполнение задачи не завершится никогда, блокируя ваше приложение. Избежать этого можно, передав таймаут:

```
1 | ExecutorService executor = Executors.newFixedThreadPool(1);
2 |
3 | Future<Integer> future = executor.submit(() -> {
4 |     try {
5 |         TimeUnit.SECONDS.sleep(2);
6 |         return 123;
7 |     }
8 |     catch (InterruptedException e) {
9 |         throw new IllegalStateException("task interrupted", e);
10 |    }
11 | });
12 |
13 | future.get(1, TimeUnit.SECONDS);
```

Выполнение этого кода вызовет `TimeoutException`:

```
1 | Exception in thread "main" java.util.concurrent.TimeoutException
2 |     at java.util.concurrent.FutureTask.get(FutureTask.java:205)
```

Вы уже, возможно, догадались, почему было выброшено это исключение: мы указали максимальное время ожидания выполнения задачи в одну секунду, в то время как ее выполнение занимает две.

InvokeAll

Исполнители могут принимать список задач на выполнение с помощью метода `invokeAll()`, который принимает коллекцию callable-задач и возвращает список из `Future`.

```

1 | ExecutorService executor = Executors.newWorkStealingPool();
2 |
3 | List<Callable<String>> callables = Arrays.asList(
4 |     () -> "task1",
5 |     () -> "task2",
6 |     () -> "task3");
7 |
8 | executor.invokeAll(callables)
9 |     .stream()
10 |    .map(future -> {
11 |        try {
12 |            return future.get();
13 |        }
14 |        catch (Exception e) {
15 |            throw new IllegalStateException(e);
16 |        }
17 |    })
18 |    .forEach(System.out::println);

```

InvokeAny

Другой способ отдать на выполнение несколько задач — метод `invokeAny()`. Он работает немного по-другому: вместо возврата `Future` он блокирует поток до того, как завершится хоть одна задача, и возвращает ее результат.

Чтобы показать, как работает этот метод, создадим метод, эмулирующий поведение различных задач. Он будет возвращать `Callable`, который вернет указанную строку после необходимой задержки:

```

1 Callable callable(String result, long sleepSeconds) {
2     return () -> {
3         TimeUnit.SECONDS.sleep(sleepSeconds);
4         return result;
5     };
6 }

```

Используем этот метод, чтобы создать несколько задач с разными строками и задержками от одной до трех секунд. Отправка этих задач исполнителю через метод `invokeAny()` вернет результат задачи с наименьшей задержкой. В данном случае это «task2»:

```

1 ExecutorService executor = Executors.newWorkStealingPool();
2
3 List<Callable<String>> callables = Arrays.asList(
4     callable("task1", 2),
5     callable("task2", 1),
6     callable("task3", 3));
7
8 String result = executor.invokeAny(callables);
9 System.out.println(result);
10
11 // => task2

```

В примере слева использован еще один вид исполнителей, который создается с помощью метода `newWorkStealingPool()`. Этот метод появился в Java 8 и ведет себя не так, как другие: вместо использования фиксированного количества потоков он создает [ForkJoinPool](#) с определенным параллелизмом (*parallelism size*), по умолчанию равным количеству ядер машины.

Исполнители с планировщиком

Мы уже знаем, как отдать задачу исполнителю и получить ее результат. Для того, чтобы периодически запускать задачу, мы можем использовать пул потоков с планировщиком.

`ScheduledExecutorService` способен запускать задачи один или несколько раз с заданным интервалом.

Этот пример показывает, как заставить исполнитель выполнить задачу через три секунды:

```

1 ScheduledExecutorService executor = Executors.newScheduledThreadPool(1);
2
3 Runnable task = () -> System.out.println("Scheduling: " + System.nanoTime());
4 ScheduledFuture<?> future = executor.schedule(task, 3, TimeUnit.SECONDS);
5
6 TimeUnit.MILLISECONDS.sleep(1337);
7
8 long remainingDelay = future.getDelay(TimeUnit.MILLISECONDS);
9 System.out.printf("Remaining Delay: %sms", remainingDelay);

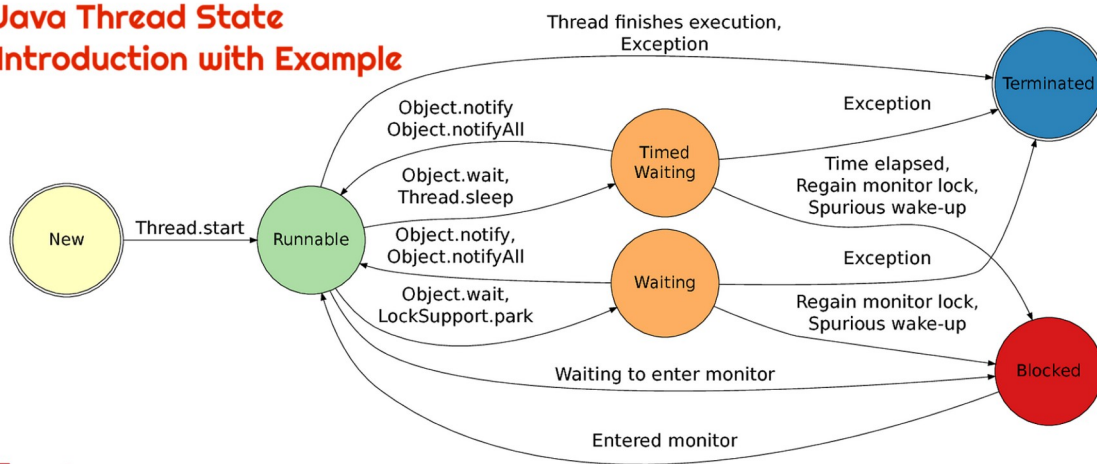
```


Когда мы передаем задачу планировщику, он возвращает особый тип Future — ScheduledFuture, который предоставляет метод getDelay() для получения оставшегося до запуска времени.

У исполнителя с планировщиком есть два метода для установки задач: scheduleAtFixedRate() и scheduleWithFixedDelay()

RUNNABLE & THREAD

Java Thread State Introduction with Example



Thread это вообще как нить переводится, а не поток, поток это Stream и совсем про другое в проганые.

Когда запускается любое приложение, то начинает выполняться поток, называемый *главным потоком* (main). От него порождаются дочерние потоки. Главный поток, как правило, является последним потоком, завершающим выполнение программы.

Несмотря на то, что главный поток создаётся автоматически, им можно управлять через объект класса **Thread**. Для этого нужно вызвать метод `currentThread()`, после чего можно управлять потоком.

Класс **Thread** содержит несколько методов для управления потоками.

- **getName()** - получить имя потока
- **getPriority()** - получить приоритет потока
- **isAlive()** - определить, выполняется ли поток
- **join()** - ожидать завершения потока
- **run()** - запуск потока. В нём пишете свой код
- **sleep()** - приостановить поток на заданное время
- **start()** - запустить поток

Синхронно (блокирующее) и Асинхронно (неблокирующее)

- ✓ А сам код синхронным не называется, это его по ошибке или для упрощения так называют. Синхронным и асинхронным называется только API ввода-вывода, т.е. операции, прерывающие исполнение кода и требующие от системы обратиться к внешнему устройству, работающему не синхронно с центральным процессором. Операции ввода-вывода, каковые есть: работа с дисками, портами, контроллерами, периферийными устройствами, как клавиатура, мышь, тачскрин, разные датчики, вебкамера, сетевые карты, блютузы и другие радиомодули, принтеры, видеокарты и прочее. Все они получают задание от программы, и исполняют его отдельно, своими мощностями. Потом внешние устройства присылают программе сигнал о статусе исполнения и, возможно, полученные данные. Программа все это время может ждать (если у нее синхронное API, т.е. блокирующее) или что-то делать (если асинхронное, т.е. не блокирующее). Если программа ждет, не переходит к выполнению следующего действия, то это синхронный ввод-вывод, потому, что осуществляется процесс синхронизации программы с внешним устройством. Внешне устройство посылает прерывание, которое обрабатывает операционная система и через несколько слоев драйверов оно попадает в программу, обычно в виде колбека или события. Если программа ждала, то вызов API не завершился, она все время слушала, когда придет событие о завершении операции ввода вывода, а получив его API отдает ответ и управление переходит к следующей команде, что и называется, синхронизацией с периферийным устройством. Если программа не ждала, то вызов API сразу завершается и не блокирует поток выполнения программ, это называется асинхронным API, потому, что процесс синхронизации не происходит явно, а ответы возвращаются через события.

Создание собственного потока Thread

Объявим внутри нашего класса внутренний класс и вызовем его, вызвав метод **start()**. Внутри **run** пишем логику

```
public class MyThread extends Thread {
    public void run() {
        Log.d(TAG, "Мой поток запущен...");
    }
}

public void onClick(View view) {
    MyThread myThread = new MyThread();
    myThread.start();
}
```

Создание потока с интерфейсом Runnable

Runnable. Вы можете создать поток из любого объекта, реализующего интерфейс **Runnable** и объявить метод **run()**.

```

class MyRunnable implements Runnable {
    Thread thread;

    // Конструктор
    MyRunnable() {
        // Создаём новый второй поток
        thread = new Thread(this, "Поток для примера");
        Log.i(TAG, "Создан второй поток " + thread);
        thread.start(); // Запускаем поток
    }

    // Обязательный метод для интерфейса Runnable
    public void run() {
        try {
            for (int i = 5; i > 0; i--) {
                Log.i(TAG, "Второй поток: " + i);
                Thread.sleep(500);
            }
        } catch (InterruptedException e) {
            Log.i(TAG, "Второй поток прерван");
        }
    }
}

```

Внутри метода **run()** вы размещаете код для нового потока. Этот поток завершится, когда метод вернёт управление.

Когда вы объявите новый класс с интерфейсом **Runnable**, вам нужно использовать конструктор:

`Thread(Runnable объект_потока, String имя_потока)`

После создания нового потока, его нужно запустить с помощью метода **start()**, который, по сути, выполняет вызов метода **run()**.

В первом параметре использовался объект **this**, что означает желание вызвать метод **run()** этого объекта. Далее вызывается метод **start()**, в результате чего запускается выполнение потока, начиная с метода **run()**. В свою очередь метод запускает цикл для нашего потока. После вызова метода **start()**, конструктор **MyRunnable()** возвращает управление приложению. Когда главный поток продолжает свою работу, он входит в свой цикл. После этого оба потока выполняются параллельно.

Можно запускать несколько потоков, а не только второй поток в дополнение к первому. Это может привести к проблемам, когда два потока пытаются работать с одной переменной одновременно. Тут нужна **СИНХРОНИЗАЦИЯ**

Ключевое слово `synchronized` - синхронизированные методы

Для решения проблемы с потоками, которые могут внести путаницу, используется синхронизация.

Метод может иметь модификатор **synchronized**. Когда поток находится внутри синхронизированного метода, все другие потоки, которые пытаются вызвать его в том же экземпляре, должны ожидать. Это позволяет исключить путаницу, когда несколько потоков пытаются вызвать метод.

Кроме того, ключевое слово **synchronized** можно использовать в качестве оператора. Вы можете заключить в блок **synchronized** вызовы методов какого-нибудь класса:

```
synchronized(объект) {  
    // операторы, требующие синхронизации  
}
```

в многопоточном программировании ввели специальное понятие **мьютекс (от англ. «mutex», «mutual exclusion» — «взаимное исключение»)**.

Задача мьютекса — обеспечить такой механизм, чтобы доступ к объекту в определенное время был только у одного потока. Если Поток-1 захватил мьютекс объекта А, остальные потоки не получают к нему доступ, чтобы что-то в нем менять. До тех пор, пока мьютекс объекта А не освободится, остальные потоки будут вынуждены ждать.

Не нужно ничего делать, чтобы создать мьютекс: он уже встроен в класс Object, а значит, есть у каждого объекта в Java.

Если один поток зашел внутрь блока кода, который помечен словом **synchronized**, он моментально захватывает мьютекс объекта, и все другие потоки, которые попытаются зайти в этот же блок или метод вынуждены ждать, пока предыдущий поток не завершит свою работу и не освободит монитор.

Эти два способа записи означают одно и то же:

```
1 public void swap() {  
2  
3     synchronized (this)  
4     {  
5         //...логика метода  
6     }  
7 }  
8  
9  
10 public synchronized void swap() {  
11  
12     }  
13 }
```

Если метод, в котором «многопоточная» логика, статический, синхронизация будет осуществляться по классу.

```
public static void swap() {  
  
    synchronized (MyClass.class) {  
        String s = name1;  
        name1 = name2;  
        name2 = s;  
    }  
}
```

по классам тоже можно синхронизироваться

Мьютекс — это специальный объект для синхронизации потоков. Он «прикреплен» к каждому объекту в Java

задача мьютекса — обеспечить такой механизм, чтобы доступ к объекту в определенное время был только у одного потока.

Монитор — это дополнительная «надстройка» над мьютексом.

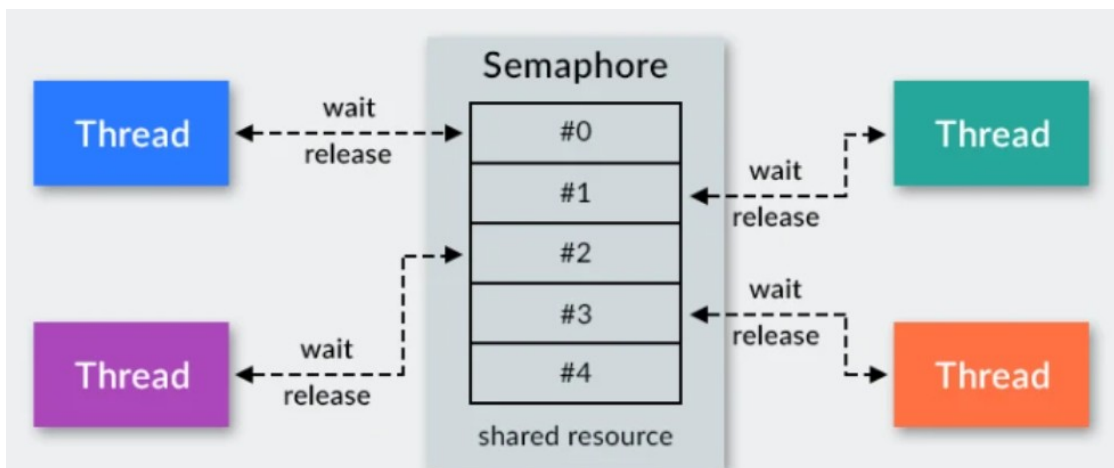
Задача монитора сказать занят или свободен кусок кода, давая понять следующему кто хочет получить доступ что надо или не надо ждать!

Семафор

Семафор — это средство для синхронизации доступа к какому-то ресурсу.

Его особенность заключается в том, что при создании механизма синхронизации он использует счетчик.

Счетчик указывает нам, сколько потоков одновременно могут получать доступ к общему ресурсу.



Семафоры в Java представлены классом Semaphore. При создании объектов-семафоров мы можем использовать такие конструкторы:

```
1 Semaphore(int permits)
2 Semaphore(int permits, boolean fair)
```

В конструктор мы передаем:

- `int permits` — начальное и максимальное значение счетчика. То есть то, сколько потоков одновременно могут иметь доступ к общему ресурсу;
- `boolean fair` — для установления порядка, в котором потоки будут получать доступ. Если `fair = true`, доступ предоставляется ожидающим потокам в том порядке, в котором они его запрашивали. Если же он равен `false`, порядок будет определять планировщик потоков.

Разница от мьютекса только в том, что

мьютекс объекта может захватить одновременно только один поток, а в случае с семафором используется счетчик потоков, и доступ к ресурсу могут получить сразу несколько из них. И это не просто случайное сходство :)

На самом деле **мьютекс — это одноместный семафор**. То есть, это семафор, счетчик которого изначально установлен в значении 1. Его еще называют «двоичным семафором», поскольку его счетчик может иметь только 2 значения — 1 («свободно») и 0 («занято»).

мьютекс и семафор - объекты ядра операционной системы, а взаимодействия с ними происходит посредством API ОС (WinAPI, Linux API). Высокоуровневые языки просто дёргают эти API функции

• КАКИМ ОБРАЗОМ МОЖНО СОЗДАТЬ ПОТОК?

Есть несколько способов создания и запуска потоков:

- С помощью класса, реализующего `Runnable`:
 - Создать объект класса `Thread`.
 - Создать объект класса, реализующего интерфейс `Runnable`.
 - Вызвать у созданного объекта `Thread` метод `start()` (после этого запустится метод `run()` у переданного объекта, реализующего `Runnable`).
- С помощью класса, расширяющего `Thread`:
 - Создать объект класса `ClassName extends Thread`.
 - Переопределить `run()` в этом классе (смотрите примере ниже, где передается имя потока `Second`).
- С помощью класса, реализующего `java.util.concurrent.Callable`:
 - Создать объект класса, реализующего интерфейс `Callable`.
 - Создать объект `ExecutorService` с указанием пула потоков.
 - Создать объект `Future`. Запуск происходит через метод `submit()`; Сигнатура: `<T> Future<T> submit(Callable<T> task)`.

- **КАКИЕ СПОСОБЫ СИНХРОНИЗАЦИИ В JAVA?**

Ниже приведены некоторые способы синхронизации в Java:

- Системная синхронизация с использованием wait/notify. Поток, который ждет выполнения каких-либо условий, вызывает у этого объекта метод wait, предварительно захватив его монитор. На этом его работа приостанавливается. Другой поток может вызвать на этом же самом объекте метод notify (опять же, предварительно захватив монитор объекта), в результате чего, ждущий на объекте поток "просыпается" и продолжает свое выполнение.
- Системная синхронизация с использованием join. Метод join, вызванный у экземпляра класса Thread, позволяет текущему потоку остановиться до того момента, как поток, связанный с этим экземпляром, закончит работу.
- Использование классов из пакета java.util.concurrent, который предоставляет набор классов для организации межпоточного взаимодействия. Примеры таких классов – Lock, семафор (Semaphore), etc. Концепция данного подхода заключается в использовании атомарных операций и переменных.

Object для синхронизации есть ряд методов:

- **wait()**: освобождает монитор и переводит вызывающий поток в состояние ожидания до тех пор, пока другой поток не вызовет метод notify()
- **notify()**: продолжает работу потока, у которого ранее был вызван метод wait()
- **notifyAll()**: возобновляет работу всех потоков, у которых ранее был вызван метод wait()

- **КАК РАБОТАЮТ МЕТОДЫ WAIT И NOTIFY/NOTIFYALL?**

Эти методы предназначены для межпоточной синхронизации, для взаимодействия потоков между собой.

Как работают эти методы. Во-первых они могут вызваны только потоком, который захватил монитор объекта, для которого эти методы вызываются. То есть они вызываются внутри блока synchronized и для объекта, монитор которого этим synchronized захвачен. Если внутри synchronized метода – то для класса, к которому относятся эти методы.

Что делает метод wait(). Метод wait() отдает (освобождает) монитор объекта, так что другие потоки теперь могут его (монитор) захватить, то есть войти в блок synchronized для этого объекта. Затем метод wait() переходит в состояние ожидания, до тех пор пока другой поток не вызовет метод notify() или notifyAll() для этого же объекта. После чего поток, в котором был вызван wait(), пытается снова захватить монитор объекта и когда монитор становится свободным, то есть когда другой поток освобождает его, захватывает монитор и продолжает выполнение со следующего после wait() оператора. Причем у потока вызвавшего wait() нет никакого преимущества перед другими потоками, ожидающими захвата того же монитора.

Что делают методы notify(), notifyAll(). Они "пробуждают" поток, ожидающий методом wait() (если такой есть), и переводят его в состояние ожидания освобождения монитора. Разница между notify() и notifyAll() в том, что notify() пробуждает только один поток, ожидающий методом wait(), какой именно будет пробужден – определить нельзя, а notifyAll() – все такие потоки.

- **ЧЕМ ОТЛИЧАЕТСЯ РАБОТА МЕТОДА WAIT С ПАРАМЕТРОМ И БЕЗ ПАРАМЕТРА?**

Разница методов в следующем:

- final void wait() – метод используется в многопоточной среде, может вызываться только потоком, владеющим объектом синхронизации. При этом объект синхронизации освобождается, а текущий поток переходит в режим ожидания сигнала освобождения объекта синхронизации другим потоком путем вызова метода notify() либо notifyAll().
- final void wait(long time) – аналогично wait() данный метод используется в многопоточной среде, переходит текущий поток в режим ожидания сигнала освобождения объекта синхронизации другим потоком путем вызова метода notify() либо notifyAll(), или ожидание происходит заданное время time, затем выполнение продолжается безусловно.

- **КАК РАБОТАЕТ МЕТОД `THREAD.YIELD()`? ЧЕМ ОТЛИЧАЮТСЯ МЕТОДЫ `THREAD.SLEEP()` И `THREAD.YIELD()`?**

Основные отличия:

- метод `yield()` – пытается сказать планировщику потоков, что нужно выполнить другой поток, что ожидает в очереди на выполнение. Метод не пытается перевести текущий поток в состояние блокировки, сна или ожидания. Он просто пытается его перевести из состояния "работающий" в состояние "работоспособный". Однако выполнение метода может вообще не произвести никакого эффекта. состояние потока остается `RUNNABLE`
- метод `sleep()` – приостанавливает поток на указанное. состояние меняется на `TIMED-WAITING`, по истечению – `RUNNABLE`
- метод `wait()` – меняет состояние потока на `WAITING` может быть вызвано только у объекта владеющего блокировкой, в противном случае выкинется исключение `IllegalMonitorStateException`. при срабатывании метода блокировка отпускается, что позволяет продолжить работу другим потокам ожидающим захватить ту же самую блокировку . в случае `wait(int)` с аргументом состояние будет `TIMED-WAITING`.

- **КАК РАБОТАЕТ МЕТОД `THREAD.JOIN()`?**

Метод `join()` вызывается для того, чтобы привязать текущий поток в конец потока для которого вызывается метод. То есть второй поток будет в режиме блокировки пока первый поток не выполнится.

- **ЧТО ТАКОЕ DEAD LOCK?**

Это когда один поток А получил блокировку на объект A1, а поток В получил блокировку на объект B1. В то время как поток А пытается получить блокировку на объект B1, а поток В на A1.

- **ДЛЯ ЧЕГО ИСПОЛЬЗУЕТСЯ КЛЮЧЕВОЕ СЛОВО `VOLATILE`, `SYNCHRONIZED`, `TRANSIENT`, `NATIVE`?**

Краткое описание ключевых слов:

- `volatile` – указывает на то, что поле синхронизировано для нескольких потоков
- `synchronized` – указывает на то что метод синхронизированный или же в методе может находится такой блок синхронизации.
- `transient` – указывает на то, что переменная не подлежит сериализации
- `native` – говорит о том, что реализация метода написана на другой программной платформе

- **ЧТО ЗНАЧИТ ПРИОРИТЕТ ПОТОКА?**

Приоритет потока – это число от 1 до 10, в зависимости от которого, планировщик потоков выбирает какой поток запускать. Однако полагаться на приоритеты для предсказуемого выполнения многопоточной программы нельзя!

- **ЧТО ТАКОЕ ПОТОКИ – ДЕМОНЫ В ДЖАВА?**

Это потоки, которые работают в фоновом режиме и не гарантируют что они завершатся. То есть если все потоки завершились, то поток демон просто обрывается вместе с закрытием приложения.

- **ЧТО ЗНАЧИТ УСЫПИТЬ ПОТОК?**

Перевести поток в спящее состояние можно с помощью метода `sleep(long ms) ms` – время в миллисекундах.

При вызове этого метода, поток переходит в спящее состояние, после сна, поток переходит в пул потоков и находится в состоянии "работоспособный", т.е. не гарантируется что после пробуждения он будет сразу выполняться. Также поток не может усыпить другой поток, так как метод `sleep` – это статический метод! Вы просто усыпите текущий поток и не более того! Также метод `sleep()` может возбуждать `InterruptedException()`.

- **ЧЕМ ОТЛИЧАЮТСЯ ДВА ИНТЕРФЕЙСА ДЛЯ РЕАЛИЗАЦИИ ЗАДАЧ `RUNNABLE` И `CALLABLE`?**

Основные различия:

- Интерфейс `Runnable` появился в Java 1.0, а интерфейс `Callable` был введен в Java 5.0 в составе библиотеки `java.util.concurrent`.
- Классы, реализующие интерфейс `Runnable` должны реализовывать метод `run()` для выполнения задачи. Классы, реализующие интерфейс `Callable` должны реализовывать метод `call()` для выполнения задачи.
- Метод `Runnable.run()` не возвращает никакого значения, его тип `void`, а метод `Callable.call()` может возвращать значение типа `T`. Интерфейс `Callable` является параметризованным `Callable<T>` и тип значения, которое будет возвращаться в методе `call()` задается этим параметром `T`.
- Метод `run()` не может бросить проверяемое исключение, в то время как метод `call()` может бросить проверяемое исключение.

- **ЧТО ТАКОЕ СОСТОЯНИЕ ГОНКИ (RACE CONDITION)?**

Состояние гонки – причина трудноуловимых багов. Как сказано в самом названии, состояние гонки возникает из-за гонки между несколькими нитями, если нить, которая должна исполняться первой, проиграла гонку и исполняется вторая, поведение кода изменяется, из-за чего возникают недетерминированные баги. Это одни из сложнейших к отлавливанию и воспроизведению багов, из-за беспорядочной природы гонок между нитями. Пример состояния гонки – беспорядочное исполнение.

- **КАК ОСТАНОВИТЬ НИТЬ?**

Java предоставляет богатые API для всего, но, по иронии судьбы, не предоставляет удобных способов остановки нити. В JDK 1.0 было несколько управляющих методов, например `stop()`, `suspend()` и `resume()`, которые были помечены как `deprecated` в будущих релизах из-за потенциальных угроз взаимной блокировки, с тех пор разработчики Java API не предприняли попыток представить стойкий, ните-безопасный и элегантный способ остановки нитей. Программисты в основном полагаются на факт того, что нить останавливается сама, как только заканчивает выполнять методы `run()` или `call()`. Для остановки вручную, программисты пользуются преимуществом `volatile boolean` переменной и проверяют её значение в каждой итерации, если в методе `run()` есть циклы, или прерывают нити методом `interrupt()` для внезапной отмены заданий.

- **ЧТО ПРОИСХОДИТ, КОГДА В НИТИ ПОЯВЛЯЕТСЯ ИСКЛЮЧЕНИЕ?**

Это один из хороших вопросов с подвохом. Простыми словами, если исключение не поймано – нить мертва, если установлен обработчик непожатых исключений, он получит колбек. `Thread.UncaughtExceptionHandler` – интерфейс, определённый как вложенный интерфейс для обработчиков, вызываемых, когда нить внезапно останавливается из-за непожатого исключения. Когда нить собирается остановиться из-за непожатого исключения, JVM проверит её на наличие `UncaughtExceptionHandler`, используя `Thread.getUncaughtExceptionHandler()`, и вызовет у обработчика метод `uncaughtException()`, передав нить и исключение в виде аргументов.

- **ПОЧЕМУ МЕТОДЫ WAIT И NOTIFY ВЫЗЫВАЮТСЯ В СИНХРОНИЗИРОВАННОМ БЛОКЕ?**

Основная причина вызова wait и notify из синхронизированного блока или метода в том, что Java API обязательно требует этого. Если вы вызовете их не из синхронизированного блока, ваш код выбросит IllegalMonitorStateException. Более хитрая причина в том, чтобы избежать состояния гонки между вызовами wait и notify.

- **ЧТО ТАКОЕ ПУЛ НИТЕЙ?**

Создание нити затратно в плане времени и ресурсов. Если вы создаёте нить во время обработки запроса, это замедлит время отклика, также процесс может создать только ограниченное число нитей. Чтобы избежать этих проблем, во время запуска приложения создаётся пул нитей и нити повторно используются для обработки запросов. Этот пул нитей называется "thread pool", а нити в нём – рабочая нить. Начиная с Java 1.5 Java API предоставляет фреймворк Executor, который позволяет вам создавать различные пулы нитей, например single thread pool, который обрабатывает только одно задание за единицу времени, fixed thread pool, пул с фиксированным количеством нитей, и cached thread pool, расширяемый пул, подходящий для приложений с множеством недолгих заданий.

- **РАЗЛИЧИЕ МЕЖДУ INTERRUPTED И ISINTERRUPTED?**

Основное различие между interrupted() и isInterrupted() в том, что первый сбрасывает статус прерывания, а второй нет. Механизм прерывания в Java реализован с использованием внутреннего флага, известного как статус прерывания. Прерывание нити вызовом Thread.interrupt() устанавливает этот флаг. Когда прерванная нить проверяет статус прерывания, вызывая статический метод Thread.interrupted(), статус прерывания сбрасывается. Нестатический метод isInterrupted(), который используется нитью для проверки статуса прерывания у другой нити, не изменяет флаг прерывания. Условно, любой метод, который завершается, выкинув InterruptedException сбрасывает при этом флаг прерывания. Однако, всегда существует возможность того, что флаг тут же снова установится, если другая нить вызовет interrupt().

- **РАЗЛИЧИЯ МЕЖДУ LIVELOCK И DEADLOCK?**

Livelock схож с deadlock, только в livelock состояния нитей или вовлечённых процессов постоянно изменяются в зависимости друг от друга. Livelock – особый случай нехватки ресурсов. Реальный пример livelock'a – когда два человека встречаются в узком коридоре и каждый, пытаясь быть вежливым, отходит в сторону, и так они бесконечно двигаются из стороны в сторону.

- **КАК ПРОВЕРИТЬ, УДЕРЖИВАЕТ ЛИ НИТЬ LOCK?**

Я и не подозревал, что можно проверять, удерживает ли нить lock в данный момент, до тех пор, пока не столкнулся с этим вопросом в одном телефонном интервью. В `java.lang.Thread` есть метод `holdsLock()`, он возвращает `true`, тогда и только тогда, когда текущая нить удерживает монитор у определённого объекта.

1. *Java*

4. *Memory Устройство памяти JVM, работа GC*

Java Heap память

Java Heap (куча) используется Java Runtime для выделения памяти под объекты и [JRE](#) классы. Создание нового объекта также происходит в куче. Здесь работает сборщик мусора: освобождает память путем удаления объектов, на которые нет каких-либо ссылок. Любой объект, созданный в куче, имеет глобальный доступ и на него могут ссылаться с любой части приложения.

Stack память в Java

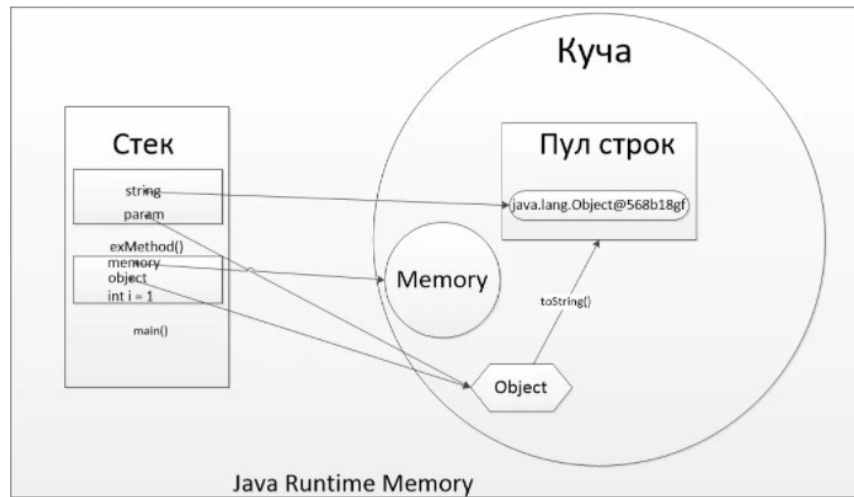
Стековая память в Java работает по схеме LIFO (Последний-зашел-Первый-вышел). Всякий раз, когда вызывается метод, в памяти стека создается новый блок, который содержит примитивы и ссылки на другие объекты в методе. Как только метод заканчивает работу, блок также перестает использоваться, тем самым предоставляя доступ для следующего метода. Размер стековой памяти намного меньше объема памяти в куче.

```

1 package ua.com.prologistic;
2
3 public class Memory {
4
5     public static void main(String[] args) { // строка 1
6         int i=1; // строка 2
7         Object object = new Object(); // строка 3
8         Memory memory = new Memory(); // строка 4
9         memory.exMethod(object); // строка 5
10    } // строка 9
11
12    private void exMethod(Object param) { // строка 6
13        String string = param.toString(); // строка 7
14        System.out.println(string );
15    } // строка 8
16
17 }

```

На картинке ниже представлена память стека и кучи для программы выше



А теперь рассмотрим шаги выполнения нашей программы:

1. Как только мы запустим программу, загружаются все классы среды выполнения в кучу. Потом метод `main()` находит строку 1 и Java Runtime создает стековую память для использования методом `main()`.
2. Далее в строке 2 создается `int`-овая переменная, которая хранится в памяти стека метода `main()`.
3. Потом мы создали объект в строке 3 и он тут же появляется в куче, а стековая память содержит ссылку на него. Точно такой же процесс происходит, когда мы создаем объект `Memory` в строке 4.
4. Теперь в строке 5 мы вызываем метод `exMethod()` и тут же сразу создается блок на вершине стека, который будет использоваться этим методом. Поскольку в Java объекты и примитивы передаются по значению, то в строке 6 будет создана новая ссылка на объект, созданный в строке 3.

5. Строка, созданная в строке 7, отправляется в [Пул строк \(String Pool\)](#), который находится в куче. На эту строку также создается ссылка в стековой памяти метода `exMethod()`.
6. Метод `exMethod()` завершается на строке 8, поэтому блок стековой памяти для этого метода становится свободным.
7. В строке 9 метод `main()` завершается, поэтому стековая память для метода `main()` будет уничтожена. Также программа заканчивается в этой строке, следовательно, Java Runtime освобождает всю память и завершает программу.

Разница между Stack и Heap памятью в Java

На основании приведенных выше объяснений, мы можем легко подытожить следующие различия между Heap и Stack памятью в Java.

- Куча используется всеми частями приложения в то время как стек используется только одним потоком исполнения программы.
- Всякий раз, когда создается объект, он всегда хранится в куче, а в памяти стека содержится ссылка на него. Память стека содержит только локальные переменные примитивных типов и ссылки на объекты в куче.
- Объекты в куче доступны с любой точки программы, в то время как стековая память не может быть доступна для других потоков.
- Управление памятью в стеке осуществляется по схеме LIFO. последний зашел – первый вышел
- Стековая память существует лишь какое-то время работы программы, а память в куче живет с самого начала до конца работы программы.
- Мы можем использовать `-Xms` и `-Xmx` опции [JVM](#), чтобы определить начальный и максимальный размер памяти в куче. Для стека определить размер памяти можно с помощью опции `-Xss`.
- Если память стека полностью занята, то Java Runtime бросает `java.lang.StackOverflowError`, а если память кучи заполнена, то бросается [исключение](#) `java.lang.OutOfMemoryError: Java Heap Space`.
- Размер памяти стека намного **меньше** памяти в куче. Из-за простоты распределения памяти (LIFO), стековая память работает **намного быстрее кучи**.

Основные особенности стека

Помимо того, что мы рассмотрели, существуют и другие особенности стека:

- Он заполняется и освобождается по мере вызова и завершения новых методов
- Переменные в стеке существуют до тех пор, пока выполняется метод в котором они были созданы
- Если память стека будет заполнена, Java бросит исключение `java.lang.StackOverflowError`
- Доступ к этой области памяти осуществляется быстрее, чем к куче
- Является потокобезопасным, поскольку для каждого потока создается свой отдельный стек

Куча

Эта область памяти используется для динамического выделения памяти для объектов и классов JRE во время выполнения. Новые объекты всегда создаются в куче, а ссылки на них хранятся в стеке.

Эти объекты имеют глобальный доступ и могут быть получены из любого места программы.

Эта область памяти разбита на несколько более мелких частей, называемых поколениями:

1. **Young Generation** — область где размещаются недавно созданные объекты. Когда она заполняется, происходит быстрая сборка мусора
2. **Old (Tenured) Generation** — здесь хранятся долгоживущие объекты. Когда объекты из Young Generation достигают определенного порога «возраста», они перемещаются в Old Generation
3. **Permanent Generation** — эта область содержит метаданные о классах и методах приложения, но начиная с Java 8 данная область памяти была упразднена.

Основные особенности кучи

Помимо рассмотренных ранее, куча имеет следующие ключевые особенности:

- Когда эта область памяти полностью заполняется, Java бросает `java.lang.OutOfMemoryError`
- Доступ к ней медленнее, чем к стеку

- Эта память, в отличие от стека, автоматически не освобождается. Для сбора неиспользуемых объектов используется сборщик мусора
- В отличие от стека, куча не является потокобезопасной и ее необходимо контролировать, правильно синхронизируя код

Stack – место под примитивы и ссылки на объекты (но не сами объекты). Хранит локальные переменные и возвращаемые значения функций. Здесь же хранятся ссылки на объекты пока те конструируются. Все данные в стеке – GC roots. Освобождается сразу на выходе из функции. Принадлежит потоку, размер по-умолчанию указывается параметром виртуальной машины -Xss, но при создании потока программно можно указать отличное значение. Подробнее.

PermGen – В этой области хранятся загруженные классы (экземпляры класса Class<T>). Здесь же с Java 7 хранится пул строк. Изначально размера - XX:PermSize, растет динамически до -XX:MaxPermSize. Не считается частью кучи.

Metaspace – с Java 8 заменяет permanent generation. Отличие в том, что по умолчанию metaspace ограничен только размерами доступной на машине памяти, но так же как PermGen может быть ограничен, параметром - XX:MaxMetaspaceSize.

Heap – куча, вся managed-память, в которой хранятся все пользовательские объекты. Все следующие разделы – части кучи. Параметры -Xms, -Xmn и -Xmx устанавливают начальный, минимальный и максимальный размеры хипа соответственно.

Eden, New Generation, Old Generation и другие – специфичные для сборщика мусора части кучи, поколения. Могут быть разные, но общий подход сохраняется: долго живущий объект постепенно двигается во всё более старое поколение; сборка мусора в разных поколениях происходит отдельно; чем поколение старше, тем сборка в нём реже, но и дороже.

Сборка мусора в Java — это процесс, с помощью которого программы Java автоматически управляют памятью.

Можно сказать, что в любой момент времени память кучи состоит из двух типов объектов.

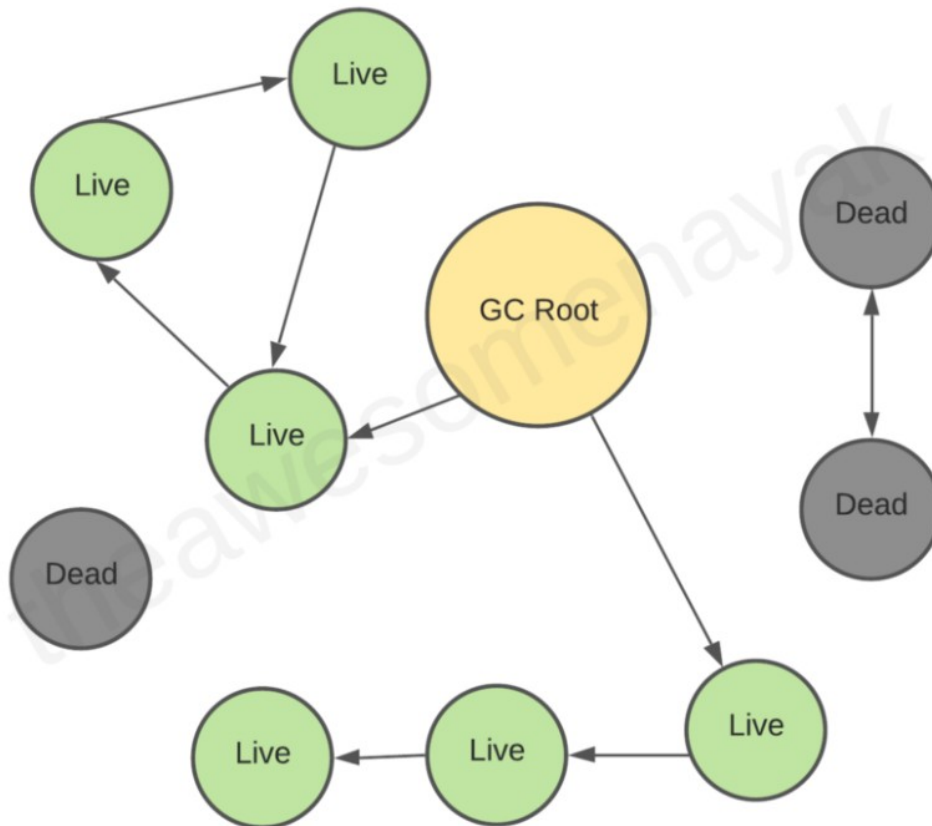
- *Живые* — эти объекты используются, на них ссылаются откуда-то еще.
- *Мертвые* — эти объекты больше нигде не используются, ссылок на них нет.

Сборщик мусора находит эти неиспользуемые объекты и удаляет их, чтобы освободить память.

Каковы источники для сборки мусора в Java?

Сборщики мусора работают с концепцией корней сбора мусора (GC Roots) для идентификации живых и мертвых объектов.

Сборщик мусора просматривает весь граф объектов в памяти, начиная с этих корней и следуя ссылкам на другие объекты.



Примеры таких корней.

- Классы, загружаемые системным загрузчиком классов (не пользовательские загрузчики классов).
- Живые потоки.
- Локальные переменные и параметры выполняемых в данный момент методов.
- Локальные переменные и параметры методов JNI.
- Глобальная ссылка на JNI.
- Объекты, применяемые в качестве монитора для синхронизации.

- Объекты, удерживаемые из сборки мусора JVM для своих целей.

3. Kotlin

1. Basics extension-функции, data-class vs POJO, модификаторы (differences доступа, поля в Kotlin, параметры по умолчанию, from Java) конструкторы в Kotlin, наследование классов, null-safety

Функции-расширения (Extension functions)

Функции-расширения позволяют создавать *видимость* внедрения в существующий класс нового метода.

Kotlin позволяет расширять класс путём добавления нового функционала без необходимости наследования от такого класса и использования паттернов, таких как Decorator.

Функции-расширения

Для того, чтобы объявить функцию-расширение, укажите в качестве префикса *расширяемый тип*, то есть тип, который мы расширяем. Следующий пример добавляет функцию `swap` к `MutableList<Int>`:

```
fun MutableList<Int>.swap(index1: Int, index2: Int) {  
    val tmp = this[index1] // 'this' даёт ссылку на список  
    this[index1] = this[index2]  
    this[index2] = tmp  
}
```

Ключевое слово `this` внутри функции-расширения соотносится с объектом расширяемого типа (этот тип ставится перед точкой). Теперь мы можем вызывать такую функцию в любом `MutableList<Int>`.

```
val list = mutableListOf(1, 2, 3)  
list.swap(0, 2) // 'this' внутри 'swap()' будет содержать значение 'list'
```

Следующая функция имеет смысл для любого `MutableList<T>`, и вы можете сделать её обобщённой:

```
fun <T> MutableList<T>.swap(index1: Int, index2: Int) {  
    val tmp = this[index1] // 'this' относится к списку  
    this[index1] = this[index2]  
    this[index2] = tmp  
}
```

Вам нужно объявлять обобщённый тип-параметр перед именем функции для того, чтобы он был доступен в получаемом типе-выражении. См. [Обобщения](#).

Ключевое слово **infix**

Если ваша функция-расширение использует только один аргумент, то можно вызвать функцию через ключевое слово **infix**.

Создадим функцию-расширение для **Int**, которое будет сравнивать число с аргументом, чтобы узнать, больше оно или меньше.

Старый способ.

```
fun Int.isGreater(value: Int): Boolean {
    return this > value
}

// вызываем функцию
3.isGreater(9) // false
```

Перепишем функцию с ключевым словом **infix**.

```
infix fun Int.isGreater(value: Int): Boolean {
    return this > value
}

// вызываем функцию
12 isGreater 9 // true
```

Расширения вычисляются статически

Расширения на самом деле не проводят никаких модификаций с классами, которые они расширяют. Объявляя расширение, вы создаёте новую функцию, а не новый член класса. Такие функции могут быть вызваны через точку, применимо к конкретному типу.

Расширения имеют *статическую* диспетчеризацию: это значит, что вызванная функция-расширение определяется типом её выражения, из которого она вызвана, а не типом выражения, вычисленным в ходе выполнения программы, как при вызове виртуальных функций.

Поскольку расширения фактически не добавляют никаких членов к классам, свойство-расширение не может иметь **теневого поля**. Вот почему *запрещено использовать инициализаторы для свойств-расширений*. Их поведение может быть определено только явным образом, с указанием геттеров/сеттеров.

data-class vs POJO

Классы данных

Нередко мы создаём классы, единственным назначением которых является хранение данных. Функционал и некоторые служебные функции таких классов зависят от самих данных, которые в них хранятся. В Kotlin они называются *классами данных* и помечены `data`.

```
data class User(val name: String, val age: Int)
```

Компилятор автоматически формирует следующие члены данного класса из свойств, объявленных в основном конструкторе:

- пару функций `equals()/hashCode()`,
- функцию `toString()` в форме `"User(name=John, age=42)"`,
- компонентные функции `componentN()`, которые соответствуют свойствам, в соответствии с порядком их объявления,
- функцию `copy()` (см. ниже).

Для обеспечения согласованности и осмысленного поведения сгенерированного кода классы данных должны удовлетворять следующим требованиям:

- Основной конструктор должен иметь как минимум один параметр;
- Все параметры основного конструктора должны быть отмечены, как `val` или `var`;
- Классы данных не могут быть абстрактными, `open`, `sealed` или `inner`;

Дополнительно, генерация членов классов данных при наследовании подчиняется следующим правилам:

- Если существуют явные реализации `equals()`, `hashCode()` или `toString()` в теле класса данных или `final` реализации в суперклассе, то эти функции не генерируются, а используются существующие реализации;
- Если суперкласс включает функции `componentN()`, которые являются открытыми и возвращают совместимые типы, соответствующие компонентные функции создаются для класса данных и переопределяют функции суперкласса. Если функции суперкласса не могут быть переопределены из-за несовместимости сигнатур или потому что они `final`, выдаётся сообщение об ошибке;
- Предоставление явных реализаций для функций `componentN()` и `copy()` не допускается.

Классы данных могут расширять другие классы (см. примеры в статье [Изолированные классы](#)).

Для того, чтобы у сгенерированного в JVM класса был конструктор без параметров, значения всех свойств должны быть заданы по умолчанию

```
data class User(val name: String = "", val age: Int = 0)
```

Свойства, объявленные в теле класса

Компилятор использует только свойства, определенные в основном конструкторе для автоматически созданных функций. Чтобы исключить свойство из автоматически созданной реализации, объявите его в теле класса:

```
data class Person(val name: String) {  
    var age: Int = 0  
}
```

Только свойство `name` будет учитываться в реализациях функций `toString()`, `equals()`, `hashCode()` и `copy()`, и будет создана только одна компонентная функция `component1()`. Даже если два объекта класса `Person` будут иметь разные значения свойств `age`, они будут считаться равными.

```
val person1 = Person("John")  
val person2 = Person("John")  
person1.age = 10  
person2.age = 20
```

Копирование

Используйте функцию `copy()` для копирования объекта, что позволит изменить только *некоторые* его свойств, оставив остальные неизменными. Для написанного выше класса `User` такая реализация будет выглядеть следующим образом:

```
fun copy(name: String = this.name, age: Int = this.age) = User(name, age)
```

Это позволяет вам писать:

```
val jack = User(name = "Jack", age = 1)  
val olderJack = jack.copy(age = 2)
```

Несколько конструкторов

Добавить второй конструктор к классу можно через ключевое слово **constructor**.

```
data class Cat(val name: String) {  
    var age = 0;  
  
    constructor(name: String, age: Int) : this(name) {  
        this.age = age;  
    }  
}
```

Есть другой способ через аннотацию.

```
data class Cat @JvmOverloads constructor(val name: String, val age: Int? = 0) {  
  
}
```

POJO (англ. Plain Old Java Object) — «простой Java-объект в старом стиле», простой Java-объект, не унаследованный от какого-то специфического объекта и не реализующий никаких служебных интерфейсов сверх тех, которые нужны для бизнес-модели.

POJO-это Plain old java object , которые берут на себя ответственность за хранение данных, а не за обработку бизнеса.

DAO - это Data access object , которые берут на себя ответственность за обработку сохраняемости/обработку базы данных.

Модификаторы доступа

Классы, объекты, интерфейсы, конструкторы, функции, свойства и их сеттеры могут иметь *модификаторы доступа*. Геттеры всегда имеют тут же видимость, что и свойства, к которым они относятся.

В Kotlin предусмотрено четыре модификатора доступа: private, protected, internal и public. Если явно не использовать никакого модификатора, то по умолчанию применяется public.

Пакеты

Функции, свойства, классы, объекты и интерфейсы могут быть объявлены на самом "высоком уровне" прямо внутри пакета.

```
// имя файла: example.kt
package foo

fun baz() { /*...*/ }
class Bar { /*...*/ }
```

- Если модификатор доступа не указан, будет использован `public`. Это значит, что весь код данного объявления будет виден из космоса;
- Если вы пометите объявление словом `private`, оно будет иметь видимость только внутри файла, где было объявлено;
- Если вы используете `internal`, видимость будет распространяться на весь *модуль*;
- `protected` запрещено использовать в объявлениях "высокого уровня".

Члены класса

Для членов, объявленных в классе:

- `private` означает видимость только внутри этого класса (включая его члены);
- `protected` — то же самое, что и `private` + видимость в subclasses;
- `internal` — любой клиент *внутри модуля*, который видит объявленный класс, видит и его `internal` члены;
- `public` — любой клиент, который видит объявленный класс, видит его `public` члены.

В Kotlin внешний класс не видит `private` члены своих вложенных классов.

Если вы переопределите `protected` или `internal` член и явно не укажете его видимость, переопределённый элемент будет иметь тот же модификатор, что и исходный.

Конструкторы

Для указания видимости основного конструктора класса используется следующий синтаксис:

Вам необходимо добавить ключевое слово `constructor`.

```
class C private constructor(a: Int) { ... }
```

В этом примере конструктор помечен `private`. По умолчанию все конструкторы имеют модификатор доступа `public`, то есть видны везде, где виден сам класс (а вот конструктор `internal` класса видно только в том же модуле).

Локальные объявления

Локальные переменные, функции и классы не могут иметь модификаторов доступа.

Модули

Модификатор доступа `internal` означает, что этот член видно в рамках его модуля. Модуль - это набор скомпилированных вместе Kotlin файлов, например:

- модуль IntelliJ IDEA;
- Maven проект;
- исходный набор Gradle (за исключением того, что исходный набор `test` может получить доступ к внутренним объявлениям `main`);
- набор скомпилированных вместе файлов с одним способом вызова `<kotlin>` задачи в Ant.

Свойства / ПОЛЯ

Объявление свойств

Свойства в классах Kotlin могут быть объявлены либо как изменяемые (mutable) и неизменяемые (read-only) — `var` и `val` соответственно.

Константы времени компиляции

Если значение константного (read-only) свойства известно во время компиляции, пометьте его как *константы времени компиляции*, используя модификатор `const`. Такие свойства должны соответствовать следующим требованиям:

- Находиться на самом высоком уровне или быть членами *объявления* `object` или *вспомогательного объекта*;
- Быть проинициализированными значением типа `String` или значением примитивного типа;
- Не иметь переопределённого геттера.

Такие свойства могут быть использованы в аннотациях.

```
const val SUBSYSTEM_DEPRECATED: String = "This subsystem is deprecated"

@Deprecated(SUBSYSTEM_DEPRECATED) fun foo() { ... }
```

Теневые поля

В Kotlin поле используется только как часть свойства для хранения его значения в памяти. Поля не могут быть объявлены напрямую. Однако, когда свойству требуется теневое поле (backing field), Kotlin предоставляет его автоматически. На это теневое поле можно обратиться в методах доступа, используя идентификатор `field`:

```
var counter = 0 // инициализатор назначает резервное поле напрямую
set(value) {
    if (value >= 0)
        field = value // значение при инициализации записывается
                        // прямоком в backing field

    // counter = value // ERROR StackOverflow: Использование 'counter' сделало бы с
    // еттер рекурсивным
}
```

Идентификатор `field` может быть использован только в методах доступа к свойству.

Теневое поле будет сгенерировано для свойства, если оно использует стандартную реализацию как минимум одного из методов доступа, либо если пользовательский метод доступа ссылается на него через идентификатор `field`.

Например, в примере ниже не будет никакого теневого поля:

```
val isEmpty: Boolean
    get() = this.size == 0
```

Теневые свойства

Если вы хотите предпринять что-то такое, что выходит за рамки вышеуказанной схемы *неявного теневого поля*, вы всегда можете использовать *теневое свойство* (backing property).

```
private var _table: Map<String, Int>? = null
public val table: Map<String, Int>
    get() {
        if (_table == null) {
            _table = HashMap() // параметры типа вычисляются автоматически
                                // (ориг.: "Type parameters are inferred")
        }
        return _table ?: throw AssertionError("Set to null by another thread")
    }
```

В JVM: доступ к приватным свойствам со стандартными геттерами и сеттерами оптимизируется таким образом, что вызов функции не происходит.

NULL

Nullable типы и Non-Null типы

Система типов в языке **Kotlin** нацелена на то, чтобы искоренить опасность обращения к *null* значениям, более известную как "Ошибка на миллиард".

и Самым распространённым подводным камнем многих языков программирования, в том числе **Java**, является попытка произвести доступ к *null* значению. Это приводит к ошибке. В **Java** такая ошибка называется `NullPointerException` (сокр. "NPE").

Kotlin призван исключить ошибки подобного рода из нашего кода. NPE могут возникать только в случае:

- Явного указания `throw NullPointerException()`
- Использования оператора `!!` (описано ниже)
- Эту ошибку вызвал внешний Java-код
- Есть какое-то несоответствие при инициализации данных (в конструкторе использована ссылка *this* на данные, которые не были ещё проинициализированы)

Система типов **Kotlin** различает ссылки на те, которые могут иметь значение *null* (nullable ссылки) и те, которые таковыми быть не могут (non-null ссылки). К примеру, переменная часто используемого типа

`String` не может быть *null*:

```
var a: String = "abc"
a = null // ошибка компиляции
```

Для того, чтобы разрешить *null* значение, мы можем объявить эту строковую переменную как `String?`:

ии
ли
ции

```
var b: String? = "abc"
b = null // ok
```

Безопасные вызовы

Вторым способом является оператор безопасного вызова `?.`:

```
b?.length
```

Этот код возвращает `b.length` в том, случае, если `b` не имеет значение *null*. Иначе он возвращает *null*.

Типом этого выражения будет `Int?`.

Проверка на *null*

Первый способ. Вы можете явно проверить `b` на *null* значение и обработать два варианта по отдельности:

```
val l = if (b != null) b.length else -1
```

Компилятор отслеживает информацию о проведённой вами проверке и позволяет вызывать `length` внутри блока *if*. Также поддерживаются более сложные конструкции:

```
if (b != null && b.length > 0) {  
    print("String of length ${b.length}")  
} else {  
    print("Empty string")  
}
```

Обратите внимание: это работает только в том случае, если `b` является неизменной переменной (ориг.: *immutable*). Например, если это локальная переменная, значение которой не изменяется в период между его проверкой и использованием. Также такой переменной может служить *val*. В противном случае может так оказаться, что переменная `b` изменила своё значение на *null* после проверки.

Элвис-оператор

Если у нас есть nullable ссылка `b`, мы можем либо провести проверку этой ссылки и использовать её, либо использовать non-null значение:

```
val l: Int = if (b != null) b.length else -1
```

Аналогом такому *if*-выражению является элвис-оператор `?:`:

```
val l = b?.length ?: -1
```

Если выражение, стоящее слева от Элвис-оператора, не является *null*, то элвис-оператор его вернёт. В противном случае, в качестве возвращаемого значения послужит то, что стоит справа. Обращаем ваше внимание на то, что часть кода, расположенная справа, выполняется ТОЛЬКО в случае, если слева получается *null*.

Так как *throw* и *return* тоже являются выражениями в **Kotlin**, их также можно использовать справа от Элвис-оператора. Это может быть крайне полезным для проверки аргументов функции:

```
fun foo(node: Node): String? {  
    val parent = node.getParent() ?: return null  
    val name = node.getName() ?: throw IllegalArgumentException("name expected")  
    // ...  
}
```

Оператор !!

Для любителей NPE существует ещё один способ. Мы можем написать `b!!` и это вернёт нам либо non-null значение `b` (в нашем примере вернётся `String`), либо выкинет NPE:

```
val l = b!!.length
```

В случае, если вам нужен NPE, вы можете заполнить её только путём явного указания.

Безопасные приведения типов (ориг.: *Safe Casts*)

Обычное приведение типа может вызвать `ClassCastException` в случае, если объект имеет другой тип. Можно использовать безопасное приведение, которое вернёт `null`, если попытка не удалась:

```
val aInt: Int? = a as? Int
```

Коллекции nullable типов

Если у вас есть коллекция nullable элементов и вы хотите отфильтровать все non-null элементы, используйте функцию `filterNotNull`.

```
val nullableList: List<Int?> = listOf(1, 2, null, 4)
val intList: List<Int> = nullableList.filterNotNull()
```

Аргументы по умолчанию

В примере выше при вызове функций `showMessage` и `displayUser` мы обязательно должны предоставить для каждого их параметра какое-то определенное значение, которое соответствует типу параметра. Мы не можем, к примеру, вызвать функцию `displayUser`, не передав ей аргументы для параметров, это будет ошибка:

```
1 displayUser()
```

Однако мы можем определить какие-то параметры функции как необязательные и установить для них значения по умолчанию:

```
1 fun displayUser(name: String, age: Int = 18, position: String="unemployed"){
2     println("Name: $name   Age: $age   Position: $position")
3 }
4
5 fun main() {
6
7     displayUser("Tom", 23, "Manager")
8     displayUser("Alice", 21)
9     displayUser("Kate")
10 }
```

Именованные аргументы

По умолчанию значения передаются параметрам по позиции: первое значение - первому параметру, второе значение - второму параметру и так далее. Однако, используя именованные аргументы, мы можем переопределить порядок их передачи параметрам:

```
1 fun main() {
2
3     displayUser("Tom", position="Manager", age=28)
4     displayUser(age=21, name="Alice")
5     displayUser("Kate", position="Middle Developer")
6 }
```

При вызове функции в скобках мы можем указать название параметра и с помощью знака равно передать ему нужное значение.

При этом, как видно из последнего случая, необязательно все аргументы передавать по имени. Часть аргументов могут передаваться параметрам по позиции. Но если какой-то аргумент передан по имени, то остальные аргументы после него также должны передаваться по имени соответствующих параметров.

Изменение параметров

По умолчанию все параметры функции равносильны val-переменным, поэтому их значение нельзя изменить. Например, в случае следующей функции при компиляции мы получим ошибку:

```
1 fun double(n: Int){
2     n = n * 2 // !Ошибка - значение параметра нельзя изменить
3     println("Значение в функции double: $n")
4 }
```

Однако если параметр представляет какой-то сложный объект, то можно изменять отдельные значения в этом объекте. Например, возьмем функцию, которая в качестве параметра принимает массив:

```
1 fun double(numbers: IntArray){
2     numbers[0] = numbers[0] * 2
3     println("Значение в функции double: ${numbers[0]}")
4 }
5
6 fun main() {
7
8     var nums = intArrayOf(4, 5, 6)
9     double(nums)
10    println("Значение в функции main: ${nums[0]}")
11 }
```

Здесь функция double принимает числовой массив и увеличивает значение его первого элемента в два раза. Причем изменение элемента массива внутри функции приведет к тому, что также будет изменено значение элемента в том массиве, который передается в качестве аргумента в функцию, так как этот один и тот же массив. Консольный вывод:

```
Значение в функции double: 8
Значение в функции main: 8
```

3. KOTLIN

var vs val, const, Unit, Any

2. Kotlin-only features

несколько условий на одно действие в when

Unit

Unit эквивалентен void в Java. В этом выражении возвращаемый тип можно не указывать, если функция ничего не возвращает. По умолчанию там будет Unit:

```
fun knockKnock(){
    println("Who's there?")
}
```

В Kotlin есть два способа объявить функцию: в теле метода (в фигурных скобках) или как выражение (через знак равенства). Можно переписать нашу функцию так, а заодно указать возвращаемое значение:

```
fun knockKnock(): Unit = println("Who's there?")
```

В стандартной библиотеке Kotlin Unit определён как объект, наследуемый от Any и содержащий единственный метод, переопределяющий toString():

```
public object Unit {
    override fun toString() = "kotlin.Unit"
```

```
}
```

Обратите внимание на ключевое слово `object`. Это значит, что `Unit` является синглтоном. `Unit` ничего не возвращает, а метод `toString` всегда будет возвращать `"kotlin.Unit"`. При компиляции в `java`-код `Unit` всегда будет превращаться в `void`.

Интересно, что в `Java` есть свой класс `Void`, который довольно слабо, но всё же соотносится с `void` и не может быть инстантирован. Это исключительно утилитарный класс, который нужен для рефлексии и дженериков в `Java`.

Nothing

С `Nothing` всё гораздо интереснее. `Nothing` — класс, который является наследником любого класса в `Kotlin`, даже класса с модификатором `final`. При этом `Nothing` нельзя создать — у него приватный конструктор. В коде он объявлен так:

```
public class Nothing private constructor()
```

Несмотря на всё это, класс `Nothing` довольно полезен. Так как невозможно передать или вернуть тип `Nothing`, он описывает результат «функции, которая никогда ничего не вернёт». Примером может быть функция, которая выбрасывает `exception` или в которой запущен бесконечный цикл: в любом из этих случаев она никогда не вернёт значения. В приложениях — независимо от того, какой тип данных возвращает функция, — она может никогда не вернуть данные, потому что произошла ошибка или вычисления затянулись на неопределённый срок. В этом случае имеет смысл использовать `Nothing`.

Посмотрим, где это используется в `Kotlin`. Вот пример: функция `TODO()`, которая часто служит заглушкой в автоматически генерируемых методах.

```
public inline fun TODO(): Nothing = throw NotImplementedError()
```

Вы можете наблюдать такую картину при автогенерации кода:

```
override fun getData(word: String): List<Data> {  
    TODO("not implemented")  
}
```

И хотя возвращаемое значение тут `List<Data>`, мы возвращаем `Nothing`. Именно потому что `Nothing` наследуется от всех классов:

```
fun doSomething(): Something = TODO()
```

Код прекрасно скомпилируется, потому что `Nothing` наследуется от `Something`. Но приложение сразу же упадёт с `NotImplementedError`, если вы вызовете метод `doSomething`.

Интересно, что в Java нельзя написать что-то подобное: код просто не скомпилируется, потому что `Void` не наследуется от `String`:

```
static Void todo(){  
  
throw new RuntimeException("Not Implemented");  
  
}
```

```
String myMethod(){  
  
return todo();  
  
}
```

Ещё один пример может касаться выполнения, например, запроса данных из БД или удалённого сервера. Если произошла ошибка, можно возвращать `null` вместо данных. И это абсолютно нормально, данных-то нет:

```
fun getData(): Data? = ...
```

А если хочется немного больше информации, чем просто `null`? Например, узнать тип ошибки. Вот тут `Nothing` приходит на помощь:

```
fun getData(onError: (SomeError) -> Nothing): Data = ...
```

Вот как это может выглядеть в коде:

```
val data = getData() { err ->  
  
when (err) {  
  
is InvalidStatement -> throw Exception(err.parseError)  
  
is NoSuchData -> ...  
  
}  
  
}  
  
return Data() //успешный сценарий
```

Закрепим:

```
//Скомпилируются нормально
```

```
fun funOne(): Unit { while (true) {} }
```



```
fun funTwo(): Nothing { while (true) {} }
```

```
//Ок
```

```
fun funThree(): Unit { println("hi") }
```

```
//He ok
```

```
fun funFour(): Nothing { println("hi") }
```

Any

Класс Any находится на вершине иерархии — все классы в Kotlin являются наследниками Any. Any — это аналог Object в Java, но с меньшим количеством методов:

```
public open class Any {  
    public open operator fun equals(other: Any?): Boolean  
    public open fun hashCode(): Int  
    public open fun toString(): String  
}
```

В java.lang.Object одиннадцать методов, и пять из них касаются многопоточности. Несмотря на меньшее количество методов, при компиляции в Java-код у любого класса появятся недостающие — тут можно быть спокойными.

Null

В Kotlin null может формировать nullable-типы (произносится как «нүллабл»). Они обозначаются добавлением знака ? в конце типа. Например, String? — это тип String + null. То есть значение может быть строковым, а может быть null. В Java такие дополнения не нужны — там любой объект может быть null, и это приводит нас к одному из преимуществ языка Kotlin перед Java — *null safety*.

В Java можно вызывать методы объекта всегда, и только в процессе работы программы вы узнаете, что объект у вас null. Тогда приложение падает с ошибкой NPE (NullPointerException). Это самая частая ошибка в Java вообще.

В Kotlin же ваш код просто не скомпилируется, если вы вызываете методы у объекта, который может быть null, то есть помечен знаком ?. Потребуется обязательная проверка на null. Если в конце типа не стоит ? — вы гарантируете, что объект никогда не может быть null. Это помогает выявить возможные ошибки, связанные с NPE, ещё на этапе написания кода — а не когда приложение уже запущено или находится в магазине, как в случае с Java.

WHEN

В выражении **when** можно использовать ключевое слово **is** для проверки принадлежности к экземпляру класса.

le

Условное выражение when

ти

`when` определяет условное выражение с несколькими "ветвями". Оно похоже на оператор `switch`, присутствующий в С-подобных языках.

```
when (x) {  
  1 -> print("x == 1")  
  2 -> print("x == 2")  
  else -> { // обратите внимание на блок  
    print("x не равен ни 1, ни 2")  
  }  
}
```

`when` последовательно сравнивает свой аргумент со всеми указанными значениями, пока не выполнится какое-либо из условий ветвей.

`when` можно использовать и как выражение, и как оператор. При использовании его в виде выражения значение первой ветки, удовлетворяющей условию, становится значением всего выражения. При использовании в виде оператора значения отдельных веток отбрасываются. В точности как `if`: каждая ветвь может быть блоком и её значением является значение последнего выражения блока.

Значение ветки `else` вычисляется в том случае, когда ни одно из условий в других ветках не удовлетворено.

Если `when` используется как *выражение*, то ветка `else` является обязательной, за исключением случаев, в которых компилятор может убедиться, что ветки покрывают все возможные значения. Так происходит, например с записями класса `enum` и с подтипами `sealed` (*изолированных классов*).

В операторах `when` ветка `else` является обязательной в следующих условиях:

- `when` имеет объект типа `Boolean`, `enum`, `sealed` или их nullable-аналоги;
- ветки `when` не охватывают все возможные случаи для этого объекта.

Также можно проверять вхождение аргумента в [интервал](#) `in` или `!in` или его наличие в коллекции:

```
when (x) {
    in 1..10 -> print("x is in the range")
    in validNumbers -> print("x is valid")
    !in 10..20 -> print("x is outside the range")
    else -> print("none of the above")
}
```

Помимо этого Kotlin позволяет с помощью `is` или `!is` проверить тип аргумента. Обратите внимание, что благодаря [умным приведениям](#) вы можете получить доступ к методам и свойствам типа без дополнительной проверки.

```
fun hasPrefix(x: Any) = when(x) {
    is String -> x.startsWith("prefix")
    else -> false
}
```

`when` можно использовать вместо условных операторов `if`, `else`, `if`. При этом в аргументе там может находиться логическое выражение.

```
import kotlin.random.Random

fun main() {
    val n = Random.nextInt(100)
    val point: Int

    when (n) {
        in 0..30, in 71..100 -> point = 1
        in 31..47, in 52..70 -> point = 2
        48, 49, 50, 51, 52 -> point = 4
        else -> point = 0
    }

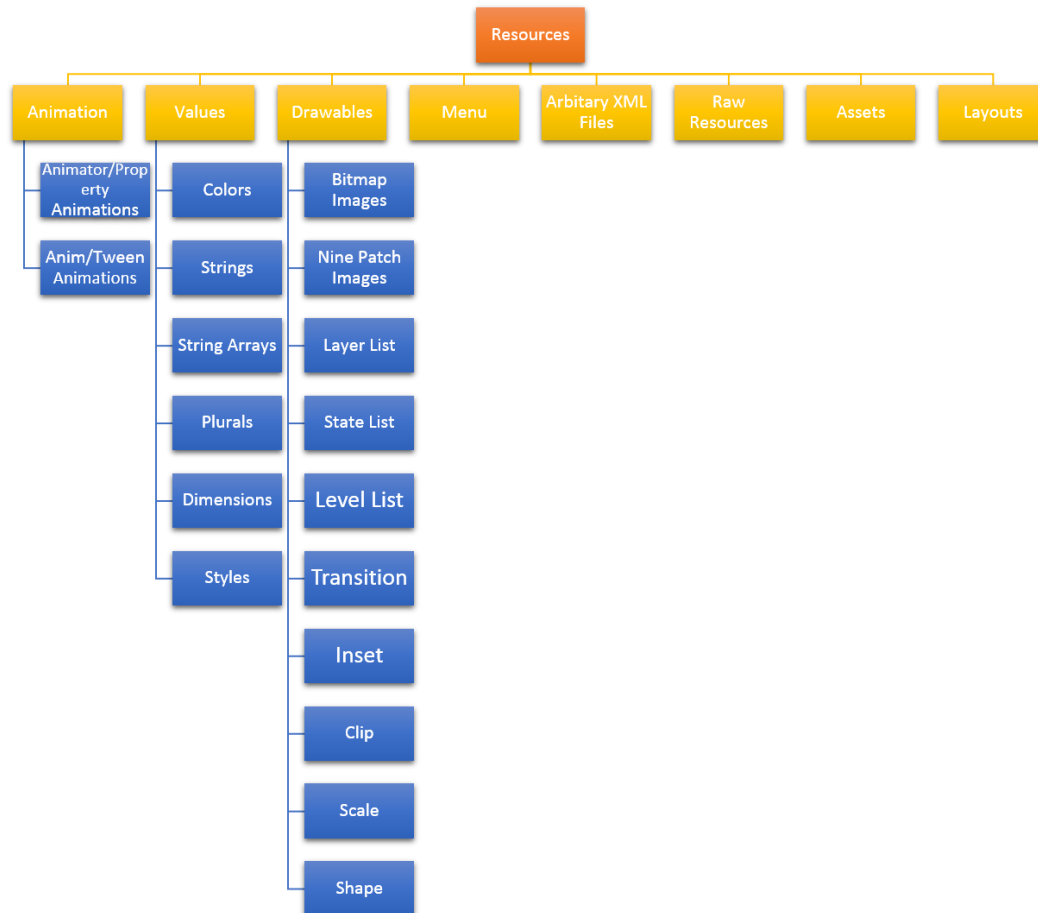
    println(point)
}
```

Логическое выражение `in 0..30` проверяет входит ли значение `n` в диапазон чисел от 0 до 30 включительно. Полная запись (не в контексте выноса `n` в заголовок оператора `when`) выглядела бы так: `n in 0..30`. Подобные выражение возвращают либо `true`, либо `false`.

4. Android Core

ресурсы, `px`; `dp`; `sp`, `R` класс, `Context`, `Manifest`, основные компоненты,

1. Fundamentals
Runtime permissions, Deep Links, Parcelable, виды context и отличия
-



В Android принято держать некоторые объекты - изображения, строковые константы, цвета, анимацию, стили и т.п. за пределами исходного кода. Система поддерживает хранение ресурсов в отдельных файлах. Ресурсы легче поддерживать, обновлять, редактировать.

Каждое приложение на Android содержит каталог для ресурсов **res** и каталог для активов **assets**. Реальное различие между ресурсами и активами заключается в следующем:

- информация в каталоге ресурсов будет доступна в приложении через класс **R**, который автоматически генерируется средой разработки. То есть хранение файлов и данных в ресурсах (в каталоге **res**) делает их легкодоступными для использования в коде программы;
- для чтения информации, помещенной в каталог активов **assets** (необработанный формат файла), необходимо использовать **AssetManager** для чтения файла как потока байтов. Assets и Raw-ресурсы – это механизмы, которые позволяют добавить дополнительные файлы произвольного формата в Android-приложение.

- Для использования Assets создается директория `src/main/assets`. Эта директория может содержать произвольные файлы и поддиректории.
- Для получения контента asset-файла используется метод `Context.getAssets()`, который возвращает объект класса `AssetManager`. Далее вызывается метод `AssetManager.open()`, который принимает имя файла и возвращает `InputStream`.

Android умеет динамически выбирать данные из дерева ресурсов, содержащие разные значения для разных конфигураций, языков и регионов. При запуске Android автоматически загрузит нужный ресурс, не требуя ни одной строчки кода.

Ресурсы в Android являются декларативными. В основном ресурсы хранятся в виде XML-файлов в каталоге **res** с подкаталогами **values**, **drawable-ldpi**, **drawable-mdpi**, **drawable-hdpi**, **layout**, но также бывают и другие типы ресурсов.

Для удобства система создает идентификаторы ресурсов и использует их в файле **R.java** (класс **R**, который содержит ссылки на все ресурсы проекта), что позволяет ссылаться на ресурсы внутри кода программы. Статический класс **R** генерируется на основе ваших заданных ресурсов и создается во время компиляции проекта. При создании класс содержит статические подклассы для всех типов ресурсов, для которых был описан хотя бы один экземпляр.

Так как файл **R** генерируется автоматически, то нет смысла его редактировать вручную, потому что все изменения будут утеряны при повторной генерации.

В общем виде ресурсы представляют собой файл (например, изображение) или значение (например, заголовок программы), связанные с создаваемым приложением. Удобств использования ресурсов заключается в том, что их можно изменять без повторной компиляции или новой разработки приложения. Имена файлов для ресурсов должны состоять исключительно из букв в нижнем регистре, чисел и символов подчёркивания.

Самыми распространёнными ресурсами являются, пожалуй, строки (`string`), цвета (`color`) и графические рисунки (`bitmap`). В приложении не рекомендуется применять жёстко написанные строки кода - вместо них следует использовать соответствующие идентификаторы, что позволяет изменять текст строкового ресурса, не изменяя исходного кода.

```

/res/values/strings.xml
    /colors.xml
        /dimens.xml
        /attrs.xml
        /styles.xml
    /drawable/*.png
        /*.jpg
        /*.gif
        /*.9.png
    /anim/*.xml
    /layout/*.xml
    /raw/*.*
    /xml/*.xml
/assets/*.*/*.*

```

Начинающие программисты не всегда до конца правильно понимают процесс создания ресурсов. В Android используются два подхода - первый подход заключается в том, что ресурсы задаются в файле, при этом имя файла значения не имеет. Второй подход - ресурс задаётся в виде самого файла, и тогда имя файла уже имеет значение (при этом нужно учитывать определённые нюансы).

Общая структура каталогов, содержащих ресурсы выглядит следующим образом:

Только в **assets** может располагаться любой набор подкаталогов разной вложенности. Файлы, находящиеся в любом другом каталоге, размещаются именно на уровне этого каталога и не глубже.

Перечисление основных ресурсов Android

Тип ресурса	Размещение	Описание
Цвета	res/values/имя_файла	Идентификатор цвета, указывающий на цветовой код. ID таких ресурсов выражаются в R.java как R.color.* . XML-узел: /resources/color
Строки	res/values/имя_файла	Строковые ресурсы. В их число также входят строки в формате java и html. ID таких ресурсов выражаются в R.java как R.string.* . XML-узел: resources/string. Можно использовать дополнительное форматирование при помощи стандартных html-тегов , <i> и <u>. (bold, italic, underscore)
		Методы, которые будут обрабатывать строковые ресурсы с HTML-форматированием, должны уметь

Тип ресурса	Размещение	Описание
		обрабатывать эти теги.
Меню	/res/menu/имя_файла	Меню в приложении можно задать как XML-ресурсы.
Параметры	/res/values/имя_файла	Представляет собой параметры или размеры различных элементов. Поддерживает пиксели, дюймы, миллиметры, не зависящие от плотности экрана пиксели (dip) и пиксели, не зависящие от масштаба. ID таких ресурсов выражаются в R.java как R.dimen.*. XML-узел: resources/dimen
Изображения	/res/drawable/ваши_файлы	Ресурсы-изображения. Поддерживает форматы JPG, GIF, PNG (самый предпочтительный) и др. Каждое изображение является отдельным файлом и получает собственный идентификатор, который формируется по имени файла без расширения. Такие ID ресурсов представлены в файле R.java как R.drawable.*. Система также поддерживает так называемые растягиваемые изображения (stretchable image), в которых можно менять масштаб отдельных элементов, а другие элементы оставлять без изменений.
Отрисовываемые цвета	/res/values/ваш_файл или /res/drawable/ваши_файлы	Представляет цветные прямоугольники, которые используются в качестве фона основных отрисовываемых объектов, например точечных рисунков. Поддержка такой функции обеспечивается тегом значения drawable , находящимся в подкаталоге значений. Такие id ресурсов выражаются в файле R.java как R.drawable.*. XML-узел для такого файла: /resources/drawable. В Android при помощи специальных XML-файлов, расположенных в /res/drawable, также поддерживаются скругленные и градиентные прямоугольники. Корневым XML-тегом для drawable является <shape>. Идентификаторы таких ресурсов выражаются в файле R.java как R.drawable.*. В таком случае, каждое имя

Тип ресурса	Размещение	Описание
		файла преобразуется в уникальный id отрисовываемого объекта
Анимация	/res/anim/ваш_файл	Android может выполнить простую анимацию на графике или на серии графических изображений. Анимация включает вращения, постепенное изменение, перемещение и протяжение.
Произвольные XML-файлы	/res/xml/*.xml	В Android в качестве ресурсов могут использоваться произвольные XML-файлы. Они компилируются в аарт. Идентификаторы таких ресурсов также выражаются в файле R.java как R.xml.*
Произвольные необработанные ресурсы	/res/raw/*.*	Любые некомпиллированные двоичные или текстовые файлы, например, видео. Каждый файл получает уникальный id ресурса. Идентификаторы таких ресурсов выражаются в файле R.java как R.raw.*
Произвольные необработанные активы	/assets /.*/*.*	Можно использовать произвольные файлы в произвольно названных каталогах, которые находятся в подкаталоге /assets. Это не ресурсы, а просто необработанные файлы. В этом каталоге, в отличие от /res, подкаталоги могут располагаться на любой глубине. Для таких файлов не создаются идентификаторы ресурсов. При работе с ними нужно использовать относительное имя пути, начиная с /assets, но не указывая этого каталгоа в имени пути

Идентификаторы

Этот тип ресурсов формируется, как правило, автоматически, и программисты даже не обращают на него внимания. Когда вы размещаете новый элемент на форме, с которым будете взаимодействовать в программе, то ему нужно присвоить идентификатор. Как правило, это происходит в виде **@+id/editText** (часто это происходит автоматически). Знак плюса + обозначает, что если идентификатора не существует, то его нужно создать в классе **R**. В программе вы можете обращаться к элементу **R.id.editText**.

Но можно заранее создать ресурс типа **item** для задания **id**, не связанного ни с каким конкретным ресурсом:

```
<resources>
  <item type="id" name="text"/>
</resources>
```

Здесь **type** описывает тип ресурса, в данном случае **id**. Когда **id** будет установлен, будет работать и следующее определение **View**:

```
<TextView android:id="@id/textView">
  </TextView>
```

Обычно идентификаторы размещают в отдельном файле **res/values/ids.xml**.

Строковые ресурсы

Строковые ресурсы помогают упростить процесс создания локализованных версий. Строковые ресурсы обозначаются тегом **<string>**.,

При [разработке первого приложения](#) вы видели, что система создала файл **strings.xml**, в котором хранились строки для заголовка приложения и выводимого сообщения. Вы можете редактировать данный файл, добавляя новые строковые ресурсы. А также вы можете создать новые файлы, которые будут содержать строковые ресурсы, например, **strings2.xml**, **catnames.xml** и т.д. Все эти файлы должны находиться в подкаталоге **/res/values**. Запомните, имена файлов и их число не важно. Но в большинстве случаев программисты используют для строковых ресурсов стандартное имя **strings.xml**. Типичный файл выглядит следующим образом.

```
<?xml version="1.0" encoding="utf-8"?>

<resources>

<string name="hello">Здравствуй, Мир!</string>

<string name="app_name">Hello, World</string>

</resources>
```

При создании или обновлении файла со строковыми ресурсами среда разработки автоматически создаёт или обновляет класс **R.java**, сообщая уникальные ID для определённых в файле строковых ресурсов (Независимо от количества файлов ресурсов, в проекте содержится только один файл **R.java**). Если открыть данный файл, то можно найти там наши ресурсы в следующем виде:

```
public static final int hello = 0x7f040000;

public static final int app_name = 0x7f040001;
```

В принципе достаточно запомнить, что **R.java** создаёт внутренний статический класс как пространство имён, в котором содержатся ID строковых ресурсов. Два **static final int**, которые используются в переменных **hello** и **app_name**, являются идентификаторами ресурсов и

соответствуют соответствующим строковым ресурсам. Вы можете использовать данные идентификаторы в своем исходном коде, используя следующий формат - **R.string.hello**.

Обратите внимание, что сгенерированные ID указывают на **int**, а не на **String**. Android при необходимости самостоятельно подставляет вместо **int** нужные строки.

Обычно принято хранить строковые ресурсы в файле **strings.xml**, но вы можете использовать несколько файлов. Главное, чтобы XML-файл имел необходимую структуру и находился в подкаталоге **res/values**.

Структура файла для строковых ресурсов довольно проста. Имеется корневой узел `<resources>`, за которым следуют один или несколько дочерних элементов `<string>`. Каждый элемент `<string>` в свою очередь имеет свойство **name**, которое в файле **R.java** представляет собой атрибут **id**.

Если вы создаёте несколько файлов с ресурсами, то следите за уникальностью создаваемых имён. Не выйдет ничего хорошего, если в двух файлах будет одна и та же переменная **app_name**.

Запомните, что пробелы в начале и в конце строк обрезаются. Если вам так нужны пробелы, то разместите строку в кавычках и строка будет выводиться как есть. Также можно попробовать использовать код `\u0020` вместо пробела.

Такая же проблема и с несколькими пробелами внутри строки - будет выводиться только один пробел (напоминает поведение в html-документе).

Продвинутые приёмы работы со строковыми ресурсами

Кроме стандартного использования строковых ресурсов, можно использовать более сложные приёмы. Посмотрим, как определять и использовать строки, написанные на HTML, а также узнаем, как происходит подстановка переменных в строковых ресурсах.

Начнём с того, как определять в XML-файлах ресурсов следующие виды строк:

- обычные строки
- строки в кавычках
- HTML-строки
- заменяемые строки

```
<resources>
<string name="simple_string">Простая строка</string>
<string name="quoted_string">"Строка в кавычках"</string>
<string name="double_quoted_string">"\Строка в двойных кавычках\"</string>
<string name="java_format_string">Привет %2$s. Здравствуй %1$s</string>
<string name="tagged_string"><b>Рыжик</b> - <i>мой любимый кот</i></string>
</resources>
```

Этот XML-файл строковых ресурсов должен находиться в подкаталоге **res/values**. Имя файла выбирается произвольно.

Обратите внимание - строки, находящиеся в кавычках, необходимо либо пропускать, либо помещать в дополнительные кавычки. При определении строк также можно использовать стандартные последовательности Java, предназначенные для форматирования строк.

Нельзя использовать символы '&', '<'. Задать эти символы можно используя специальную последовательность < — < или & — &. Если текст содержит html-теги и в нем встречается неразрывный пробел , то его надо заменить на .

Также можно использовать простые элементы HTML, предназначенные для форматирования, используя теги (полужирный шрифт), <i> (наклонный шрифт), <u> (подчеркнутый шрифт). Вы можете использовать такую HTML-строку для оформления текста, перед тем как выводить текст на экран.

Каждый вариант использования проиллюстрирован в листинге на примере кода Java.

```
// Считывание обычной строки и помещение ее в текстовый вид
String simpleString = activity.getString(R.string.simple_string);
textView.setText(simpleString);

// Считывание строки в кавычках и помещение ее в текстовый вид
String quotedString = activity.getString(R.string.quoted_string);
textView.setText(quotedString);

// Считывание строки в двойных кавычках и помещение ее в текстовый вид
String doubleQuotedString = activity.getString(R.string.double_quoted_string);
textView.setText(doubleQuotedString);

// Считывание строки форматирования Java
String javaFormatString = activity.getString(R.string.java_format_string);
// Преобразование отформатированной строки при помощи данных
// передаваемых в аргументах
String substitutedString = String.format(javaFormatString,
    "Рыжик", "Барсик");
// помещение вывода в текстовый вид
textView.setText(substitutedString);

// Считывание строки html_string из ресурса и помещение ее в текстовый вид
String htmlTaggedString = activity.getString(R.string.tagged_string);
// Преобразование строки во фрагмент текста,
// который может быть помещён в текстовом виде
// Класс android.text.Html допускает рисование строк "html" (не все теги)
Spanned textSpan = Html.fromHtml(htmlTaggedString); // Устарело в API 24
Spanned textSpan = Html.fromHtml(htmlTaggedString, Html.FROM_HTML_MODE_LEGACY); // начиная с API 24

// Поместить информацию в текстовый вид
textView.setText(textSpan);
```

wh (3)

После того, как вы определите строки в виде ресурсов, вы можете вставить их прямо в вид. Например, воспользуемся строкой на HTML в элементе **TextView**:

```
<TextView
    android:layout_width="match_parent"
    ...
    android:text="@string/tagged_string" />
```

TextView автоматически определяет, что эта строка написана на HTML, и соответствующим образом обрабатывает форматирование этой строки.

Можно использовать строковые ресурсы в качестве входящих параметров для метода **String.format**. Однако данный метод не поддерживает стилизацию текста через теги `<b>`, `<i>`, `<u>`. Что выйти из этого положения, экранируйте HTML-теги следующим образом:

```
<string name="my_message">&lt;b>Жирный кот&lt;/b></string>
```

А в коде используйте метод **Html.fromHtml** для преобразования строки в нужном виде:

```
String rString = getString(R.string.my_message);
String fString = String.format(rString, "Bla-bla-bla");
CharSequence styledString = Html.fromHtml(fString);
```

Сложный текст с проблемными символами можно загнать в особый контейнер **CDATA**, который знаком веб-мастерам.

```

<string name="long_message">
<![CDATA[
<html>
<body>
<h1>Котик: {{birthDate}}</h1>
<p>Здесь текст о котиках:</p>
<p>{{message}}</p>
<p>Если хотите поговорить об этом, то обращайтесь к <b>Котовскому</b>.</p>
</body>
</html>
]]>
</string>

```

Существует особый вид ресурсов для строк - [Строковые массивы](#)

Ресурсы массивов

Существует еще один тип ресурсов для хранения значений массивов. Принято хранить данные ресурсы в файле **arrays.xml** папки **res/values**. Вот как может выглядеть файл:

```

<?xml version="1.0" encoding="utf-8"?>
<resources>

    <array name="choose">
        <item>@string/red_pill_label</item>
        <item>@string/blue_pill_label</item>
    </array>

    <string-array name="catnames">
        <item>Рыжик</item>
        <item>Барсик</item>
        <item>Мурзик</item>
    </string-array>

    <integer-array name="years">
        <item>2009</item>
        <item>2010</item>
        <item>2011</item>
    </integer-array>

</resources>

```

Как видите, есть типы ресурсов **array**, **string-array**, **integer-array**.

Программно получить доступ к ресурсам массива можно так:

```
// загрузка массива строк из res/values/arrays.xml в текстовое поле textStrings
String[] names = getResources().getStringArray(R.array.names);
for(int i = 0; i < names.length; i++) {
    textStrings.append("Name[" + i + "]: " + names[i] + "\n");
}

// загрузка массива целых чисел из res/values/arrays.xml в текстовое поле textDigits
int[] digits = getResources().getIntArray(R.array.digits);
for(int i = 0; i < digits.length; i++) {
    textDigits.append("Digit[" + i + "]: " + digits[i] + "\n");
}
```

Ещё один вариант, который может вам встретиться. Цвета заданы в массиве и доступ к ним через особый объект **TypedArray**

```
<array name="rainbow">
    <item>#ff0000</item>
    <item>#ff9900</item>
    <item>#ffff00</item>
    <item>#00ff00</item>
    <item>#0000ff</item>
    <item>#0099ff</item>
</array>

TypedArray arrayColors = getResources().obtainTypedArray(R.array.rainbow);
ImageView imageView = findViewById(R.id.imageView);
imageView.setBackgroundColor(arrayColors.getColor(1, 0xFF000000));
arrayColors.recycle();
```

Есть ещё один вид ресурсов, который редко используется, но может пригодиться - **plurals**, позволяющий задавать строки для разных ситуаций. Например, в английском языке мужчина в единственном числе - man, а во множественном - men.

```
<plurals name="NumberOfMan">
    <item quantity="one">man</item>
    <item quantity="other">men</item>
</plurals>

<plurals name="cats">
    <item quantity="one">One Cat</item>
    <item quantity="other">%d Cats</item>
</plurals>
```

Два варианта представлены в виде двух отдельных строк одного числительного. Теперь можно использовать Java-код для применения этого множественного числа при выводе строки с указанием количества.

```
Resources res = getResources();
String s1 = res.getQuantityString(R.plurals.NumberOfMan, 0, 0);
```

Первый параметр в методе `getQuantityString()` - это идентификатор ресурса множественного числа, второй указывает на нужную строку, если значение равно 1, то строка берётся без изменений. Третий параметр - строка, которая подставляется, если второй параметр не равен 1.

В английском языке можно использовать только два значения: **one** и **other**. В русском возможны три варианта: **one** (для 1), **few** (для 2-4), **other** (в остальных случаях).

Ресурсы plurals. Множественное число

Когда нам нужно вывести какое-то количество, то возникает проблема с окончаниями. Самый простой способ решения проблемы заключается в использовании двоеточия:

Всего котиков: 5

Но хочется более элегантного решения. Для подобных случаев есть специальный вид ресурсов **plurals**. Следует отметить, что в старых версиях русский язык использовать было проблематично из-за ошибок. Но в версии Android 3.0 (API 11) всё починили.

В обычном файле строковых ресурсов **strings.xml** создаём ресурс **plurals**, в котором задаётся набор правил склонения нужного выражения. В коде нужно указать нужный вариант для заданных чисел.

Дочерний элемент **item** имеет обязательный атрибут **quantity**, который указывает на диапазон чисел.

- **zero** — строка для нуля, отсутствия чего-либо; в некоторых языках — ещё и для чисел, оканчивающихся нулём;
- **one** — строка для чисел, заканчивающихся на единицу; в некоторых языках — только для единицы;
- **two** — для чисел, заканчивающихся на двойку, или только для двойки;
- **few** — зависит от языковых правил; например, для русского языка это относится к числам, оканчивающимся на 2, 3 и 4 (несмотря на наличие правила «two»);
- **many** — также зависит от языковых правил; в русском языке — от пятёрки и выше;
- **other** — остальные варианты

Чтобы лучше уяснить механизм работы с множественными числами, создадим ресурс и проверим на практике.

```

<!-- Этот код мы используем для отладки: когда какой item выбирается системой -->
<plurals name="plurals_1">
    <item quantity="zero">%1$d = zero</item>
    <item quantity="one">%1$d = one</item>
    <item quantity="two">%1$d = two</item>
    <item quantity="few">%1$d = few</item>
    <item quantity="many">%1$d = many</item>
    <item quantity="other">%1$d = other</item>
</plurals>

<!-- А этот код мы используем для проверки того, как выглядит реальное склонение -->
<plurals name="plurals_2">
    <item quantity="zero">нет котов</item>
    <item quantity="one">%1$d кот</item>
    <item quantity="two">%1$d кота</item>
    <item quantity="few">%1$d кота</item>
    <item quantity="many">%1$d котов</item>
    <item quantity="other">%1$d котов</item>
</plurals>

```

В коде вызываем метод **Resources.getQuantityString()**, где **id** — идентификатор нашего ресурса **plurals** (**R.plurals.xxx**), **quantity** — число, для которого нужно найти словоформу и дополнительные параметры **formatArgs** для подстановки в строку по шаблону.

Протестируем вывод чисел от нуля до тридцати трёх:

```

// Выведем данные в формате: 8 = many : 8 котов
for (int i = 0; i <= 33; i++) {
    String systemPlural = this.getResources().getQuantityString(R.plurals.plurals_1, i, i);
    String catPlural = this.getResources().getQuantityString(R.plurals.plurals_2, i, i);
    Log.i("Plurals", systemPlural + " : " + catPlural);
}

```

В цикле получаем из ресурсов строку, согласующуюся (по мнению системы) со счётчиком цикла, на место шаблона **%1\$d** подставляем значение этого счётчика.

Посмотрим, как система выводит количество 33 котов.


```
I/Plurals: 0 = many : 0 котов
I/Plurals: 1 = one : 1 кот
I/Plurals: 2 = few : 2 кота
I/Plurals: 3 = few : 3 кота
I/Plurals: 4 = few : 4 кота
I/Plurals: 5 = many : 5 котов
I/Plurals: 6 = many : 6 котов
I/Plurals: 7 = many : 7 котов
I/Plurals: 8 = many : 8 котов
I/Plurals: 9 = many : 9 котов
I/Plurals: 10 = many : 10 котов
I/Plurals: 11 = many : 11 котов
I/Plurals: 12 = many : 12 котов
I/Plurals: 13 = many : 13 котов
I/Plurals: 14 = many : 14 котов
I/Plurals: 15 = many : 15 котов
I/Plurals: 16 = many : 16 котов
I/Plurals: 17 = many : 17 котов
I/Plurals: 18 = many : 18 котов
I/Plurals: 19 = many : 19 котов
I/Plurals: 20 = many : 20 котов
I/Plurals: 21 = one : 21 кот
I/Plurals: 22 = few : 22 кота
I/Plurals: 23 = few : 23 кота
I/Plurals: 24 = few : 24 кота
I/Plurals: 25 = many : 25 котов
I/Plurals: 26 = many : 26 котов
I/Plurals: 27 = many : 27 котов
I/Plurals: 28 = many : 28 котов
I/Plurals: 29 = many : 29 котов
I/Plurals: 30 = many : 30 котов
I/Plurals: 31 = one : 31 кот
I/Plurals: 32 = few : 32 кота
I/Plurals: 33 = few : 33 кота
```

Система справилась. Мы получили правильные окончания: 1 кот, 2 кота, 5 котов, 31 кот и т.д.

Русский язык требует указать строки для **one**, **few**, **many** и **other** (на всякий случай). По стандарту не используется **zero**, и не используется **two** — но, ничего страшного в указании этих строк нет. На всякий случай можно их всё-таки указывать — кто знает, а вдруг что-то поменяется в будущем.

Системные строковые ресурсы

Во многих случаях можно задействовать системные строковые ресурсы - это строки типа OK, Cancel и др. В таких ситуациях используется схожий формат (добавляется слово android):

```
android:text="@android:string/cancel"
```

xliff

В строковых ресурсах может применяться особый способ с использованием специального формата **xliff**. На практике мне не встречался, но вдруг вам пригодится.

Создадим ресурсы следующим образом.

```
<?xml version="1.0" encoding="utf-8"?>
<resources xmlns:xliff="rn:oasis:names:tc:xliff:document:1.2">

    ...
    <string name="title">Ваше имя: <xliff:g id="name" example="Александр" >%2$s</xliff:g>,
        Имя кота: <xliff:g id="cat_name" example="Рыжик" >%1$s</xliff:g>
    </string>
</resources>
```

Обратите внимание на пространство имён, определяющий тип документа.

Ресурс **title** содержит несколько вставок, обрамленных в тег **xliff**, которые будут заполняться динамически. Каждый тег **xliff** включает в себя атрибуты **id** с уникальным именем тега и **example** с примером его содержимого. Далее заполняем участки с **xliff**:

```
TextView textView = findViewById(R.id.textView);
textView.setText(getString(R.string.title, "Васька", "Толя"));
```

С помощью метода **getString()** достаём нужный ресурс, где первый параметр — это **id** ресурса, второй параметр «Васька» заменит плейсхолдер **%1\$s**, а третий — «Толя» заменит **%2\$s**.

Вы можете использовать любой порядок для вывода текста, который будет зависеть от чисел **%1**, **%2** и т.д. В строке можно использовать несколько раз один и тот же плейсхолдер, например, **%1\$s**, и он везде будет заполнен одним и тем же параметром — «Васька».

Булевы ресурсы

Можно также хранить в ресурсах булевы значения **true** или **false** в файле с произвольным именем в папке **res/values**.

В файле с корневым элементом **<resources>** вы определяете элемент **bool** с нужным значением. У элемента есть атрибут **name** - строка, определяющая имя булевого ресурса.

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <bool name="autostart">true</bool>
    <bool name="sound">no</bool>
</resources>
```

Получить значение через код:

```
Resources res = getResources();
boolean autostartSetting = res.getBoolean(R.bool.autostart);
```

В макете для булевых атрибутов

```
<ImageView
    android:layout_height="fill_parent"
    android:layout_width="fill_parent"
    android:adjustViewBounds="@bool/adjust_view_bounds" />
```

Числовые ресурсы

В ресурсах можно хранить числа типа **Integer**. Хранить можно в произвольном имени XML-файла в папке **res/values/** в корневом элементе **<resources>**

У элемента **<integer>** есть атрибут **name**, определяющий имя числового значения.

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <integer name="max_speed">75</integer>
    <integer name="min_speed">5</integer>
</resources>
```

Для работы в коде:

```
Resources res = getResources();
int maxSpeed = res.getInteger(R.integer.max_speed);
```

Таким образом, для работы с типами **boolean** и **int** следует применять код (в общем виде):

```
Resources resources = context.getResources();
boolean myBooleanResource = resources.getBoolean(R.bool.my_boolean_resource);
int myIntegerResource = resources.getInteger(R.integer.my_integer_resource);
```

Нет необходимости писать избыточный код (хотя он и будет работать) типа такого:

```
boolean myBooleanResource = Boolean.parseBoolean(context.getString(R.string.my_boolean_resource));
int myIntegerResource = Integer.parseInt(context.getString(R.string.my_integer_resource));

// или так (совсем никуда не годится)
if ("false".equals(context.getString(R.string.my_boolean_resource))){
    // ...
}
```

Ресурсы меню

Не создавайте меню в коде приложения, а используйте отдельные ресурсы для меню в формате XML. Можно использовать как для описания обычного и контекстного меню. Меню, описанное в формате XML, загружается в виде программного объекта с помощью метода **inflate**, принадлежащего сервису **MenuInflater**. Как правило, это происходит внутри обработчика **onOptionsItemSelected** (смотри урок [Меню](#)).

```
<menu xmlns:android="http://schemas.android.com/apk/res/android"
      xmlns:app="http://schemas.android.com/apk/res-auto"
      xmlns:tools="http://schemas.android.com/tools"
      tools:context=".MainActivity">
    <item
        android:id="@+id/action_settings"
        android:orderInCategory="100"
        android:title="@string/action_settings"
        app:showAsAction="never"/>

    <item
        android:id="@+id/action_cat1"
        android:orderInCategory="100"
        android:title="@string/action_cat_male"
        app:showAsAction="never"/>

    <item
        android:id="@+id/action_cat2"
        android:orderInCategory="100"
        android:title="@string/action_cat_female"
        app:showAsAction="never"/>

    <item
        android:id="@+id/action_cat3"
        android:orderInCategory="100"
        android:title="@string/action_kitten"
        app:showAsAction="never"/>
</menu>
```

Описание меню

хранится в отдельном файле в каталоге **res/menu**. Имена файлов без расширения автоматически становятся идентификаторами ресурсов.

В XML-файле меню есть три элемента:

- **<menu>** — корневой элемент файла меню;
- **<group>** — контейнерный элемент, определяющий группу меню;
- **<item>** — элемент, определяющий пункт меню

Элементы **<item>** и **<group>** могут быть дочерними элементами **<group>**.

Элемент **item** может иметь несколько атрибутов:

id

Идентификатор пункта меню, по которому приложение может распознать при выделении пункта меню пользователем

title

Текст, который будет выводиться в меню

icon

Значок для пункта меню. Можно использовать графический ресурс

Ресурсы разметки

Еще один из важных видов ресурсов - ресурсы разметки, которые отвечают за внешний вид приложения. Данные ресурсы представлены в формате XML. Ресурс разметки формы (layout resource) - это ключевой тип ресурсов, применяемый при программировании пользовательских интерфейсов в Android. Каждый ресурс, описывающий разметку, хранится в отдельном файле каталога **res/layout**. Имя файла без расширения выступает как идентификатор ресурса.

Ниже представлен фрагмент исходного кода для разметки:

```
setContentView(R.layout.main);
```

Строка **setContentView(R.layout.main);** указывает на то, что у нас имеется статический класс, называемый **R.layout**, в котором есть константа **main** (некое число), указывающая на **View**, определяемый в XML-файле ресурса разметки формы. XML-файл будет иметь имя **main.xml**, и он должен быть размещен в подкаталоге ресурсов **res/layout** и содержать необходимое определение разметки формы.

Содержимое самого файла **main.xml** может быть таким:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:layout_width="fill_parent"
    android:layout_height="fill_parent"
    >
    <TextView
        android:layout_width="fill_parent"
        android:layout_height="wrap_content"
        android:text="@string/hello"
        />
</LinearLayout>
```

В данном файле разметки формы имеется корневой узел **<LinearLayout>**, в котором содержится дочерний элемент **<TextView>**.

Для каждого варианта разметки требуется специальный файл. Если вы рисуете два экрана, вам понадобится два файла разметки,

например, **res/layout/screen1_layout.xml** и **res/layout/screen2_layout.xml**. Каждый файл в подкаталоге **res/layout** генерирует уникальную константу на основе имени файла (без расширения). При работе с ресурсами разметки здесь важно количество файлов, а при работе со строковыми ресурсами важно количество отдельных строковых ресурсов внутри файлов. Например, если в подкаталоге **res/layout** у нас есть два файла **file1.xml** и **file2.xml**, то в файле **R.java** будут содержаться следующие записи:

```
public static final int file1=0x7f030000;
```

```
public static final int file2=0x7f030001;
```

Элементы управления, которые используются в данных файла компоновки, например, **TextView**, будут доступны в коде через ID, генерируемым в **R.java**:

```
TextView tvInfo = this.findViewById(R.id.textView);
```

В данном примере мы находим класс **TextView** при помощи метода **findViewById()** класса **Activity**. Константа **R.id.textView** соответствует ID, заданному в **TextView**. Идентификатор для **TextView** в файле разметки выглядит следующим образом:

```
<TextView android:id="@+id/textView" />
```

Значение атрибута **id** указывает, что константа **textView** будет использоваться для идентификации этого вида среди других, используемых данным окном. Знак **+** в **@+id/textView** означает, что будет создан **ID** с именем **textView**, если он еще не существует.

Синтаксис ссылок на ресурсы

Все ресурсы Android идентифицируются по их **id**, содержащемуся в исходном коде Java. Синтаксис, используемый при связывания **id** с ресурсом в файле XML, называется синтаксис ссылок на ресурс (resource-reference syntax). Синтаксис атрибута **id** в предыдущем примере **@+id/textView** имеет следующую структуру:

```
@[package:]type/name
```

Параметр **type** соответствует одному из пространств имен:

- R.drawable
- R.id
- R.layout
- R.string
- R.attr
- и др.

Параметр **name** - это имя ресурса. Оно также представлено в виде константы **int** в файле **R.java**.

Если не указывать пакет (package), то разрешение пары **type/name** будет производиться на основе локальных ресурсов и локального пакета **R.java**. Если указать **android:type/name**, то связывание ID ссылки будет производиться с применением пакета Android, в частности с использованием файла **android.R.java**. Вы можете использовать имя любого java-пакета вместо подстановочного слова **package**, чтобы использовать файл **R.java**, подходящий для связывания ссылки.

Рассмотрим несколько примеров

TextView android:id="text" - Ошибка компиляции, так как id не принимает необработанные текстовые строки.

TextView android:id="@text" - Неправильный синтаксис. Не хватает названия типа. Вы получите сообщение об ошибке "No Resource type specified"

TextView android:id="@id/text" - Ошибка: не найдено ни одного ресурса, соответствующего id "text". Возможно, вы не задали "text" как один из видов ID

TextView android:id="@android:id/text" - Ошибка: Ресурс не является общедоступным". Означает, что такой id отсутствует в android.R.id. Чтобы такая запись была действительной, необходимо в файле Android.R.java задать id с таким именем

TextView android:id="@+id/text" - Успешно: создает id с названием "text" в файле R.java локального пакета.

Определение собственных идентификационных номеров ресурсов для последующего использования

Общий принцип присвоения **id** предполагает либо создание нового id для ресурса, либо использование одного из id, созданных в пакете Android. Однако **id** можно создавать и заранее, а потом использовать их в собственных пакетах.

Строка <TextView android:id="@+id/text"> в предыдущем фрагменте кода указывает, что **id** с названием **text**, будет использоваться в том случае, если этот **id** уже создан. Если **id** еще не существует, нужно создать новый идентификатор. В связи с этим возникает вопрос: может ли **id**, например, **text** уже существовать в файле **R.java**, чтобы такой идентификатор можно было использовать многократно?

Можно предположить, что такую задачу могла бы выполнять константа, например **R.id.text**, находящаяся в файле **R.java**, но **R.java** не поддается редактированию. Даже, если бы было возможно внести такие изменения, файл приходилось бы заново генерировать после добавления, изменения или удаления любой информации из подкаталога **res/**.

Решение проблемы заключается в использовании тега ресурса под названием **item** для задания **id**, не связанного ни с каким конкретным ресурсом. Ниже приведен соответствующий пример:

```
<resources>
<item type="id" name="text" />
</resources>
```

Здесь **type** описывает тип ресурса, в данном случае **id**. Когда **id** будет установлен, будет работать и следующее определение **View**:

```
<TextView android:id="@id/text"/>
```

Цветовые ресурсы

Для работы со цветом используется тег **<color>**, а цвет указывается в специальных значениях.

- **#RGB**;
- **#RRGGBB**;
- **#ARGB**;
- **#AARRGGBB**;

Также существуют predefined названия цветов. Такие ID доступны в пространстве имен **android.R.color**. Посмотреть цветовые значения цветов можно в документации <http://developer.android.com/reference/android/R.color.html>.

Например, там есть оранжевый цвет **holo_orange_dark**, а также оранжевый-светлый, синий-темный, синий-светлый и т.д.

Обычно для цветовых ресурсов используют файл **colors.xml** в подкаталоге **res/values**. Но можно использовать любое произвольное имя файла, или даже вставить их в файл вместе со строковыми ресурсами **strings.xml**. Android прочтет все файлы, а затем обработает их, присвоив им нужные ID.


```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <color name="red">#f00</color>
    <color name="yellow">#ffff00</color>
    <color name="green">#ff00ff</color>
</resources>
```

Для программного использования цветовых ресурсов можно использовать следующий код:

```
int myRedColor = activity.getResources.getColor(R.color.red); // получаем значение красного цвета

linearLayout.setBackgroundResource(R.color.yellow); // устанавливаем фон в желтый цвет

linearLayout.setBackgroundResource(android.R.color.holo_orange_dark); // устанавливаем фон в оранжевый цвет
```

Более длинный вариант без склейки методов.

```
Resources res = getResources(); // получим объект для работы с ресурсами
int title_color = res.getColor(R.color.title_red); // получим значение цвета, заданного в ресурсах
```

При использовании в xml-файлах (например, файл разметки) используется следующий формат:

```
<TextView
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="@string/anytext"
    android:color="@color/red" />
```

Обратите внимание на использование префикса ****@**** для того, чтобы ввести ссылку ресурса — текст после этого префикса — имя ресурса. В этом случае мы не должны были указывать пакет, потому что мы ссылаемся на ресурс в нашем собственном пакете. Для ссылки на системный ресурс мы должны записать: **android:textColor="@android:color/black"** (чёрный цвет).

Ресурсы размеров

В Android используются следующие единицы измерения: пиксели, дюймы, точки. Все они могут входить в состав XML-шаблонов и кода Java. Данные единицы измерения также можно использовать в качестве ресурсов при помощи тега **<dimen>** (обычно используют файл **dimens.xml**):

```
<resources>
<dimen name="in_pixels">1px</dimen>
<dimen name="in_dp">5dp</dimen>
<dimen name="in_sp">100sp</dimen>
</resources>
```

Список разрешённых единиц измерения можно прочитать в отдельной статье [Единицы измерения](#)

Метод Java использует в названии целое слово **Dimension**, а в коде и в XML используется сокращённая версия **dimen**

Android поддерживает несколько стандартных единиц измерения. Вкратце перечислим их.

- px (pixels) — пиксели. Точки на экране - минимальные единицы измерения;
- dp (density-independent pixels) — независимые от плотности пиксели. Абстрактная единица измерения, основанная на физической плотности экрана с разрешением 160 dpi. В этом случае 1dp = 1px;
- dip - синоним для dp. Иногда используется в примерах Google;
- sp (scale-independent pixels) — независимые от масштабирования пиксели. Допускают настройку размеров, производимую пользователем. Полезны при работе с шрифтами;
- in (inches) — дюймы, базируются на физических размерах экрана. Можно измерить обычной линейкой;
- mm (millimeters) — миллиметры, базируются на физических размерах экрана. Можно измерить обычной линейкой;
- pt (points) — 1/72 дюйма, базируются на физических размерах экрана;

Вы должны иметь доступ к каждому экземпляру объекта **Resources**, чтобы найти значения его параметров. Это можно сделать, применив метод **getResources()** к объекту **Activity**. Когда у вас будет объект **Resources**, его можно запросить по **id**, чтобы узнать значение этого параметра.

```
float dimen = activity.getResources().getDimension(R.dimen.in_pixels);
```

В XML-файлах используется следующий синтаксис

```
<TextView
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:textSize="@dimen/in_dp"/>
```

Ресурсы визуальных стилей и тем

Ресурсы со стилями позволяют поддерживать единство внешнего вида приложения. Чаще всего визуальные стили и темы используются для хранения цветовых значений и шрифтов. Обычно используют **файл styles.xml**.

Вместо описания каждого стиля, вы можете использовать ссылки, предоставляемые Android, с помощью которых вы можете использовать стили из текущей темы.

Чтобы создать стиль, используйте тег **<style>**, включающий атрибут **name**, а также один или несколько вложенных узлов **item**. Каждый тег **item** в свою очередь также должен иметь атрибут **name**, содержащий тип описываемого значения (например, размер шрифта или цвет). Внутри тега должно находиться само значение.

```
<resources>
    <drawable name="black_rectangle">#000000</drawable>
    <drawable name="white_rectangle">#ffffff</drawable>
</resources>
```

В XML-шаблонах ресурсы используются следующим образом

```
<TextView
    android:layout_width="fill_parent"
    ...
    android:background="@drawable/white_rectangle" />
```

Программным способом:

```
// Получение отрисовываемого объекта
ColorDrawable whiteDrawable =
    (ColorDrawable)activity.getResources().getDrawable(R.drawable.white_rectangle);

// установление его в качестве фона для текстового вида
textView.setBackground(whiteDrawable);
```

Mipmap

С появлением Nexus 6 и Nexus 9, Гугл добавила в Android новые типы ресурсов - **Mipmap**. По сути, это замена ресурсам **Drawable** для значков приложения. Нужно подготовить значки и расположить их в папках **mipmap-mdpi**, **mipmap-hdpi** и т.д. Основная идея заключается в том, что при компиляции неиспользуемые drawable-ресурсы могут быть удалены в целях оптимизации. Перенос значков приложения в новые папки с разными разрешениями позволяет избежать потенциальной проблемы с удалением нужных файлов.

Ресурсы анимации

Android поддерживает два типа анимации. Первый тип основан на расчете промежуточных кадров и может использоваться для поворота, перемещения, растягивания, затемнения элементов. Второй тип - пошаговая анимация, т.е. последовательный вывод заранее подготовленных изображений.

При использовании анимации промежуточных кадров каждый экземпляр анимации хранится в отдельном XML-файле внутри каталога **res/anim**. Имена файлов без расширения являются идентификаторами для ресурсов.

Анимацию можно задать в виде изменений параметров **alpha** (затемнение), **scale** (масштабирование), **translate** (перемещение) или **rotate** (вращение).

Вы можете комбинировать различные экземпляры анимации, используя тег **set** с атрибутами:

- **duration** - продолжительность анимации в секундах

- `startOffset` - миллисекундная задержка перед началом анимации
- `fillBefore` - при значении `true` преобразование происходит перед началом анимации
- `fillAfter` - при значении `true` преобразование происходит после завершения анимации
- `interpolator` - описывает изменения в скорости эффекта

Если не использовать атрибут **`startOffset`**, все анимационные эффекты внутри набора происходят одновременно.

Пошаговая анимация подразумевает создание последовательности объектов **`Drawable`**, каждый из которых будет отображаться в качестве фона на протяжении указанного промежутка времени. Пошаговая анимация хранится в виде ресурсов **`Drawable`** в каталоге **`res/drawable`** (см. выше). Имена файлов без расширения используются в качестве идентификаторов.

Скомпилированные и нескомпилированные ресурсы Android

Большинство ресурсов компилируются в двоичные файлы, но некоторые используются без дополнительной обработки. В Android поддержка ресурсов осуществляется преимущественно при помощи файлов двух типов - XML и RAW-файлов (к последним относятся изображения, аудио и видео). При работе с XML-файлами мы видели, что в большинстве случаев ресурсы определяются как значения внутри файла XML (это касается, например, строк), а иногда весь XML-файл является ресурсом (например, файл ресурса разметки).

Файлы, созданные в XML, также подразделяются на два типа: первые компилируются в двоичный формат, а вторые копируются на устройство без изменений. Например, XML-файлы строковых ресурсов и ресурсов разметки компилируются в двоичный формат. Эти XML-файлы имеют заданный формат, в котором узлы XML преобразуются в ID.

XML-документы

Вы также можете специально выбрать некоторые XML-файлы и задать для них структуру формата, отличающуюся от принятой по умолчанию, - чтобы эти файлы не интерпретировались и для них не генерировались идентификаторы ресурсов. Однако в данном случае нам как раз нужно, чтобы они компилировались в двоичный формат и чтобы их было удобно локализовывать. Для достижения этой цели файлы XML можно поместить в подкаталог **`/res/xml`** - тогда они будут скомпилированы в двоичный формат. В таком случае вы можете воспользоваться поставляемыми вместе с Android инструментами для считывания XML, которые могут считывать XML-узлы.

Ниже приведен пример XML-файла в подкаталоге **res/xml/test.xml**:

Ниже приведен пример XML-файла в подкаталоге **res/xml/test.xml**:

```
<rootelement>
  <subelement>
    Коты - наше всё!
  </subelement>
</rootelement>
```

Как и при работе с другими XML-файлами ресурсов, ААРТ компилирует такой XML-файл, перед тем как поместить его в пакет прикладных программ. Для синтаксического разбора подобных файлов используйте экземпляр **XmlPullParser**:

```
Resources res = activity.getResources();
XmlResourceParser xpp = res.getXml(R.xml.test);
```

Возвращённый **XmlResourceParser** является экземпляром **XmlPullParser**, а также реализует **java.util.AttributeSet**.

Использование необработанных ресурсов RAW

Если разместить файлы, в том числе написанные на XML, в каталоге **res/raw**, они не будут скомпилированы в двоичном формате, а попадают в пакет прикладных программ как есть. Для считывания таких файлов нужно использовать явные API с поддержкой потоков. К категории **raw** относятся аудио- и видеофайлы.

Каждый такой файл, помещённый в папку **res/raw**, имеет свой идентификатор, генерируемый в **R.java**. Чтобы получить доступ к ресурсам, предназначенным только для чтения, вызовите метод **openRawResource()**, принадлежащий объекту **Resource** приложения. Таким образом, вы получите объект **InputStream**, основанный на указанном файле. В качестве имени переменной, принадлежащей **R.raw**, задайте имя файла (без расширения).

Если бы вы поместили текстовый файл в **res/raw/test.txt**, то его можно было бы прочитать при помощи следующего кода, используя идентификатор **test**:

```

public void onClick(View view) {
    TextView infoTextView = findViewById(R.id.textViewInfo);
    infoTextView.setText(getStringFromRawFile(MainActivity.this));
}

private String getStringFromRawFile(Activity activity) {
    Resources r = activity.getResources();
    InputStream is = r.openRawResource(R.raw.test);
    String myText = null;
    try {
        myText = convertStreamToString(is);
    } catch (IOException e) {
        e.printStackTrace();
    }
    try {
        is.close();
    } catch (IOException e) {
        e.printStackTrace();
    }
    return myText;
}

private String convertStreamToString(InputStream is) throws IOException {
    ByteArrayOutputStream baos = new ByteArrayOutputStream();
    int i = is.read();
    while( i != -1)
    {
        baos.write(i);
        i = is.read();
    }
    return baos.toString();
}

```

Мы уже рассматривали структуру папок в каталоге **res**. Компилятор ресурсов, входящий в состав инструмента Android Asset Packaging Tool (AAPT), собирает все ресурсы, кроме **raw**, и помещает все их в итоговый файл APK. Этот файл, содержащий код и ресурсы приложения Android, аналогичен файлу JAR, который применяется в Java (APK означает Android package - пакет Android). Именно файл APK устанавливается на устройство.

Хотя инструмент синтаксического разбора (parser) ресурсов XML допускает такие имена ресурсов, как `hello-string`, в файле R.java во время компиляции такое название вызовет ошибку. Для ее устранения можно переименовать ресурс в `hello_string`, заменив дефис нижним подчеркиванием.

Имена файлов, в которых дублируются базовые имена, вызывают ошибку компиляции (build error). Так происходит со всеми идентификаторами ресурсов, которые сгенерированы для ресурсов, созданных на основе файлов.

Использование ресурсов в коде программы

Подведём воедино информацию об использовании ресурсов в коде программы.

Во время компиляции генерируется статический класс **R** на основе ваших ресурсов и содержит идентификаторы всех ресурсов в программе. Класс **R** имеет несколько вложенных классов, один для каждого типа ресурса, поддерживаемого системой Android, и для которого в проекте существует файл ресурса.

Класс **R** может содержать следующие вложенные классы:

- R.anim — идентификаторы для файлов из каталога res/anim/ (анимация);
- R.array — идентификаторы для файлов из каталога res/values/ (массивы);
- R.bool — идентификаторы для битовых массивов в файлах arrays.xml из каталога res/values/;
- R.integer — идентификаторы для целочисленных массивов в файлах arrays.xml из каталога res/values/;
- R.color — идентификаторы для файлов colors.xml из каталога res/values/ (цвета);
- R.dimen — идентификаторы для файлов dimens.xml из каталога res/values/ (размеры);
- R.drawable — идентификаторы для файлов из каталога res/drawable/ (изображения);
- R.id — идентификаторы представлений и групп представлений для файлов XML-разметки из каталога res/layout/;
- R.layout — идентификаторы для файлов разметки из каталога res/layout/;
- R.raw — идентификаторы для файлов из каталога res/raw/;
- R.string — идентификаторы для файлов strings.xml из каталога res/values/ (строки);
- R.style — идентификаторы для файлов styles.xml из каталога res/values/ (стили);
- R.xml — идентификаторы для файлов из каталога res/xml/.

Синтаксис для обращения к ресурсу:

```
R.resource_type.resource_name
```

При использовании системных ресурсов используется класс **android.R**.

```
android.R.drawable.sym_def_app_icon; // стандартный значок приложения
android.R.style.Theme_Black; // загрузить стандартный стиль:
```

Если в коде вам понадобится идентификатор ресурса для конструктора или метода, то можете использовать данные свойства:

```
setContentView(R.layout.activity_main); // загрузка ресурса разметки

// Строковый ресурс используется при выводе Toast-сообщения
Toast.makeText(this, R.string.mytext, Toast.LENGTH_LONG).show();
```

Если вам нужен не идентификатор, а сам экземпляр ресурса, то используйте метод **getResources** для доступа к экземпляру класса **Resources**:

```
Resources myResources = getResources();
```

Принцип работы класса **Resources** заключается в передаче идентификатора ресурса, чей экземпляр вам нужен для работы. Вот несколько примеров получения экземпляров ресурсов:

```

Resources myResources = getResources();

CharSequence myText = myResources.getText(R.string.hello_app); // получим строку
Drawable myIcon = myResources.getDrawable(R.drawable.app_icon); // получим значок
int myColor = myResources.getColor(R.color.color_blue); // получим значение цвета
float myWidth = myResources.getDimension(R.dimen.width_border); // получим размер

// Получим массив строк
String[] stringArray;
stringArray = myResources.getStringArray(R.array.string_array);

// Ресурс, содержащий пошаговую анимацию, возвращается в виде объекта AnimationResources
// Также можно вернуть значение с помощью метода getDrawable и привести к нужному типу
AnimationDrawable cat;
cat = (AnimationDrawable)myResources.getDrawable(R.drawable.frame_cat);

```

Итак, к графическим ресурсам можно обратиться через **R.drawable.cat** (файл cat.png), а к музыкальным трекам через **R.raw.meow** (файл meow.mp3) и по аналогии с другими типами ресурсов.

Иногда требуется большей гибкости при использовании файлов из ресурсов. Например, в приложении нужно использовать имена ресурсов. В этом случае есть два подхода для получения информации о ресурсе.

Первый способ:

```
android.resource://[package]/[res type]/[res name]
```

```
Uri.parse("android.resource://ru.alexanderklimov.test/raw/filename");
```

Второй способ с использованием идентификатора (не особо нужен):

```
android.resource://[package]/[resource_id]
```

```
Uri.parse("android.resource://ru.alexanderklimov.test/" + R.raw.filename);
```

Допустим, у вас файлы имеют схожие имена **meow1.mp3**, **meow2.mp3**, **meow3.mp3**. По приведённой выше схеме не составит труда создать переменную типа **String fileName = "meow" + n**. Такой подход может пригодиться в циклах, когда счётчик можно сопоставить с именем файла.

Можете заменить имя пакета на программное извлечение с помощью метода **Context.getPackageName()**.

Получить идентификатор ресурса по его имени

Ещё один полезный способ, который может пригодиться. Получить идентификатор по имени файла (без расширения) при помощи метода **getIdentifier()**:


```
int resourceID = this.getResources().getIdentifier("filename",
    "raw", this.getPackageName());
MediaPlayer = MediaPlayer.create(getApplicationContext(),
    resourceID);
```

Пример для графического ресурса.

```
String mDrawableName = "cat1"; // файл cat1.jpg в папке drawable
int resID = getResources().getIdentifier(mDrawableName, "drawable", getPackageName());
```

Пример для строкового ресурса в виде функции на Kotlin. Сам идентификатор в виде числа может и не очень нужен, но он поможет узнать имя ресурса.

```
// узнать идентификатор
val resId = resources.getIdentifier(
    "app_name", "string", packageName
)
println("resId: $resId")

// функция для получения имя ресурса из идентификатора
fun getResIdFromResourcesByName(resourceName: String): String? {
    return try {
        // get the string resource id by name
        val resourceId = resources.getIdentifier(
            resourceName,
            "string",
            packageName
        )

        // return the string value
        getString(resourceId)
    } catch (e: Resources.NotFoundException) {
        null
    }
}

println(getResIdFromResourcesByName("app_name"))
```

Иногда нужно получить не сам идентификатор, а его имя (не значение), чтобы сохранить его, скажем, в базе данных. Воспользуйтесь следующим приёмом, используя метод **getResourceEntryName()**:

```
// вернёт cat1
Log.i("id", this.getResources().getResourceEntryName(R.drawable.cat1));
```

Или так

```
// вернёт ru.alexanderklimov.test:drawable/cat1
Log.i("id", getResources().getResourceName(R.drawable.cat1));
```

```
// вернёт res/drawable/cat1.jpg
Log.i("id", getResources().getString(R.drawable.cat1));
// вернёт тот же результат
Log.i("getText()", this.getResources().getText(R.drawable.cat1).toString());
```

Пример для строкового ресурса на Kotlin

```
// вернёт app_name
println(resources.getResourceEntryName(R.string.app_name))
```

Вложенные ресурсы

Можно использовать ссылки на ресурсы в качестве значений для атрибутов внутри других ресурсов (разметка, стили), что позволяет создавать специальные варианты визуальных тем, локализованных строк и графических объектов. Чтобы сослаться на один ресурс внутри другого, используйте символ @ в следующем виде:

```
attribute = "[packageName:]resourcetype/resourceID"
```

Полное имя **packageName** нужно указывать только при использовании сторонних пакетов, при использовании собственных ресурсов данный параметр можно опустить, так как Android предполагает, что вы используете ресурсы пакета со своим приложением.

```
...
android:text="@string/hello_app"
android:textColor="@color/color_blue"
...
```

Использование системных ресурсов

Сама система имеет собственные ресурсы (строки, изображения, анимация, разметки, стили), которые используются стандартными приложениями, входящими в состав Android. Вы также можете использовать данные системные ресурсы в своих приложениях, добиваясь единообразного стиля и дизайна. Получение доступа к системным ресурсам внутри кода программы ничем не отличается от приведенных выше примеров. Единственно, в чем состоит отличие, это использование не вашего класса R, а системного класса **android.R**.

Например, для получения строки, которая хранит сообщение об ошибке, используется следующий код:

```
CharSequence errorMessage = getString(android.R.string.httpErrorBadUrl);
```

Чтобы получить доступ к системным ресурсам внутри XML-файла, используйте значение **android** следующим образом:

```
...
android:text="@android:string/httpErrorBadUrl"
android:textColor="@android:color/darker_gray"
...
```

Создание ресурсов для локализации и аппаратных конфигураций

Применение ресурсов позволяет использовать механизм динамического выбора нужного ресурса в программе. Можно задать определенную структуру каталогов в проекте, чтобы создавать ресурсы для конкретных языков, регионов и аппаратных конфигураций. Во время выполнения программы Android выберет нужные значения для конкретного телефона пользователя.

Вы можете указывать альтернативные значения, создавая собственные структуры каталогов внутри каталога **res** при помощи дефиса (-). Например, мы хотим создать дополнительные строковые ресурсы для французского языка, франкоканадского региона и для русского языка. Тогда структура каталогов в проекте будет выглядеть следующим образом:

```
Project/
  res/
    values/
      strings.xml
    values-fr/
      strings.xml
    values-fr-rCA
      strings.xml
    values-ru/
      strings.xml
```

Как видите, мы создали несколько файлов strings.xml, которые содержат текст на французском и русском языках и раскидали их по нужным каталогам. Далее приводится список возможных идентификаторов, которые можно использовать для создания альтернативных значений в ресурсах.

Регионы и язык можно указывать без всякой связи друг с другом. Так в папке values-ru-rJP будут храниться русские тексты для жителей Японии.

- **Мобильный код страны и код мобильного оператора (MCC/MNC)** - содержит информацию о стране и опционально о мобильной сети, которая привязана к SIM-карте. MCC (Mobile Country Code) состоит из символов mcc, за которыми следует трехзначный код страны. Также можно добавить MNC (Mobile Network Code), используя символы mnc и двухзначный код мобильной сети, например, **mcc250-mnc99** (Россия-Билайн). Список кодов MCC/MNC можно посмотреть в Википедии.
- **Язык и регион** - указывает на язык при помощи языкового кода в формате ISO 639-1. Состоит из двух символов в нижнем регистре. В случае необходимости можно добавить обозначение региона в виде символа r и двухсимвольного кода в формате ISO 3166-1-alpha-2, записанного в верхнем регистре, например, en-rUS, en-rGB, fr-rCA.
- **Размер экрана** - может иметь одно из следующих значений:
 - small - меньше, чем HVGA, 3.2 дюйма
 - medium - HVGA или меньше, чем HVGA, типичный размер
 - large - VGA или больше, планшет или нетбук
- **Высота и ширина экрана** - можно использовать значения
 - long — для экранов, которые в альбомном режиме значительно шире, чем на стандартных смартфонах (таких как G1);
 - notlong — для экранов с обычным соотношением сторон
- **Ориентация экрана** - возможны значения port (Портретный), land (Альбомный), square (Квадратные экраны).
- **Плотность пикселей на экране** - возможны значения:
 - ldpi — предназначен для хранения ресурсов, рассчитанных на экраны с низкой плотностью пикселей (100–140 dpi);
 - mdpi — для экранов со средней плотностью пикселей (140–180 dpi);
 - hdpi — для экранов с высокой плотностью пикселей (190–250 dpi);
 - xdpi - новый тип для очень высокой плотности для планшетов
 - nodpi - Вы можете использовать значение nodpi для растровых ресурсов, которые не должны масштабироваться. В этом случае система не требует точного совпадения.

Подбирая подходящий каталог, Android выберет тот спецификатор, который наиболее точно описывает плотность пикселей экрана

устройства и откорректирует масштаб объекта Drawable. По умолчанию, в новом проекте создаются папки drawable-ldpi, drawable-mdpi, drawable-hdpi, drawable-xhdpi, в которых содержатся значки для приложения.

- **Тип сенсорного экрана** - возможны варианты: notouch, stylus, finger.
- **Наличие клавиатуры** - возможны варианты: keysexposed, keyshidden, keysoft.
- **Тип ввода** - возможны варианты: nokeys, qwerty, 12key.
- **Способ навигации** - возможны варианты: nonav, dpad, trackball, wheel.

Вы можете комбинировать указанные спецификаторы, разделяя их дефисами. Поддерживаются любые сочетания, однако они должны идти в том порядке, как указано в списке, т.е. язык всегда указывается раньше ориентации экрана и т.п. В одном спецификаторе может применяться не более одного значения.

Корректные варианты

```
drawable-en-rUS  
drawable-en-keyshidden  
drawable-long-land-notouch-nokeys
```

Некорректные варианты

```
drawable-rUS-en (нарушен порядок)  
drawable-nonav-dpad (несколько значений для одного спецификатора)
```

Если на устройстве не будет обнаружено соответствующих ресурсов, при попытке получения доступа к ним ваша программа сгенерирует исключение. Чтобы избежать подобной ситуации, необходимо всегда добавлять значения по умолчанию для всех типов ресурсов в каталог без спецификаторов.

Пример с локализацией программы можно посмотреть в статье [Локализация приложений](#).

Псевдонимы (alias)

Чтобы избежать дублирования ресурсов, можно использовать псевдонимы, которые будут ссылаться на один и тот же ресурс. Предположим, вы создали два файла `res/layout-land/activity_main.xml` и `res/layout-large/activity_main.xml` с одинаковым содержанием для разметки с альбомной ориентацией для смартфонов и планшетов. Создайте теперь ещё один файл с таким же содержанием, например под именем `res/layout/activity_main_horizontal.xml`.

Теперь два одинаковых файла можете удалить. Вместо них создайте два файла `res/values-land/refs.xml` и `res/values-large/refs.xml`.

```
<resources>  
  <item type="layout" name="activity_main">@layout/activity_main_horizontal</item>  
</resources>
```

Получение идентификаторов ресурсов приложения

Врядли вам пригодится этот код в жизни. Для общего развития.

```
public void getAllResourceId() {
    final R.id idResources = new R.id();
    final Class<R.id> c = R.id.class;
    final java.lang.reflect.Field[] fields = c.getDeclaredFields();

    for (Field field : fields) {
        final int resourceId;
        try {
            resourceId = field.getInt(idResources);
            Log.i("ID", "resourceId: " + resourceId + " Name:" + field.getName());
        } catch (Exception e) {
            Log.e("Exception", e.getLocalizedMessage());
        }
    }
}
```

Context

Context – это объект, который предоставляет доступ к базовым функциям приложения: доступ к ресурсам, к файловой системе, вызов активности и т.д. **Activity** является подклассом **Context**, поэтому в коде мы можем использовать её как **ИмяАктивности.this** (напр. MainActivity.this), или укороченную запись **this**. Классы **Service**, **Application** и др. также работают с контекстом.

Доступ к контексту можно получить разными способами. Существуют такие методы как **getApplicationContext()**, **getContext()**, **getBaseContext()** или **this**, который упоминался выше, если используется в активности.

На первых порах не обязательно понимать, зачем он нужен. Достаточно помнить о методах, которые позволяют получить контекст и использовать их в случае необходимости, когда какой-нибудь метод или конструктор будет требовать объект **Context** в своих параметрах.

В свою очередь **Context** имеет свои методы, позволяющие получать доступ к ресурсам и другим объектам.

- `getAssets()`
- `getResources()`
- `getPackageManager()`
- `getString()`
- `getSharedPreferences()`

Возьмём к примеру метод **getAssets()**. Ваше приложение может иметь ресурсы в папке **assets** вашего проекта. Чтобы получить доступ к данным ресурсам, приложение использует механизм контекста, который и отвечает доступность ресурсов для тех, кто запрашивает доступ - активность, служба и т.д. Аналогично происходит с методом **getResources**. Например, чтобы получить доступ к ресурсу цвета используется конструкция **getResources().getColor()**, которая может получить доступ к данным из файла **res/colors.xml**.

Таким образом, создавая, например, вторую активность, мы можем сразу обеспечить ей доступ к своим ресурсам, так как активность относится к контексту. При создании собственных компонентов **View** также используется контекст в конструкторах, так как компонент тоже может использовать ваши ресурсы. При создании собственных классов, если вам нужно будет обращаться к контексту, то необходимо создать конструктор:

```
// В классе сделайте конструктор, куда будет передаваться контекст:
public MyClass(Context context) {
    this.context = context;
}

// в активности
MyClass cat = new MyClass(this);
```

Через контекст можно узнать практически всю информацию о вашем приложении - имя пакета, класса и т.п.

```
// имя вашего пакета
String info = this.getApplicationInfo().packageName;
```

Тем не менее, следует различать контекст в разных ситуациях. Допустим, у вас есть приложение с несколькими активностями. В манифесте можно прописать используемую тему как для всего приложения, так и для каждой активности в отдельности. Соответственно, выбор контекста повлияет на результат. Как правило, при использовании собственной темы предпочтительнее использовать контекст активности, а не приложения.

Очень часто начинающие программисты впадают в ступор, когда ключевое слово **this** не работает в анонимных классах, например, при щелчке кнопки. В этом случае, используйте полное имя класса перед ним.

```
Button button = (Button) findViewById(R.id.button);
button.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View view) {
        Toast.makeText(MainActivity.this, "Мая", Toast.LENGTH_SHORT).show();
    }
});
```

При создании адаптеров для списков также обращаются к контексту.

```
if (convertView == null) {
    convertView = LayoutInflater.from(getContext()).inflate(R.layout.item_user, parent, false);
}
```

Или ещё пример для адаптера в фрагменте **ListFragment**:

```
@Override
public View onCreateView(LayoutInflater inflater, ViewGroup container,
    Bundle savedInstanceState) {

    String[] names; // имена котов
    // бла-бла-бла, т.е. мяу-мяу-мяу
    ArrayAdapter<String> adapter = new ArrayAdapter<>(
        inflater.getContext(), android.R.layout.simple_list_item_1,
        names);
    setListAdapter(adapter);

    return super.onCreateView(inflater, container, savedInstanceState);
}
```

Здесь тоже следует быть внимательным, если используется своя тема для списка.

Последнее замечание относится к опытным программистам. Неправильный контекст может послужить источником утечки памяти. Если вы создадите собственный класс, в котором содержится статическая переменная, обращающая к контексту активности, то система будет держать ссылку на переменную. Если активность будет закрыта, то сборщик мусора не сможет очистить память от переменной и самой неиспользуемой активности. В таких случаях лучше использовать контекст приложения через метод **getApplicationContext()**.

ContextCompat

В библиотеки совместимости появился свой класс для контекста **ContextCompat**. Он может вам пригодиться, когда студия вдруг подчеркнёт метод в старом проекте и объявит его устаревшим.

```
context.getResources().getColor(R.color.some_color_resource_id);
```

Допустим, мы хотим поменять цвет текста на кнопки.

```
public void onClick(View view) {
    int color = getResources().getColor(R.color.colorPrimary); // ругается
    Button button = (Button) findViewById(R.id.button);
    button.setTextColor(color);
}
```

Студия ругается, что нужно использовать новый вариант **getColor(int, Theme)**. Заменим строчку.

```
// Теперь не ругается
int color = ContextCompat.getColor(this, R.color.colorPrimary);
```


Если посмотреть на исходники этого варианта, то увидим, что там тоже идёт вызов нового метода. Поэтому можно сразу использовать правильный вариант, если вы пишете под Marshmallow и выше.

```
// Только для API 23 и выше
int color = getResources().getColor(R.color.colorPrimary, getTheme());
```

Контекст (Context) – это базовый абстрактный класс, реализация которого обеспечивается системой Android. Этот класс имеет методы для доступа к специфичным для конкретного приложения ресурсам и классам и служит для выполнения операций на уровне приложения, таких, как запуск активностей, отправка широковещательных сообщений, получение намерений и прочее.

От класса Context наследуются такие крупные и важные классы, как Application, Activity и Service, поэтому все его методы доступны из этих классов.

Методы получения контекста и их различие

Получить контекст внутри кода можно одним из следующих методов:

- **getBaseContext** (получить ссылку на базовый контекст)
- **getApplicationContext** (получить ссылку на объект приложения)
- **getContext** (внутри активности или сервиса получить ссылку на этот объект)
- **this** (то же, что и getContext)
- **MainActivity.this** (внутри вложенного класса или метода получить ссылку на объект MainActivity)
- **getActivity** (внутри фрагмента получить ссылку на объект родительской активности)

Кроме того, Context является проводником в систему, он может предоставлять ресурсы, получать доступ к базам данных, предпочтениям и т.д. Ещё в Android приложениях есть Activity. Это похоже на проводник в среду, в которой выполняется ваше приложение. Объект Activity наследует объект Context. Он позволяет получить доступ к конкретным ресурсам и информации о среде приложения.

Контекст приложения

Это singleton-экземпляр (единственный на всё приложение), и к нему можно получить доступ через функцию `getApplicationContext()`. Этот контекст привязан к жизненному циклу приложения. Контекст приложения может использоваться там, где вам нужен контекст, жизненный цикл которого

не связан с текущим контекстом или когда вам нужно передать контекст за пределы Activity.

Например, если вам нужно создать singleton-объект для вашего приложения, и этому объекту нужен какой-нибудь контекст, всегда используйте контекст приложения.

Если вы передадите контекст Activity в этом случае, это приведет к утечке памяти, так как singleton-объект сохранит ссылку на Activity и она не будет уничтожена сборщиком мусора, когда это потребуется.

В случае, когда вам нужно инициализировать какую-либо библиотеку в Activity, всегда передавайте контекст приложения, а не контекст Activity.

Таким образом, `getApplicationContext()` нужно использовать тогда, когда известно, что вам нужен контекст для чего-то, что может жить дольше, чем любой другой контекст, который есть в вашем распоряжении.

Контекст Activity

Этот контекст доступен в Activity и привязан к её жизненному циклу. Контекст Activity следует использовать, когда вы передаете контекст в рамках Activity или вам нужен контекст, жизненный цикл которого привязан к текущему контексту.

`getContext()` в `ContentProvider`

Этот контекст является контекстом приложения и может использоваться аналогично контексту приложения. К нему можно получить доступ через метод `getContext()`.

Когда нельзя использовать `getApplicationContext()`?

- Это не полноценный контекст, поддерживающий всё, что может Activity. Некоторые вещи, которые вы попытаетесь сделать с этим контекстом, потерпят неудачу, в основном связанные с графическим интерфейсом.
- Могут появляться утечки памяти, если контекст из `getApplicationContext()` удерживается на каком-то объекте, который вы не очищаете впоследствии. Если же где-то удерживается контекст Activity, то как только Activity уничтожается сборщиком мусора, всё остальное тоже уничтожается. Объект Application остается на всю жизнь вашего процесса.



70

В основных программах Java нам нужен метод `main()`, потому что при выполнении байтового кода JVM будет искать метод `main()` в классе и начнет выполнение там.

В случае Android виртуальная машина Dalvik (после android 5.0 DVM заменяется Android Runtime) предназначена для поиска класса, который является подклассом `Activity` и который установлен как LAUNCHER для запуска выполнения приложения из его метода `onCreate()`, поэтому нет необходимости в методе `main()`.

Дополнительные сведения см. в разделе Жизненный цикл `Activity`.

Поделиться

Chandra Sekhar 15 февраля 2012 в 12:35



23

На самом деле метод `main()` является классом Android framework `android.app.ActivityThread`. Этот метод создает Main (UI) `Thread` для процесса OS, устанавливает в нем `Looper` и запускает цикл событий.

Фреймворк Android отвечает за создание и уничтожение процессов OS, запуск приложений, запуск активностей, служб и других компонентов. `ActivityManager` является частью структуры Android и отвечает за координацию и управление различными компонентами.

Архитектура Android немного отличается от архитектуры автономных приложений Java. Самая большая разница заключается в том, что все ваши компоненты (`Activity`, `Service`, `BroadcastReceiver` и т. Д.) Не обязательно выполняются в одном и том же процессе OS или в одной и той же виртуальной машине (VM). Можно иметь компоненты из одного приложения, запущенного в разных процессах OS, а также можно иметь компоненты из разных приложений, запущенных в одном и том же процессе OS. В традиционном Java метод `main()` - это метод, вызываемый виртуальной машиной после создания процесса OS и завершения инициализации виртуальной машины.

Dalvik — регистровая виртуальная машина для выполнения программ, написанных на языке программирования Java, созданная группой разработчиков Google во главе с Дэном Борнштейном (англ. *Dan Bornstein*). Входит в мобильную операционную систему Android.

Dalvik оптимизирован для низкого потребления памяти, это нестандартная регистр-ориентированная виртуальная машина, хорошо подходящая для исполнения на процессорах RISC-архитектур, часто используемых в мобильных и встраиваемых устройствах, таких как коммуникаторы и планшетные компьютеры (большинство виртуальных машин, используемых в настольных системах, является стек-ориентированным, включая стандартную виртуальную машину Java, принадлежащую Oracle).

Программы для Dalvik пишутся на языке Java. Несмотря на это, стандартный байт-код Java не используется, вместо него Dalvik исполняет байт-код собственного формата. После компиляции исходных текстов программы на Java (при помощи `javac`) утилита `dx` из Android SDK преобразует файлы классов (расширение `.class`) в файлы собственного формата (с расширением `.dex`), которые и включаются в пакет приложения (`.apk`).

В версиях, начиная с Android 4.4 Kitkat, имеется возможность переключиться с Dalvik на более быстрый ART (Android Runtime). В Android 5.0 Dalvik был полностью заменён на ART.

Android Runtime — среда выполнения Android-приложений, разработанная компанией Google как замена Dalvik. ART впервые появился в Android 4.4 как тестовая функция, а в Android 5.0 полностью заменил Dalvik. В отличие от Dalvik, который использует JIT-компиляцию (во время выполнения приложения), ART компилирует^[1] приложение во время его установки. За счет этого планируется повышение скорости работы программ и одновременно увеличение времени работы от батареи. Недостатком является более долгая загрузка устройства.

Android 7.0 Nougat представила JIT-компилятор с профилированием кода для ART, который позволяет постоянно повышать производительность приложений Android при их запуске. Компилятор JIT дополняет нынешний компилятор Ahead of Time от ART и помогает улучшить производительность во время выполнения.

Для обеспечения обратной совместимости ART использует тот же байт-код, что и Dalvik.

AndroidManifest.xml

Каждый проект в Android имеет файл манифеста, называемый **AndroidManifest.xml**, который хранится в корневом каталоге. Файл манифеста является важной частью нашего приложения, поскольку он определяет структуру и метаданные приложения. В частности, манифест приложения выполняет следующие задачи:

- Задаёт имя пакета Java для приложения. Имя пакета служит уникальным идентификатором для приложения.

- Описывает компоненты приложения (активности, службы, широковестьательные приёмники и контент-провайдеры), из которых состоит приложение. Он содержит названия классов, которые реализуют каждый из компонентов, и задаёт им различные свойства (например, какие интенды они могут обрабатывать). Эти объявления позволяют Android знать, какие компоненты и при каких условиях могут быть запущены.
- Определяет, в каких процессах будут размещаться компоненты приложения.
- Объявляет, каких разрешения должно иметь приложение для доступа к защищенным частям API и взаимодействия с другими приложениями.
- Перечисляет классы Instrumentation, которые предоставляют профилирование и другую информацию во время работы приложения. Эти объявления присутствуют в манифесте только тогда, когда приложение находится на стадии разработки и отладки, и удаляются перед публикацией.
- Объявляет минимальный уровень API, который требуется приложению.
- Перечисляет библиотеки, с которыми приложение должно быть связано.

Файл манифеста также указывает метаданные приложения, которые включают в себя иконку, номер версии, темы и так далее.

Манифест приложения содержит корневой элемент **<manifest>** с именем пакета, заданным в атрибуте **package**. Он также должен включать атрибут **xmlns:android**, который будет предоставлять несколько системных атрибутов, используемых в файле.

В **<manifest>** можно добавить атрибут **android:versionCode**, который используется для определения текущей версии приложения в виде целого числа, которое увеличивается с каждым обновлением. Также атрибут **android:versionName** используется для указания публичной версии, которая показывается пользователям.

Также можно указать, куда должно устанавливаться приложение: на SD-карту или внутреннюю память, используя атрибут **android:installLocation**.

Тег **<manifest>**

Это самый главный тег в котором вложена вся конфигурация проекта.

По умолчанию он создается вместе с файлом AndroidManifest и изначально имеет начальный набор параметров:

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android"  
package="com.example.helloworld"  
android:versionCode="1"  
android:versionName="1.0" >
```

Что же значат эти параметры?

xmlns:android – определяет пространство имен Android;

package – определяет уникальное имя пакета приложения, которое вы задали при создании проекта.

Для чего нужно указывать package? Если вы захотите загрузить ваше приложение на **Google Play**, то он проверяет уникальность при приеме приложения, поэтому рекомендуется использовать свое имя для избежания конфликтов с другими разработчиками.

android:versionCode – по сути это версия вашего приложения. Выпуская новую версию вы указываете её в этом поле, оно должно быть целым числом.

Выше мы говорили про уникальность пакета, так вот если вы ранее загрузили на **Google Play** ваше приложение, то когда вы решите загрузить обновленную версию приложения, то вам нужно придерживаться нескольких правил. Имя пакета должно совпадать с тем, что уже загружено Google Play и указать версию android:versionCode на порядок выше. Но это при условии что вы выпускаете новую версию приложения, в случае если вы хотите добавить немного исправленную версию, то это читайте ниже.

Изменив данный параметр и загрузив приложение на Google Play все пользователям вашего приложения будет предложено обновиться до новой версии приложения.

android:versionName – указывает номер пользовательской версии. Если вы нашли несколько недоработок в вашем приложении и исправили их, то в этом случае можно указать для этого поля новую версию, что будет говорить Google Play, что это не новая версия приложения, а улучшенная. Для именованной версии можно использовать строку или строковый ресурс.

Изменив данный параметр и загрузив приложение на Google Play все пользователям вашего приложения будет предложено обновиться до модифицированной версии приложения.

Тег <uses-permission>

Тег **<uses-permission>** позволяет вам запрашивать разрешение, которые приложению должны быть предоставлены системой для его нормального функционирования.

Разрешения предоставляются во время установки приложения, а не во время его работы. **<uses-permission>** имеет единственный атрибут с именем разрешения `android:name`.

android:name – позволяет дать разрешения на использование ресурсов системы. Например:

android:name='android.permission.CAMERA'

или

android:name='android.permission.READ_CONTACTS'

Наиболее распространенные разрешения

- INTERNET – доступ к интернету;
- READ_CONTACTS – чтение данных из адресной книги пользователя;
- WRITE_CONTACTS – запись данных из адресной книги пользователя;
- RECEIVE_SMS – обработка входящих SMS;
- ACCESS_COARSE_LOCATION – использование приблизительного определения местонахождения при помощи вышек сотовой связи или точек доступа Wi-Fi;
- ACCESS_FINE_LOCATION – точное определение местонахождения при помощи GPS.

Тег <permission>

<permission> – позволяет установить разрешения на использования ресурсов системы или запретить использование компонентов приложения.

android:name – название разрешения

android:label – имя разрешения, отображаемое пользователю

android:description – описание разрешения

android:icon – значок разрешения

android:permissionGroup – определяет принадлежность к группе разрешений

android:protectionLevel – уровень защиты

Ter <permission-tree>

<permission-tree> – объявляет базовое имя для дерева разрешений.

Этот элемент объявляет не само разрешение, а только пространство имен, в которое могут быть помещены дальнейшие разрешения.

Ter <permission-group>

<permission-group> – определяет имя для набора логически связанных разрешений.

Этот элемент не объявляет разрешение непосредственно, только категорию, в которую могут быть помещены разрешения.

Разрешение можно поместить в группу, назначив имя группы в атрибуте **permissionGroup** элемента <permission>.

Ter <instrumentation>

<instrumentation> – объявляет объект *instrumentation*, который дает возможность контролировать взаимодействие приложения с системой.

Обычно используется при отладке и тестировании приложения и удаляется из release-версии приложения.

Ter <uses-sdk>

<uses-sdk> – позволяет объявлять совместимость приложения с указанной версией (или более новыми версиями API) платформы Android.

Уровень API, объявленный приложением, сравнивается с уровнем API системы мобильного устройства, на который устанавливается данное приложение.

Данный тег имеет следующие атрибуты:

android:minSdkVersion – определяет минимальный уровень API, требуемый для работы приложения.

Система Android будет препятствовать тому, чтобы пользователь установил приложение, если уровень API системы будет ниже, чем значение, определенное в этом атрибуте. Вы должны всегда объявлять этот атрибут, например:

android:minSdkVersion="17"

android:maxSdkVersion – позволяет определить самую позднюю версию, которую готова поддерживать ваше приложение.

Ваше приложение будет невидимым в Google Play для устройств с более свежей версией.

targetSDKVersion – позволяет указать платформу, для которой вы разрабатывали и тестировали приложение.

Устанавливая значение для этого атрибута, вы сообщаете системе, что для поддержки этой конкретной версии не требуется никаких изменений.

Ter <uses-configuration>

<uses-configuration> – указывает требуемую для приложения аппаратную и программную конфигурацию мобильного устройства.

Например, приложению для работы нужно наличие фронтальной камеры или USB порт. Спецификация используется, чтобы избежать установки приложения на устройствах, которые не поддерживают требуемую конфигурацию.

Если приложение может работать с различными конфигурациями устройства, необходимо включить в манифест отдельные элементы **<uses-configuration>** для каждой конфигурации. Вы можете задать любую комбинацию, содержащие следующие устройства

- **reqHardKeyboard** – используйте значение **true**, если приложению нужна аппаратная клавиатура;

- **reqKeyboardType** – позволяет задать тип клавиатуры: **nokeys**, **qwerty**, **twelvekey**, **undefined**;

- **reqNavigation** – укажите одно из значений: **nonav**, **dpad**, **trackball**, **wheel** или **undefined**, если требуется устройство для навигации;

Приложение не будет устанавливаться на устройстве, которое не соответствует заданной вами конфигурации.

В идеале, вы должны разработать такое приложение, которое будет работать с любым сочетанием устройств ввода. В этом случае **<uses-configuration>** не нужен.

Ter <uses-feature>

<uses-feature> объявляет определенную функциональность, требующуюся для работы приложения.

Таким образом, приложение не будет установлено на устройствах, которые не имеют требуемую функциональность. Например, приложение могло бы определить, что оно требует фронтальную камеру с автофокусом. Если устройство не имеет встроенную фронтальную камеру с автофокусом, приложения не будет установлено.

android.hardware.camera.front – требуется аппаратная камера

android.hardware.camera.autofocus – требуется камера с автоматической фокусировкой

В офф. документации Android можно посмотреть все параметры, вот [ССЫЛКА](#).

Можно переопределить требование по умолчанию, добавив атрибут **required** со значением **false**.

Например, если вашей программе не нужно, чтобы камера поддерживала автофокус:

```
<uses-feature android:name="android.hardware.camera.autofocus"
android:required="false" />
```

Тег <supports-screens>

<supports-screens> – определяет разрешение экрана, требуемое для функционирования устройства.

Данный тег позволяет указать размеры экран, для которого было создано приложение. Система будет масштабировать ваше приложение на основе ваших макетов на тех устройствах, которые поддерживают указанные вами разрешения экран.

Для других случаев система будет растягивать макет по мере возможности.

Значения:

smallScreen – QVGA экраны

normalScreen – стандартные экраны HVGA и WQVGA

largeScreen – большие экраны

xlargeScreen – очень большие экраны, которые превосходят размеры планшетов

anyDensity – установите значение *true*, если ваше приложение способно масштабироваться для отображения на экране с любым разрешением.

По умолчанию, для каждого атрибута установлено значение *true*. Вы можете указать, какие размеры экранов ваше приложение не поддерживает.

Начиная с API 13 – **Android 3**, у тега появились новые атрибуты:

requiresSmallestWidthDp – указываем минимальную поддерживаемую ширину экрана в аппаратно-независимых пикселях. С его помощью можно отфильтровать устройства при размещении приложения в Google Play

compatibleWidthLimitDp – задаёт верхнюю границу масштабирования для вашего приложения. Если экран устройства выходит за указанную границу, система включит режим совместимости.

largestWidthLimitDp – задаёт абсолютную верхнюю границу, за пределами которой ваше приложение точно не может быть масштабировано. В этом случае приложение запускается в режиме совместимости, которую нельзя отключить.

Пример использования:

```
<supports-screens android:smallScreens="false"
android:normalScreens="true"
android:largeScreens="true"
android:requiresSmallestWidthDp="480"
android:compatibleWidthLimitDp="600"
android:largestWidthLimitDp="720" />
```

Следует избегать подобных ситуаций и разрабатывать макеты для любых экранов.

Тег **<compatible-screens>**

<compatible-screens> – указывает для каждого экрана конфигурацию, с которой оно совместимо. Только один экземпляр **<compatible-screens>** элемента допускается в манифесте, но он может содержать несколько элементов **<screen>**.

Каждый элемент **<screen>** указывает конкретный размер экрана с которой приложение совместимо.

Этот элемент является информационным и может быть использован внешние услуги (например, Google Play), чтобы лучше понять совместимость приложения с конкретными конфигурациями экрана и включить фильтрацию для пользователей.

Данный тег имеет два атрибута:

android:screenSize – указывает размер экрана.

Значения:

small – маленький;

normal – средний;

large – большой;

xlarge – очень большой;

android:screen – указываем dpi

Значения:

ldpi – ~120dpi

mdpi – ~160dpi

hdpi – ~240dpi

xhdpi – ~320dpi

Пример использования:

```
<compatible-screens>
  <screen android:screenSize="small" android:screenDensity="ldpi" />
  <screen android:screenSize="small" android:screenDensity="mdpi" />
  <screen android:screenSize="small" android:screenDensity="hdpi" />
  <screen android:screenSize="small" android:screenDensity="xhdpi" />

  <screen android:screenSize="normal" android:screenDensity="ldpi" />
  <screen android:screenSize="normal" android:screenDensity="mdpi" />
  <screen android:screenSize="normal" android:screenDensity="hdpi" />
  <screen android:screenSize="normal" android:screenDensity="xhdpi" />
</compatible-screens>
```

Ter <supports-gl-texture>

<supports-gl-texture> – указывает одну текстуру GL сжатия, который поддерживается приложением.

Пример использования:

<supports-gl-texture android:name="GL_OES_compressed_ETC1_RGB8_texture" />

<supports-gl-texture android:name="GL_OES_compressed_paletted_texture" />

Значения:

GL_OES_compressed_ETC1_RGB8_texture – указано в OpenGL ES 2.0 и доступна во всех Android-устройствах, которые поддерживают OpenGL ES 2.0.

GL_OES_compressed_paletted_texture – общая палитра сжатия текстур.

GL_AMD_compressed_3DC_texture – ATI 3DC сжатия текстур.

Все значения можно посмотреть по этой [ссылке](#).

Тег <application>

<application> – один из основных элементов манифеста, содержащий описание компонентов приложения, доступных в пакете: стили, иконки и др.

Содержит дочерние элементы, которые объявляют каждый из компонентов, входящих в состав приложения. В манифесте может быть только один элемент **<application>**.

Ниже описание тегов которые вложены в тег <application>.

Тег <activity>

<activity> – указывает активность.

Если приложение содержит несколько активностей, они должны быть объявлены в манифесте, создавая для каждой из них свой элемент **<activity>**. Если активность не объявлена в манифесте, она не будет видна системе и не будет запущена при выполнении приложения или будет выводиться сообщение об ошибке.

Пример использования:

```
<activity android:name="com.devcolibri.HelloWorld.MainActivity"
android:label="@string/app_name">
```

android:name – имя класса.

Имя должно включать полное обозначение пакета, но если имя пакета уже включено в <manifest>, имя класса, реализующего деятельность, можно записывать в сокращенном виде, опуская имя пакета.

android:label – текстовая метка, отображаемая пользователю.

Тег <intent-filter>

Каждый тег **<activity>** поддерживает вложенные узлы **<intent-filter>**. Элемент **<intent-filter>** определяет типы намерений, на которые могут ответить активность, сервис или приемник намерений. Фильтр намерений предоставляет для компонентов-клиентов возможность получения намерений объявляемого типа, отфильтровывая те, которые не значимы для компонента, и содержит дочерние элементы **<action>**, **<category>**, **<data>**.

Тег <action>

Элемент **<action>** добавляет действие к фильтру намерений. Элемент **<intent-filter>** должен содержать один или более элементов **<action>**. Если в элементе **<intent-filter>** не будет этих элементов, то объекты намерений не пройдут через фильтр. Пример объявления действия:

```
<action android:name="android.intent.action.MAIN">
```

Ter <category>

<category> – определяет категорию компонента, которую должно обработать намерение. Это строковые константы, определенные в классе `intent`, например:

```
<category android:name="android.intent.category.LAUNCHER">
```

Ter <data>

<data> – добавляет спецификацию данных к фильтру намерений.

Спецификация может быть только типом данных (атрибут `mimeType`), URI или типом данных вместе с URI.

Значение URI определяется отдельными атрибутами для каждой из его частей, т. е. URI делится на части:

`android:scheme`;

`android:host`;

`android:port`;

`android:path` или `android:pathPrefix`;

`android:pathPattern`.

Ter <meta-data>

<meta-data> – определяет пару “имя-значение” для элемента дополнительных произвольных данных, которыми можно снабдить родительский компонент.

Составляющий элемент может содержать любое число элементов `<meta-data>`.

Ter <activity-alias>

<activity-alias> — это псевдоним для Activity, определенной в атрибуте `targetActivity`.

Целевая деятельность должна быть в том же самом приложении, что и псевдоним, и должна быть объявлена перед псевдонимом деятельности в манифесте. Псевдоним представляет целевую деятельность как независимый объект. У псевдонима может быть свой собственный набор фильтров намерений, определяющий, какие намерения могут активизировать целевую деятельность и как система будет обрабатывать эту деятельность.

Например, фильтры намерений на псевдониме деятельности могут определить флаги:

android:name='android.intent.action.MAIN'

и

android:name='android.intent.category.LAUNCHER'

заставляя целевую деятельность загружаться при запуске приложения даже в том случае, когда в фильтрах намерений на целевой деятельности эти флаги не установлены.

Ter <service>

<service> – объявляет службу как один из компонентов приложения. Службы, которые не были объявлены, не будут обнаружены системой и никогда не будут запущены.

Ter <receiver>

<receiver> – объявляет приемник широковещательных намерений как один из компонентов приложения.

Приемники широковещательных намерений дают возможность приложениям получить намерения, которые переданы системой или другими приложениями, даже когда другие компоненты приложения не работают.

Ter <provider>

Элемент **<provider>** объявляет контент-провайдера (источник данных) для управления доступом к базам данных.

Элемент <provider> содержит свой набор дочерних элементов для установления разрешений доступа к данным:

<grant-uri-permission>

<path-permission>

<meta-data>

Ter <grant-uri-permission>

Элемент **<grant-uri-permission>** является дочерним элементом для <provider>.

Он определяет, для кого можно предоставить разрешения на подмножества данных контент-провайдера.

Предоставление разрешения является способом допустить к подмножеству данных, предоставляемым контент-провайдером, клиента, у которого нет разрешения для доступа к полным данным.

Если атрибут **grantUriPermissions** контент-провайдера имеет значение **true**, то разрешение предоставляется для любых данных, поставляемых контент-провайдером. Однако, если атрибут поставлен в **false**, разрешение можно предоставить только подмножествам данных, которые определены этим элементом.

Контент-провайдер может содержать любое число элементов `<grant-uri-permission>`.

Ter `<path-permission>`

`<path-permission>` – дочерний элемент для `<provider>`. Определяет путь и требуемые разрешения для определенного подмножества данных в пределах поставщика оперативной информации. Этот элемент может быть определен многократно, чтобы поставлять множественные пути.

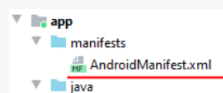
Ter `<uses-library>`

`<uses-library>` – определяет общедоступную библиотеку, с которой должно быть скомпоновано приложение.

Этот элемент указывает системе на необходимость включения кода библиотеки в загрузчик классов для пакета приложения. Каждый проект связан по умолчанию с библиотеками Android, в которые включены основные пакеты для сборки приложений.

Однако некоторые пакеты находятся в отдельных библиотеках, которые автоматически не компонуются с приложением. Если же приложение использует пакеты из этих библиотек или других, от сторонних разработчиков, необходимо сделать явное связывание с этими библиотеками и манифест обязательно должен содержать отдельный элемент `<uses-library>`.

Каждое приложение содержит файл манифеста **AndroidManifest.xml**. Данный файл определяет важную информацию о приложении - название, версию, иконки, какие разрешения приложение использует, регистрирует все используемые классы activity, сервисы и т.д. Данный файл можно найти в проекте в папке **manifests**:



```

1 <?xml version="1.0" encoding="utf-8"?>
2 <manifest xmlns:android="http://schemas.android.com/apk/res/android"
3     package="com.example.viewapp">
4
5     <application
6         android:allowBackup="true"
7         android:icon="@mipmap/ic_launcher"
8         android:label="@string/app_name"
9         android:roundIcon="@mipmap/ic_launcher_round"
10        android:supportRtl="true"
11        android:theme="@style/Theme.ViewApp">
12        <activity android:name=".MainActivity">
13            <intent-filter>
14                <action android:name="android.intent.action.MAIN" />
15
16                <category android:name="android.intent.category.LAUNCHER" />
17            </intent-filter>
18        </activity>
19    </application>
20
21 </manifest>

```

- **android:allowBackup** указывает, будет ли для приложения создаваться резервная копия. Значение `android:allowBackup="true"` разрешает создание резервной копии.
- **android:icon** устанавливает иконку приложения. При значении `android:icon="@mipmap/ic_launcher"` иконка приложения берется из каталога **res/mipmap**
- **android:roundIcon** устанавливает круглую иконку приложения. Также берется из каталога **res/mipmap**
- **android:label** задает название приложения, которое будет отображаться на мобильном устройстве в списке приложений и в заголовке. В данном случае оно хранится в строковых ресурсах - `android:label="@string/app_name"`.
- **android:supportRtl** указывает, могут ли использоваться различные RTL API - специальные API для работы с правосторонней ориентацией текста (например, для таких языков как арабский или фарси).
- **android:theme** устанавливает тему приложения. Подробно темы будут рассмотрены далее, а пока достаточно знать, что тема определяет общий стиль приложения. Значение `@style/Theme.ViewApp` берет тему "Theme.ViewApp" из каталога **res/values/themes**

Установка версии SDK

Для управления версией android sdk в файле манифеста определяется элемент `<uses-sdk>`. Он может использовать следующие атрибуты:

- `minSdkVersion`: минимальная поддерживаемая версия SDK
- `targetSdkVersion`: оптимальная версия
- `maxSdkVersion`: максимальная версия

Версия определяется номером API, например, Jelly Beans 4.1 имеет версию 16, а Android 11 имеет версию 30:

```

1 <?xml version="1.0" encoding="utf-8"?>
2 <manifest xmlns:android="http://schemas.android.com/apk/res/android"
3     package="com.example.viewapp"
4     android:versionName="1.0"
5     android:versionCode="1">
6     <uses-sdk android:minSdkVersion="22" android:targetSdkVersion="30" />
7     <!-- остальное содержимое -->
8
9 </manifest>

```


Установка разрешений

Иногда приложению требуются разрешения на доступ к определенным ресурсам, например, к списку контактов, камере и т.д. Чтобы приложение могло работать с тем же списком контактов, в файле манифеста необходимо установить соответствующие разрешения. Для установки разрешений применяется элемент `<uses-permission>`:

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <manifest xmlns:android="http://schemas.android.com/apk/res/android"
3     package="com.example.viewapp">
4     <uses-permission android:name="android.permission.READ_CONTACTS" />
5     <uses-permission android:name="android.permission.CAMERA" android:maxSdkVersion="30" />
6     <!-- остальное содержимое -->
7
8 </manifest>
```

Атрибут `android:name` устанавливает название разрешения: в данном случае на чтение списка контактов и использование камеры. Опционально можно установить максимальную версию sdk посредством атрибута `android:maxSdkVersion`, который принимает номер API.

Поддержка разных разрешений

Мир устройств Android очень сильно фрагментирован, здесь встречаются как гаджеты с небольшим экраном, так и большие широкоэкранные телевизоры. И бывают случаи, когда надо ограничить использование приложения для определенных разрешений экранов. Для этого в файле манифеста определяется элемент `<supports-screens>`:

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <manifest xmlns:android="http://schemas.android.com/apk/res/android"
3     package="com.example.viewapp">
4
5     <supports-screens
6         android:largeScreens="true"
7         android:normalScreens="true"
8         android:smallScreens="false"
9         android:xlargeScreens="true" />
```

Операционная система Android

Android — операционная система, основанная на Linux с интерфейсом программирования Java. Это предоставляет нам такие инструменты, как компилятор, дебаггер и эмулятор устройства, а также его (Андроида) собственную виртуальную машину Java (Dalvik Virtual Machine — DVM). Android создан альянсом Open Handset Alliance, возглавляемым компанией Google.

Android использует специальную виртуальную машину, так называемую Dalvik Virtual Machine. Dalvik использует свой, особенный байткод.

Следовательно, Вы не можете запускать стандартный байткод Java на Android. Android предоставляет инструмент «dx», который позволяет конвертировать файлы Java Class в файлы «dex» (Dalvik Executable). Android-приложения пакуются в файлы .apk (Android Package) программой «aapt» (Android Asset Packaging Tool) Для упрощения разработки Google предоставляет Android Development Tools (ADT) для Eclipse. ADT выполняет автоматическое преобразование из файлов Java Class в файлы dex, и создает apk во время развертывания.

Android Runtime — среда выполнения [Android](#)-приложений, разработанная компанией [Google](#) как замена [Dalvik](#). ART впервые появился в [Android 4.4](#) как тестовая функция, а в Android 5.0 полностью заменил Dalvik. В отличие от Dalvik, который использует [JIT-компиляцию](#) (во время выполнения приложения), ART компилирует^[1] приложение во время его установки. За счет этого планируется повышение скорости работы программ и одновременно увеличение времени работы от батареи. Недостатком является более долгая загрузка устройства.

Основные компоненты Android

- **Activity/Деятельность** (далее Активности) — представляет собой схему представления Android-приложений. Например, экран, который видит пользователь. Android-приложение может иметь несколько активности и может переключаться между ними во время выполнения приложения.
- **Views/Виды** — Пользовательский интерфейс активности, создаваемый виджетами классов, наследуемых от «android.view.View». Схема views управляется через «android.view.ViewGroups».
- **Services/Службы** — выполняют фоновые задачи без предоставления пользовательского интерфейса. Они могут уведомлять пользователя через систему уведомлений Android.
- **Content Provider/Контент-провайдеры** — предоставляет данные приложениям, с помощью контент-провайдера Ваше приложение может обмениваться данными с другими приложениями. Android содержит базу данных SQLite, которая может быть контент-провайдером
- **Intents/Намерения** (далее Интененты) — асинхронные сообщения, которые позволяют приложению запросить функции из других служб или активности. Приложение может делать прямые интенеты службе или активности (явное намерение) или запросить у Android зарегистрированные службы и приложения для интенета (неявное намерение). Для примера, приложение может запросить через интенет контакт из приложения контактов (телефонной/записной книги) аппарата. Приложение регистрирует само себя в интенетах через IntentFilter. Интенеты — мощный концепт, позволяющий создавать слабосвязанные приложения.
- **Broadcast Receiver/Широковещательный приемник** (далее просто Приемник) — принимает системные сообщения и неявные интенеты, может использоваться для реагирования на изменение состояния системы. Приложение может регистрироваться как приемник определенных событий и может быть запущено, если такое событие произойдет.

Explicit intent явно содержит информацию о классе компонента. Это может быть объект Class, переданный в конструкторе [Intent\(context: Context, cls: Class<*>\)](#) или методом [setClass\(context: Context, cls: Class<*>\)](#), или объект класса ComponentName, переданный методом [setComponent\(componentName: ComponentName\)](#).

Явные интенеты часто используются для старта компонентов внутри приложения, т.к. имена классов известны.

Например `startActivity(Intent(context, MyHomeActivity::class.java))` – это старт активности с явным интенетом.

Implicit intent не содержит информацию о конкретном компоненте. Система использует косвенные атрибуты, такие как action, type и category для выбора стартового компонента. Механизм поиска компонента по атрибутам неявного интенета называется *Intent Resolution*.

Неявные интенеты часто используются для старта компонентов других приложений.

Например `startActivity(Intent(Intent.ACTION_CALL, Uri.parse("tel:$number")))` – неявный интенет, стартовый активности, которая выполнит звонок по заданному номеру.

Механизм поиска компонента по неявному интенету называется *Intent Resolution*. Intent resolution сводится к тому, что система проверяет интенет фильтры всех компонентов, которые прописаны в андроид манифестах установленных приложений, и интенет фильтры бродкаст ресиверов, [зарегистрированных динамически](#).

Для поиска компонента проверяется совпадение по заданным в интенете атрибутам action, category и data.

● Если в интенет добавлен action, то этот же action должен быть объявлен в [intent-filter](#) компонента. Если фильтр компонента не содержит никакой action, то этот компонент сможет обрабатывать только интенеты без action.

● Если интенет содержит категории (атрибут category), то интенет фильтр компонента должен иметь все заданные категории.

● Data в интенте – это [URI](#). При проверке совпадения по data сравнивается data type и data scheme+authority+path. Type может быть добавлен в интент явно методом `setType()` или разрешен по схеме (scheme) объекта data.

[BroadcastReceiver](#) можно зарегистрировать статически и динамически. *Статическая регистрация* – это добавление элемента `<receiver>` в `AndroidManifest`.

```
<receiver android:name=".MyBroadcastReceiver">
    <intent-filter>
        <action android:name="android.intent.action.LOCALE_CHANGED"/>
        <action android:name="android.intent.action.BOOT_COMPLETED"/>
    </intent-filter>
</receiver>
```

Элемент `<intent-filter>` объявляет действия (`<action>`) на которые реагирует `BroadcastReceiver`. Система регистрирует ресиверы, прописанные в манифесте и вызывает их независимо от того запущено приложение или нет.

Динамическая регистрация – это регистрация на контексте в коде приложения. Выполняется методом `context.registerReceiver(receiver: BroadcastReceiver, intentFilter: IntentFilter)`. Этот метод принимает параметром объект класса `IntentFilter`, который определяет на какие действия будет реагировать зарегистрированный ресивер.

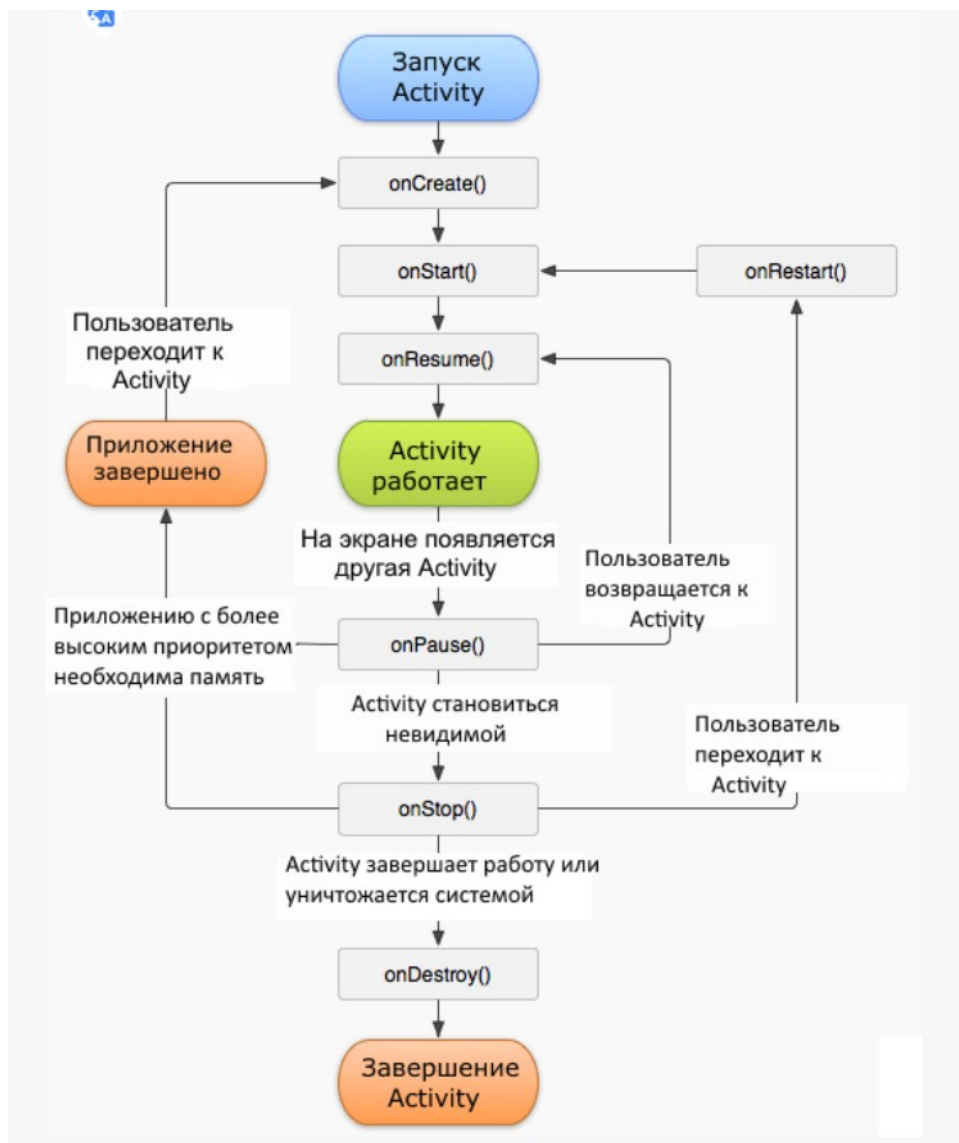
```
val receiver: BroadcastReceiver = MyBroadcastReceiver()
val filter = IntentFilter().apply {
    addAction(ConnectivityManager.CONNECTIVITY_ACTION)
    addAction(Intent.LOCALE_CHANGED)
    addAction("my_custom_action")
}
context.registerReceiver(receiver, filter)
```

Activity

Ключевым компонентом для создания визуального интерфейса в приложении Android является activity (активность). Нередко activity ассоциируется с отдельным экраном или окном приложения, а переключение между окнами будет происходить как перемещение от одной activity к другой. Приложение может иметь одну или несколько activity. Например, при создании проекта с пустой Activity в проект по умолчанию добавляется один класс Activity - MainActivity, с которого и начинается работа приложения:

Каждое приложение запускается в виде отдельного процесса, что позволяет системе давать одним процессам более высокой приоритет, в отличие от других. Благодаря этому, например, при работе с одними приложениями Android позволяет не блокировать входящие звонки. После прекращения работы с приложением, система освобождает все связанные ресурсы и переводит приложение в разряд низкоприоритетного и закрывает его.

Все объекты activity, которые есть в приложении, управляются системой в виде стека activity, который называется **back stack**. При запуске новой activity она помещается поверх стека и выводится на экран устройства, пока не появится новая activity. Когда текущая activity заканчивает свою работу (например, пользователь уходит из приложения), то она удаляется из стека, и возобновляет работу та activity, которая ранее была второй в стеке.



Подробный разбор каждого метода жизненного цикла (`onCreate`, `onStart`, `onResume`, `onPause`, `onStop`, `onDestroy`) и сценариев переходов между ними — см. ниже в разделе «Activity, Fragments» (Android Core).

СЕРВИСЫ Service

Итак, сервис – это некая задача, которая работает в фоне и не использует UI. Запускать и останавливать сервис можно из приложений и других сервисов. Также можно подключиться к уже работающему сервису и взаимодействовать с ним.

Android Services - это компонент, который помогает выполнять **длительные процессы**, такие как обновление базы данных или сервера, запуск обратного отсчета или воспроизведение звука. По умолчанию Android Services запускаются **в том же** потоке, что и основные процессы приложения. Такой вид службы также называют **local service**.

Все сервисы наследуются от класса **Service** и проходят следующие этапы жизненного цикла:

- Метод `onCreate()`: вызывается при создании сервиса
- Метод `onStartCommand()`: вызывается при получении сервисом команды, отправленной с помощью метода `startService()`
- Метод `onBind()`: вызывается при закреплении клиента за сервисом с помощью метода `bindService()`
- Метод `onDestroy()`: вызывается при завершении работы сервиса

Service – это **компонент приложения**, который используется для выполнения долгих фоновых операций без взаимодействия с пользователем.

Любой компонент приложения может запустить сервис, который продолжит работу, даже если пользователь перейдет в другое приложение.

Примеры использования сервисов: проигрывание музыки, трекинг локации водителя в приложении такси, загрузка файла из сети.

Сервисы делятся на два вида по способу использования: **Started** и **Bound**, и на два вида по способу взаимодействия с пользователем: **Background** и **Foreground**.

Когда спрашивают о видах сервисов обычно имеют в виду способ использования.

Started Service Запускается методом `startService(Intent intent)`. **Intent** должен быть явным (**explicit**), это значит, что при создании объекта **Intent** было передано имя класса сервиса.

После запуска сервиса вызывается метод `onStartCommand()`.

Остановить сервис можно вызовом метода `stopSelf()` из самого сервиса или методом `stopService()` из другого компонента.

Bound Service привязывается к компоненту вызовом метода `bindService(Intent service, ServiceConnection serviceConnection, int flags)`.

Аргумент `serviceConnection` используется для взаимодействия с привязанным сервисом. Сервис стартует после вызова `bindService()`, если аргумент `flags` имеет значение **BIND_AUTO_CREATE**.

После вызова `bindService()` у сервиса вызывается метод `onBind()`.

Для отвязывания компонента от сервиса используется метод `unbindService()`. У сервиса вызывается метод `onUnbind()`. Если у сервиса больше нет привязанных компонентов, вызывается метод `onDestroy()`.

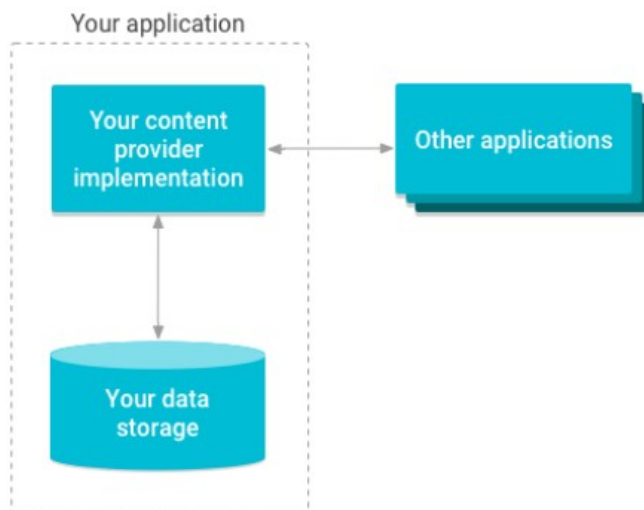
Компонент может быть привязан к сервису, запущенному методом `startService()`. В этом случае сервис относится сразу и к *Started* и к *Bound*.

Bound и *Started* сервисы важно различать, потому что у них разные жизненные циклы.

ContentProvider – один из **четырёх компонентов** андроид приложения. ContentProvider используется для предоставления доступа к хранилищу данных приложения *другим приложениям*. ContentProvider прописывается в андроид манифесте, как элемент `<provider>`.

Для доступа к данным контент провайдера используется класс **ContentResolver**.

ContentResolver предоставляет **CRUD API** для доступа к хранилищу данных. ContentResolver определяет к какому провайдеру направить запрос. Выбранный ContentProvider обрабатывает запрос и возвращает данные обратно резолверу. ContentResolver передает результат пользователю, вызвавшему метод.



Контент-провайдер или "Поставщик содержимого" (Content Provider) - это оболочка (wrapper), в которую заключены данные. Если ваше приложение использует базу данных SQLite, то только ваше приложение имеет к ней доступ. Но бывают ситуации, когда данные желательно сделать общими. Простой пример - ваши контакты из телефонной книги тоже содержатся в базе данных, но вы хотите иметь доступ к данным, чтобы ваше

приложение тоже могло выводить список контактов. Так как вы не имеете доступа к базе данных чужого приложения, был придуман специальный механизм, позволяющий делиться своими данными всем желающим.

Поставщик содержимого применяется лишь в тех случаях, когда вы хотите использовать данные совместно с другими приложениями, работающих в устройстве. Но даже если вы не планируете сейчас делиться данными, то всё-равно можно подумать об реализации этого способа на всякий случай.

В Android используются встроенные поставщики содержимого (пакет **android.provider**)

- Browser
- CallLog
- Contacts
 - People
 - Phones
 - Photos
 - Groups
- MediaStore
 - Audio
 - Albums
 - Artists
 - Genres
 - Playlists
 - Images
 - Thumbnails
 - Video
- Settings

На верхних уровнях иерархии располагаются базы данных, на нижних - таблицы. Так, Browser, CallLog, Contacts, MediaStore и Settings - это отдельные базы данных SQLite, инкапсулированные в форме поставщиков. Обычно такие базы данных SQLite имеют расширение DB и доступ к ним открыт только из специальных пакетов реализации (implerentation package). Любой доступ к базе данных из-за пределов этого пакета осуществляется через интерфейс поставщика содержимого.

Для создания собственного контент-провайдера нужно унаследоваться от абстрактного класса **ContentProvider**:

```
public class MyContentProvider extends ContentProvider {  
}
```

В классе необходимо реализовать абстрактные методы **query()**, **insert()**, **update()**, **delete()**, **getType()**, **onCreate()**. Прослеживается некоторое сходство с созданием обычной базы данных.

А также его следует зарегистрировать в манифесте с помощью тега **provider** с атрибутами **name** и **authorities**. Тег **authorities** служит для описания базового пути URI, по которому **ContentResolver** может найти базу данных для взаимодействия. Данный тег должен быть уникальным, поэтому рекомендуется использовать имя вашего пакета, чтобы не произошло путаницы с другими приложениями, например:

```
<provider  
    android:name=".MyContentProvider"  
    android:authorities="ru.alexanderklimov.provider.notepad" />
```

Источник поставщика содержимого аналогичен доменному имени сайта. Если источник уже зарегистрирован, эти поставщики содержимого будут представлены гиперссылками, начинающимися с соответствующего префикса источника:

```
content://ru.alexanderklimov.provider.notepad/
```

Итак, поставщики содержимого, как и веб-сайты, имеют базовое доменное имя, действующее как стартовая URL-страница.

UriMatcher

Провайдер имеет специальный объект класса **UriMatcher**, который получает данные снаружи и на основе полученной информации создает нужный запрос к базе данных.

Вам нужно задать специальные константы, по которым провайдер будет понимать дальнейшие действия. Если используется одна таблица, то обычно применяют две константы - любые два целых числа, например, 100 для таблицы и 101 для отдельного ряда таблицы. Схематично можно изобразить так.

URI pattern	Code	Constant name
content://ru.alexanderklimov.provider.notepad/notes	100	NOTES
content://ru.alexanderklimov.provider.notepad/notes/#	101	NOTES_ID

Broadcast Receiver - это механизм для отсылки и получения сообщений в Android. Другими словами - это почта, через которую мы можем отправить письмо, а также можем попросить эту почту, доставлять нам письма определенного содержания (как буд-то купили подписку на журнал).

Например: мы можем попросить **BroadcastReceiver** уведомлять нас обо всех изменениях с сетью интернет. И как только у нас пропадет интернет или включится, мы получим соответствующее уведомление.

Также, мы можем самостоятельно отсылать сообщения. Например, отправить сообщение из одной части приложения в другую, при этом пересылка сообщений потоко-безопасна, т. е. мы без проблем можем отослать сообщение в одном потоке, а словить его в другом. Более того, можно отправлять сообщения из одного приложения на телефоне в другое.

Пересылка сообщений и их получение происходит с помощью определенного идентификатора (почтового адреса).

Задача: мы хотим получать сообщения всегда, даже когда приложение не запущено. В этом варианте наша подписка на получение уведомлений будет работать всегда и мы не сможем ее отключить.

Это удобно, если необходимо получать уведомления даже в тех случаях, когда приложение не запущено (т. е. мы будем выполнять некоторый код и пользователь даже ничего не узнает хе-хе:).

Итак, во первых мы должны создать получателя сообщений. Это будет метод, который будет срабатывать, как только мы получим уведомление от

BroadcastReceiver. Для этого необходимо создать реализацию абстрактного класса *BroadcastReceiver* и переопределить метод *onReceive*.

Вот так:

```
public class MyBroadcastReceiver extends BroadcastReceiver {
    @Override
    public void onReceive(Context context, Intent intent) {
        Log.d("MyBroadcastReceiver", " ПРИШЛО УВЕДОМЛЕНИЕ!!!");
    }
}
```

Мы создали свой класс *MyBroadcastReceiver.java* и наследовались от *BroadcastReceiver* в котором находится абстрактный метод ***onReceive()***, его мы должны обязательно реализовать. В данном примере мы просто выводим в консоль LogCat сообщение.

Теперь нам необходимо как-то сообщить *BroadcastReceiver* какие сообщения мы хотим получать и куда присылать уведомление (т. е. нам надо указать, чтобы уведомления приходили в наш класс *MyBroadcastReceiver.java*, который мы создали выше).

Для этого необходимо сделать дополнительную запись в файле *AndroidManifest.xml*. Внутри тега - там, где у нас прописаны наши Активности, необходимо дописать следующие строки:

```
<receiver android:name="by.kiparo.test.MyBroadcastReceiver">
    <intent-filter>
        <action android:name="android.net.conn.CONNECTIVITY_CHANGE"/>
    </intent-filter>
</receiver>
```

by.kiparo.test.MyBroadcastReceiver - это класс который мы создали для получения уведомления. Этой строчкой мы говорим Андроиду, куда необходимо присылать уведомления. То есть, теперь Андроид знает, что необходимо отсылать уведомления в класс *MyBroadcastReceiver*, а в нем уже будет вызван метод *onReceive()*.

Дальше в теге прописывается идентификатор(ы) (почтовый адрес) того, какие уведомления мы хотим получать. В данном случае, мы хотим получать уведомления с адресом: **android.net.conn.CONNECTIVITY_CHANGE** - это уведомления, которые происходят при изменении состояния сети (например отключился или включился интернет).

На этом все. Вот полный код *AndroidManifest.xml*, что-бы видеть куда вписать *MyBroadcastReceiver*.

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="by.kiparo.test">

    <application
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:theme="@style/AppTheme">

        <activity android:name=".MainActivity" android:label="@string/app_name">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>

        <receiver android:name="by.kiparo.test.MyBroadcastReceiver">
            <intent-filter>
                <action android:name="android.net.conn.CONNECTIVITY_CHANGE"/>
            </intent-filter>
        </receiver>

    </application>

    <uses-permission android:name="android.permission.ACCESS_NETWORK_STATE"/>
</manifest>
```

ПЛЮСЫ:

мы получаем уведомление всегда, даже если приложение не запущено (отчасти это минус, так как мы подписаны на уведомление всегда, а это потребляет дополнительные ресурсы телефона).

МИНУСЫ:

мы не можем остановить получение уведомлений

из метода `onReceive()` мы не имеем доступа к интерфейсу, так как приложение может быть не запущено в момент, когда пришло уведомление, да и у нас нет никакой ссылки на Activity

КОГДА ИСПОЛЬЗОВАТЬ ДАННЫЙ МЕТОД

только тогда, когда нам необходимо получать уведомления всегда и всюду, даже если приложение не запущено.

Теория Broadcast Receivers

Широковещательные приемники — это компоненты в вашем приложении Android, которые прослушивают широковещательные сообщения (или события) из разных точек:

- Из других приложений
- Из самой системы
- Из вашего приложения

Это означает, что они вызываются, когда происходит определенное действие, которое они запрограммированы на прослушивание, например, трансляция (**broadcast**).

Трансляция (**broadcast**) — это просто сообщение, заключенное в объект Intent. Трансляция может быть неявной или явной.

- Неявная широковещательная трансляция (**implicit broadcast**) — это такая, которая не предназначена специально для вашего приложения, поэтому она не является эксклюзивной для вашего приложения. Чтобы зарегистрироваться, вам нужно использовать [IntentFilter](#) и объявить его в своем манифесте. Вы должны сделать все это, потому что операционная система Android просматривает все объявленные фильтры намерений (IntentFilter) в вашем манифесте и проверяет, есть ли совпадение. Из-за этого поведения неявные широковещательные сообщения не имеют целевого атрибута. Примером неявной трансляции может быть действие входящего SMS-сообщения.
- Явная трансляция (**explicit broadcast**) — это то, что предназначено специально для вашего приложения на заранее известном компоненте. Это происходит из-за атрибута target, который содержит имя пакета приложения или имя класса компонента.

[Intent](#) это объект, который обеспечивает связывание отдельных компонент во время выполнения (например, двух активити). Intent представляет «намерение что-то сделать». Вы можете использовать интент для широкого круга задач, но чаще всего они используются, чтобы начать другую активити.

Создание неявного интента

Неявные интенты не объявляют имя класса компонента для запуска, а вместо этого объявляют выполняемое действие. Действие определяет, что вы хотите сделать, например, *просмотреть, редактировать, послать*, или *получить* что-нибудь. Интенты часто включают в себя также данные, связанные с действием, такие как адрес, который вы хотите просмотреть, или сообщения электронной почты, которое требуется передать. В зависимости от интента, которое вы хотите создать, данные могут быть [Uri](#), каким-либо другим типом данных, или интент может не нуждаться в данных вообще.

Если ваши данные это [Uri](#), есть простой [Intent\(\)](#) конструктор, который можно использовать для определения действия и данных.

Например, вот как создать интент, инициирующее телефонный звонок с помощью [Uri](#), указывающий номер телефона:

```
1 Uri number = Uri.parse("tel:5551234");
2 Intent callIntent = new Intent(Intent.ACTION_DIAL, number);
```

Когда ваше приложение вызывает это интентс помощью `startActivity()`, приложение Телефон инициирует вызов по данному номеру телефона.

Вот несколько других интентов и их действий и `Uri` пар данных:

Открыть карту:

```
    // Map point based on address
1 Uri location = Uri.parse( "geo:0,0?q=1600+Amphitheatre+Parkway,+Mountain+View,
2 +California");
3 // Or map point based on latitude/longitude
4 // Uri location = Uri.parse("geo:37.422219,-122.08364?z=14"); // z param is zoom level
5 Intent mapIntent = new Intent(Intent.ACTION_VIEW, location);
```

Открыть веб-страницу:

```
1 Uri webpage = Uri.parse("http://www.android.com");
2 Intent webIntent = new Intent(Intent.ACTION_VIEW, webpage);
```

Другие виды неявных интентов требуют «дополнительные» данные, которые предоставляют данные различных типов, таких как строки. Вы можете добавить один или несколько кусков дополнительных данных, используя различные `putExtra()` методы.

По умолчанию, система определяет соответствующий MIME тип, необходимый интенту, на основе `Uri` данных. Если вы не добавили `Uri` в интент, вы должны, как правило, использовать `setType()` для указания тип данных, связанных с интентом. Установка MIME типа в дальнейшем определяет, какие виды activity должны получить это намерение.

Вот еще несколько интентов, которые добавляют дополнительные данные для указания нужного действия:

Отправить письмо с вложением:

```
Intent emailIntent = new Intent(Intent.ACTION_SEND);  
1 // The intent does not have a URI, so declare the "text/plain" MIME type  
2 emailIntent.setType(HTTP.PLAIN_TEXT_TYPE);  
3 emailIntent.putExtra(Intent.EXTRA_EMAIL, new String[] {"jon@example.com"}); //  
  recipients  
4 emailIntent.putExtra(Intent.EXTRA_SUBJECT, "Email subject");  
5 emailIntent.putExtra(Intent.EXTRA_TEXT, "Email message text");  
6 emailIntent.putExtra(Intent.EXTRA_STREAM,  
7 Uri.parse("content://path/to/email/attachment"));  
8 // You can also attach multiple items by passing an ArrayList of Uris
```

Создание события календаря:

```
Intent calendarIntent = new Intent(Intent.ACTION_INSERT, Events.CONTENT_URI);  
1 Calendar beginTime = Calendar.getInstance().set(2012, 0, 19, 7, 30);  
2 Calendar endTime = Calendar.getInstance().set(2012, 0, 19, 10, 30);  
3 calendarIntent.putExtra(CalendarContract.EXTRA_EVENT_BEGIN_TIME,  
  beginTime.getTimeInMillis());  
4 calendarIntent.putExtra(CalendarContract.EXTRA_EVENT_END_TIME,  
5 endTime.getTimeInMillis());  
6 calendarIntent.putExtra(Events.TITLE, "Ninja class");  
7 calendarIntent.putExtra(Events.EVENT_LOCATION, "Secret dojo");
```

(Что такое Context — см. определение выше. Нуже — отдельно про способы его получить.)

Методы получения контекста и их различие

Получить контекст внутри кода можно одним из следующих методов:

- ***getBaseContext*** (получить ссылку на базовый контекст)
- ***getApplicationContext*** (получить ссылку на объект приложения)

- **getContext** (внутри активности или сервиса получить ссылку на этот объект)
- **this** (то же, что и getContext)
- **MainActivity.this** (внутри вложенного класса или метода получить ссылку на объект MainActivity)
- **getActivity** (внутри фрагмента получить ссылку на объект родительской активности)

Все эти способы дадут нам возможность получить ссылку на объект, содержащий методы класса Context.

Как было сказано выше, контекст является базовым классом для классов Application, Activity и Service, а значит его методы входят в их состав. Именно поэтому для передачи контекста в качестве параметра можно использовать как ссылку на сам контекст (getBaseContext), так и ссылки на наследуемые классы (getApplicationContext, getContext, this, MainActivity.this, getActivity).

Но тут важно понимать, что время жизни этих ссылок будет разное. Ссылка на переданный объект будет работать, пока будет жить этот объект. Поэтому в качестве контекста важно передать такую ссылку, которая будет рабочей на всём протяжении работы вызываемого метода.

Например, если вызвать сообщение с помощью Toast, используя разные context, то:

Сообщение умрёт вместе с активностью:

```
Toast.makeText(this, «Text», Toast.LENGTH_SHORT).show();
```

Сообщение умрёт вместе с приложением:

```
Toast.makeText(getApplicationContext(), «Text », Toast.LENGTH_SHORT).show();
```

Будет видно даже после завершения приложения:

```
Toast.makeText(getBaseContext(), «Text », Toast.LENGTH_SHORT).show();
```

 **Уточнение:** на практике это распространённое заблуждение. Toast показывается системным сервисом и не привязан к жизненному циклу Activity — однажды показанный Toast отобразится положенное время независимо от того, какой Context был передан в `makeText()`, пока жив процесс приложения. Тип Context здесь влияет не на «время жизни» тоста, а на корректность вызова (например, на каком потоке/состоянии Activity это безопасно сделать) и на риск утечки памяти при хранении ссылки на Activity Context.

То есть, мы видим, что хотя контекст одинаков у разных объектов, сами эти объекты могут жить разное время.

Контекст можно представить, как набор функций для работы на уровне приложения, вошедший в состав таких крупных классов, как **Application**, разных видов **Activity**, **Service**.

Runtime permission

Речь в статье пойдет о такой известной всем разработчикам вещи как Runtime permission, а именно – о возможности введения в заблуждение конечного пользователя путем демонстрации диалогового окна выдачи разрешения со своим текстом и иконкой поверх системного. Нетрудно догадаться, что подобный подход позволил бы разработчикам запрашивать у пользователя разрешение, скажем, к файловой системе, а по факту – к выдаче доступа к геопозиционированию, камере или чему-то еще.

Это невозможно

Не один раз задавался подобный вопрос на специализированных форумах, в частности на [StackOverflow](#). Единственным правильным ответом было то, что это невозможно. И это действительно так: невозможно подменить текст в самом системном диалоге, но возможно его перекрыть своим.

Что под капотом

Runtime Permission впервые появились в Android 6.0 в ответ на потребность повышенного внимания в области выдачи dangerous-разрешений. Фактически основная идея состоит в том, чтобы взаимодействовать с пользователем при запросе разрешений через всплывающее окно. Поэтому теперь разрешения из списка *dangerous* необходимо запрашивать у пользователя как только они понадобятся приложению.

Dangerous permissions

- *android.permission_group.CALENDAR*
 - android.permission.READ_CALENDAR
 - android.permission.WRITE_CALENDAR
- *android.permission_group.CAMERA*
 - android.permission.CAMERA
- *android.permission_group.CONTACTS*
 - android.permission.READ_CONTACTS
 - android.permission.WRITE_CONTACTS
 - android.permission.GET_ACCOUNTS

- *android.permission_group.LOCATION*
 - android.permission.ACCESS_FINE_LOCATION
 - android.permission.ACCESS_COARSE_LOCATION
- *android.permission_group.MICROPHONE*
 - android.permission.RECORD_AUDIO
- *android.permission_group.PHONE*
 - android.permission.READ_PHONE_STATE
 - android.permission.CALL_PHONE
 - android.permission.READ_CALL_LOG
 - android.permission.WRITE_CALL_LOG
 - android.permission.ADD_VOICEMAIL
 - android.permission.USE_SIP
 - android.permission.PROCESS_OUTGOING_CALLS
- *android.permission_group.SENSORS*
 - android.permission.BODY_SENSORS
- *android.permission_group.SMS*
 - android.permission.SEND_SMS
 - android.permission.RECEIVE_SMS
 - android.permission.READ_SMS
 - android.permission.RECEIVE_WAP_PUSH
 - android.permission.RECEIVE_MMS
 - android.permission.READ_CELL_BROADCASTS
- *android.permission_group.STORAGE*
 - android.permission.READ_EXTERNAL_STORAGE
 - android.permission.WRITE_EXTERNAL_STORAGE

Для отображения системного диалога в Android есть [GrantPermissionsActivity](#), который запускается после запроса на выдачу разрешений и отображает нам диалог со знакомым интерфейсом.

ActivityCompat.requestPermissions(


```
MainActivity.this,  
arrayOf(Manifest.permission.READ_CONTACTS),  
PERMISSION_REQUEST_CODE  
)
```

С выходом Android 6 механизм подтверждения поменялся.

Теперь при установке приложения пользователь больше не видит списка запрашиваемых разрешений. Приложение автоматически получает все требуемые `normal` разрешения, а `dangerous` разрешения необходимо будет программно запрашивать в процессе работы приложения.

Т.е. теперь недостаточно просто указать в манифесте, что вам нужен, например, доступ к контактам. Когда вы в коде попытаетесь запросить список контактов, то получите ошибку `SecurityException: Permission Denial`. Потому что вы явно не запрашивали это разрешение, и пользователь его не подтверждал.

Перед выполнением операции, требующей разрешения, необходимо спросить у системы, есть ли у приложения разрешение на это. Т.е. подтверждал ли пользователь, что он дает приложению это разрешение. Если разрешение уже есть, то выполняем операцию. Если нет, то запрашиваем это разрешение у пользователя.

Давайте посмотрим, как это выглядит на практике.

Проверка текущего статуса разрешения выполняется методом `checkSelfPermission`

```
1 int permissionStatus = ContextCompat.checkSelfPermission(this,  
Manifest.permission.READ_CONTACTS);
```

На вход метод требует `Context` и название разрешения. Он вернет константу `PackageManager.PERMISSION_GRANTED` (если разрешение есть) или `PackageManager.PERMISSION_DENIED` (если разрешения нет).

Если разрешение есть, значит мы ранее его уже запрашивали, и пользователь подтвердил его. Можем получать список контактов, система даст нам доступ.

Если разрешения нет, то нам надо его запросить. Это выполняется методом [requestPermissions](#). Схема его работы похожа на метод

startActivityForResult. Мы вызываем метод, передаем ему данные и request code, а ответ потом получаем в определенном onActivityResult методе.

Добавим запрос разрешения к уже имеющейся проверке.

```
1  int permissionStatus = ContextCompat.checkSelfPermission(this,
    Manifest.permission.READ_CONTACTS);
2  if (permissionStatus == PackageManager.PERMISSION_GRANTED) {
3      readContacts();
4  } else {
5      ActivityCompat.requestPermissions(this, new String[]
6      {Manifest.permission.READ_CONTACTS},
7          REQUEST_CODE_PERMISSION_READ_CONTACTS);
8  }
```

Проверяем разрешение READ_CONTACTS. Если оно есть, то читаем контакты. Иначе запрашиваем разрешение READ_CONTACTS методом [requestPermissions](#). На вход метод требует Activity, список требуемых разрешений, и request code. Обратите внимание, что для разрешений используется массив. Т.е. вы можете запросить сразу несколько разрешений.

Что такое и как работает deep linking?

[Deep linking](#) – это концепция перемещения по ссылке на конкретный ресурс или страницу веб-сайта или приложения. В андроиде под deep link подразумевают URL, который открывает экран приложения, если приложение установлено на устройстве.

Реализация механизма deep linking для Android-приложения состоит из двух частей:

1. Сделать так, чтобы операционная система запускала ваше приложение, когда пользователь кликает на ссылку.
2. Реализовать навигацию внутри приложения на нужный экран.

Один из способов запуска приложения по ссылке – использование [intent-filter](#) с кастомной схемой.

В интент фильтре на скриншоте задана схема itsobes. Это значит, что когда пользователь кликнет на ссылку вида itsobes://<authority>/<path>, система

запустит SplashActivity. Если на устройстве установлены несколько приложений или активити с интендом, обрабатывающим данную схему, то система покажет пользователю диалог выбора приложения.

Ссылки с кастомной схемой работают в нативных приложениях, если поддерживаются операционной системой. Например ссылка для телеграмма: <tg://resolve?domain=JavaSobes> будет работать в приложениях под Android, iOS и Mac OS, но не откроется в браузере. Такие ссылки работают в телеграмме, несмотря на блокировку РКН, а вот ссылка на веб-клиент <https://web.telegram.org/#/im?p=@JavaSobes> заблокирована в России.

```
<activity android:name=".SplashActivity">
    <intent-filter>
        <action android:name="android.intent.action.VIEW"/>
        <category android:name="android.intent.category.DEFAULT"/>
        <category android:name="android.intent.category.BROWSABLE"/>

        <data android:scheme="itsobes"/>
    </intent-filter>
</activity>
```

Как было сказано в предыдущем посте, ссылки с кастомной схемой не работают в браузерах. Обычно этого не достаточно и требуется использовать одну и ту же ссылку, которая будет открывать приложение или Play store на Android, приложение или App store на iOS, веб-сайт на десктопе. Например как эта ссылка на Google Maps <https://goo.gl/maps/8JTWxetcw7PutAfu8>.

Разберем алгоритм обработки таких ссылок на примере <https://itsobes.ru/app/AndroidSobes/7.html> (ссылка не рабочая):

1. Когда пользователь переходит по ссылке, открывается пустая веб-страница, которая по User-Agent определяет платформу пользователя.
2. Если это браузер на Android, происходит редирект на ссылку с кастомной схемой <itsobes://AndroidSobes/7>.
3. Если браузер на iOS, то редиректитесь на [Universal link](#).
4. Если браузер на десктопе, то редиректим на сайт <https://itsobes.ru/AndroidSobes/7.html>.

Самая сложная часть – это определение платформы по User-Agent, потому что под Android есть тысячи браузеров. Существуют готовые сервисы, реализующие механизм обработки deep link, например <https://docs.adjust.com/en/deeplinking>.

В API level 23 были добавлены App Links, упрощающие обработку deep links. Мы разберем App Links в одном из следующих постов.

После того, как система обработала deep link и запустила активити, необходимо выполнить второй шаг и открыть экран, заданный в deep link. Для этого в запущенной активити получаем интенд методом `getIntent()` и

далее получаем URI методом `intent.getData()`. Полученный URI – это и будет deep link, по которому запущена активити, в нашем примере это `itsobes://AndroidSobes/7`.

Дальше нет никакой магии, нужно руками распарсить URI и реализовать логику перехода на экран, который должен быть отображен.

Что такое Deeplink

Глубинная ссылка (deeplink) — по сути тоже самое что и обычная ссылка. Разница лишь в том, как обрабатывается ее открытие.

К примеру у нас есть URL:

<https://website.domain/catalog?search=Android&page=3>.

Из параметров мы видим, что данная ссылка ведет на каталог на страницу 3.

Если у нас кроме сайта есть мобильное приложение, то мы хотим, чтобы наш пользователь открыл его а не браузер, верно? Если просто открыть программу на главном экране, то теряется пользовательский контекст, а именно то, что он был в каталоге на третьей странице.

Deeplink в мобильное приложение подразумевает то, что мы откроем приложение, перехватим параметры и покажем экран релевантный контексту пользователя.

Чтобы перехватить нужный вам домен, необходимо в `AndroidManifest.xml` установить intent-filter на `Activity`, которую нужно открыть при переходе по ссылке.

Что такое Parcelable?

Parcelable – это интерфейс, который совместно с классом `Parcel` реализует механизм сериализации в Android.

Если класс реализует интерфейс `Parcelable`, поля класса сериализуются в методе `writeToParcel()`.

Также `Parcelable` класс обязан иметь статическое ненулевое поле, названное `CREATOR` типа `Creator<T>`.

Интерфейс `Creator<T>` имеет два метода `createFromParcel(parcel: Parcel): T` и `newArray(size: Int): Array<T>`. Эти методы обратные `writeToParcel()` и используются для чтения данных из `Parcel` и создания объекта.

Объект `Parcelable` записывается в контейнер `Parcel`, который имеет метод `marshall(): Array<Byte>` для представления объекта в виде массива байтов.

Parcel предназначен для передачи данных при [межпроцессовой коммуникации](#).

При изменении структуры объекта или реализации метода `writeToParcel()` байтовое представление, которое возвращается методом `marshall()`, будет изменено. Поэтому строго не рекомендуется записывать его в персистентное хранилище.

Очень часто программисту приходится передавать данные из одной активности в другую. Когда речь идёт о простых типах, то используется метод `intent.putExtra()` и ему подобные. Данный способ годится для типов **String**, **int**, **long** или массивов. А вот объекты таким образом передать не получится.

В Java существует специальный интерфейс **Serializable**, но он оказался слишком неповоротлив для мобильных устройств и пользоваться им настоятельно не рекомендуется. Для передачи объектов следует использовать другой интерфейс **Parcelable**.

В интерфейсе **Parcelable** используются два метода `describeContents()` и `writeToParcel()`:

```
@Override
public int describeContents() {

}

@Override
public void writeToParcel(Parcel dest, int flags) {

}
```

Метод `describeContents()` описывает различного рода специальные объекты, описывающие интерфейс.

Метод `writeToParcel()` упаковывает объект для передачи.

Также необходимо реализовать статическое поле **Parcelable.Creator<T> CREATOR**, которое генерирует объект класса-передатчика.

На основе тестов сделано заключение, что на таких данных Parcelable работает быстрее, чем Serializable в среднем в 16 раз.
Техника Serializable удобна, но может использоваться только для небольшого количества данных.

4. Android Core

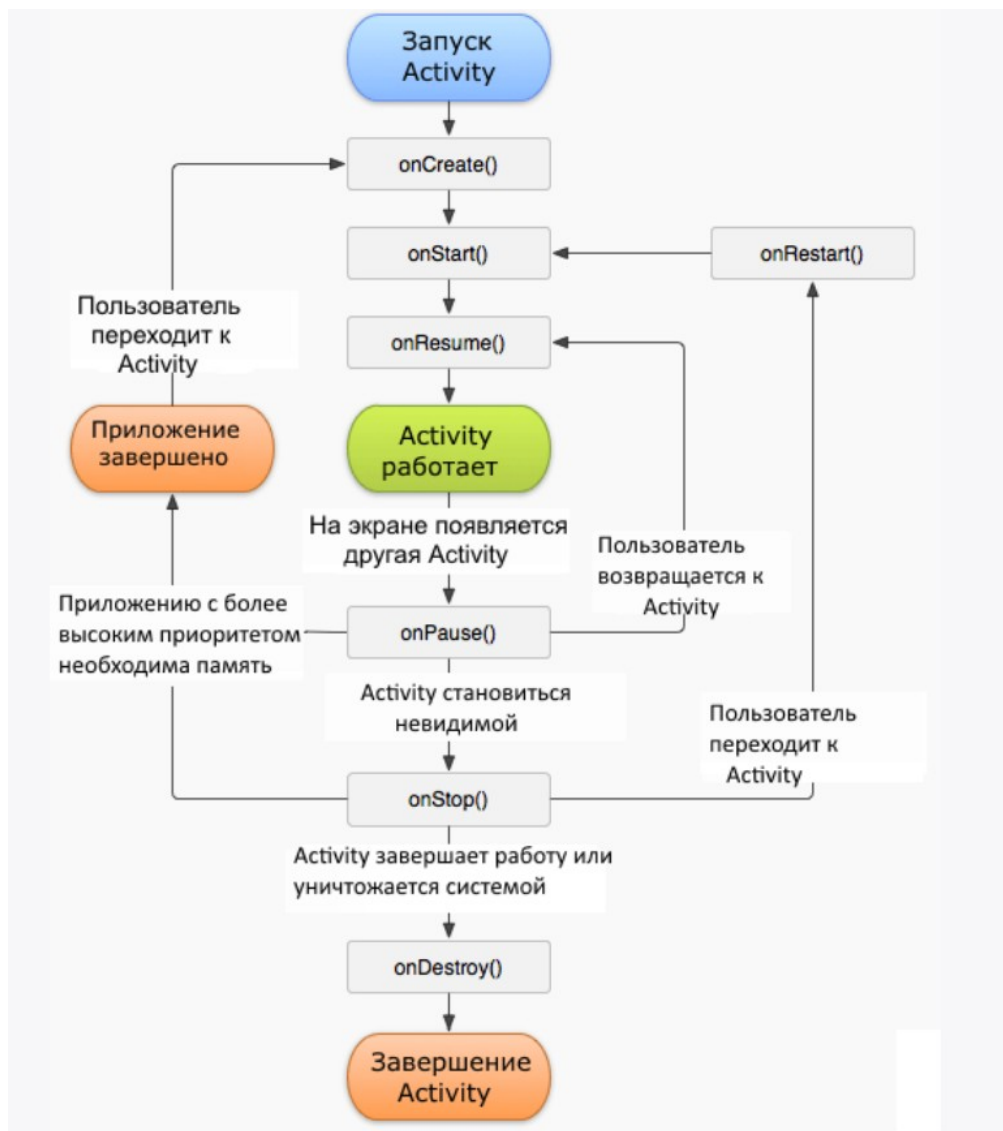
2. жизненный цикл Activity и Fragment, Intent, startActivity, Activity, startActivityForResult, IntentFilter, PendingIntent,

Fragments backStack, fragmentManager, добавление фрагмента на экран,
commit vs commitNow

Все объекты activity, которые есть в приложении, управляются системой в виде стека activity, который называется **back stack**. При запуске новой activity она помещается поверх стека и выводится на экран устройства, пока не появится новая activity. Когда текущая activity заканчивает свою работу (например, пользователь уходит из приложения), то она удаляется из стека, и возобновляет работу та activity, которая ранее была второй в стеке.

После запуска activity проходит через ряд событий, которые обрабатываются системой и для обработки которых существует ряд обратных вызовов:

```
1 protected void onCreate(Bundle savedInstanceState);
2 protected void onStart();
3 protected void onRestart();
4 protected void onResume();
5 protected void onPause();
6 protected void onStop();
7 protected void onDestroy();
```



onCreate()

onCreate - первый метод, с которого начинается выполнение activity. В этом методе activity переходит в состояние Created. Этот метод обязательно должен быть определен в классе activity. В нем производится первоначальная настройка activity. В частности, создаются объекты визуального интерфейса. Этот метод получает объект Bundle, который содержит прежнее состояние activity, если оно было сохранено. Если activity заново создается, то данный объект имеет значение null. Если же activity уже ранее была создана, но находилась в приостановленном состоянии, то bundle содержит связанную с activity информацию.

После того, как метод `onCreate()` завершил выполнение, activity переходит в состояние Started, и система вызывает метод `onStart()`

onStart

В методе **onStart()** осуществляется подготовка к выводу activity на экран устройства. Как правило, этот метод не требует переопределения, а всю работу производит встроенный код. После завершения работы метода activity отображается на экране, вызывается метод **onResume**, а activity переходит в состояние Resumed.

onResume

При вызове метода onResume activity переходит в состояние Resumed и отображается на экране устройства, и пользователь может с ней взаимодействовать. И собственно activity остается в этом состоянии, пока она не потеряет фокус, например, вследствие переключения на другую activity или просто из-за выключения экрана устройства.

onPause

Если пользователь решит перейти к другой activity, то система вызывает метод **onPause**, а activity переходит в состояние Paused. В этом методе можно освобождать используемые ресурсы, приостанавливать процессы, например, воспроизведение аудио, анимаций, останавливать работу камеры (если она используется) и т.д., чтобы они меньше сказывались на производительности системы.

Но надо учитывать, что в этом состоянии activity по-прежнему остается видимой на экране, и на работу данного метода отводится очень мало времени, поэтому не стоит здесь сохранять какие-то данные, особенно если при этом требуется обращение к сети, например, отправка данных по интернету, или обращение к базе данных - подобные действия лучше выполнять в методе onStop().

После выполнения этого метода activity становится невидимой, не отображается на экране, но она все еще активна. И если пользователь решит вернуться к этой activity, то система вызовет снова метод onResume, и activity снова появится на экране.

Другой вариант работы может возникнуть, если вдруг система видит, что для работы активных приложений необходимо больше памяти. И система может сама завершить полностью работу activity, которая невидима и находится в фоне. Либо пользователь может нажать на кнопку Back (Назад). В этом случае у activity вызывается метод **onStop**.

onStop

В этом методе activity переходит в состояние Stopped. В этом состоянии activity полностью невидима. В методе onStop следует освобождать используемые ресурсы, которые не нужны пользователю, когда он не

взаимодействует с activity. Здесь также можно сохранять данные, например, в базу данных.

При этом во время состояния Stopped activity остается в памяти устройства, сохраняется состояние всех элементов интерфейса. К примеру, если в текстовое поле EditText был введен какой-то текст, то после возобновления работы activity и перехода ее в состояние Resumed мы вновь увидим в текстовом поле ранее введенный текст.

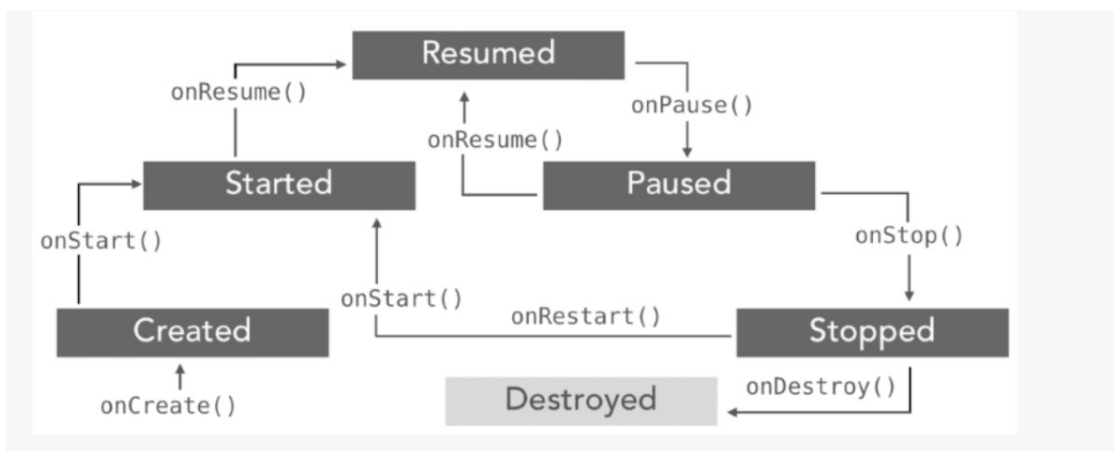
Если после вызова метода **onStop** пользователь решит вернуться к прежней activity, тогда система вызовет метод **onRestart**. Если же activity вовсе завершила свою работу, например, из-за закрытия приложения, то вызывается метод **onDestroy()**.

onDestroy

Ну и завершается работа activity вызовом метода **onDestroy**, который возникает либо, если система решит убить activity в силу конфигурационных причин (например, поворот экрана или при многоконном режиме), либо при вызове метода **finish()**.

Также следует отметить, что при изменении ориентации экрана система завершает activity и затем создает ее заново, вызывая метод **onCreate**.

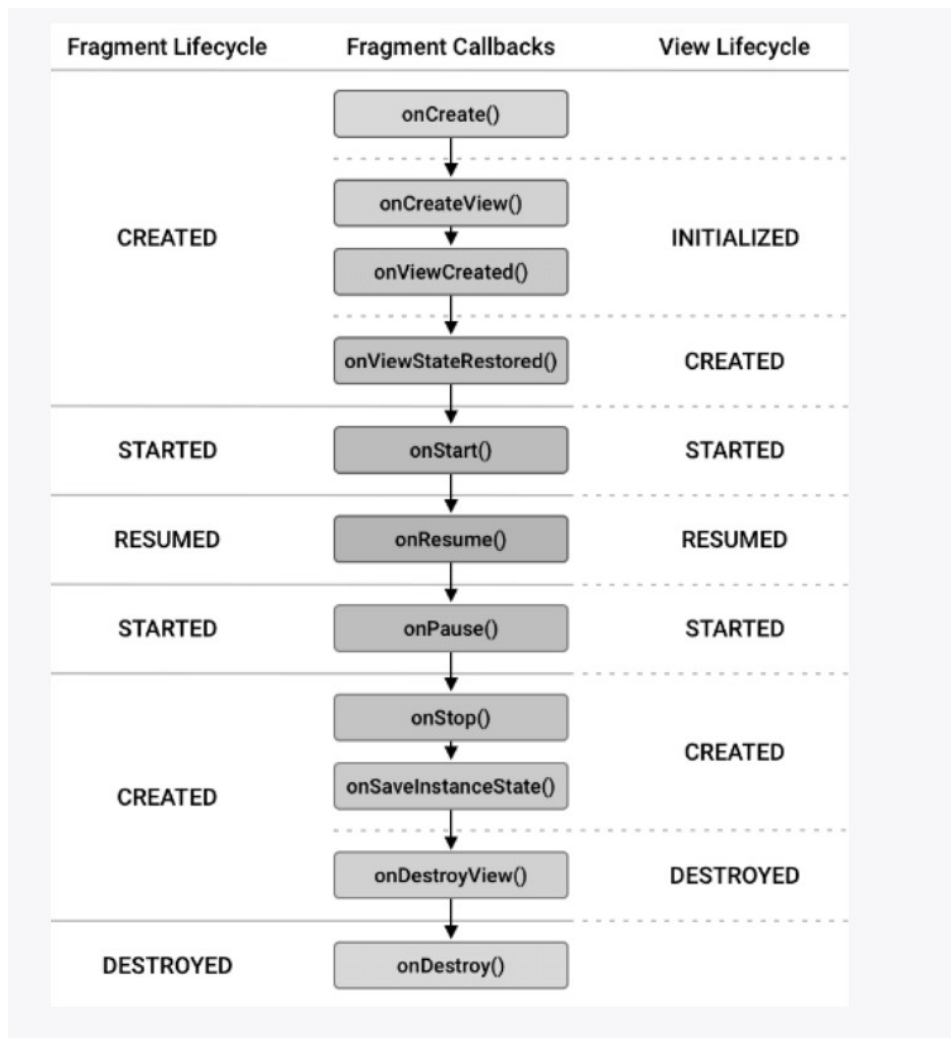
В целом переход между состояниями activity можно выразить следующей схемой:



Рассмотрим несколько ситуаций. Если мы работаем с Activity и затем переключаемся на другое приложение, либо нажимаем на кнопку Home, то у Activity вызывается следующая цепочка методов: **onPause -> onStop**. Activity оказывается в состоянии Stopped. Если пользователь решит вернуться к Activity, то вызывается следующая цепочка методов: **onRestart -> onStart -> onResume**.

Другая ситуация, если пользователь нажимает на кнопку Back (Назад), то вызывается следующая цепочка **onPause -> onStop -> onDestroy**. В результате

Activity уничтожается. Если мы вдруг захотим вернуться к Activity через диспетчер задач или заново открыв приложение, то activity будет заново пересоздаваться через методы **onCreate -> onStart -> onResume**



Каждый этап жизненного цикла описывается одной из констант перечисления **Lifecycle.State**:

- **INITIALIZED**
- **CREATED**
- **STARTED**
- **RESUMED**
- **DESTROYED**

Стоит отметить, что представление фрагмента (его визуальный интерфейс или View) имеет отдельный жизненный цикл.

- При создании фрагмент находится в состоянии **INITIALIZED**. Чтобы фрагмент прошел все остальные этапы жизненного цикла, фрагмент необходимо передать в объект **FragmentManager**, который далее определяет состояние фрагмента и переводит фрагмент из одного состояния в другое.
- Когда фрагмент добавляется в **FragmentManager** и прикрепляется к определенному классу **Activity**, у фрагмента вызывается метод **onAttach()**. Данный метод вызывается до всех остальных методов жизненного цикла. После этого фрагмент переходит в состояние **CREATED**
- **onCreate()**: происходит создание фрагмента. В этом методе мы можем получить ранее сохраненное состояние фрагмента через параметр метода **Bundle savedInstanceState**. (Если фрагмент создается первый раз, то этот объект имеет значение **null**) Этот метод вызывается после вызова соответствующего метода **onCreate()** у **activity**.

```
public void onCreate(Bundle savedInstanceState)
```

- **onCreateView()**: фрагмент создает представление (**View** или визуальный интерфейс). В этом методе мы можем установить, какой именно визуальный интерфейс будет использовать фрагмент. При выполнении этого метода представление фрагмента переходит в состояние **INITIALIZED**. А сам фрагмент все еще находится в состоянии **CREATED**

```
1 public View onCreateView(LayoutInflater inflater, ViewGroup container,  
Bundle savedInstanceState)
```

- Первый параметр - объект **LayoutInflater** позволяет получить содержимое ресурса **layout** и передать его во фрагмент.
- Вторым параметром - объект **ViewGroup** представляет контейнер, в которой будет загружаться фрагмент.
- Третьим параметром - объект **Bundle** представляет состояние фрагмента. (Если фрагмент загружается первый раз, то равен **null**)
- На выходе метод возвращает созданное с помощью **LayoutInflater** представление в виде объекта **View** - собственно представление фрагмента
- **onViewCreated()**: вызывается после создания представления фрагмента.

```
1 public void onViewCreated (View view, Bundle savedInstanceState)
```

- Первый параметр - объект **View** - представление фрагмента, которое было создано посредством метода `onCreateView`.
- Второй параметр - объект **Bundle** представляет состояние фрагмента. (Если фрагмент загружается первый раз, то равен `null`)
- **onViewStateRestored()**: получает состояние представления фрагмента. После выполнения этого метода представление фрагмента переходит в состояние **CREATED**

```
1 public void onViewStateRestored (Bundle savedInstanceState)
```

- **onStart()**: вызывается, когда фрагмент становится видимым и вместе с представлением переходит в состояние **STARTED**

```
1 public void onStart ()
```

- **onResume()**: вызывается, когда фрагмент становится активным, и пользователь может с ним взаимодействовать. При этом фрагмент и его представление переходят в состояние **RESUMED**

```
1 public void onResume ()
```

- **onPause()**: фрагмент продолжает оставаться видимым, но уже не активен и вместе с представлением переходит в состояние **STARTED**

```
1 public void onPause ()
```

- **onStop()**: фрагмент больше не является видимым и вместе с представлением переходит в состояние **CREATED**

```
1 public void onStop ()
```

- На этом этапе жизненного цикла мы можем сохранить состояние фрагмента с помощью метода **onSaveInstanceState()**. Однако стоит учитывать, что вызов этого метода зависит от версии API. До API 28 `onSaveInstanceState()` вызывается до `onStop()`, а начиная API 28 после `onStop()`.
- **onDestroyView()**: уничтожается представление фрагмента. Представление переходит в состояние **DESTROYED**

- **onDestroy()**: окончательно уничтожение фрагмента - он также переходит в состояние **DESTROYED**
- Метод **onDetach()** вызывается, когда фрагмент удаляется из `FragmentManager` и открепляется от класса `Activity`. Этот метод вызывается после всех остальных методов жизненного цикла.

INTENT

Для взаимодействия между различными объектами `activity` ключевым классом является **`android.content.Intent`**. Он представляет собой задачу, которую надо выполнить приложению.

Intent представляет собой объект описания операции, которую необходимо выполнить через систему Android. Т.е. необходимо сначала описать некоторую операцию в виде объекта *Intent*, после чего отправить её на выполнение в систему вызовом одного из методов активности *Activity*.

Объекты *Intent* в основном используются в трёх операциях :

1. Старт операции

В android-приложении один экран представлен компонентом *Activity*, описанном в файле манифеста. Если необходимо в приложении иметь несколько экранов (страниц/форм), то все они должны быть зарегистрированы в манифесте приложения. Переход от одной активности к другой выполняется с помощью объекта *Intent*, который передается методу *startActivity()*. Объект *Intent* наряду с описанием операции может дополнительно включать все необходимые данные для выполнения операции.

Если необходимо получить результат выполнения операции, то следует использовать метод *startActivityForResult()*, который вернет отдельный объект *Intent* в обратном вызове метода *onActivityResult()*.

2. Запуск сервиса

Android-приложение может выполнять действия в фоновом режиме без пользовательского интерфейса с помощью определенного сервиса, в качестве которого используется компонент *Service*. Сервис можно запускать для выполнения какого-либо действия, например, чтение файла. Для старта сервиса необходимо вызвать метод активности *startService()* и передать ему объект *Intent*.

Если сервис имеет интерфейс типа клиент-сервер, то можно с ним установить связь через вызов метода *bindService()*, с передачей ему в качестве параметра объекта *Intent*.

3. Рассылка широковещательных сообщений

Широковещательное сообщение может принимать любое приложение android. Система генерирует различные широковещательные сообщения о системных событиях, например, зарядка устройства.

Широковещательные сообщения отправляются другим приложениям передачей объекта *Intent* методам `sendBroadcast()`, `sendOrderedBroadcast()` или `sendStickyBroadcast()`.

Типы объектов *Intent*

Существует два типа объектов *Intent* : явные и неявные.

Явные объекты *Intent*

Явные объекты *Intent* , как правило, используются для запуска компонента из собственного приложения, где известно наименование запускаемых классов и сервисов. В примере рассмотрения жизненных циклов [двух активностей](#) был использован следующий вызов 2-ой активности :

```
Intent intent = new Intent(this, SecondActivity.class);  
  
startActivity(intent);
```

В качестве первого параметра конструктора *Intent* указывается **Context**. Поскольку активность является наследником *Context*, то можно использовать укороченную запись **this** или полную как **Class_name.this**. Во втором параметре конструктора указывается наименование класса активности. Этот класс зарегистрирован в манифесте приложения. Таким образом, приложение может иметь несколько активностей, каждую из которых можно вызвать по наименованию класса. После вызова метода *startActivity* будет создана новая активность, которая запустится или возобновит свою работу, переместившись на вершину стека активностей.

Неявные объекты *Intent*

Неявные объекты *Intent* не содержат имени конкретного класса. Вместо этого они включают действие (action), которое требуется выполнить. Неявный *Intent* может включать дополнительно наименование категории (category) и тип данных (data). Такой набор параметров позволяют компоненту из другого приложения обработать этот запрос. Например, если необходимо пользователю показать место на карте, то с помощью неявного объекта *Intent* можно попросить это сделать другое приложение, в котором данная функция предусмотрена.

Когда android получает неявный объект *Intent* для выполнения, то система ищет подходящие компоненты путем сравнения содержимого *Intent* с фильтрами *Intent* других приложений, зарегистрированных в файлах манифестов. Если параметры объекта *Intent* совпадают с параметрами одного из фильтров *Intent*, то система запускает этот компонент и передает ему объект *Intent*. При наличии нескольких подходящих фильтров система открывает диалоговое окно, где пользователь может выбрать подходящее приложение.

Фильтр *Intent* представляет собой секцию в файле манифеста приложения, описывающую типы объектов *Intent*, которые компонент мог бы выполнить. Таким образом, наличие фильтра *Intent* в описании активности в манифесте позволяет другим приложениям напрямую запускать данную операцию с помощью некоторого объекта *Intent*. Если фильтр *Intent* не описан, то операцию можно будет запустить только с помощью явного объекта *Intent*.

*Механизм использования неявных объектов **Intent** для вызова компонентов других приложений напоминает вызовы API. Основное отличие между вызовами компонентов и вызовами API заключается в том, что вызовы компонентов с помощью неявных Intent являются асинхронными, а API-вызовы являются синхронными. Привязка API-вызовов выполняется на этапе компиляции, тогда как вызовы неявных намерений являются привязкой ко времени выполнения. Намерение, которое работает синхронно и имеет привязку к конкретной активности, становится явным.*

В случае, если система не сможет выполнить неявное намерение при вызове метода *startActivity* или *startActivityForResult*, то приложение завершится с ошибкой. Чтобы не допустить этого следует использовать метод намерения **resolveActivity** для проверки возможности выполнения заданного действия.

Фильтры намерений Intent

В секциях фильтров намерений в файле манифеста можно объявить только три составляющих объекта *Intent* : действие, данные, категория. Рассмотрим пример описания фильтров намерений для приложения работы с социальными сетями.

```

<activity android:name="MainActivity">
    <!-- This activity is the main entry,
         should appear in app launcher -->
    <intent-filter>
        <action android:name=
            "android.intent.action.MAIN" />
        <category android:name=
            "android.intent.category.LAUNCHER" />
    </intent-filter>
</activity>

<activity android:name="ShareActivity">
    <!-- This activity handles "SEND" actions
         with text data -->
    <intent-filter>
        <action android:name="android.intent.action.SEND"/>
        <category android:name=
            "android.intent.category.DEFAULT"/>
        <data android:mimeType="text/plain"/>
    </intent-filter>

    <!-- This activity also handles "SEND" and
         "SEND_MULTIPLE" with media data -->
    <intent-filter>
        <action android:name="android.intent.action.SEND"/>
        <action android:name=
            "android.intent.action.SEND_MULTIPLE"/>
        <category android:name=
            "android.intent.category.DEFAULT"/>
        <data android:mimeType=
            "application/vnd.google.panorama360+jpg"/>
        <data android:mimeType="image/*"/>
        <data android:mimeType="video/*"/>
    </intent-filter>
</activity>

```

Категория намерения, category

Категория – это строка, содержащая дополнительные сведения о том, каким компонентом должна выполняться обработка объекта *Intent*. В объект *Intent* можно поместить любое количество категорий. Однако большинству объектов *Intent* описание категории не требуется.

Класс **Intent** для работы с категориями имеет группу методов :

- `addCategory()` — добавить категорию в объект *Intent*;
- `removeCategory()` — удалить ранее добавленную категорию из объекта *Intent*;
- `getCategories()` — получить набор категорий объекта *Intent*.

При описании категории в файле манифеста используются стандартные значения, предоставляемые системой :

BROWSABLE	Операция запускает веб-браузер для отображения данных, указанных по ссылке (url), например, изображение или сообщение электронной почты.
LAUNCHER	Операция (активность) является начальной операцией задачи. Активность с данной категорией помещается в окно для запуска приложений.
DEFAULT	Данная категория позволяет объявить компонент обработчиком по умолчанию для действия, выполняемого с указанным типом данных внутри фильтра намерений.
HOME	Активность с данной категорией отображает главный экран (Home Screen), который открывается после включения устройства и загрузки системы, или когда нажимается клавиша HOME.

startActivity начнет новую деятельность и не будет заботиться о том, где и как завершится эта деятельность.

очевидно,

startActivityForResult ждет ответных запросов, когда начальная активность решила закончить

startActivity() запустит действие, которое вы хотите запустить, не беспокоясь о получении какого-либо результата от новой дочерней активности, запущенной с помощью **startActivity** для родительской активности.

startActivityForResult() запускает другое действие из вашей активности и рассчитывает получить некоторые данные из недавно начатой дочерней активности **startActivityForResult()** и вернуть ее родительской активности.

Бывает необходимость **вызвать Activity, выполнить** на нем какое-либо **действие** и **вернуться с результатом**. Например – при создании

SMS. Вы жмете кнопку «добавить адресата», система показывает экран со списком из адресной книги, вы выбираете нужного вам абонента и возвращаетесь в экран создания SMS. Т.е. вы **вызвали экран выбора** абонента, а он **вернул** вашему экрану **результат**.

Фильтры намерений и запуск заданий

Если намерение запрашивает выполнение какого-либо действия с определённым набором данных, то системе нужно уметь выбрать приложение (или компонент) для обслуживания этого запроса. На помощь приходят фильтры намерений, которые используются для регистрации активностей, сервисов и широковещательных приёмников в качестве компонентов, способных выполнять заданные действия с конкретным видом данных. С помощью этих фильтров также регистрируются широковещательные приёмники, настроенные на трансляцию намерением заданного действия или события.

Задействуем фильтры намерений, приложения объявляют, что они могут отвечать на действия, запрашиваемые любой другой программой, установленной на устройстве. Чтобы зарегистрировать компонент приложения в качестве потенциального обработчика намерений, нужно добавить тег `<intent-filter>` в узел компонента в манифесте.

В фильтре намерений декларируется только три составляющих объекта **Intent**: действие, данные, категория. Дополнения и флаги не играют никакой роли в принятии решения, какой компонент получает намерение.

Можете заметить что в объявлении Activity на `AndroidManifest.xml` может быть тег `<intent-filter>`, этот тег определяет вид намерения Intent отправленный ему (Activity), который можно принять.

Когда вы создаете неявное намерение (Implicit Intent), система Android находит подходящие компоненты чтобы начать сравнивать содержания намерений (Intent) с фильтром намерений объявленные в файле `manifest` других приложений на устройстве. Если Intent подходит к фильтру Intent, система вызывает этот компонент и передает в объект Intent.

Что такое PendingIntent?

PendingIntent используется для описания интента, выполнение которого отложено во времени.

Инстанс класса PendingIntent создается одним из статических методов: [PendingIntent.getActivity\(\)](#), [PendingIntent.getActivities\(\)](#), [PendingIntent.getBroadcast\(\)](#), [PendingIntent.getService\(\)](#).

При создании в PendingIntent записывается информация о желаемом

интенте и о том, какой [компонент](#) будет запущен. Объекты PendingIntent переживают остановку процесса, поэтому система может использовать PendingIntent для старта приложения.

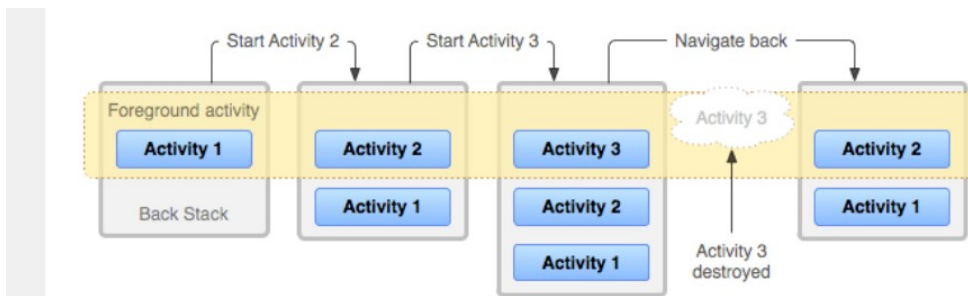
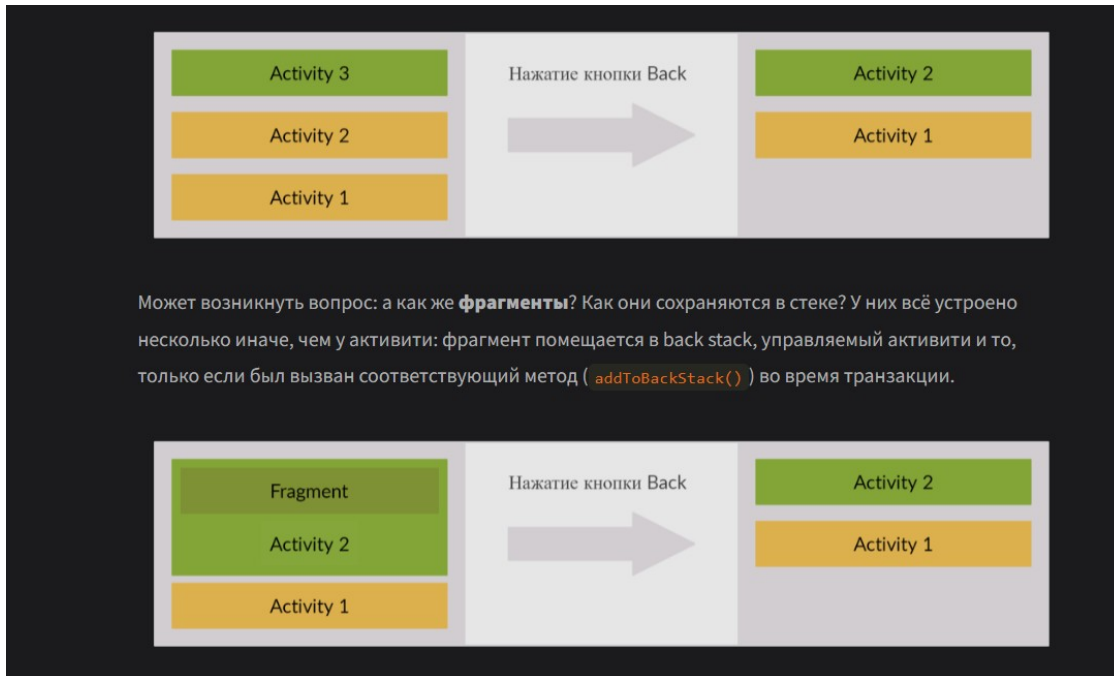
Один из частых примеров использования PendingIntent – это создание нотификации. Метод [NotificationCompat.Builder.setContentIntent\(\)](#) принимает PendingIntent, который выполняется, когда пользователь кликает на нотификацию.

Tasks и Back Stack

24 июля 2020 г.

Время чтения 6 мин.

Task - это набор активити, с которыми пользователь взаимодействует при использовании какого-либо приложения. У каждого task'a есть свой **back stack** - это что-то вроде способа организации открытых пользователем активити, который устроен по принципу [LIFO](#) - “последним вошел - первым вышел”. То есть при открытии новой активити, она становится вершиной стека, а предыдущая уходит в состояние “остановлена”. При нажатии пользователем на кнопку **Back**, новая активити уничтожается и удаляется из стека, а предыдущая восстанавливается (возвращается в состояние “возобновлена”). Если продолжать нажимать на кнопку **Back**, то в итоге пользователь вернётся на главный экран устройства, а task перестанет существовать. Если же пользователь свернёт приложение, то task продолжит существовать в фоне и хранить весь свой стек, при этом все активити перейдут в состояние “остановлена”. Поэтому пользователь в любой момент сможет вновь открыть приложение и продолжить работу с того, на чём остановился. Однако такой task может быть удален системой при нехватке ресурсов.



- При запуске новая Activity становится на вершину стека, смещая текущую
- При сворачивании приложения таск уходит в background и хранит свое состояния. После очередного запуска приложения таск восстанавливается вместе со своим стеком
- При нажатии back текущая Activity безвозвратно удаляется. На вершину стека ставится предыдущая
- Одна и та же Activity может иметь сколько угодно экземпляров в стеке

FragmentManager

Для взаимодействия между фрагментами используется класс **android.app.FragmentManager** - специальный менеджер по фрагментам.

Как в любом офисе, спецменеджер не делает работу своими руками, а использует помощников. Например, для транзакций (добавление,

удаление, замена) используется класс-помощник **android.app.FragmentTransaction**.

FragmentManager

Класс **FragmentManager** имеет два метода, позволяющих найти фрагмент, который связан с активностью:

findFragmentById(int id)

Находит фрагмент по идентификатору

findFragmentByTag(String tag)

Находит фрагмент по заданному тегу

Методы транзакции

Мы уже использовали некоторые методы класса **FragmentTransaction**. Познакомимся с ними поближе

add()

Добавляет фрагмент к активности

remove()

Удаляет фрагмент из активности

replace()

Заменяет один фрагмент на другой

hide()

Прячет фрагмент (делает невидимым на экране)

show()

Выводит скрытый фрагмент на экран

detach() (API 13)

Отсоединяет фрагмент от графического интерфейса, но экземпляр класса сохраняется

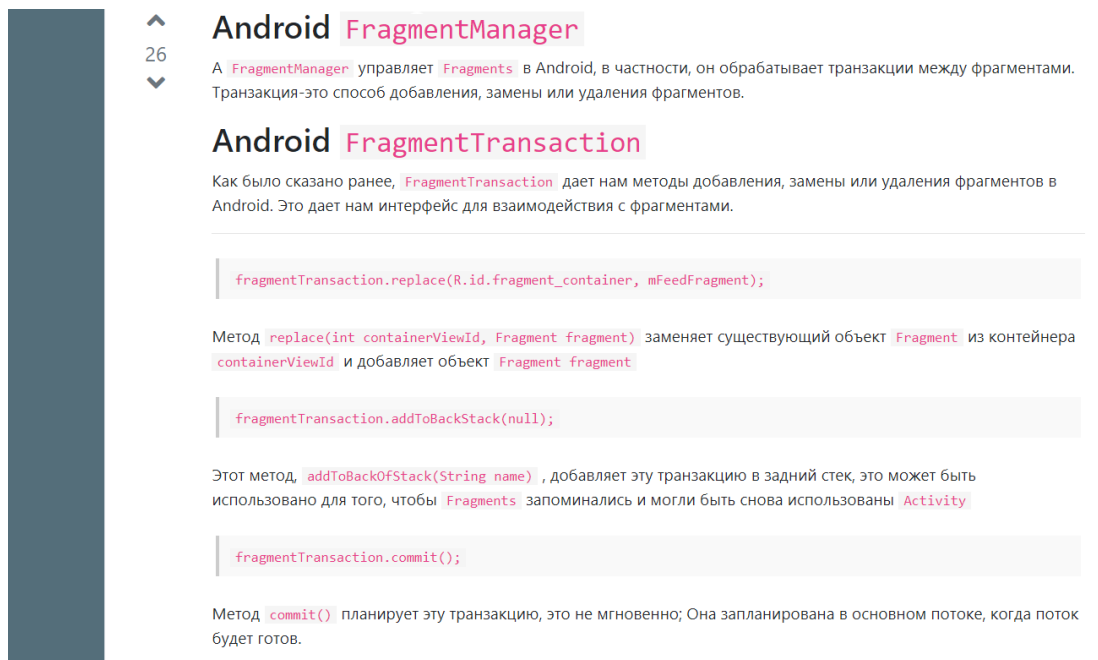
attach() (API 13)

Присоединяет фрагмент, который был отсоединён методом **detach()**

Методы **remove()**, **replace()**, **detach()**, **attach()** не применимы к статичным фрагментам.

FragmentManager, который используется для создания транзакций для добавления, удаления или замены фрагментов.

fragmentManager.beginTransaction()



The screenshot shows the Android Studio interface with the documentation for `FragmentManager` and `FragmentTransaction`. On the left, there is a dark sidebar with a vertical bar and the number 26. The main content area has a light blue header with the title "Android FragmentManager". Below the header, there is a paragraph explaining that `FragmentManager` manages `Fragments` in Android and processes transactions between them. A transaction is defined as a way to add, replace, or remove fragments. Below this, there is another header "Android FragmentTransaction". A paragraph explains that `FragmentTransaction` provides methods for adding, replacing, or removing fragments in Android, giving an interface for interacting with fragments. There are three code blocks: 1. `fragmentTransaction.replace(R.id.fragment_container, mFeedFragment);` 2. A paragraph explaining the `replace(int containerViewId, Fragment fragment)` method, which replaces an existing `Fragment` in the container `containerViewId` and adds the `Fragment fragment`. 3. `fragmentTransaction.addToBackStack(null);` 4. A paragraph explaining the `addToBackStack(String name)` method, which adds the transaction to the back stack, useful for remembering `Fragments` and reusing them in an `Activity`. 5. `fragmentTransaction.commit();` 6. A paragraph explaining the `commit()` method, which schedules the transaction and is not immediate; it is executed in the main thread when the thread is ready.

Метод `FragmentManager.commit()` – синхронный или нет?

– Асинхронный.

Это значит, что транзакция не выполняется во время вызова метода. `commit()` добавляет транзакцию в очередь главного потока и транзакция выполняется при первой возможности.

`commitNow()` – синхронный!

Чтобы выполнить транзакцию синхронно, можно воспользоваться методом `commitNow()` вместо `commit()` или вызвать `executePendingTransactions()` после метода `commit()`.

4. Android Core

3. State переверот экрана и смена конфигурации, Bundle, handling SharedPreferences, способы сохранения данных, жизненный цикл при перевероте, обработка смены конфигурации

Для избежания проблем отображения планшетов нужен специальный лейаут в специальной папке:

Воспользуемся контейнером **TableLayout (устаревшее)**. Сейчас любой можно. С его помощью мы можем разбить кнопки на две колонки и поместить их в три ряда.

Для этой операции нам понадобится сделать несколько важных шагов. Сначала нужно создать новую подпапку в папке **res**. Выделяем папку **res**, вызываем из него контекстное меню и последовательно выбираем команды **New | Android resource directory**. В диалоговом окне из выпадающего списка **Resource type**: выбираем **layout**. В списке **Available qualifiers**: находим элемент **Orientation** и переносим его в правую часть **Chosen qualifiers**: с помощью кнопки с двумя стрелками. По умолчанию у вас появится имя папки **layout-port** в первой строке **Directory Name**:. Но нам нужен альбомный вариант, поэтому в выпадающем списке **Screen orientation** выбираем **Landscape**. Теперь название папки будет **layout-land**.

Узнать ориентацию программно

Чтобы из кода узнать текущую ориентацию, можно создать следующую функцию:

```
// Kotlin
private fun getScreenOrientation(): String {
    return when (resources.configuration.orientation) {
        Configuration.ORIENTATION_PORTRAIT -> "Портретная ориентация"
        Configuration.ORIENTATION_LANDSCAPE -> "Альбомная ориентация"
        else -> ""
    }
}
```

Кручу-верчу, запутать хочу!

Хорошо, мы можем определить текущую ориентацию, но в какую сторону повернули устройство? Ведь его можно повернуть влево, вправо или вообще вверх тормашками. Напишем другую функцию:

```
// Kotlin
private fun getRotateOrientation(): String {
    return when (windowManager.defaultDisplay.rotation) {
        Surface.ROTATION_0 -> "Не поворачивали"
        Surface.ROTATION_90 -> "Повернули на 90 градусов по часовой стрелке"
        Surface.ROTATION_180 -> "Повернули на 180 градусов"
        Surface.ROTATION_270 -> "Повернули на 90 градусов против часовой стрелки"
        else -> "Не понятно"
    }
}
```

Установить ориентацию программно и через манифест

Если вы большой оригинал и хотите запустить приложение в стиле "вид сбоку", то можете сделать это программно. Разместите код в методе **onCreate()**:

```
// Kotlin
import android.content.pm.ActivityInfo

requestedOrientation = ActivityInfo.SCREEN_ORIENTATION_LANDSCAPE
```

Вы можете запретить приложению менять ориентацию, если добавите нужный код в **onCreate()**.

```
setRequestedOrientation (ActivityInfo.SCREEN_ORIENTATION_LANDSCAPE); //для альбомного режима
// или
setRequestedOrientation (ActivityInfo.SCREEN_ORIENTATION_PORTRAIT); //для портретного режима
```

Но указанный способ не совсем желателен. Лучше установить нужную ориентацию через манифест, прописав в элементе <activity> параметр **android:screenOrientation**:

```
android:screenOrientation="portrait"
android:screenOrientation="landscape"
```

Кстати, существует ещё один вариант, когда устройство полагается на показания сенсора и некоторые другие:

```
android:screenOrientation="sensor"
```

В Android 4.3 (API 18) появились новые значения (оставлю пока без перевода):

- **userLandscape** - Behaves the same as "sensorLandscape", except if the user disables auto-rotate then it locks in the normal landscape orientation and will not flip. [userLandscape - ведет себя так же, как 'sensorLandscape', за исключением того, что если пользователь отключает автоповорот, он фиксируется в нормальной альбомной ориентации и не переворачивается.]
- **userPortrait** - Behaves the same as "sensorPortrait", except if the user disables auto-rotate then it locks in the normal portrait orientation and will not flip. [userPortrait - ведет себя так же, как «sensorPortrait», за исключением того, что если пользователь отключает автоповорот, он фиксируется в нормальной портретной ориентации и не переворачивается.]
- **fullUser** - Behaves the same as "fullSensor" and allows rotation in all four directions, except if the user disables auto-rotate then it locks in the user's preferred orientation. [fullUser - ведет себя так же, как «fullSensor», и разрешает вращение во всех четырех направлениях, за исключением случаев, когда пользователь отключает автоповорот, тогда он фиксируется в предпочтительной ориентации пользователя.]
- **locked** - to lock your app's orientation into the screen's current orientation. [заблокировано - чтобы заблокировать ориентацию вашего приложения в соответствии с текущей ориентацией экрана.]

После появления Android 5.0 зашёл на страницу документации и пришёл в ужас. Там появились новые значения.

```
android:screenOrientation=["unspecified" | "behind" |
    "landscape" | "portrait" |
    "reverseLandscape" | "reversePortrait" |
    "sensorLandscape" | "sensorPortrait" |
    "userLandscape" | "userPortrait" |
    "sensor" | "fullSensor" | "nosensor" |
    "user" | "fullUser" | "locked"]
```

Запоминаем значения переменных

Что же делать? Для этих целей у активности существует специальный метод **onSaveInstanceState()**, который вызывается системой перед методами **onPause()**, **onStop()** и **onDestroy()**. Метод позволяет сохранить значения простых типов в объекте **Bundle**. Класс **Bundle** - это простой способ хранения данных ключ/значение.

Жизненный цикл при повороте

При повороте активность проходит через цепочку различных состояний. Порядок следующий.

1. **onPause()**

2. onStop()
3. onDestroy()
4. onCreate()
5. onStart()
6. onResume()

Когда работа Activity приостанавливается (**onPause** или **onStop**), она остается в памяти и хранит все свои объекты и их значения. И при возврате в Activity, все остается, как было.

Но если приостановленное Activity уничтожается, например, при нехватке памяти, то соответственно удаляются и все его объекты. И если к нему снова вернуться, то системе надо заново его создавать и восстанавливать данные, которые были утеряны при уничтожении. Для этих целей Activity предоставляет нам для реализации пару методов: первый позволяет сохранить данные – [onSaveInstanceState](#), а второй – восстановить – [onRestoreInstanceState](#).

Эти методы используются в случаях, когда Activity уничтожается, но есть вероятность, что оно еще будет востребовано в своем текущем состоянии. Т.е. при нехватке памяти или при повороте экрана. Если же вы просто нажали кнопку Back (назад) и тем самым явно сами закрыли Activity, то эти методы не будут выполнены.

Но даже если не реализовать эти методы, у них есть реализация по умолчанию, которая сохранит и восстановит данные в экранных компонентах. Это выполняется для всех экранных компонентов, у которых есть ID.

В чем разница между Bundle и Intent?

Bundle – это класс, реализующий [ассоциативный массив](#), т.е. хранящий пары ключ-значение. Имеет `get()` и `put()` методы для [примитивов](#), строк и объектов, которые реализуют интерфейсы [Parcelable](#) и [Serializable](#). Bundle используется для передачи данных между [базовыми компонентами](#).

Также рекомендуется использовать Bundle для передачи данных между процессами, потому что Bundle оптимизирован под [маршалинг/демаршалинг](#).

Intent описывает операцию к исполнению. Интенды используются при старте базовых компонент, например `startActivity(intent: Intent)` и `startService(intent: Intent)`.

Intent так же как и Bundle имеет `get()` и `put()` методы и используется для

передачи данных. Но Intent не реализует ассоциативный массив, а лишь предоставляет интерфейс. Intent имеет внутри объект Bundle, куда [делегируются](#) переданные пары и уже Bundle используется для хранения и передачи данных.

SharedPreferences. Сохранение данных в постоянное хранилище Android

SharedPreferences — постоянное хранилище на платформе Android, используемое приложениями для хранения своих настроек, например. Это хранилище является относительно постоянным, пользователь может зайти в настройки приложения и очистить данные приложения, тем самым очистив все данные в хранилище.

Для работы с данными постоянного хранилища нам понадобится экземпляр класса **SharedPreferences**, который можно получить у любого объекта, унаследованного от класса *android.content.Context* (например, **Activity** или **Service**). У объектов этих классов (унаследованных от **Context**) есть метод **getSharedPreferences**, который принимает 2 параметра:

- **name** — выбранный файл настроек. Если файл настроек с таким именем не существует, он будет создан при вызове метода `edit()` и фиксации изменений с помощью метода `commit()`.
- **mode** — режим работы. Возможные значения:
 - **MODE_PRIVATE** — используется в большинстве случаев для приватного доступа к данным приложением-владельцем
 - **MODE_WORLD_READABLE** — только для чтения
 - **MODE_WORLD_WRITEABLE** — только записи
 - **MODE_MULTI_PROCESS** — несколько процессов совместно используют один файл **SharedPreferences**.

ВНИМАНИЕ! Все модификаторы кроме **MODE_PRIVATE** в настоящий момент объявлены **deprecated** и не рекомендуются к использованию в целях безопасности. Если необходимо реализовать использование общих данных несколькими приложениями, это можно сделать через сервисы или контент-провайдеры. Подробнее можно почитать, например, [здесь](#).

Чтобы получить значение необходимой переменной, используйте следующие методы объекта **SharedPreferences**:

- **getBoolean**(String key, boolean defValue),
- **getFloat**(String key, float defValue),

- **getInt**(String key, int defValue),
- **getLong**(String key, long defValue),
- **getString**(String key, String defValue),
- **getStringSet**(String key, Set defValues).

Второй параметр — значение, которое вернется в случае если значение по ключу key отсутствует в **SharedPreferences**. Также, методом **getAll()** можно получить все доступные значения.

Чтобы записать значение переменной необходимо:

1. получить объект **SharedPreferences.Editor** выполнив метод **edit()** объекта класса **SharedPreferences**
2. записать значение с помощью методов:
 - **putBoolean**(String key, boolean value),
 - **putFloat**(String key, float value),
 - **putInt**(String key, int value),
 - **putLong**(String key, long value),
 - **putString**(String key, String value),
 - **putStringSet**(String key, Set values)
3. выполнить метод **commit()**

Также есть возможность удалить конкретное значение (**remove(String key)**) или все значения (**clear()**)

Приведенный ниже код демонстрирует запись переменной типа String в хранилище:

```
1 SharedPreferences settings = context.getSharedPreferences(PERSISTANT_STORAGE_NAME, Context.MODE_PRIVATE);
2 SharedPreferences.Editor editor = settings.edit();
3 editor.putString( "name", "John" );
4 editor.commit();
```

— context — объект, унаследованный от android.content.Context.

Перед записью значений в хранилище или получением значений из хранилища, класс нужно проинициализировать, вызвав метод **init()** и передав ему объект, унаследованный от **android.content.Context** (например, **Activity** или **Service**).

```

1 import android.content.Context;
2 import android.content.SharedPreferences;
3
4 public class PersistantStorage {
5     public static final String STORAGE_NAME = "StorageName";
6
7     private static SharedPreferences settings = null;
8     private static SharedPreferences.Editor editor = null;
9     private static Context context = null;
10
11     public static void init( Context cntxt ){
12         context = cntxt;
13     }
14
15     private static void init(){
16         settings = context.getSharedPreferences(STORAGE_NAME, Context.MODE_PRIVATE);
17         editor = settings.edit();
18     }
19
20     public static void addProperty( String name, String value ){
21         if( settings == null ){
22             init();
23         }
24         editor.putString( name, value );
25         editor.commit();
26     }
27
28     public static String getProperty( String name ){
29         if( settings == null ){
30             init();
31         }
32         return settings.getString( name, null );
33     }
34 }

```

В этом классе реализована работа только со строковыми данными. Вы легко можете самостоятельно его расширить, используя стандартные методы PersistantStorage или написав собственные сериализаторы для сложных объектов.

ВНИМАНИЕ! Метод `commit()` устарел и объявлен **deprecated**. Вместо него следует использовать метод `apply()` — он появился в API 9 и работает в асинхронном режиме, что является более предпочтительным вариантом.

СПОСОБЫ ХРАНЕНИЯ ДАННЫХ

- **Shared Preferences:** хорош для хранения примитивов (booleans, floats, ints, longs и strings) в виде ключ-значение
- **SQLite Databases:** хранилище структурированных данных в базе данных SQLite (довольно шустрая БД). Любая БД, созданная в вашем аппликейшене, будет доступна по имени из любого класса этого приложения и только внутри этого приложения
- **Internal Storage:** хранение приватных файлов вашей аппликухи в памяти девайса. Другие приложения не будут иметь доступа к этим файлам
- **External Storage:** внешние носители (всякого рода флэшки, доп харддрайвы и т.д.). Файлы сохранённые здесь не защищены от удаления пользователем и использования другими приложениями
- **Network Connection:** хранение данных в сети. Можно использовать свои вебсервисы для хранения и получения каких-либо данных из сети

Shared Preferences, SQLite Databases, API - способы получения доступа к данным, и их представление.

Internal/External Storage и сеть - места хранения данных. Файлы SharedPreferences и локальные базы данных создаются в Internal storage.

- 1) Shared Preferences для примитивов ключ-значение.
- 2) Internal Storage для сохранения в памяти телефона.
- 3) External Storage для сохранения на внешней карте.
- 4) SQLite Databases
- 5) Network Connection, чтобы сохранить на веб-серевере.

4. Android Core

4. Background work Service vs IntentService vs AsyncTask

Службы (Сервисы) в Android работают как фоновые процессы и представлены классом **android.app.Service**. Они не имеют пользовательского интерфейса и нужны в тех случаях, когда не требуется вмешательства пользователя. Сервисы работают в фоновом режиме, выполняя сетевые запросы к веб-серверу, обрабатывая информацию, запуская уведомления и т.д. Служба может быть запущена и будет продолжать работать до тех пор, пока кто-нибудь не остановит её или пока она не остановит себя сама. Сервисы предназначены для длительного существования, в отличие от активностей. Они могут работать, постоянно перезапускаясь, выполняя постоянные задачи или выполняя задачи, требующие много времени.

Клиентские приложения устанавливают подключение к службам и используют это подключение для взаимодействия со службой. С одной и той же службой могут связываться множество клиентских приложений.

Android даёт службам более высокий приоритет, чем бездействующим активностям, поэтому вероятность того, что они будут завершены из-за нехватки ресурсов, заметно уменьшается. По сути, если система должна преждевременно завершить работу запущенного сервиса, он может быть настроен таким образом, чтобы запускаться повторно, как только станет доступно достаточное количество ресурсов. В крайних случаях прекращение работы сервиса (например, задержка при проигрывании музыки) будет заметно влиять на впечатления пользователя от приложения, и в подобных ситуациях приоритет сервиса может быть повышен до уровня активности, работающей на переднем плане.

Используя сервис, можете быть уверены, что ваши приложения продолжают работать и реагировать на события, даже если они в неактивном состоянии. Для работы службам не нужен отдельный графический интерфейс, как в случае с активностями, но они по-прежнему выполняются в главном потоке хода приложения. Чтобы повысить отзывчивость вашего приложения, нужно уметь переносить

трудоёмкие процессы (например, сетевые запросы) в фоновые потоки, используя классы **Thread** и **AsyncTask**.

Службы идеально подходят для проведения постоянных или регулярных операций, а также для обработки событий даже тогда, когда активности вашего приложения невидимы, работают в пассивном режиме или закрыты.

Сервисы запускаются, останавливаются и контролируются из различных компонентов приложения, включая другие сервисы, активности и приёмники широковещательных намерений. Если ваше приложение выполняет задачи, которые не зависят от прямого взаимодействия с пользователем, сервисы могут стать хорошим выбором.

Запущенные сервисы всегда имеют больший приоритет, чем бездействующие или невидимые активности, поэтому менее вероятно, что их работа завершится преждевременно при распределении ресурсов. Единственная причина, почему Android может досрочно остановить Сервис, — выделение дополнительных ресурсов для компонентов, работающих на переднем плане (как правило, для активностей). Если такое случится, ваш сервис автоматически перезапустится, когда будет достаточно доступных ресурсов.

Когда сервис напрямую взаимодействует с пользователем (например, проигрывая музыку), может понадобиться повысить его приоритет до уровня активностей, работающих на переднем плане. Это гарантия того, что сервис завершится только в крайнем случае, но при этом снижается его доступность во время выполнения, мешая управлять ресурсами, что может испортить общее впечатление от приложения.

Приложения, которые регулярно обновляются, но очень редко или нерегулярно взаимодействуют с пользователем, можно назвать первыми кандидатами на реализацию в виде сервисов. Проигрыватели MP3 и приложения, отслеживающие спортивные результаты, — примеры программ, которые должны постоянно работать и обновляться без необходимости отображать активность.

Что такое Сервис?

Сервис (Service) это фоновые процессы в системе выполняющие длительные операции и не нуждаются в взаимодействии с пользователем и работает даже при уничтожении приложения. Сервис имеет два состояния.

Состояние	Описание
Started (Запущенный)	Сервис называется started (запущенный) когда компонент приложения, например Activity запускает его вызовом startService() . При вызове, этот сервис может работать в фоновом режиме в течении неограниченного времени, даже при уничтожении запущенного компонента. Этот сервис так же называет непривязанным (Un Bounded Service).
Bound (Привязанный)	Сервис является привязанной (bound) когда компонент приложения привязывается к ней вызовом bindService() . Привязанный сервис предлагает интерфейс клиент-сервер client-server , который позволяет компонентам взаимодействовать со службой, отправлять запросы, получать результаты и даже делать это между разными процессами посредством межпроцессного взаимодействия Interprocess communication (IPC).

В каком потоке работает сервис? В главном или фоновом?

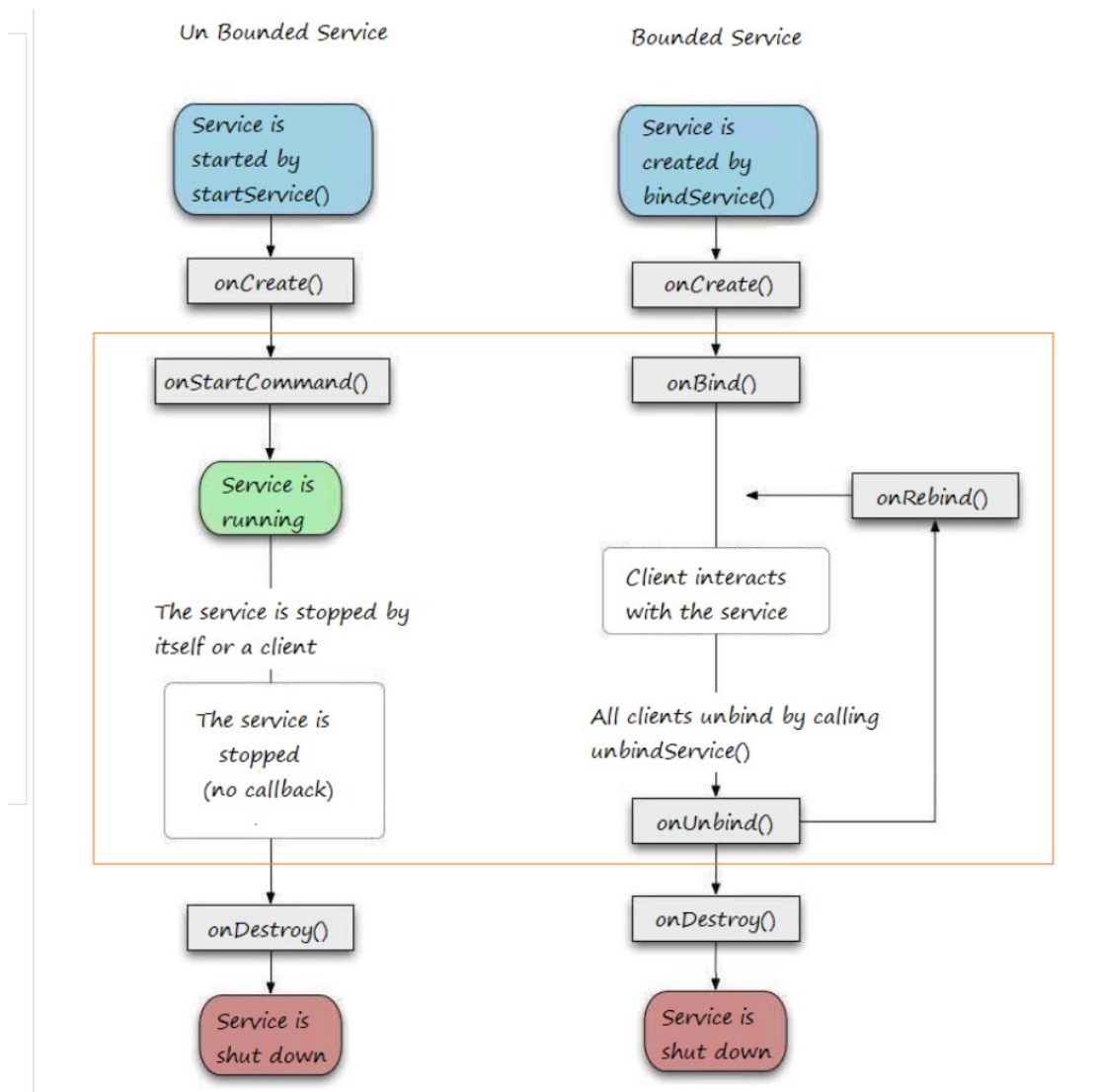
Service по умолчанию работает в главном потоке, в андроиде это UI thread. На этот вопрос часто отвечают неправильно, потому что путают понятия фоновой задачи (**background task**) и фонового потока (**background thread**).

Андроид приложение может работать в фоновом режиме. Это значит, что пользователь не видит UI компоненты приложения, а активити на верхушке бэкстека находится в **stopped состоянии**.

Для выполнения асинхронной операции в сервисе можно использовать **multithreading api** андроида.

Для выполнения всего кода сервиса асинхронно используется специальный вид сервиса, **JobIntentService**, который по умолчанию работает в фоновом потоке.

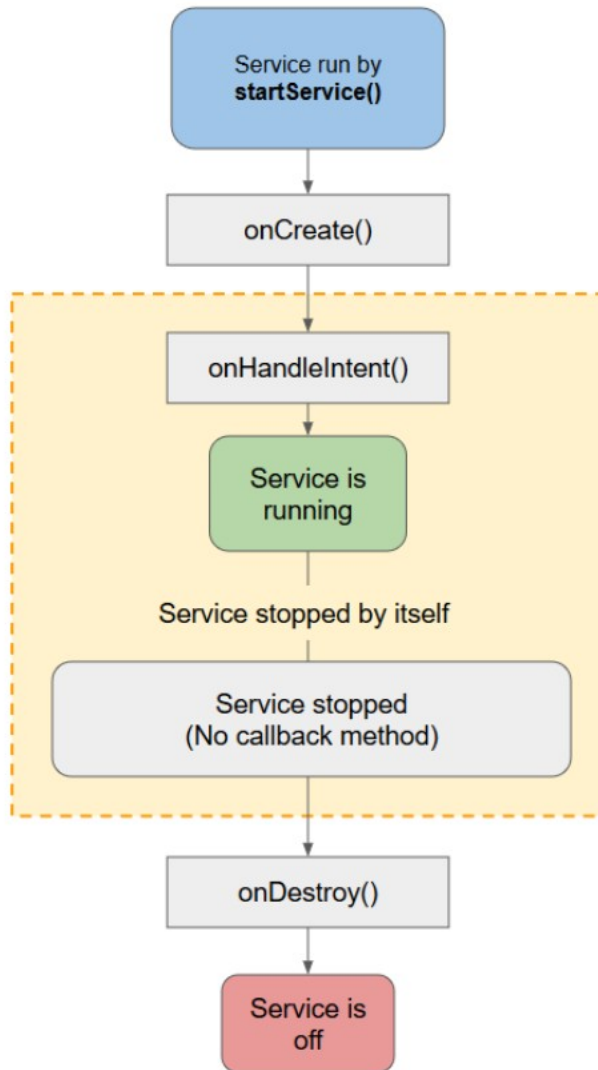
Сервис содержит методы обратного вызова жизненного цикла (life cycle callback methods) которые можно реализовать (implement) для контроля изменений состояния службы и выполнения работы в соответствующие моменты времени. Диаграмма ниже показывает жизненный цикл когда сервис создан с **startService()**, диаграмма справа показывает жизненный цикл когда сервис создан с **bindService()**.



Чтобы создать сервис, создайте класс Java, который расширит класс `Service` или один из его подклассов. Класс `Service` определяет разные методы callback и самое главное описывается далее. Вам не нужно выполнять (implements) все методы callbacks. При этом, важно понять, что делает каждый метод и удостовериться, что приложение ведет себя так, как ожидает пользователь.

Помимо 2 сервисов выше, есть другой сервис, который называется `IntentService`. `Intent Service` используется при выполнении одного задания, т.е. при завершении задания сервис самоуничтожается.

IntentService Service



Сравнение сервисов:

Unbound Service (Непривязанный)	Bound Service (Привязанный)	Intent Service
Unbounded Service используется для выполнения долгосрочных и повторяющихся заданий.	Bounded Service используется для выполнения заданий в фоновом режиме (background) и связанный с компонентом интерфейса	Intent Service используется для выполнения заданий 1 раз, т.е. при завершении задания сервис самоуничтожается.
Unbound Service запускается при вызове startService() .	Bounded Service запускается при вызове bindService() .	Intent Service запускается при вызове startService() .
Unbound Service останавливается или уничтожается при точном вызове stopService() .	Bounded Service снимает ограничения или уничтожается при вызове unbindService() .	IntentService вызывается неточно stopSelf() для уничтожения
Unbound Service независим от запускающего компонента.	Bound Service зависим от запускающего компонента.	Intent Service независим от запускающего компонента.

методы Callback сервисов

Callback [Перезвони]	Description [описание]
onStartCommand() [onstartcommand ()]	The system calls this method when another component, such as an activity, requests that the service be started, by calling startService() . [Система вызывает этот метод, когда другой компонент, например действие, запрашивает запуск службы, вызывая startService ().] If you implement this method, it is your responsibility to stop the service when its work is done, by calling stopSelf() or stopService() methods. [Если вы реализуете этот метод, вы обязаны остановить службу по завершении ее работы, вызвав методы stopSelf () или stopService ().]
onBind() [отвязать ()]	The system calls this method when another component wants to bind with the service by calling bindService() . [Система вызывает этот метод, когда другой компонент хочет выполнить привязку к службе, вызвав bindService ().] If you implement this method, you must provide an interface that clients use to communicate with the service, by returning an IBinder object. [Если вы реализуете этот метод, вы должны предоставить интерфейс, который клиенты используют для связи со службой, путем возврата объекта IBinder.] You must always implement this method, but if you don't want to allow binding, then you should return null. [Вы всегда должны реализовывать этот метод, но если вы не хотите разрешать привязку, вы должны вернуть null.]
onUnbind() [itsbind ()]	The system calls this method when all clients have disconnected from a particular interface published by the service. [Система вызывает этот метод, когда все клиенты отключились от определенного интерфейса, опубликованного службой.]
onRebind() [onrebind ()]	The system calls this method when new clients have connected to the service, after it had previously been notified that all had disconnected in its onUnbind(Intent) . [Система вызывает этот метод, когда новые клиенты подключились к службе, после того, как она ранее была уведомлена о том, что все отключились в ее onUnbind (Intent).]
onCreate() [oncreate ()]	The system calls this method when the service is first created using onStartCommand() or onBind() . [Система вызывает этот метод при первом создании службы с помощью onStartCommand () или onBind ().] This call is required to perform one-time set-up. [Этот вызов требуется для выполнения одноразовой настройки.]
onDestroy() [ondestroy ()]	The system calls this method when the service is no longer used and is being destroyed. [Система вызывает этот метод, когда служба больше не используется и уничтожается.] Your service should implement this to clean up any resources such as threads, registered listeners, receivers, etc [Ваша служба должна реализовать это для очистки любых ресурсов, таких как потоки, зарегистрированные слушатели, приемники и т. Д.] .

Создание новой асинхронной задачи

Предназначен для разгрузки длительных операций из UI-потока в фоновые потоки, а затем доставки результатов этих операций обратно в UI-поток

Класс **AsyncTask** предлагает простой и удобный механизм для перемещения трудоёмких операций в фоновый поток. Благодаря ему вы получаете удобство синхронизации обработчиков событий с графическим потоком, что позволяет обновлять элементы пользовательского интерфейса для отчета о ходе выполнения задачи или для вывода результатов, когда задача завершена.

Следует помнить, что **AsyncTask** не является универсальным решением для всех случаев жизни. Его следует использовать для не слишком продолжительных операций - загрузка небольших изображений, файловые операции, операции с базой данных и т.д.

Напрямую с классом **AsyncTask** работать нельзя (абстрактный класс), вам нужно наследоваться от него (`extends`). Ваша реализация должна предусматривать классы для объектов, которые будут переданы в качестве параметров методу `execute()`, для переменных, что станут использоваться для оповещения о ходе выполнения, а также для переменных, где будет храниться результат. Формат такой записи следующий:

```
AsyncTask<[Input_Parameter Type], [Progress_Report Type], [Result Type]>
```

Если не нужно принимать параметры, обновлять информацию о ходе выполнения или выводить конечный результат, просто укажите тип **Void** во всех трёх случаях. В параметрах можно использовать только обобщённые типы (Generic), т.е. вместо **int** используйте **Integer** и т.п.

Соответственно, варианты могут быть самыми разными. Вот несколько из них

```
AsyncTask<Void, Void, Void>
```

```
AsyncTask<String, Integer, Integer>
```

```
AsyncTask<String, Void, Integer>
```

```
AsyncTask<String, Integer, String>
```

Для запоминания можно смотреть на схему.

```
private class DownloadImage extends AsyncTask<String, Integer, Bitmap> {

    @Override
    protected void onPreExecute() {...}

    @Override
    protected Bitmap doInBackground(String... params) {...}

    @Override
    protected void onProgressUpdate(Integer... values) {
        super.onProgressUpdate(values);
    }

    @Override
    protected void onPostExecute(Bitmap bitmap) {...}
}
```

Каркас реализации **AsyncTask**, в котором используются строковый параметр и два целочисленных значения, нужных для оповещения о выполнении работы и о конечном результате:

```
private class MyAsyncTask extends AsyncTask<String, Integer, Integer> {

    @Override
    protected Integer doInBackground(String... parameter) {
        int myProgress = 0;
        // [... Выполните задачу в фоновом режиме, обновите переменную myProgress...]
        publishProgress(myProgress);
        // [... Продолжение выполнения фоновой задачи ...]
        // Верните значение, ранее переданное в метод onPostExecute
        return result;
    }

    @Override
    protected void onProgressUpdate(Integer... progress) {
        // [... Обновите индикатор хода выполнения, уведомления или другой
        // элемент пользовательского интерфейса ...]
    }

    @Override
    protected void onPostExecute(Integer... result) {
        // [... Сообщите о результате через обновление пользовательского
        // интерфейса, диалоговое окно или уведомление ...]
    }

}
```

У **AsyncTask** есть несколько основных методов, которые нужно освоить в первую очередь. Обязательным является метод **doInBackground()**, остальные используются исходя из логики вашего приложения.

- **doInBackground()** – основной метод, который выполняется в новом потоке. **Не имеет доступа к UI**. Именно в этом методе должен находиться код для тяжёлых задач. Принимает набор параметров тех типов, которые определены в реализации вашего класса. Этот метод выполняется в фоновом потоке, поэтому в нем не должно быть никакого взаимодействия с элементами пользовательского интерфейса. Размещайте здесь трудоёмкий код, используя метод **publishProgress()**, который позволит

обработчику **onProgressUpdate()** передавать изменения в пользовательский интерфейс. Когда фоновая задача завершена, данный метод возвращает конечный результат для обработчика **onPostExecute()**, который сообщит о нём в поток пользовательского интерфейса.

- **onPreExecute()** – выполняется перед **doInBackground()**. **Имеет доступ к UI**
- **onPostExecute()** – выполняется после **doInBackground()** (может не вызываться, если **AsyncTask** был отменен). **Имеет доступ к UI**. Используйте его для обновления пользовательского интерфейса, как только ваша фоновая задача завершена. Данный обработчик при вызове синхронизируется с потоком GUI, поэтому внутри него вы можете безопасно изменять элементы пользовательского интерфейса.
- **onProgressUpdate()**. **Имеет доступ к UI**. Переопределите этот обработчик для публикации промежуточных обновлений в пользовательский интерфейс. При вызове он синхронизируется с потоком GUI, поэтому в нём вы можете безопасно изменять элементы пользовательского интерфейса.
- **publishProgress()** - можно вызвать в **doInBackground()** для показа промежуточных результатов в **onProgressUpdate()**
- **cancel()** - отмена задачи
- **onCancelled()** - **Имеет доступ к UI**. Задача была отменена. Имеются две перегруженные версии.

Вкратце о том, что значит имеет/не имеет доступ к UI. Все ваши кнопки, текстовые метки, **ImageView** (всё, что отображается на экране) являются частью пользовательского интерфейса - User Interface (UI). Ваша задача - не допустить, чтобы в методе **doInBackground()** было обращение к какому-нибудь элементу. Например, нельзя установить текст в **TextView** через метод **setText()** или поменять цвет шрифта в **EditText**. В примерах вы увидите, как нужно делать подобные вещи.

На заметку: Несмотря на то, что студия генерирует строки **super.onPreExecute()** и **super.onPostExecute()** для соответствующих методов, вы их можете удалить. В исходниках суперкласса методы ничего не делают (это просто заглушка), поэтому во многих примерах в интернете они опущены. Пусть вас это не пугает.

AsyncTask разработан, чтобы быть классом-оберткой (helper class) вокруг классов **Thread** и **Handler**, и он не составляет универсальную платформу для поточной обработки. **AsyncTasks** идеально подходит для коротких операций (занимающих по времени около нескольких секунд). Если Вам

нужны потоки, работающие длительный промежуток времени, то рекомендуется использовать разное API, представленное в пакете `java.util.concurrent` с такими классами как [Executor](#), [ThreadPoolExecutor](#) и [FutureTask](#).

Как понятно уже из названия, класс `AsyncTask` предназначен для выполнения асинхронных по отношению к UI thread задач. Асинхронная задача - это некое вычисление, которое работает в фоновом потоке, и свои результаты публикует в UI thread. Асинхронная задача определена 3 стандартными типами, которые называются `Params`, `Progress` и `Result`, и 4 шагами, называемыми `onPreExecute`, `doInBackground`, `onProgressUpdate` и `onPostExecute`. Дополнительную информацию по использованию процессов и потоков

4. [Android Core](#)

[5. [основные контейнеры, `EditText` - отслеживание ввода, `Views`]{.underline}*** `RecyclerView` vs `ListView` - основные компоненты, виды `LayoutManager`]{.underline}***

Все описываемые разметки являются подклассами **`ViewGroup`** и наследуют свойства, определённые в классе **`View`**.

[Комбинирование](#)

Компоновка ведёт себя как элемент управления и их можно группировать. Расположение элементов управления может быть вложенным. Например, вы можете использовать **`RelativeLayout`** в **`LinearLayout`** и так далее. Но будьте осторожны: слишком большая вложенность элементов управления вызывает проблемы с производительностью.

[<include>](#)

Можно внедрить готовый файл компоновки в существующую разметку при помощи тега `<include>`:

```
<include layout="@layout/newfile"/>
```

Подробнее в отдельной статье [Include Other Layout](#)

[Программный способ создания разметки](#)

Для подключения созданной разметки используется код в методе **`onCreate()`**:

```
// activity_main.xml - имя вашей основной разметки
```

```
setContentView(R.layout.activity_main);
```

Естественно, вы можете придумать и свое имя для файла, а также в приложениях с несколькими экранами у вас будет несколько файлов разметки: **game.xml**, **activity_settings.xml**, **fragment_about.xml** и т.д.

В большинстве случаев вы будете использовать XML-способ задания разметки и подключать его способом, указанным выше. Но, иногда бывают ситуации, когда вам понадобится программный способ (или придётся разбираться с чужим кодом). Вам доступны для работы классы **android.widget.LinearLayout**, **LinearLayout.LayoutParams**, а также **Android.view.ViewGroup.LayoutParams**, **ViewGroup.MarginLayoutParams**. Вместо стандартного подключения ресурса разметки через метод **setContentView()**, вы строите содержимое разметки в Java, а затем уже в самом конце передаёте методу **setContentView()** родительский объект макета:

```
public void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    // закомментируем строку подключения xml-компоновки
    // setContentView(R.layout.activity_main);

    // Подключаем компоновку программно
    // Сначала объявим все необходимые элементы управления
    TextView label = new TextView(this);
    label.setText(R.string.my_text_label);
    label.setTextSize(20);
    label.setGravity(Gravity.CENTER_HORIZONTAL);

    ImageView pic = new ImageView(this);
    pic.setImageResource(R.drawable.matterhorn);
    pic.setLayoutParams(new LayoutParams(LayoutParams.WRAP_CONTENT, LayoutParams.WRAP_CONTENT));
    pic.setAdjustViewBounds(true);
    pic.setScaleType(ScaleType.FIT_XY);
    pic.setMaxHeight(250);
    pic.setMaxWidth(250);

    LinearLayout ll = new LinearLayout(this);
    ll.setOrientation(LinearLayout.VERTICAL);
    ll.setLayoutParams(new LayoutParams(LayoutParams.FILL_PARENT, LayoutParams.FILL_PARENT));
    ll.setGravity(Gravity.CENTER);
    ll.addView(label);
    ll.addView(pic);
    setContentView(ll); // все готово, можно подключать компоновку
}
```

Виды разметок

Существует несколько стандартных типов разметок:

- [FrameLayout](#)
- [LinearLayout](#)
- [TableLayout](#)

- [RelativeLayout](#)
- [GridLayout](#)
- [SwipeRefreshLayout](#)
- [ConstraintLayout](#)
- [CoordinatorLayout](#)

FrameLayout

FrameLayout является самым простым типом разметки. Обычно это пустое пространство на экране, которое можно заполнить только дочерними объектами **View** или **ViewGroup**. Все дочерние элементы **FrameLayout** прикрепляются к верхнему левому углу экрана.

В разметке **FrameLayout** нельзя определить различное местоположение для дочернего объекта. Последующие дочерние объекты **View** будут просто рисоваться поверх предыдущих компонентов, частично или полностью затеняя их, если находящийся сверху объект непрозрачен, поэтому единственный дочерний элемент для **FrameLayout** обычно растянут до размеров родительского контейнера и имеет атрибуты:

```
android:layout_width="match_parent"
```

```
android:layout_height="match_parent"
```

Также можно использовать свойства **Gravity** для управления порядком размещения.

LinearLayout

В студии макет **LinearLayout** представлен двумя вариантами - **Horizontal** и **Vertical**. Макет **LinearLayout** выравнивает все дочерние объекты в одном направлении — вертикально или горизонтально. Направление задается при помощи атрибута ориентации **android:orientation**:

- `android:orientation="horizontal"`
- `android:orientation="vertical"`

Все дочерние элементы помещаются в стек один за другим, так что вертикальный список компонентов будет иметь только один дочерний элемент в ряду независимо от того, насколько широким он является. Горизонтальное расположение списка будет размещать элементы в одну строку с высотой, равной высоте самого высокого дочернего элемента списка.

У разметки **LinearLayout** есть интересный атрибут **android:layout_weight**, который назначает индивидуальный вес для дочернего элемента. Этот атрибут определяет "важность" представления и позволяет этому элементу расширяться, чтобы заполнить любое оставшееся пространство в родительском представлении. Заданный по умолчанию вес является нулевым.

Также можно указать атрибут **android:weightSum**. Если атрибуту присвоить значение 100, то можно указывать вес дочерних элементов в удобном виде, как в процентах. Такой способ широко используется веб-мастерами при вёрстке.

Здесь нам придёт на помощь свойство **Layout weight**. вес нужно распределить таким образом: 0.08, 0.10, 0.12, 0.14, 0.16, 0.18, 0.22.

Градиентный фон

Если вам нужен градиентный фон для **LinearLayout**, то создайте в папке **res/drawable** xml-файл, например, **gradient.xml**:

```
<?xml version="1.0" encoding="utf-8"?>
<shape xmlns:android="http://schemas.android.com/apk/res/android"
    android:shape="rectangle" >

    <gradient
        android:angle="90"
        android:endColor="#00FF00"
        android:startColor="#FF0000" />

</shape>
```

Далее остаётся только прописать файл в свойстве **Background**:

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="fill_parent"
    android:layout_height="fill_parent"
    android:background="@drawable/gradient"
    android:orientation="vertical" >
```

Разделители

Начиная с API 11, у **LinearLayout** появился новый атрибут **android:divider**, позволяющий задать графический разделитель между кнопками. Также нужно явно включить использование разделителей через атрибут **android:showDividers**, в котором можно указать, каким образом использовать разделители - только в середине, в начале, в конце - можно комбинировать эти значения.

RelativeLayout

RelativeLayout (относительная разметка) находится в разделе **Layouts** и позволяет дочерним компонентам определять свою позицию относительно родительского компонента или относительно соседних дочерних элементов (по идентификатору элемента).

В **RelativeLayout** дочерние элементы расположены так, что, если первый элемент расположен по центру экрана, другие элементы, выровненные относительно первого элемента, будут выровнены относительно центра экрана. При таком расположении, при объявлении разметки в XML-файле, элемент, на который будут ссылаться для позиционирования другие объекты представления, должен быть объявлен раньше, чем другие элементы, которые обращаются к нему по его идентификатору.

Важные атрибуты, которые связаны с родителем.

- **android:layout_alignParentBottom** - выравнивание относительно нижнего края родителя
- **android:layout_alignParentLeft** - выравнивание относительно левого края родителя
- **android:layout_alignParentRight** - выравнивание относительно правого края родителя
- **android:layout_alignParentTop** - выравнивание относительно верхнего края родителя
- **android:layout_centerInParent** - выравнивание по центру родителя по вертикали и горизонтали
- **android:layout_centerHorizontal** - выравнивание по центру родителя по горизонтали
- **android:layout_centerVertical** - выравнивание по центру родителя по вертикали

Некоторые атрибуты можно использовать вместе, чтобы добиться нужного эффекта, например, разместить компонент в верхнем правом углу.

Компонент можно размещать не только относительно родителя, но и относительно других компонентов. Для этого все компоненты должны иметь свой идентификатор, по которому их можно будет отличать друг от друга. В этом случае вы можете задействовать другие атрибуты.

- **android:layout_above** - размещается над указанным компонентом
- **android:layout_below** - размещается под указанным компонентом

- **android:layout_alignLeft** - выравнивается по левому краю указанного компонента
- **android:layout_alignRight** - выравнивается по правому краю указанного компонента
- **android:layout_alignTop** - выравнивается по верхнему краю указанного компонента
- **android:layout_alignBottom** - выравнивается по нижнему краю указанного компонента
- **android:layout_toLeftOf** - правый край компонента размещается слева от указанного компонента
- **android:layout_toRightOf** - левый край компонент размещается справа от указанного компонента

Чтобы компоненты "не прилипали" друг к другу, используются атрибуты, добавляющие пространство между ними.

- **android:layout_marginTop**
- **android:layout_marginBottom**
- **android:layout_marginLeft**
- **android:layout_marginRight**

Создаём неподвижную полосу

С помощью **RelativeLayout** можно создать неподвижную полосу внизу экрана, которая не будет прокручиваться вместе со списком. Для этого используется атрибут **android:layout_alignParentBottom** и родственные ему атрибуты для верхней части

TableLayout и TableRow

Разметка **TableLayout** (Табличная разметка) позиционирует свои дочерние элементы в строки и столбцы, как это привыкли делать веб-мастера в теге **table**. **TableLayout** не отображает линии обрамления для их строк, столбцов или ячеек. **TableLayout** может иметь строки с разным количеством ячеек. При формировании разметки таблицы некоторые ячейки при необходимости можно оставлять пустыми. При создании разметки для строк используются объекты **TableRow**, которые являются дочерними классами **TableLayout** (каждый **TableRow** определяет единственную строку в таблице). Строка может не иметь ячеек или иметь одну и более ячеек, которые являются контейнерами для других объектов. В ячейку допускается вкладывать другой **TableLayout** или **LinearLayout**.

TableLayout удобно использовать, например, при создании логических игр типа Судоку, Крестики-Нолики и т.п.

Вот несколько правил для TableLayout. Во-первых, ширина каждой колонки определяется по наиболее широкому содержимому в колонке. Дочерние элементы используют в атрибутах значение **match_parent**. Атрибут TableRow для **layout_height** всегда *wrap_content*. Ячейки могут объединять колонки, но не ряды. Достигается слияние колонок через атрибут **layout_span**.

Если атрибуту **android:stretchColumns** компонента TableLayout присвоить значение "*", то содержимое каждого компонента TableRow может растягиваться на всю ширину макета.

Усадка, усушка, утруска

Если текст в ячейке таблицы слишком длинный, то он может растянуть ячейку таким образом, что часть текста просто выйдет за пределы видимости. Чтобы избежать данной проблемы, у контейнера TableLayout есть атрибут **android:shrinkColumns**. Мы рассмотрим программное применение данного атрибута через метод **setColumnShrinkable()**.

GridLayout

В Android 4.0 появился новый вид макета под именем **GridLayout** (раздел **Layouts** на панели инструментов). На первый взгляд он может показаться похожим на **TableLayout**. Но на самом деле он гораздо удобнее и функциональнее. И очень рекомендуется изучить и использовать его в своих новых проектах, которые разрабатываются под новую платформу.

Разметка относится к классу **android.widget.GridLayout** и имеет колонки, ряды, клетки как в **TableLayout**, но при этом элементы могут гибко настраиваться.

В **GridLayout** для любого компонента можно указать строку и колонку, и в этом месте таблицы он будет размещён. Указывать элемент для каждой ячейки не понадобится, это нужно делать только для тех ячеек, где действительно должны быть компоненты. Компоненты могут растягиваться на несколько ячеек таблицы. Более того, в одну ячейку можно поместить несколько компонентов.

В данной разметке нельзя использовать атрибут веса, поскольку он не сработает в дочерних представлениях **GridLayout**. Используйте атрибут **layout_gravity**.

Обратите внимание на атрибуты **layout_column** и **layout_columnSpan**, используемые для указания самой левой колонки и количества

занимаемых компонентом колонок. Также доступны атрибуты **layout_row** и **layout_rowSpan**.

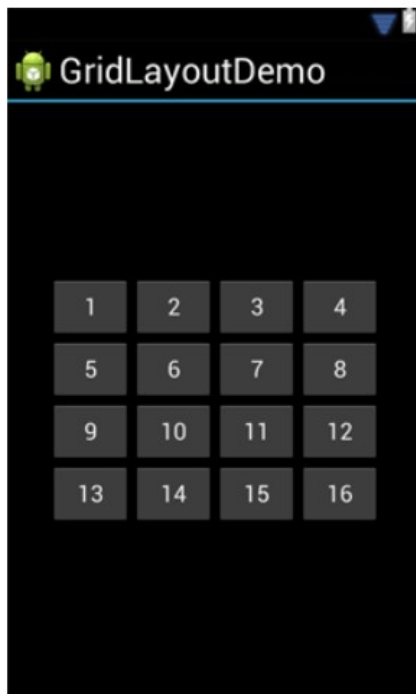
Для дочерних элементов не нужно указывать атрибуты **layout_height** и **layout_width**, по умолчанию они устанавливаются в значение **wrap_content**.

Количество колонок и рядов используются атрибуты **android:columnCount="number"** и **android:rowCount="number"**.

GridLayout. Во многом его поведение схоже с **LinearLayout** - у него есть горизонтальная и вертикальная ориентации, которые определяют вывод следующего элемента.

```
<?xml version="1.0" encoding="utf-8"?>
<GridLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_gravity="center"
    android:columnCount="4"
    android:orientation="horizontal" >
    <Button android:text="1" />
    <Button android:text="2" />
    <Button android:text="3" />
    <Button android:text="4" />
    <Button android:text="5" />
    <Button android:text="6" />
    <Button android:text="7" />
    <Button android:text="8" />
    <Button android:text="9" />
    <Button android:text="10" />
    <Button android:text="11" />
    <Button android:text="12" />
    <Button android:text="13" />
    <Button android:text="14" />
    <Button android:text="15" />
    <Button android:text="16" />
</GridLayout>
```

А так будет выглядеть форма в графическом дизайнере:



SwipeRefreshLayout (обертка добавляющая обновлялку)

интерфейс **OnRefreshListener** с методом **onRefresh()**, в котором следует обновить поступающие данные

Контейнер **SwipeRefreshLayout** представляет собой **ViewGroup**, который может работать только с одним дочерним элементом. Этим элементом может быть, например, [ScrollView](#), [ListView](#) или **RecyclerView**.

Контейнер **SwipeRefreshLayout** нужен для того, чтобы пользователи могли обновлять экран вручную. Раньше отдельного компонента, который бы отвечал за это не было, поэтому приходилось создавать свои велосипеды с обнаружением вертикальных свайпов (движение пальца по экрану). Пример этого можно посмотреть при обновлении ленты в приложении Twitter.

ConstraintLayout

ConstraintLayout представляет контейнер, который позволяет создавать гибкие и масштабируемые визуальные интерфейсы.

Для позиционирования элемента внутри **ConstraintLayout** необходимо указать ограничения (**constraints**). Есть несколько типов ограничений. В частности, для установки позиции относительно определенного элемента используются следующие ограничения:

- **layout_constraintLeft_toLeftOf**: левая граница позиционируется относительно левой границы другого элемента
- **layout_constraintLeft_toRightOf**: левая граница позиционируется относительно правой границы другого элемента
- **layout_constraintRight_toLeftOf**: правая граница позиционируется относительно левой границы другого элемента
- **layout_constraintRight_toRightOf**: правая граница позиционируется относительно правой границы другого элемента
- **layout_constraintTop_toTopOf**: верхняя граница позиционируется относительно верхней границы другого элемента
- **layout_constraintTop_toBottomOf**: верхняя граница позиционируется относительно нижней границы другого элемента
- **layout_constraintBottom_toBottomOf**: нижняя граница позиционируется относительно нижней границы другого элемента
- **layout_constraintBottom_toTopOf**: нижняя граница позиционируется относительно верхней границы другого элемента

- **layout_constraintBaseline_toBaselineOf:** базовая линия позиционируется относительно базовой линии другого элемента
- **layout_constraintStart_toEndOf:** элемент начинается там, где завершается другой элемент
- **layout_constraintStart_toStartOf:** элемент начинается там, где начинается другой элемент
- **layout_constraintEnd_toStartOf:** элемент завершается там, где начинается другой элемент
- **layout_constraintEnd_toEndOf:** элемент завершается там, где завершается другой элемент

Возможно, по поводу четырех последних свойств возникло некоторое непонимание, что подразумевается под началом или завершением элемента. Дело в том, что некоторые языки (например, арабский или фарси) имеют правостороннюю ориентацию, то есть символы идут справа налево, а не слева направо, как в европейских языках. И в зависимости от текущей ориентации - левосторонняя или правосторонняя - будет изменяться то, где именно начало, а где завершение элемента. Например, при левосторонней ориентации начало - слева, а завершение - справа, поэтому атрибут `layout_constraintStart_toEndOf` фактически будет аналогичен атрибуту `layout_constraintLeft_toRightOf`. А при правосторонней ориентации - атрибуту `layout_constraintRight_toLeftOf`, так как начало справа, а завершение - слева

Каждое ограничение устанавливает позиционирование элемента либо по горизонтали, либо по вертикали. И для определения позиции элемента в `ConstraintLayout` необходимо указать как минимум одно ограничение по горизонтали и одно ограничение по вертикали.

Позиционирования может производиться относительно границ самого контейнера `ContentLayout` (в этом случае ограничение имеет значение **parent**), либо же относительно любого другого элемента внутри `ConstraintLayout`

CoordinatorLayout

Автоматически определяет как должны взаимодействовать дочерние элементы

Как и предполагает его название, цель и философия этой `ViewGroup` является **координация** view элементов, которые находятся внутри него.

В библиотеке дизайна представлен *CoordinatorLayout*, макет, который обеспечивает дополнительный уровень контроля над событиями касания между дочерними представлениями, что позволяет многим из компонентов библиотеки *Design*.

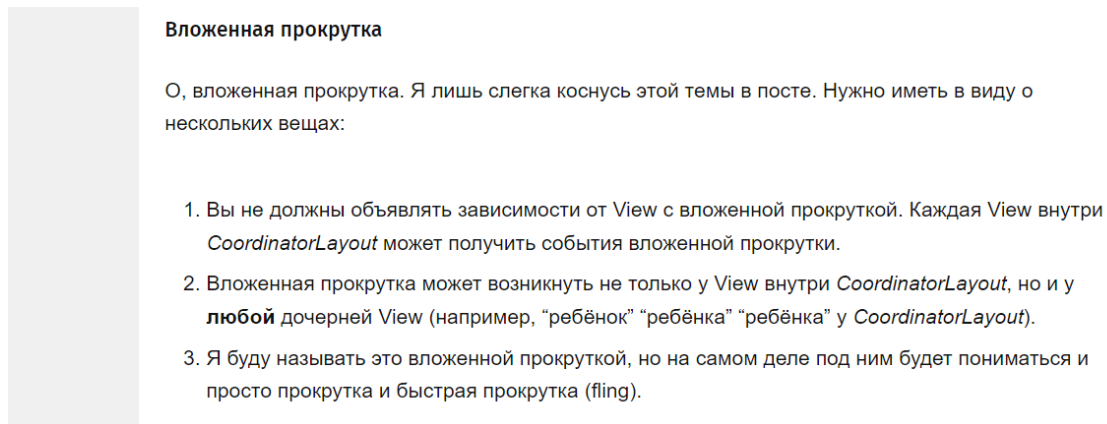
CoordinatorLayout – супермощный *FrameLayout*.

CoordinatorLayout позволяет контролировать взаимодействие дочерних *View* между собой. Одна из задач, которая решается с помощью *CoordinatorLayout* – скрыть *ActionBar* при скролле вниз и показать при скролле вверх.

Для настройки поведения дочерних *View* используются *Behaviors*-атрибуты. Кроме существующих реализаций *Behaviors*, таких как [AppBarLayout.ScrollingViewBehavior](#), можно создавать кастомные. Для этого необходимо создать наследника класса [CoordinatorLayout.Behavior](#) и указать полное имя класса в атрибуте `app:layout_behavior`.

Подключив *Behavior* к дочерней *View* у *CoordinatorLayout*, вы сможете перехватывать касания, оконные вставки (*window insets*), изменения размеров и макета (*measurement* и *layout*), а также вложенную прокрутку.

Создать *Behavior* довольно просто: наследуем наш класс от *Behavior*



EditText

listeners

`onTextChanged` выполняется во время изменения текста.

`afterTextChanged` выполняется сразу после изменения текста.

`beforeTextChanged` запускает момент перед изменением текста.

1.beforeTextChanged: Это означает, что символы будут заменены новым текстом. Текст не может быть изменен. Это событие используется, когда вам нужно взглянуть на старый текст, который вот-вот изменится.

2.onTextChanged: Были внесены изменения, некоторые символы только что были заменены. Текст не может быть изменен. Это событие используется, когда вам нужно увидеть, какие символы в тексте являются новыми.

3.afterTextChanged: То же, что и выше, за исключением того, что текст доступен для редактирования. Это событие используется, когда вам нужно увидеть и, возможно, отредактировать новый текст.

Компонент **EditText** — это текстовое поле для пользовательского ввода, которое используется, если необходимо редактирование текста. Следует заметить, что **EditText** является наследником **TextView**.

Для быстрой разработки текстовые поля снабдили различными свойствами и дали разные имена:

Plain Text, Person Name, Password, Password (Numeric), E-mail, Phone, Postal Address, Multiline Text, Time, Date, Number, Number (Signed), NumberDecimal.

Plain Text

Plain Text - самый простой вариант текстового поля без наворотов. При добавлении в разметку его XML-представление будет следующим:

Person Name

При использовании элемента **Person Name** в XML добавляется атрибут **inputType**, который отвечает за вид клавиатуры (только буквы) при вводе текста.

Password и Password (Numeric)

При использовании **Password** в **inputType** используется значение **textPassword**. При вводе текста сначала показывается символ, который заменяется на звездочку. Если используется элемент **Password (Numeric)**, то у атрибута **inputType** используется значение **numberPassword**. В этом случае на клавиатуре будут только цифры вместо букв. Вот и вся разница.

E-mail

У элемента **E-mail** используется атрибут **android:inputType="textEmailAddress"**. В этом случае на клавиатуре появляется дополнительная клавиша с символом ****@****, который обязательно используется в любом электронном адресе.

Phone

У элемента **Phone** используется атрибут **android:inputType="phone"**. Клавиатура похожа на клавиатуру из старого кнопочного сотового телефона с цифрами, а также с кнопками звёздочки и решётки.

Postal Address

Атрибут **android:inputType="textPostalAddress"**.

Multiline Text

У **Multiline Text** используется атрибут **android:inputType="textMultiLine"** позволяющий сделать текстовое поле многострочным. Дополнительно можете установить свойство **Lines** (атрибут **android:lines**), чтобы указать количество видимых строк на экране.

Time и Date

Атрибут **android:inputType="time"** или **android:inputType="date"**. На клавиатуре цифры, точка, запятая, тире.

Number, Number (Signed), Number (Decimal)

Атрибут **android:inputType="number"** или **numberSigned** или **numberDecimal**. На клавиатуре только цифры и некоторые другие символы.

Текст-подсказка

В Android у многих элементов есть свойство **Hint** (атрибут **hint**), который работает аналогичным образом. Установите у данного свойства нужный текст и у вас появится текстовое поле с подсказкой.

Вызов нужной клавиатуры

У элемента **EditText** на этот случай есть атрибут **inputType**

В данном случае с атрибутом **inputType="textCapWords"** каждый первый символ каждого слова при вводе текста автоматически будет преобразовываться в прописную. Удобно, не так ли?

Значение **textCapSentences** делает прописным каждый первый символ предложения.

Если вам нужен режим CapsLock, то используйте значение **textCapCharacters** и все буквы сразу будут большими при наборе.

Для набора телефонного номера используйте **phone**, и тогда вам будут доступны только цифры, звёздочка (*), решётка (#).

Для ввода веб-адресов удобно использовать значение **textUri**. В этом случае у вас появится дополнительная кнопка **.com** (при долгом нажатии на нее появятся альтернативные варианты .net, .org и др.).

Вот вам целый список доступных значений (иногда различия очень трудно различимы)

text

textCapCharacters (клавиатура с символами в верхнем регистре)

textCapWords

textCapSentences

textAutoCorrect

textAutoComplete

textMultiLine

textImeMultiLine

textNoSuggestions (без подсказок при вводе текста)

textUri

textEmailAddress

textEmailSubject

textShortMessage

textLongMessage

textPersonName

textPostalAddress

textPassword

textVisiblePassword (без автокоррекции)

textWebEditText

textFilter

textPhonetic

number

numberSigned

numberDecimal

phone

datetime

date

time

Интерфейс `InputType`

Кроме использования атрибута **`android:inputType`** мы можем добиться нужного поведения от текста при помощи интерфейса **`InputType`**.

Атрибут `android:imeOptions` - параметры для текущего метода ввода

У текстовых полей есть атрибут **`android:imeOptions`**, с помощью которого настраиваются параметры для текущего метода ввода. Например, когда **`EditText`** получает фокус и отображается виртуальная клавиатура, эта клавиатура содержит кнопку "Next" (Далее), если атрибут **`android:imeOptions`** содержит значение **`actionNext`**. Если пользователь касается этой кнопки, фокус перемещается к следующему компоненту, который принимает пользовательский ввод. Если компонент **`EditText`** получает фокус и на виртуальной клавиатуре появляется кнопка "Done" (Готово), значит использовался атрибут **`android:imeOptions`** со значением **`actionDone`**. Как только пользователь касается этой кнопки, система скрывает виртуальную клавиатуру.

Заблокировать текстовое поле

Для блокировки текстового поля присвойте значения **`false`** свойствам **`Focusable`**, **`Long clickable`** и **`Cursor visible`**.

Другие свойства

`minLines` и `maxLines`

Позволяют ограничить количество строк текста, которое можно ввести в текстовом поле

`maxLength`

Позволяет задать максимальное количество символов для ввода

Методы

Основной метод класса **`EditText`** — **`getText()`**, который возвращает текст, содержащийся в текстовом поле. Возвращаемое значение имеет специальный тип **`Editable`**, а не **`String`**.

Выделение текста

У **`EditText`** есть специальные методы для выделения текста:

- **selectAll()** — выделяет весь текст;
- **setSelection(int start, int stop)** — выделяет участок текста с позиции start до позиции stop;
- **setSelection(int index)** — перемещает курсор на позицию index;

Предположим, нам нужно выделить популярное слово из трёх букв в большом слове (это слово "кот", а вы что подумали?).

// выделяем 4, 5, 6 символы

```
editText.setSelection(3, 6);
```

Ещё есть метод **setSelectAllOnFocus()**, который позволяет выделить весь текст при получении фокуса.

```
editText.setSelectAllOnFocus(true)
```

Переопределяем кнопку Enter

Кроме атрибута **InputType** можно также использовать атрибут **android:imeOptions** в компоненте **EditText**, который позволяет заменить кнопку **Enter** на клавиатуре на другие кнопки, например, **Next**, **Go**, **Search** и др. Возможны следующие значения:

- **actionUnspecified**: Используется по умолчанию. Система сама выбирает нужный вид кнопки (IME_NULL)
- **actionGo**: Выводит надпись **Go**. Действует как клавиша Enter при наборе адреса в адресной строке браузера (IME_ACTION_GO)
- **actionSearch**: Выводит значок поиска (IME_ACTION_SEARCH)
- **actionSend**: Выводит надпись **Send** (IME_ACTION_SEND)
- **actionNext**: Выводит надпись **Next** (IME_ACTION_NEXT)
- **actionDone**: Выводи надпись **Done** (IME_ACTION_DONE)

RecyclerView и ListView

ListView может создать только вертикально прокручиваемый список, и чтобы он прокручивался плавно, приходилось делать все надлежащим способом. Кроме того, класс ListView довольно тяжелый - у него множество обязанностей. Всякий раз, когда нам требовалось обработать список, т.е. сконфигурировать его, единственный путь сделать это был через объект ListView внутри адаптера.

ViewHolder

Паттерн ViewHolder помогает сделать прокрутку нашего списка плавной. Он сохраняет ссылки на элементы списка, чтобы затем адаптер мог их переиспользовать. Благодаря этому надувание View элемента (inflating) и вызов дорогого метода `findViewById()` происходит всего пару раз, а не для каждого элемента списка.

Адаптер RecyclerView в обязательном порядке заставляет нас реализовать надувание элемента списка с помощью двух методов - `onCreateViewHolder()` и `onBindViewHolder()`.

ListView, с другой стороны, не делает обязательным реализацию ViewHolder. И, если мы не реализуем надувание сами, внутри `getView()`, мы останемся без эффективной прокрутки в списке.

ItemDecoration

Обязанности [ItemDecoration](#) просты в теории - декорировать элементы списка (рисование за ними, перед ними, между ними) - но нетривиальны в реализации. Вы должны наследоваться от класса `ItemDecoration` и реализовать ваше решение. У RecyclerView по-умолчанию нет разделителей между строками - это соответствует гайдлайнам [Material Design](#). Однако, если нам необходим разделитель, мы можем использовать [DividerItemDecoration](#) и добавить его в RecyclerView.

В случае ListView мы можем добавить разделитель прямо в описании layout ListView в файле xml. Но у нас нет гибкого механизма, наподобие `ItemDecoration`, для реализации разделителя.

Notifying adapter

У нас есть несколько классов механизмов уведомления (notify) в адаптере RecyclerView. Мы все еще можем использовать `notifyDataSetChanged()`, но, если нам необходимо изменить только один элемент списка, к нашим услугам `notifyItemInserted()`, `notifyItemRemoved()` или даже `notifyItemChanged()` и т.д. Мы должны использовать подходящее уведомление, чтобы сработала надлежащая анимация. Используя ListView мы можем использовать лишь `notifyDataSetChanged()`, а со всем остальным мы должны были справляться сами.

RecyclerView был создан как улучшенная замена ListView. Основные отличия следующие:

1. RecyclerView переиспользует ячейки списка при скроллинге. Для реализации этой логики используется класс ViewHolder. В ListView тоже можно реализовать адаптер с ViewHolder, но это необязательно и требует написания бойлерплейт кода.

2. RecyclerView разделяет хранение данных и логику отображения. С RecyclerView легко изменить лэйаут в рантайме, используя различные реализации абстрактного класса `LayoutManager`.

3. Логика отображения анимации элементов вынесена из RecyclerView в класс `ItemAnimator`.

4. Android Core

6. Broadcast Receivers что это и где может использоваться

Приёмник широковещательных сообщений — это компонент для получения внешних событий и реакции на них. Инициализировать передачи могут:

- другие приложения или службы
- сама система
- ваше собственное приложение

Широковещательные сообщения можно разделить на две группы:

- Неявная широковещательная трансляция (`implicit broadcast`) — сообщения рассылаются всем желающим, а не конкретно вашему приложению. Вам нужно только зарегистрироваться для получения этих сообщений через фильтр **`IntentFilter`** в манифесте. Система просматривает все объявленные фильтры намерений в вашем манифесте и проверяет, есть ли совпадение. Из-за этого поведения неявные широковещательные сообщения не имеют целевого атрибута
- Явная трансляция (`explicit broadcast`) предназначена для конкретных приложений. В атрибуте **`target`** указывают имя пакета приложения или имя класса компонента, по которому можно найти получателя

Класс **`BroadcastReceiver`** является базовым для класса, в котором должны происходить получение и обработка сообщений, посылаемых клиентским приложением с помощью вызова метода **`sendBroadcast()`**.

Зарегистрировать экземпляр класса **`BroadcastReceiver`** можно динамически в коде или статически в манифесте.

Для статической регистрации в файле манифеста в секции **`application`** следует создать секцию **`receiver`** и указать класс приёмника. Атрибут **`android:exported="true"`** сообщает получателю, что он может принимать широковещательные сообщения вне области приложения. Внутри секции указывается фильтр намерений в виде

строки, чтобы определить, какие сообщения приёмник должен прослушивать.

```
<receiver
    android:name=".YourReceiver"
    android:enabled="true"
    android:exported="true" >
    <intent-filter>
        <action android:name="ru.alexanderklimov.action.CAT" />
    </intent-filter>
</receiver>
```

Динамическая регистрация происходит с помощью метода **Context.registerReceiver()**.

```
this.registerReceiver(mTimeBroadcastReceiver, new IntentFilter(
    "android.intent.action.TIME_TICK"));
```

Перед этим создаётся класс, расширяющий базовый класс **BroadcastReceiver** и реализуется метод обратного вызова **onReceive()** обработчика событий.

```
public class TimeBroadcastReceiver extends BroadcastReceiver {
    public TimeBroadcastReceiver() {
    }

    @Override
    public void onReceive(Context context, Intent intent) {
        ...
    }
}
```

Метод **onReceive()** будет выполнен при получении широковещательного намерения, если полученное намерение соответствует фильтру. Приложения с зарегистрированными приёмниками широковещательных намерений будут запущены автоматически при получении соответствующего намерения. Метод должен быть завершён в течение пяти секунд, иначе появится диалоговое окно о принудительном закрытии.

Когда широковещательное сообщение прибывает для получателя сообщения, Android вызывает его методом **onReceive()** и передаёт в него объект **Intent**, содержащий сообщение. Приёмник широковещательных сообщений является активным только во время выполнения этого метода. Процесс, который в настоящее время выполняет **BroadcastReceiver**, т. е. выполняющийся в настоящее время код в методе обратного вызова **onReceive()**, как полагает система, является приоритетным процессом и будет сохранён, кроме случаев критического недостатка памяти в системе.

Когда программа возвращается из метода **onReceive()**, приёмник становится неактивным и система полагает, что работа объекта **BroadcastReceiver** закончена. Процесс с активным широковещательным получателем защищён от уничтожения системой. Однако процесс, содержащий неактивные компоненты, может быть уничтожен системой в любое время, когда память, которую он потребляет, будет необходима другим процессам.

Это представляет проблему, когда ответ на широковещательное сообщение занимает длительное время. Если метод **onReceive()** порождает отдельный поток, а затем возвращает управление, то полный процесс,

включая и порождённый поток, система Android считает неактивным (если другие компоненты приложения не активны в процессе), и считает этот процесс кандидатом на уничтожение.

В частности, вы не можете отобразить диалог или осуществить связывание со службой внутри экземпляра **BroadcastReceiver**. Для первого случая необходимо вместо этого использовать методы класса **NotificationManager**. Во втором случае можно использовать вызов метода **Context.startService()**, чтобы послать команду для запуска службы.

Решение этой проблемы возможно, если запустить в методе **onReceive()** отдельную службу вместе с **BroadcastReceiver** и позволить службе выполнять задание, чтобы сохранить содержание процесса активным в течение всего времени вашей операции.

Также можно зарегистрировать широковещательный приёмник не через манифест, а программно. Приёмник, зарегистрированный таким способом, будет отвечать на поступающие намерения только в том случае, если компонент приложения, которому он принадлежит, выполняется в этот момент.

Это может быть полезным, когда приёмник используется для обновления элементов пользовательского интерфейса внутри активности, запуска служб или уведомления через **NotificationManager**. В таких случаях вы можете отменять регистрацию широковещательного приёмника, если активность не отображается на экране или находится в неактивном состоянии.

Приёмники системных событий

Android использует широковещательные сообщения для системных событий, таких как уровень зарядки батареи, сетевые подключения, входящие звонки, изменения часового пояса, состояние подключения данных, входящие сообщения SMS или обращения по телефону. Вы можете использовать эти сообщения, чтобы добавлять к вашим собственным проектам новые функциональные возможности, основанные на системных событиях.

Следующий список показывает некоторые из встроенных действий, представленных как константы в классе **Intent**, которые используются для того, чтобы проследить изменения состояния устройства:

- **ACTION_BOOT_COMPLETED** — передаётся один раз, когда устройство завершило свою загрузку. Требуется разрешения **RECEIVE_BOOT_COMPLETED**
- **ACTION_CAMERA_BUTTON** — передаётся при нажатии пользователем клавиши **Camera**

- **ACTION_DATE_CHANGED** и **ACTION_TIME_CHANGED** - запускаются при изменении даты или времени на устройстве вручную пользователем
- **ACTION_SCREEN_OFF** и **ACTION_SCREEN_ON** — передаются, когда экран выключается или включается
- **ACTION_TIMEZONE_CHANGED** — передаётся при изменении текущего часового пояса

```
String requiredPermission = "ru.alexanderklimov.MY_BROADCAST_PERMISSION";
sendOrderedBroadcast(intent, requiredPermission);
```

С помощью этого метода ваше намерение дойдёт до всех зарегистрированных приёмников, обладающих необходимым доступом (если он был указан), в порядке их приоритета. Приоритет задаётся с помощью атрибута **android:priority** внутри узла фильтра намерений в манифесте. Чем больше значение, тем выше приоритет.

Производить упорядоченные трансляции с использованием приоритетов рекомендуется только для тех приёмников, которым необходим конкретный порядок приёма сообщений.

Передача сообщений весьма проста в реализации. В вашем приложении необходимо создать сообщение, которое вы хотите передать. Установите при необходимости поля **action**, **data** и **category** (действие, данные и категорию) вашего сообщения и путь, который позволяет приёмникам широковещательных сообщений точно определять "свое" сообщение. В этом сообщении строка действия **ACTION** должна быть уникальной, чтобы идентифицировать передаваемое действие. В таких случаях создают строку-идентификатор действия по правилам именования пакетов Java. Например, для обнаружения кота в большом здании:

```
public static final String NEW_CAT_DETECTED = "ru.alexanderklimov.action.NEW_CAT";
```

Далее вы создаёте объект **Intent**, загружаете в него нужную информацию и вызываете метод **sendBroadcast()**, передав ему в качестве параметра созданный объект **Intent**. Дополнительные данные можно использовать в **extras** как необязательные параметры.

Виртуальный код для обнаружения кота:

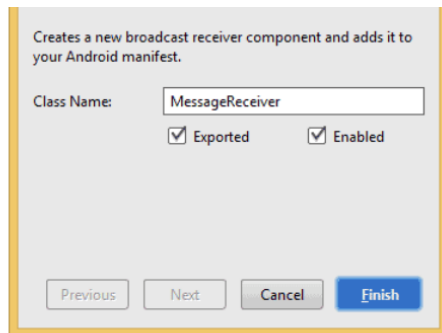
```
Intent intent = new Intent(NEW_CAT_DETECTED);
// Или так
// Intent intent = new Intent();
// intent.setAction(NEW_CAT_DETECTED);

intent.putExtra("catname", CatName);
intent.putExtra("longitude", currentLongitude);
intent.putExtra("latitude", currentLatitude);
sendBroadcast(intent);
```

Существуют также родственные методы **sendStickyBroadcast()** и **sendOrderedBroadcast()**.

Приёмник представляет собой обычный Java-класс на основе **BroadcastReceiver**. Вы можете создать вручную класс и наполнить его необходимыми методами. Раньше так и поступали. Но в студии есть готовый шаблон

Создание приемника и метода onReceive



Студия создаст изменения в двух местах. Во-первых, будет создан класс **MessageReceiver**:

```
package ru.alexanderklimov.testapplication;

import android.content.BroadcastReceiver;
import android.content.Context;
import android.content.Intent;

public class MessageReceiver extends BroadcastReceiver {
    public MessageReceiver() {
    }

    @Override
    public void onReceive(Context context, Intent intent) {
        // TODO: This method is called when the BroadcastReceiver is receiving
        // an Intent broadcast.
        throw new UnsupportedOperationException("Not yet implemented");
    }
}
```

Во-вторых, в манифесте будет добавлен новый блок.

```
<receiver
    android:name=".MessageReceiver"
    android:enabled="true"
    android:exported="true" >
</receiver>
```

В него следует добавить фильтр, по которому он будет ловить сообщения.

```
<receiver
    android:name=".MessageReceiver"
    android:enabled="true"
    android:exported="true">
    <intent-filter>
        <action android:name="ru.alexanderklimov.action.CAT" />
    </intent-filter>
</receiver>
```

Вернёмся в класс приёмника и модифицируем метод **onReceive()**.

Приёмники широковещательных сообщений

Вот плавно мы перешли к приёмникам широковещательных сообщений. На самом деле вам не так часто придётся рассылать сообщения, гораздо чаще встречается потребность принимать сообщения. В первую очередь, сообщения от системы. Примерами таких сообщений могут быть:

- Низкий заряд батареи
- Нажатие на кнопку камеры
- Установка нового приложения

Приёмник можно создать двумя способами - через манифест (мы использовали этот способ в примере) и программно через метод **registerReceiver()**. Между двумя способами есть существенная разница. Приёмник, заданный в манифесте, известен системе, которая сканирует файлы манифеста всех установленных приложений. Поэтому, даже если ваше приложение не запущено, оно всё равно сможет отреагировать на поступающее сообщение.

Приёмник, созданный программно, может работать только в том случае, когда активность вашего приложения активна. Казалось, это является недостатком и нет смысла использовать такой подход. Но всё не так просто. Некоторые системные сообщения могут обрабатываться только приёмниками, созданными программно. И в этом есть свой резон. Например, если ваше приложение не запущено, ему нет смысла

принимать сообщения о заряде батареи. Иначе заряд батареи будет расходоваться ещё быстрее при лишней бесполезной работе. Информацию о заряде батареи ваше приложение может получить, когда в этом есть необходимость. Следует сверяться с документацией, какой вид приёмника следует использовать.

При программной регистрации приёмника мы можем также снять регистрацию, когда больше не нуждаемся в нём с помощью метода `unregisterBroadcastReceiver()`.

Следим за питанием

Нет, речь пойдёт не о правильном питании кота. Имеется в виду питание от электричества. Если ваше устройство отключить от зарядки, то система оповещает об этом событии через широковещательное намерение `android.intent.action.ACTION_POWER_DISCONNECTED`.

Не станем заводить новый приёмник, а откроем манифест и добавим дополнительный фильтр к приёмнику сообщений от радистки Кэт.

```
<receiver
    android:name=".MessageReceiver"
    android:enabled="true"
    android:exported="true">
    <intent-filter>
        <action android:name="ru.alexanderklimov.action.CAT" />
    </intent-filter>
    <intent-filter>
        <action android:name="android.intent.action.ACTION_POWER_DISCONNECTED"></action>
    </intent-filter>
</receiver>
```

Автозапуск приложения при загрузке

В манифесте указать разрешение:

```
<uses-permission android:name="android.permission.RECEIVE_BOOT_COMPLETED" />
```

В манифесте в блоке **application** зарегистрировать приёмник на приём сообщения `ACTION_BOOT_COMPLETED`:

```
<receiver android:name=".MyBroadcastReceiver">
    <intent-filter>
        <action android:name="android.intent.action.BOOT_COMPLETED" />
    </intent-filter>
</receiver>
```

В описании без необходимости не указывайте атрибуты «enabled», «exported» и т.д. Вполне достаточно настроек и атрибутов по умолчанию.

Код вашего широковещательного приёмника:

```
public class BootCompletedReceiver extends BroadcastReceiver {
    public BootCompletedReceiver() {
    }

    public void onReceive(Context context, Intent intent) {
        if (intent.getAction().equals(Intent.ACTION_BOOT_COMPLETED)) {
            // ваш код здесь
        }
    }
}
```

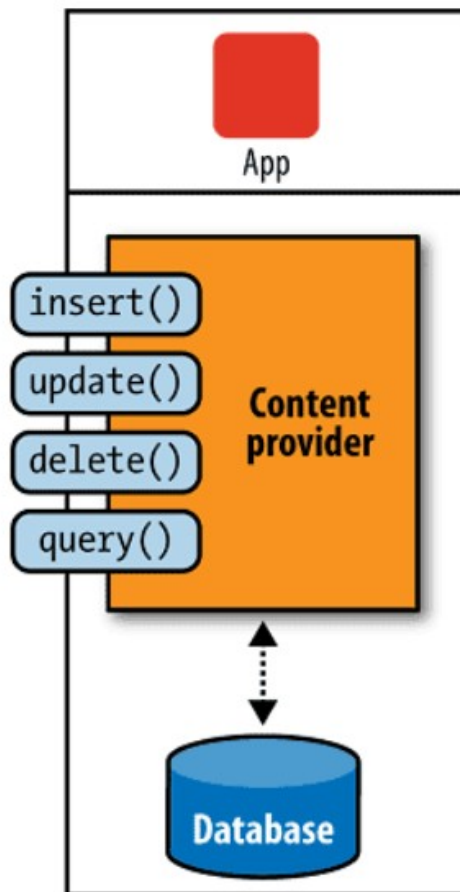
Если ваш приёмник используется только для сообщения `ACTION_BOOT_COMPLETED`, то проверка «if» не обязательна. Однако иногда разработчики используют один и тот же ресивер для разных сообщений. В этом случае фильтруйте сообщения, проверяя их внутри метода `onReceive()`.

4. Android Core

7. Content что это и где может использоваться, как передаются Providers данные

Что такое контент-провайдер

ContentProvider — это «обёртка» (wrapper) над данными приложения, позволяющая безопасно делиться ими с другими приложениями через единый интерфейс (полное определение и иерархия БД на примере системных провайдеров — см. выше, в разделе «ContentProvider»).



REST (Representational state transfer) – это стиль архитектуры программного обеспечения для распределенных систем

В общем случае REST является очень простым интерфейсом управления информацией без использования каких-то дополнительных внутренних прослоек. Каждая единица информации однозначно определяется глобальным идентификатором, таким как URL. Каждая URL в свою очередь имеет строго заданный формат.

А теперь тоже самое более наглядно:

Отсутствие дополнительных внутренних прослоек означает передачу данных в том же виде, что и сами данные. Т.е. мы не заворачиваем данные в XML, как это делает SOAP и XML-RPC, не используем AMF, как это делает Flash и т.д. Просто отдаем сами данные.

Каждая единица информации однозначно определяется URL – это значит, что URL по сути является первичным ключом для единицы данных. Т.е. например третья книга с книжной полки будет иметь вид /book/3, а 35 страница в этой книге — /book/3/page/35. Отсюда и получается строго заданный формат. Причем совершенно не имеет значения, в каком формате находятся данные по адресу /book/3/page/35 – это может быть и HTML, и отсканированная копия в виде jpeg-файла, и документ Microsoft Word.

Как происходит управление информацией сервиса – это целиком и полностью основывается на протоколе передачи данных. Наиболее распространенный протокол конечно же HTTP. Так вот, для HTTP действие над данными задается с помощью методов: GET (получить), PUT (добавить, заменить), POST (добавить, изменить, удалить), DELETE (удалить). Таким образом, действия CRUD (Create-Read-Update-Delete) могут выполняться как со всеми 4-мя методами, так и только с помощью GET и POST.

Вот как это будет выглядеть на примере:

GET /book/ — получить список всех книг
GET /book/3/ — получить книгу номер 3
PUT /book/ — добавить книгу (данные в теле запроса)
POST /book/3 – изменить книгу (данные в теле запроса)
DELETE /book/3 – удалить книгу

В Android существует возможность выражения источников данных (или поставщиков данных) при помощи передачи состояния представления - REST, в виде абстракций, называемых поставщиками содержимого. Базу данных SQLite можно заключить в поставщик содержимого. Чтобы получить данные из поставщика содержимого или сохранить в нём новую информацию, нужно использовать набор REST-подобных идентификаторов URI. Например, если бы вам было нужно получить набор книг из поставщика содержимого, в котором заключена электронная библиотека, вам понадобился бы такой URI (по сути запрос к получению всех записей таблицы books):

content://com.android.book.bookprovider/books

Чтобы получить из библиотеки конкретную книгу (например, книгу №23), будет использоваться следующий URI (отдельный ряд таблицы):

content://com.android.book.bookProvider/books/23

Любая программа, работающая в устройстве, может использовать такие URI для доступа к данным и осуществления с ними определенных операций. Следовательно, поставщики содержимого играют важную роль при совместном использовании данных несколькими приложениями.

Встроенные поставщики

В Android используются встроенные поставщики содержимого (пакет **android.provider**). Вот неполный список поставщиков содержимого:

- Browser
- CallLog
- Contacts
 - People
 - Phones
 - Photos
 - Groups
- MediaStore
 - Audio
 - Albums
 - Artists
 - Genres
 - Playlists
 - Images
 - Thumbnails
 - Video
- Settings

На верхних уровнях иерархии располагаются базы данных, на нижних - таблицы. Так, Browser, CallLog, Contacts, MediaStore и Settings - это отдельные базы данных SQLite, инкапсулированные в форме поставщиков. Обычно такие базы данных SQLite имеют расширение DB и доступ к ним открыт только из специальных пакетов реализации (implementation package). Любой доступ к базе данных из-за пределов этого пакета осуществляется через интерфейс поставщика содержимого.

Создание собственного контент-провайдера

Для создания собственного контент-провайдера нужно унаследоваться от абстрактного класса **ContentProvider**:

```
public class MyContentProvider extends ContentProvider {  
  
}
```

В классе необходимо реализовать абстрактные методы **query()**, **insert()**, **update()**, **delete()**, **getType()**, **onCreate()**.

Прослеживается некоторое сходство с созданием обычной базы данных.

А также его следует зарегистрировать в манифесте с помощью тега **provider** с атрибутами **name** и **authorities**. Тег **authorities** служит для описания базового пути URI, по которому **ContentResolver** может найти базу данных для взаимодействия. Данный тег должен быть уникальным, поэтому рекомендуется использовать имя вашего пакета, чтобы не произошло путаницы с другими приложениями, например:

```
<provider  
android:name=".MyContentProvider"  
android:authorities="ru.alexanderklimov.provider.notepad" />
```

Источник поставщика содержимого аналогичен доменному имени сайта. Если источник уже зарегистрирован, эти поставщики содержимого будут представлены гиперссылками, начинающимися с соответствующего префикса источника:

```
content://ru.alexanderklimov.provider.notepad/
```

Итак, поставщики содержимого, как и веб-сайты, имеют базовое доменное имя, действующее как стартовая URL-страница.

Необходимо отметить, что поставщики содержимого, используемые в Android, могут иметь неполное имя источника. Полное имя источника рекомендуется использовать только со сторонними поставщиками содержимого. Поэтому вам иногда могут встретиться поставщики содержимого, состоящие из одного слова, например **contacts**, в то время как полное имя такого поставщика содержимого - **com.google.android.contacts**.

В поставщиках содержимого также встречаются REST-подобные гиперссылки, предназначенные для поиска данных и работы с ними. В случае описанной выше регистрации унифицированный идентификатор ресурса, предназначенный для обозначения каталога или коллекции записей в базе данных NotePadProvider, будет иметь имя:

```
content://ru.alexanderklimov.provider.notepad/notes
```

URI для идентификации отдельно взятой записи будет иметь вид:

content://ru.alexanderklimov.provider.notepad/notes/#

Символ # соответствует конкретной записи (ряд таблицы). Ниже приведено еще несколько примеров URI, которые могут присутствовать в поставщиках содержимого:

content://media/internal/images

content://media/external/images

content://contacts/people/

content://contacts/people/23

Обратите внимание - здесь поставщики содержимого **content://media** и **content://contacts** имеют неполную структуру. Это обусловлено тем, что данные поставщики содержимого не являются сторонними и контролируются Android.

Структура унифицированных идентификаторов содержимого (Content URI)

Для получения данных из поставщика содержимого нужно просто активировать URI. Однако при работе с поставщиком содержимого найденные таким образом данные представлены как набор строк и столбцов и образуют объект Android **cursor**. Рассмотрим структуру URI, которую можно использовать для получения данных.

Унифицированные идентификаторы содержимого (Content URI) в Android напоминают HTTP URI, но начинаются с **content** и строятся по следующему образцу:

content://*/**/*

или

content://authority-name/path-segment1/path-segment2/etc...

Вот пример URI, при помощи которого в базе данных идентифицируется запись, имеющая номер 23:

content://ru.alexanderklimov.provider.notepad/notes/23

После **content:** в URI содержится унифицированный идентификатор источника, который используется для нахождения поставщика содержимого в соответствующем реестре. Часть URI **ru.alexanderklimov.provider.notepad** представляет собой источник.

/notes/23 - это раздел пути (path section), специфичный для каждого отдельного поставщика содержимого. Фрагменты **notes** и **23** раздела пути называются сегментами пути (path segments). Одной из функций поставщика содержимого является документирование и интерпретация раздела и сегментов пути, содержащихся в URI.

UriMatcher

Провайдер имеет специальный объект класса **UriMatcher**, который получает данные снаружи и на основе полученной информации создаёт нужный запрос к базе данных.

Вам нужно задать специальные константы, по которым провайдер будет понимать дальнейшие действия. Если используется одна таблица, то обычно применяют две константы - любые два целых числа, например, 100 для таблицы и 101 для отдельного ряда таблицы. Схематично можно изобразить так.

URI pattern	Code	Contant name
content://ru.alexanderklimov.provider.notepad/notes	100	NOTES
content://ru.alexanderklimov.provider.notepad/notes/#	101	NOTES_ID

В коде с помощью **switch** создаётся ветвление - хотим ли мы получить информацию о всей таблице (код 100) или к конкретному ряду (код 101).

Приложение может быть сложным и иметь несколько таблиц. Тогда и констант будет больше. Например, так.

URI pattern	Code	Contant name
content://com.android.contacts/contacts	1000	CONTACTS
content://com.android.contacts/contacts/#	1001	CONTACTS_ID
content://com.android.contacts/lookup/*	1002	CONTACTS_LOOKUP
content://com.android.contacts/lookup/*/#	1003	CONTACTS_LOOKUP_ID
...	...	...
content://com.android.contacts/data	3000	DATA
content://com.android.contacts/data/#	3001	DATA_ID
...	...	...

Символ решётки (#) отвечает за число, а символ звёздочки (*) за строку.

Метод query()

Метод **query()** является обязательным для класса **ContentProvider**. Если мы используем контент-провайдер для обращения к базе данных, то в нём вызывает одноимённый метод **SQLiteDatabase**. Состав метода практически идентичен.

```
@Override
public Cursor query(Uri uri, String[] projection, String selection, String[] selectionArgs, String sortOrder) {
    ...

    cursor = database.query(GuestEntry.TABLE_NAME, projection, selection, selectionArgs,
        null, null, sortOrder);
    ...
}
```

Краткая подсказка:

```
URI: content://com.example.android.cathouse/cats/3
Projection: {"_id", "name"}
Selection: "_id=?"
Selection Args: {"3"}
```

Соответствует SQL-выражению.

```
SELECT id, name FROM cats WHERE _id=3
```

Вам нужно программно получить необходимые данные для аргументов метода. Обратите внимание на метод `ContentUri.parseId(uri)`, который возвращает последний сегмент адреса, в нашем случае число 3, для Selection Args.

Метод insert()

Метод **insert()** содержит два параметра - **URI** и **ContentValues**. Первый параметр работает аналогично, как в методе **query()**. Вторая вставляет данные в нужные колонки таблицы.

Для вставки используется вспомогательный метод **insertGuest()**.

Структурирование MIME-типов в Android

Как веб-сайт возвращает тип MIME для заданной гиперссылки (это позволяет браузеру активировать программу, предназначенную для просмотра того или иного типа контента), так и в поставщике содержимого предусмотрена возможность возвращения типа MIME для заданного URI. Благодаря этому достигается определенная гибкость при просмотре данных. Если мы знаем, данные какого именно типа получим, то можем выбрать одну или несколько программ, предназначенных для представления таких данных. Например, если на жестком диске компьютера есть текстовый файл, мы можем выбрать несколько редакторов, которые способны его отобразить.

Типы MIME работают в Android почти так же, как и в HTTP. Вы запрашиваете у контент-провайдера тип MIME определенного поддерживаемого им URI, и поставщик содержимого возвращает двухчастную последовательность символов, идентифицирующую тип MIME в соответствии с принятыми стандартами.

Обозначение MIME состоит из двух частей: типа и подтипа.

Ниже приведены примеры некоторых известных пар типов и подтипов MIME:

```
text/html
text/css
```

text/xml
image/jpeg
audio/mp3
video/mp4
application/pdf
application/msword

Основные зарегистрированные типы содержимого:

application
audio
image
message
model
multipart
text
video

В Android применяется схожий принцип для определения типов MIME. Обозначение **vnd** в типах MIME в Android означает, что данные типы и подтипы являются нестандартными, зависящими от производителя. Для обеспечения уникальности в Android типы и подтипы разграничиваются при помощи нескольких компонентов, как и доменные имена. Кроме того, типы MIME в Android, соответствующие каждому типу содержимого, существуют в двух формах: для одиночной записи и для нескольких записей.

Типы MIME широко используются в Android, в частности при работе с намерениями, когда система определяет по MIME-типу данных, какое именно явление следует активировать. Типы MIME всегда воспроизводятся контент-провайдерами на основании соответствующих URI. Работая с типами MIME, необходимо не упускать из виду три аспекта.

1. Тип и подтип должны быть уникальными для того типа содержимого, который они представляют. Обычно это каталог с элементами или отдельный элемент. В контексте Android разница между каталогом и элементом может быть не такой очевидной, как кажется на первый взгляд.
2. Если тип или подтип не являются стандартными, им должен предшествовать префикс **vnd** (обычно это касается конкретных видов записи).
3. Обычно типы и подтипы относятся к определенному пространству имен в соответствии с вашими нуждами.

Необходимо еще раз подчеркнуть этот момент: основной тип MIME для коллекции элементов, возвращаемый командой **cursor** в Android, всегда должен иметь вид **vnd.android.cursor.dir**, а основной тип MIME для

одионого элемента, находимый командой **cursor** в Android, - вид **vnd.android.cursor.item**. Если речь идет о подтипе, то поле для маневра расширяется, как в случае с **vnd.google.note**; после компонента **vnd**. вы можете свободно выбирать любой устраивающий вас подтип.

ContentResolver

Каждый объект **Content**, принадлежащий приложению, включает в себя экземпляр класса **ContentResolver**, который можно получить через метод **getContentResolver()**.

```
ContentResolver contentResolver = getContentResolver();
```

ContentResolver используется для выполнения запросов и транзакций от активности к контент-провайдеру. **ContentResolver** включает в себя методы для запросов и транзакций, аналогичные тем, что содержит **ContentProvider**. Объекту **ContentResolver** не нужно знать о реализации контент-провайдера, с которым он взаимодействует - любой запрос всего лишь принимают путь **URI**, в котором указано, к какому объекту **ContentProvider** необходимо обратиться.

- **ContentResolver** — статический прокси-сервер, который взаимодействует с, чтобы получить доступ к его данным из того же приложения или из другого приложения.

4. Android Core

- 8. состав APK, app bundle, для чего используется proguard, Platform поток vs процесс, подпись приложения

APK (Android Package) – это формат файла, который используется для распространения и установки мобильных приложений в операционной системе Android.

APK представляет собой архив, который содержит следующие файлы:

- Папка META-INF:
- MANIFEST.MF – манифест-файл, который содержит **SHA-хэши** всех файлов в APK-пакете;
- Сертификат приложения;
- Файлы с дополнительной метаданной.
- lib – папка, содержащая скомпилированный код платформенно-зависимых библиотек. lib содержит подкаталоги для соответствующих платформ: armeabi, armeabi-v7a, arm64-v8a, x86, x86_64, mips.
- res – папка, в которой лежат андроид-ресурсы в **Binary XML** формате.

- assets – папка, содержащая **ассеты приложения**.
- AndroidManifest.xml – манифест Андроид-приложения. Этот файл хранится в скомпилированном Binary XML формате.
- classes.dex – один или несколько файлов, которые представляют собой код приложения, скомпилированный в Dalvik-байткод.
- resources.arsc – файл, в котором хранится таблица маппинга id ресурсов в соответствующие файлы.

SHA-1 был разработан в рамках проекта Capstone правительства США. Первоначальная спецификация алгоритма - теперь обычно называемая SHA-0 - была опубликована в 1993 году под названием Secure Hash Standard, FIPS PUB 180, правительственным агентством стандартов США NIST (Национальный институт стандартов и технологий). Он был отозван Агентством национальной безопасности вскоре после публикации и заменен пересмотренной версией, опубликованной в 1995 году в FIPS PUB 180-1 и обычно обозначаемой как SHA-1. Конфликты с полным алгоритмом SHA-1 могут быть произведены с использованием разбитой атаки, и хеш-функция должна считаться неработающей. SHA-1 создает хеш-дайджест из 160 бит (20 байтов).

В документах SHA-1 может называться просто «SHA», даже если это может конфликтовать с другими алгоритмами безопасного хеширования, такими как SHA-0, SHA-2 и SHA-3.

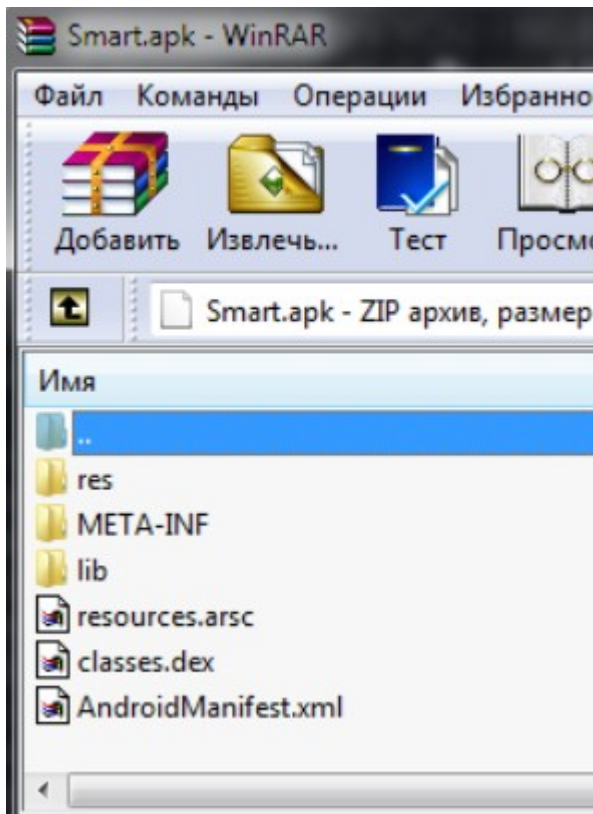
Двоичный XML

В качестве компактного представления XML (Extensible Markup Language) были предложены различные двоичные форматы. Использование двоичного формата XML обычно снижает многословность XML-документов, тем самым также снижая стоимость синтаксического анализа [1], но затрудняет использование обычных текстовых редакторов и сторонних инструментов для просмотра и редактирования документа. Существует несколько конкурирующих форматов, но ни один из них еще не стал стандартом де-факто, хотя Консорциум World Wide Web принял EXI в качестве Рекомендации 10 марта 2011 года [2].

Двоичный XML обычно используется в приложениях, где производительность стандартного XML недостаточна, но ценится возможность преобразовать документ в форму (XML), которая легко просматривается и редактируется, и обратно. Другие преимущества могут включать в себя возможность произвольного доступа и индексации XML-документов.

АССЕТЫ

Предоставляет доступ к файлам необработанных ресурсов приложения; Этот класс представляет API нижнего уровня, который позволяет вам открывать и читать необработанные файлы, которые были связаны с приложением в виде простого потока байтов.



- Файл **AndroidManifest.xml** – чтобы сразу стало ясно - это «Паспорт», внутри него описывается что находится в приложении:
 1. Требования к приложению.
 2. Структура приложения.
 3. Версия приложения.
- Папка **META-INF** – содержит MANIFEST.MF(открывается текстовым редактором "блокнот"), открыв его можно увидеть контрольные суммы SHA-1 и пути к данным, версию стандарта Manifest-Version, файлы сертификата RSA или DSA, файл SF содержит пути к ресурсам и их контрольные суммы. META-INF – это файлы метаданных - данные о данных.
- Папка **res** - В этой папке все ресурсы программы «Начинка», которые разнесены по разным поддиректориям. Например папки ~ drawable~ находятся графические элементы приложения (иконки, статусы и другие картинки), layout – XML-файлы о размещении элементов графического интерфейса (GUI).

- **classes.dex** - программный код, который выполняется в Dalvik VM. Для тех кто не в курсе Android это смесь Linux ядра с виртуальной машиной Java.
- **resources.arsc** - скомпилированный XML-файл, содержит данные о ресурсах, которые использует программа.
- Папка **assets** – может и не быть в арк приложение, также содержат ресурсы.
- Папка **lib** – может и не быть в арк приложение. Приложение написано с использованием NDK (native development kit) , а в папке располагаются нативные библиотеки, к примеру можно писать приложения на c++.
- Папка **com** – может и не быть в арк приложение.
- Папка **org** – может и не быть в арк приложение.
- Папка **udk** – может и не быть в арк приложение.

About Android App Bundles

Android App Bundle — это формат публикации, который включает в себя весь скомпилированный код и ресурсы вашего приложения, а также откладывает создание APK и подписку в Google Play.

Google Play использует ваш набор приложений для создания и обслуживания оптимизированных APK-файлов для каждой конфигурации устройства, поэтому для запуска вашего приложения загружаются только код и ресурсы, необходимые для конкретного устройства. Вам больше не нужно создавать, подписывать и управлять несколькими APK-файлами, чтобы оптимизировать поддержку различных устройств, а пользователи получают более мелкие и оптимизированные загрузки.

Большинство проектов приложений не требуют больших усилий для создания наборов приложений, поддерживающих обслуживание оптимизированных APK. Например, если вы уже организовали код и ресурсы своего приложения в соответствии с установленными соглашениями, просто создайте подписанные пакеты Android App Bundle с помощью Android Studio или командной строки и загрузите их в Google Play. Оптимизированное обслуживание APK становится автоматическим преимуществом.

Когда вы используете формат пакета приложений для публикации своего приложения, вы также можете при желании воспользоваться функцией Play Feature Delivery, которая позволяет вам добавлять функциональные

модули в проект вашего приложения. Эти модули содержат функции и ресурсы, которые включены в ваше приложение только в соответствии с указанными вами условиями или доступны позже во время выполнения для загрузки с помощью основной библиотеки Play.

Разработчики игр, которые публикуют свои приложения с помощью наборов приложений, могут использовать Play Asset Delivery: решение Google Play для доставки больших объемов игровых ресурсов, которое предлагает разработчикам гибкие методы доставки и высокую производительность.

ProGuard

ProGuard - утилита, которая удаляет из готового кода неиспользуемые фрагменты и изменяет имена переменных и методов для усложнения реверс-инжиниринга приложения. Также позволяет уменьшить размер загружаемых на устройство файлов.

Во время своей работы утилита совершает несколько последовательных шагов.

На первом шаге **Proguard** рекурсивно определяет, какие классы и члены классов (переменные, методы, константы) используются. Все другие классы или члены классов будут удалены из приложения.

Следующий шаг - оптимизация. Proguard может поменять модификаторы классов и методов, удалить неиспользуемые параметры и т.д. Не всегда подобная оптимизация идёт на пользу, поэтому часто этот шаг пропускают.

Важная часть - обфускация. ProGuard переименовывает классы и члены классов, которые не являются точками входа. Точки входа сохраняют свое оригинальное название. Это затрудняет декомпиляцию и исследование работы приложения (reverse engineering).

В Android не используется, но для обычных приложений Java также используется дополнительный шаг - проверка, что код не может случайно или намеренно вырваться из песочницы виртуальной машины Java. Для того, чтобы эта проверка проходила быстрее, компилятор добавляет к файлам классов дополнительную информацию. Соответственно, ProGuard должен в конце своей работы сформировать новую информацию для этой проверки.

В приложении под Android пользовательский интерфейс чаще всего описывается с помощью XML. Часть классов, использующихся при описании интерфейса, нигде в коде не используется. Поэтому, если не сообщить об этих классах ProGuard, они будут удалены. XML с описаниями интерфейса в процессе сборки анализируются аналогично

AndroidManifest.xml, и все перечисленные там классы добавляются в файл конфигурации для ProGuard.

Сформированный файл сохраняется в **app/build/intermediates/proguard-rules/debug/aapt_rules.txt**.

Второй файл конфигурации Proguard, который по умолчанию используется при сборке приложения под Android, приходит в составе SDK. Этот файл явно указан в **build.gradle** нашего приложения.

proguard-android.txt — это дефолтный файл конфигурации, поставляемый с SDK. Экспериментаторы могут заменить его на **proguard-android-optimize.txt**.

proguard-rules.pro — файл, в который предлагается писать свои директивы для ProGuard. Изначально пустой, находится в каталоге приложения.

Вы можете включить ProGuard не только для release, но и для debug версий. Это очень полезно, потому что поведение приложения с включенным ProGuard может отличаться от приложения с выключенным ProGuard совершенно неожиданным образом. Чем раньше все эти неожиданности будут обнаружены и исправлены, тем лучше. За включение ProGuard отвечает директива **minifyEnabled**.

Как получить подпись для приложений Android?

Вы можете сгенерировать ключ разработчика двумя способами:

1. **Генерация ключа подписи с помощью Android Studio**
2. **Генерация ключа подписи с помощью Keytool**

Любое приложение, выкладываемое в магазин, должно иметь подписанный сертификат. Сертификат позволяет идентифицировать вас как автора программы. И если кто-то попытается выложить программу с таким же именем как у вас, то ему будет отказано из-за конфликта имён. Под именем приложения имеется в виду полное название пакета.

Когда вы запускали свои приложения на эмуляторе или своём телефоне, то среда разработки автоматически подписывала программу отладочным сертификатом. Для распространения через магазин отладочный сертификат не подходит, и вам нужно подписать приложение своим уникальным сертификатом.

Build | Generate Signed APK

Далее вы вернётесь обратно и продолжаете заполнять поля. Поля **Password** и **Confirm** в объяснении не нужны.

Теперь создаёте ключ для приложения. В поле **Alias** (Псевдоним) вводите понятное вам и котам название ключа. Не обязательно создавать

псевдоним для каждого приложения, можете использовать один псевдоним для своих приложений и отдельные псевдонимы для приложений под заказ.

Для ключа также нужно создать пароль и подтвердить его.

Ключ рассчитан на 25 лет. Поле **Validity (years)** оставляем без изменений (если у вас нет весомых причин в обратном).

Приложения для Android имеют криптографическую подпись разработчика. С ее помощью менеджер пакетов на устройстве пользователя может проверить, что каждое обновление приложения происходит из одного и того же источника, и что оно не было подделано. Google Play также применяет эту проверку подписи, когда вы загружаете свой APK-файл на консоль Google Play, так что даже если бы у кого-то были ваши учетные данные для входа, было бы невозможно отправить вредоносное обновление, не имея также доступа к вашему закрытому ключу.

В настоящее время у разработчиков есть привлекательная альтернатива для управления ключами самостоятельно: [подписание приложения на Google Play](#), где ключ загрузки (тот, который вы используете, чтобы загрузить свои артефакты в Google Play) и ключ для подписи приложения (который используется для подписи APK-файлов, распространяющихся на устройства) могут быть разными, и второй ключ хранится на Google инфраструктуре.

что такое процесс (**Process**) и что такое поток (**Thread**),

И процесс, и поток относятся к работающему коду, которое выполняется операционной системой Android.

Процесс более широкое логическое понятие, олицетворяющее работающее приложение или службу.

Процесс может объединять в себе несколько потоков, т. е. в одном процессе может работать несколько потоков (в процессе должен быть как минимум 1 поток), т. е. без потока не может быть работающего процесса. Поток — это элементарная единица, выполняющая какие-то действия.

[Процессы (Process)]

По умолчанию все компоненты в одном и том же приложении работают в одном и том же процессе, и для большинства приложений это менять не нужно. Однако если Вы решили, что нужно управлять, какому процессу какой компонент принадлежит, то Вы можете сделать это в файле манифеста приложения.

Запись в манифесте для каждого типа элемента компонента — `< activity >`, `< service >`, `< receiver >` и `< provider >` — поддерживает атрибут `android:process`,

который указывает процесс, которым этот компонент должен работать. Вы можете установить этот атрибут так, что каждый компонент будет работать в своем собственном процессе, или так, что некоторые компоненты будут разделять процесс с другими, в то время как другие не будут этого делать. Вы также можете установить `android:process` так, чтобы компоненты различных приложений работали в одном и том же процессе — при условии, что приложения используют один и тот же идентификатор пользователя (Linux user ID), и что они подписаны одними и теми же сертификатами.

Элемент `< application >` также поддерживает атрибут `android:process` для установки значения по умолчанию, которое будет применено ко всем компонентам.

В некоторой точке Android может решить прибить процесс, когда памяти мало и она требуется другим процессам, которые более интенсивно эксплуатирует пользователь. Компоненты приложения, работающие в уничтоженном процессе, также будут уничтожены. Процесс запустится снова для тех компонентов, которые по какой-то причине должны работать.

Когда система Android принимает решение, какие процессы прибить, она взвешивает относительную их важность для пользователя. Например, вероятнее всего будет закрыт процесс, у которого активности не видны на экране - по сравнению с видимыми активностями. Поэтому решение о завершении процесса зависит от состояния компонентов, работающих в этом процессе. Ниже будут рассмотрены правила, используемые для принятия решения завершения процессов.

[Жизненный цикл процесса]

Система Android пытается удерживать процесс приложения в работе так долго, как это возможно, но в конечном итоге нужно удалять старые процессы, чтобы высвободить память для новых или более важных процессов. Чтобы определить, какой процесс оставить, а какой прибить, система помещает каждый процесс в "список важности", который составляется на базе наличия запущенных в процессе компонентов и состояния этих компонентов. Процессы, у которого самая низкая важность, будут удалены в первую очередь, затем те, которые следующие по уровню значимости, и так далее, пока не будет освобождено нужное количество ресурсов системы.

В иерархии важности есть 5 уровней. Следующий список представляет разные типы процессов в порядке уменьшения важности (первый в списке тип самый важный и будет прибит в последнюю очередь):

1. Foreground process (процесс верхнего уровня)

Если буквально перевести название, то получится "процесс на переднем плане". Это процесс, который должен работать, потому что сейчас с ним активно взаимодействует пользователь. Считается, что процесс является foreground, если верно любое из следующих условий:

- Если в этом процессе размещена активность (Activity [2]), в которой работает пользователь (в Activity был вызван её метод `onResume()`).
- Если в этом процессе размещена служба (**Service** [4]), связанная с активностью, с которой взаимодействует пользователь.
- Если в этом процессе размещена служба (Service), запущенная "на верхнем уровне" (running "in the foreground") — служба вызвала `startForeground()`.
- Если в этом процессе размещена служба (Service), которая выполняет один из методов обратного вызова (callback), обслуживающих жизненный цикл активности (`onCreate()`, `onStart()` или `onDestroy()`).
- Если в этом процессе размещен [BroadcastReceiver](#), который выполняет свой метод `onReceive()`.

Обычно в каждый момент времени только небольшое количество процессов находится в состоянии foreground. Такие процессы будут убиты в последнюю очередь — если памяти станет так мало, что они все не смогут выполняться одновременно. Если такое произошло, то устройство находится в состоянии использования файла подкачки, и будут прибиты некоторые foreground-процессы для того, чтобы сохранить работоспособность интерфейса пользователя.

2. Visible process (видимый процесс)

Процесс, который не имеет каких-то foreground-компонентов, но который содержит нечто, что может повлиять на видимое содержимое экрана. Считается, что процесс является visible, если верно любое из следующих условий:

- Если в этом процессе размещена активность (Activity [2]), которая не находится на переднем плане, но все еще видима для пользователя (был вызван метод `onPause()` активности). Это может произойти, например, если foreground-активность запустила диалог, в результате чего запустившая диалог активность переместилась на экране на задний план, но все еще видна за окном диалога.
- Если в этом процессе размещена служба (Service), связанная с активностью на переднем плане или видимой (foreground или visible).

Visible-процесс считается очень важным, и он не будет прибит, пока это не потребуется для сохранения работоспособности foreground-процессов.

3. Service process (процесс службы)

Процесс, в котором работает служба, запущенная методом `startService()`, и которая не попадает в одну из предыдущих более важных категорий (Foreground или Visible). Поскольку процессы службы могут быть не

привязаны напрямую к тому, что сейчас видит пользователь на экране, они обычно делают вещи, которые все же нужны пользователю (такие как фоновое проигрывание музыки или загрузка данных через сеть). Поэтому система сохранит работу Service-процессов, пока есть ресурсы для работы всех более важных (foreground и visible) процессов.

4. Background process

Если буквально перевести название, то получится "процесс на заднем плане". Это процесс, в котором размещена активность, которая сейчас невидима для пользователя (был вызван метод `onStop()` активности). Эти процессы не влияют напрямую на работу с пользователем, и система убьет их в любой момент, когда нужно освободить память для процессов видов foreground, visible или service. Обычно есть множество запущенных на заднем плане процессов, и они удерживаются в списке LRU (least recently used, используемые недавно). Список LRU нужен для того, чтобы можно было прибавлять Background-процессы, основываясь на порядке, в котором работал с процессами пользователь, т. е. те Background-процессы, с которыми позже всего работал пользователь, будут прибавлены в последнюю очередь. Если в активности корректно реализованы методы жизненного цикла (lifecycle methods, подробнее см. [2]), и активность сохраняет свое текущее состояние, то завершение её процесса пользователь не заметит, потому что когда пользователь вернется обратно к этой активности, активность восстановит все свое видимое состояние. См. документ Activities, раздел посвященный сохранению и восстановлению состояния.

5. Empty process (пустой процесс)

Это процесс, в котором не размещены какие-либо активные компоненты приложения. Причина, по которой эти процессы будут сохранены в памяти - только лишь для кэширования, чтобы уменьшить время загрузки в следующий раз, когда понадобится этот процесс запустить. Система часто убивает эти процессы, чтобы поддерживать баланс общих ресурсов системы между кэшированием процессов и кэшированием нижележащих подсистем ядра.

Android оценивает важность процесса на базе самом важном из компонентов, которые в настоящее время активны в процессе. Например, если в процессе размещена служба и видимая активность, то важность процесса будет visible, а не service.

Кроме того, важность процесса может быть увеличена в зависимости от того, какие процессы зависят от него — процесс, который обслуживает другой процесс, не может быть понижен в ранге ниже, чем процесс, который он обслуживает. Например, если провайдер контента (content provider) в процессе А обслуживает клиента в процессе В, или если служба (service) в процессе А привязана к компоненту в процессе В, то процесс А всегда рассматривается как минимум важным на столько же, как и процесс В.

Поскольку процесс со службой (service) оценивается важнее, чем процесс с background активностями, то активность, которая инициирует длительную операцию, могла бы для улучшения обслуживания пользователя запустить службу (service) для этой длительной операции, вместо того, чтобы просто создать отдельный рабочий поток — особенно если эта операция должна пережить активность. К примеру, если активность выгружает картинку на сайт, то она должна запустить службу для этого действия, чтобы выгрузка могла продолжаться и тогда, когда активность ушла в background (т. е. даже если пользователь оставил эту активность и перешел к другим действиям). Использование службы гарантирует, что операция получит приоритет как минимум процесса службы "service process", независимо от того, что случится с самой активностью. По той же самой причине приемники широковещательных сообщений (broadcast receiver) должны использовать службы, а не просто помещать в поток потребляющие время процессора действия.

[**Потоки (Thread)**]

Когда запускается приложение, система Android создает для него поток, называемый "main". Этот поток очень важен, потому что он отвечает за обработку событий (dispatching events) и направление их к виджетам интерфейса пользователя, включая события отрисовки. Это также поток, в котором Ваше приложение взаимодействует с компонентами тулкита интерфейса пользователя (Android UI toolkit, компоненты из пакетов android.widget и android.view). Также main thread иногда называют потоком интерфейса пользователя (**UI thread**).

Система не создает отдельный поток для каждого экземпляра компонента. Таким образом, все компоненты, запущенные в главном потоке UI, работают в том же самом потоке, и системные вызовы каждого компонента диспетчеризированы из этого потока. Следовательно методы, которые отвечают за системные обратные вызовы (system callbacks, такие как onKeyDown() для сообщения о действиях пользователя, или метод жизненного цикла), всегда работают в потоке UI процесса.

Например, когда пользователь касается кнопки на экране, UI thread приложения перенаправляет событие касания виджету, который в свою очередь устанавливает свое нажатое состояние (pressed state), и помещает запрос в очередь событий. Затем UI thread обрабатывает запрос и оповещает виджет о том, что он должен перерисовать сам себя.

Когда Ваше приложение выполняет интенсивную работу в ответ на взаимодействие с пользователем, эта однопоточная модель может привести к низкой производительности, если Вы не построите свое приложение должным образом. В частности, если все происходит в потоке интерфейса (UI thread), то долгие операции наподобие доступа к сети или запросы к базе данных будут блокировать весь пользовательский интерфейс. Когда поток UI заблокирован (например на

циклах или ожидании), то события интерфейса не могут быть обработаны, включая события перерисовки. С точки зрения пользователя приложение зависнет. Еще хуже, если UI будет заблокирован больше, чем несколько секунд (в настоящее время этот таймаут составляет около 5 секунд), тогда пользователю будет представлено позорное окно диалога **ANR** (application not responding, приложение не отвечает). Пользователь может решить выйти из программы, и даже деинсталлировать её, потому что программа работает странно и его это не устраивает [3].

Кроме того, **Android UI toolkit** не является потокобезопасным (not **thread-safe**). Так что Вы не должны манипулировать интерфейсом пользователя из дополнительного рабочего потока (**worker thread**) — все манипуляции с графикой интерфейса должны происходить только из UI thread. Таким образом, отсюда вытекают 2 правила для однопоточной модели приложения Android:

1. Нельзя блокировать выполнение потока обработки интерфейса пользователя (UI thread).
2. Нельзя обращаться к Android UI toolkit из других потоков. Можно работать с Android UI toolkit только из кода UI thread.

[Дополнительные рабочие потоки (Worker thread)]

По причине применения вышеописанной однопоточной модели, для скорости отклика UI приложения жизненно важно, чтобы Вы не блокировали выполнение UI thread. Если Вам нужно выполнить операции, которые не завершатся немедленно, то нужно их перенести в отдельные потоки (так называемые "background", фоновые потоки, или "worker", рабочие потоки).

5. Android 1. что это такое, как показать и для чего
Features Notifications используются

Уведомление — это сообщение, которое Android отображает за пределами пользовательского интерфейса вашего приложения, чтобы предоставить пользователю напоминания, сообщения от других людей или другую своевременную информацию из вашего приложения.

Уведомления отображаются в виде значка в строке состояния, подробной записи в панели уведомлений, в виде значка (например счетчик новых сообщений) на значке приложения и на спаренных носимых устройствах автоматически.

У уведомлений в панели уведомлений могут быть кнопки для взаимодействия, например для плеера кнопка паузы и переключения трека, для мессенджера кнопка для быстрого ответа прямо из уведомлений, или просто крестик для закрытия приложения

Heads-up notification Предупреждающее уведомление

Это когда уведомления могут на короткое время появляться в плавающем окне, называемом хедз-апом.

Например, пришло новое сообщение, или аккумулятор разряжен при этом это уведомление появляется только на разблокированном экране

Потом хедз ап исчезает и остается в панели уведомлений

ЛОК СКРИН НОТИФИКАЙШН – уведомления при ЗАБЛОКИРОВАННОМ ЭКРАНЕ

Вы можете программно установить уровень детализации уведомлений, отправляемых вашим приложением на безопасном экране блокировки, или даже то, будет ли уведомление отображаться на экране блокировки вообще.

Пользователи могут использовать настройки системы, чтобы выбрать уровень детализации, отображаемый в уведомлениях на заблокированном экране, включая возможность отключения всех уведомлений на заблокированном экране.

App icon badge

это уведомление на иконке, например количество новых сообщений

значки приложений обозначают новые уведомления с помощью цветного «значка» (также известного как «точка уведомления») на соответствующем значке средства запуска приложения.

Пользователи могут долгим нажатием на значок увидеть уведомления для этого приложения.

Затем пользователи могут отклонять уведомления или действовать в соответствии с ними из этого меню, как и в панели уведомлений.

Notification anatomy

The design of a notification is determined by system templates—your app simply defines the contents for each portion of the template. Some details of the notification appear only in the expanded view.

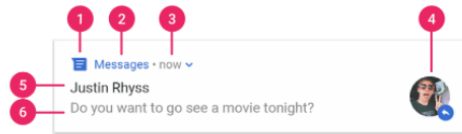


Figure 7. A notification with basic details

The most common parts of a notification are indicated in figure 7 as follows:

- 1 Small icon: This is required and set with `setSmallIcon()`.
- 2 App name: This is provided by the system.
- 3 Time stamp: This is provided by the system but you can override with `setWhen()` or hide it with `setShowWhen(false)`.
- 4 Large icon: This is optional (usually used only for contact photos; do not use it for your app icon) and set with `setLargeIcon()`.
- 5 Title: This is optional and set with `setContentTitle()`.
- 6 Text: This is optional and set with `setContentText()`.

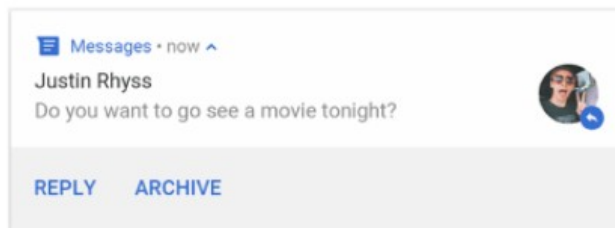
We strongly recommend using system templates to ensure proper design compatibility on all devices. If necessary, however, you can [create a custom notification layout](#).

Notification actions

Действия с уведомлением

Хотя это не обязательно, каждое уведомление должно открывать соответствующее действие приложения при нажатии.

В дополнение к этому действию уведомления по умолчанию вы можете добавить кнопки действий, которые завершают задачу, связанную с приложением, из уведомления (часто без открытия действия), как показано на рисунке



Require an unlocked device

Требовать разблокированное устройство

При попытке запустить действие на уведомлении появится запрос на разблокировку устройства

Чтобы потребовать разблокировку устройства до того, как ваше приложение вызовет заданное действие уведомления, передайте `true` в `setAuthenticationRequired()`

```
val moreSecureNotification = Notification.Builder ( context,  
NotificationListenerVerifierActivity.TAG )
```

```
.addAction (...)
```

// Это уведомление всегда запрашивает аутентификацию при вызове с экрана блокировки.

```
.setAuthenticationRequired (true)
```

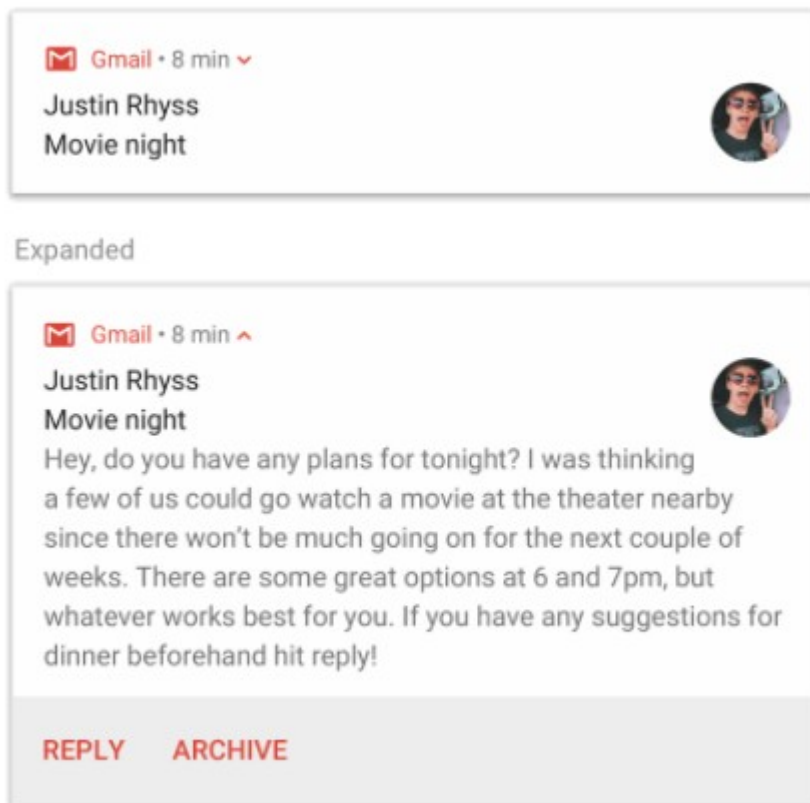
```
.build ()
```

Expandable notification

Расширяемое уведомление

По умолчанию текстовое содержимое уведомления обрезается до размера одной строки.

Если вы хотите, чтобы ваше уведомление было длиннее, вы можете включить большую текстовую область, которую можно расширить, применив дополнительный шаблон, как показано на рисунке



Notification updates and groups Уведомления об обновлениях и группах

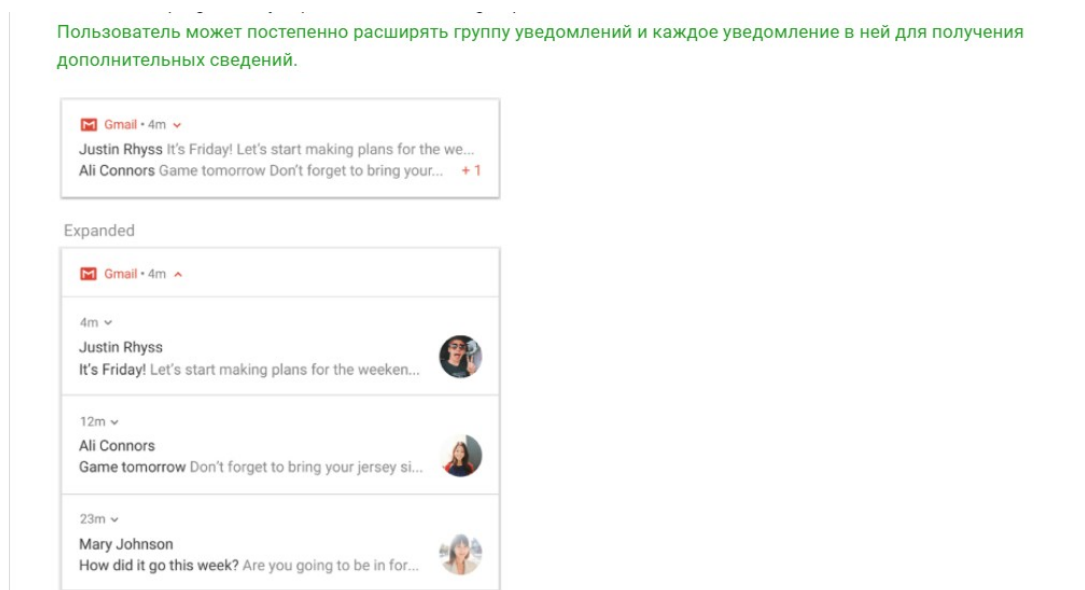
Чтобы избежать бомбардировки ваших пользователей множественными или избыточными уведомлениями,

можно сделать в стиле почтового ящика для отображения обновлений беседы.

Однако, если необходимо доставить несколько уведомлений, вам следует подумать о том, чтобы сгруппировать эти отдельные уведомления в группу

Группа уведомлений позволяет вам свернуть несколько уведомлений в одну публикацию в панели уведомлений со сводкой.

Затем пользователь может развернуть уведомление, чтобы раскрыть подробности каждого отдельного уведомления.

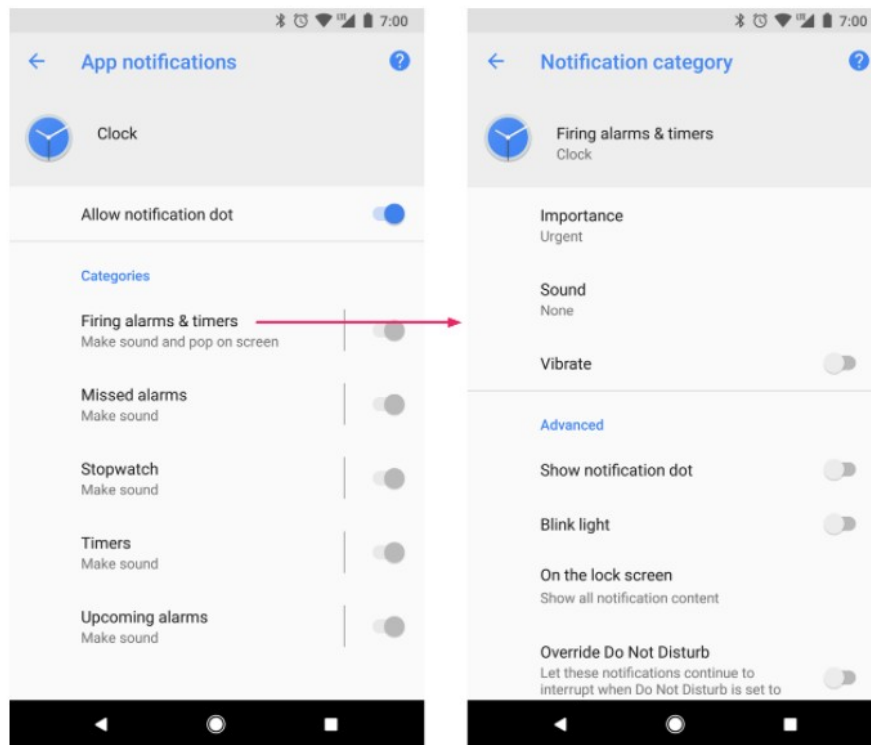


Примечание. Если одно и то же приложение отправляет четыре или более уведомлений и не указывает группировку, система автоматически группирует их вместе.

Notification channels Каналы уведомлений

все уведомления должны быть назначены каналу, иначе он не появится.

Распределяя уведомления по каналам, пользователи могут отключать определенные каналы уведомлений для вашего приложения (вместо отключения всех ваших уведомлений), а пользователи могут управлять визуальными и слуховыми параметрами для каждого канала - и все это в системных настройках Android



Примечание. В пользовательском интерфейсе каналы называются «категориями».

Одно приложение может иметь несколько каналов уведомлений - отдельный канал для каждого типа уведомлений, которые выдает приложение.

Приложение также может создавать каналы уведомлений в ответ на выбор, сделанный пользователями вашего приложения.

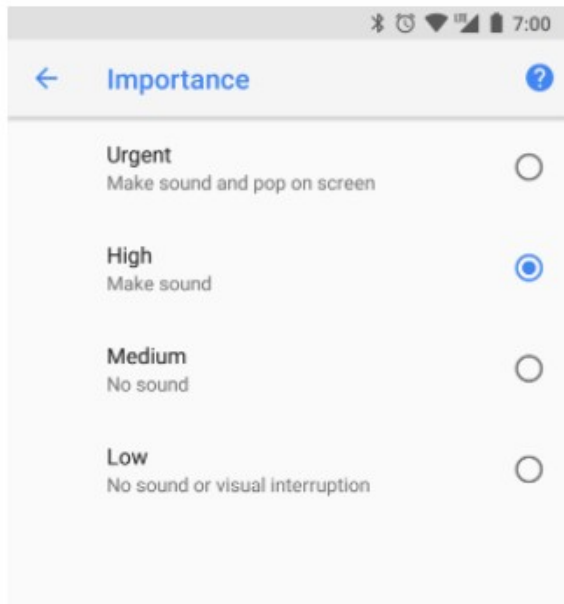
Например, вы можете настроить отдельные каналы уведомлений для каждой группы разговоров, созданной пользователем в приложении для обмена сообщениями.

В этом канале вы также указываете уровень важности для ваших уведомлений на Android 8.0 и выше.

Notification importance Важность уведомления

Android использует важность уведомления, чтобы определить, насколько уведомление должно прерывать пользователя (визуально и звуком).

Пользователи могут изменить важность канала уведомлений в настройках системы (рисунок

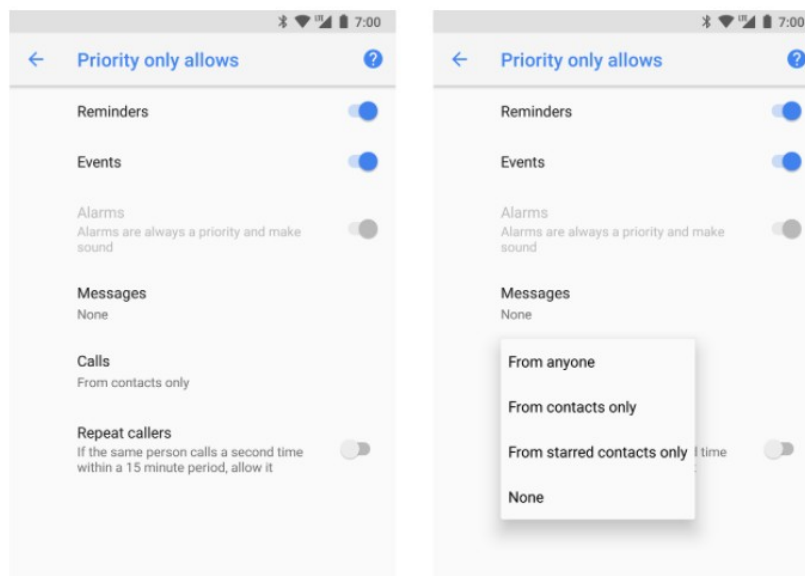


Do Not Disturb mode Режим 'Не беспокоить'

пользователи могут включить режим «Не беспокоить», который отключает звуки и вибрацию для всех уведомлений.

В режиме «Не беспокоить» доступны три разных уровня:

•



Tot

al silence: blocks all sounds and vibrations, including from alarms, music, videos, and games.

Полная тишина: блокирует все звуки и вибрации, в том числе от сигналов будильника, музыки, видео и игр.

- Alarms only: blocks all sounds and vibrations, except from alarms.
Только будильники: блокирует все звуки и вибрации, кроме сигналов будильника.
- Priority only: users can configure which system-wide categories can interrupt them (such as only alarms, reminders, events, calls, or messages).
Только приоритет: пользователи могут настроить, какие общесистемные категории могут их прерывать (например, только сигналы тревоги, напоминания, события, звонки или сообщения). Для сообщений и звонков пользователи также могут выбрать фильтрацию на основе отправителя или вызывающего абонента
рисунок

Notifications for foreground services Уведомления для служб переднего плана

Уведомление требуется, когда ваше приложение запускает «службу переднего плана» - службу, работающую в фоновом режиме, которая долговечна и заметна для пользователя, например, медиаплеер.

Это уведомление нельзя закрыть, как другие уведомления.

Чтобы удалить уведомление, службу необходимо остановить или вывести из состояния «переднего плана»

Posting limits Ограничения на публикацию

приложения не могут издавать звуковое уведомление чаще одного раза в секунду.

Set the notification content

To get started, you need to set the notification's content and channel using a `NotificationCompat.Builder` object. The following example shows how to create a notification with the following:

- A small icon, set by `setSmallIcon()`. This is the only user-visible content that's required.
- A title, set by `setContentTitle()`.
- The body text, set by `setContentText()`.
- The notification priority, set by `setPriority()`. The priority determines how intrusive the notification should be on Android 7.1 and lower. (For Android 8.0 and higher, you must instead set the channel importance—shown in the next section.)

```
Kotlin      Java
var builder = NotificationCompat.Builder(this, CHANNEL_ID)
    .setSmallIcon(R.drawable.notification_icon)
    .setContentTitle(textTitle)
    .setContentText(textContent)
    .setPriority(NotificationCompat.PRIORITY_DEFAULT)
```

Notice that the `NotificationCompat.Builder` constructor requires that you provide a channel ID. This is required for compatibility with Android 8.0 (API level 26) and higher, but is ignored by older versions.

Set the notification's tap action

Every notification should respond to a tap, usually to open an activity in your app that corresponds to the notification. To do so, you must specify a content intent defined with a `PendingIntent` object and pass it to `setContentIntent()`.

The following snippet shows how to create a basic intent to open an activity when the user taps the notification:

```
Kotlin  Java

// Create an explicit intent for an Activity in your app
val intent = Intent(this, AlertDetails::class.java).apply {
    flags = Intent.FLAG_ACTIVITY_NEW_TASK or Intent.FLAG_ACTIVITY_CLEAR_TASK
}
val pendingIntent: PendingIntent = PendingIntent.getActivity(this, 0, intent, 0)

val builder = NotificationCompat.Builder(this, CHANNEL_ID)
    .setSmallIcon(R.drawable.notification_icon)
    .setContentTitle("My notification")
    .setContentText("Hello World!")
    .setPriority(NotificationCompat.PRIORITY_DEFAULT)
// Set the intent that will fire when the user taps the notification
    .setContentIntent(pendingIntent)
    .setAutoCancel(true)
```

Notice this code calls `setAutoCancel()`, which automatically removes the notification when the user taps it.

The `setFlags()` method shown above helps preserve the user's expected navigation experience after they open your app via the notification. But whether you want to use that depends on what type of activity you're starting, which may be one of the following:

- An activity that exists exclusively for responses to the notification. There's no reason the user would navigate to this activity during normal app use, so the activity starts a new task instead of being added to your app's existing task and back stack. This is the type of intent created in the sample above.

Show the notification

To make the notification appear, call `NotificationManagerCompat.notify()`, passing it a unique ID for the notification and the result of `NotificationCompat.Builder.build()`. For example:

```
Kotlin  Java

with(NotificationManagerCompat.from(this)) {
    // notificationId is a unique int for each notification that you must define
    notify(notificationId, builder.build())
}
```

Remember to save the notification ID that you pass to `NotificationManagerCompat.notify()` because you'll need it later if you want to update or remove the notification.

6. Technologies

1. Network

основа работы с Retrofit, что такое JSON, какой http-client используется в Retrofit, что такое Interceptor, как объявлять запросы (POST, GET) с помощью Retrofit, коды состояний HTTP

Retrofit упрощает взаимодействие с REST API сайта, беря на себя часть рутинной работы.

REST расшифровывается как REpresentational State Transfer. Это был термин, первоначально введен Роем Филдингом (Roy Fielding), который также был одним из создателей протокола HTTP. Отличительной особенностью сервисов REST является то, что они позволяют наилучшим образом использовать протокол HTTP.

Авторами библиотеки **Retrofit** являются разработчики из компании "Square", которые написали множество полезных библиотек, например, [Picasso](#), [Okhttp](#), [Otto](#).

Библиотекой удобно пользоваться для запроса к различным веб-сервисам с командами GET, POST, PUT, DELETE. Может работать в асинхронном режиме, что избавляет от лишнего кода. HTTP, GET, POST, PUT, PATCH, DELETE, OPTIONS, HEAD

В основном вам придётся работать с методами GET и POST. Если вы будете создавать собственный API, то будете использовать и другие команды.

Библиотека может работать с GSON и XML, используя специальные конвертеры, которые следует указать отдельно.

в коде конвертер добавляется с помощью метода **addConverterFactory()**.

Список готовых конвертеров:

- Gson: com.squareup.retrofit2:converter-gson
- Jackson: com.squareup.retrofit2:converter-jackson
- Moshi: com.squareup.retrofit2:converter-moshi
- Protobuf: com.squareup.retrofit2:converter-protobuf
- Wire: com.squareup.retrofit2:converter-wire
- Simple XML: com.squareup.retrofit2:converter-simplexml
- Scalars (primitives, boxed, and String): com.squareup.retrofit2:converter-scalars

Также вы можете создать свой собственный конвертер, реализовав интерфейс на основе абстрактного класса **Converter.Factory**.

Можно подключить несколько конвертеров (порядок важен).

```
Retrofit retrofit = Retrofit.Builder()
    .baseUrl("https://your.api.url/v2/");
    .addConverterFactory(PrettyConverterFactory.create())
    .addConverterFactory(GsonConverterFactory.create())
    .build();
```

Если вы хотите изменить формат какого-нибудь JSON-объекта, то это можно сделать с помощью **GsonConverterFactory.create()**:

```
Gson gson = new GsonBuilder()
    .setDateFormat("yyyy-MM-dd'T'HH:mm:ssZ")
    .create();

Retrofit retrofit = new Retrofit.Builder()
    .baseUrl("http://api.example.com/base/")
    .addConverterFactory(GsonConverterFactory.create(gson))
    .build();
```

```
service = retrofit.create(APIService.class);
```

Базовый URL всегда заканчивается слешем /. Задаётся в методе **baseUrl()**.

Можно указать полный URL в запросе, тогда базовый URL будет проигнорирован:

```
public interface ApiService {  
    @GET("http://api.sample.com/users/user/list")  
    Call<Users> getUsers();  
}
```

Retrofit против OkHttp Причина проста : OkHttp-это чистый клиент HTTP/SPDY, отвечающий за любые низкоуровневые сетевые операции, кэширование, манипуляции с запросами и ответами и многое другое. Напротив, Retrofit-это высокоуровневая абстракция REST, построенная поверх OkHttp. Retrofit 2 сильно связана с OkHttp и интенсивно использует ее.

OkHttp Функции: объединение соединений, gzipping, кэширование, восстановление после сетевых проблем, синхронизация и асинхронные вызовы, перенаправление, повторные попытки ... и так далее.

Retrofit Функции: URL манипулирование, запрос, загрузка, кэширование, потоковая передача, синхронизация... Он позволяет синхронизировать и асинхронные вызовы.

HTTP, GET,

POST подстановка переменных для конечной точки API (т.е. имя пользователя будет заменено на {username} в конечной точке URL),

PUT, PATCH, DELETE, OPTIONS,

HEAD указывает заголовок со значением аннотированного параметра

@Query указывает имя ключа запроса со значением аннотированного параметра.

Можно использовать замещающие блоки и параметры запроса для настройки URL-адреса. Замещающий блок добавляется к относительному URL-адресу с помощью {}. С помощью аннотации @ Path для параметра метода значение этого параметра привязывается к конкретному замещающему блоку.

```
@GET("users/{name}/commits")  
Call<List<Commit>> getCommitsByName(@Path("name") String name);
```

Параметры запроса добавляются с помощью аннотации @ Query к параметру метода. Они автоматически добавляются в конце URL-адреса.

```
@GET("users")  
Call<User> getUserById(@Query("id") Integer id);
```

Аннотация `@Body` к параметру метода говорит Retrofit использовать объект в качестве тела запроса для вызова.

```
@POST("users")
Call<User> postUser(@Body User user)
```

Retrofit Адаптеры

Retrofit также может быть расширен адаптерами для взаимодействия с другими библиотеками, такими как RxJava 2.x, Java 8 и Guava.

Retrofit аутентификация

Retrofit поддерживает вызовы API, требующие аутентификации. Аутентификацию можно выполнить, используя имя пользователя и пароль (аутентификация Http Basic) или API токен.

Существует два способа управления аутентификацией. Первый метод — управлять заголовком запроса с помощью аннотаций. Другой способ — использовать для этого OkHttp перехватчик.

5.1. Аутентификация с аннотациями

Предположим, что вы хотите запросить информацию о пользователе, для которой требуется аутентификация. Вы можете сделать это, добавив новый параметр в определение API, например:

```
@GET("user")
Call<UserDetails> getUserDetails(@Header("Authorization") String credentials)
```

С помощью аннотации `@Header("Authorization")` вы говорите Retrofit добавить заголовок `Authorization` в запрос со значением, которое вы передаете.

Для работы с **Retrofit** понадобятся три класса.

1. POJO (Plain Old Java Object) или Model Class - json-ответ от сервера нужно реализовать как модель
2. Retrofit - класс для обработки результатов. Ему нужно указать базовый адрес в методе `baseUrl()`

3. Interface - интерфейс для управления адресом, используя команды GET, POST и т.д.

Работу с **Retrofit** можно разбить на отдельные задачи.

Задача первая. POJO

Задача первая - посмотреть на структуру ответа сайта в виде JSON (или других форматов) и создать на его основе Java-класс в виде POJO.

POJO удобнее создавать с помощью готовых веб-сервисов в автоматическом режиме. Либо можете самостоятельно создать класс, если структура не слишком сложная.

В классе часто используются аннотации. Иногда они необходимы, иногда их можно пропустить. В некоторых случаях аннотации помогают избежать ошибок. Список аннотаций зависит от типа используемого конвертера, их список можно посмотреть в соответствующей документации.

Задача вторая. Интерфейс

Задача вторая - создать интерфейс и указать имя метода. Добавить необходимые параметры, если они требуются.

В интерфейсе задаются команды-запросы для сервера. Команда комбинируется с базовым адресом сайта (**baseUrl()**) и получается полный путь к странице. Код может быть простым и сложным. Можно посмотреть примеры в документации.

Запросы размещаются в обобщённом классе **Call** с указанием желаемого типа.

```
import retrofit2.Call;

public interface APIService {
    @POST("list")
    Call<Repo> loadRepo();
}
```

В большинстве случаев вы будете возвращать объект **Call<T>** с нужным типом, например, **Call<User>**. Если вас не интересует тип ответа, то можете указать **Call<Response>**.

В большинстве случаев вы будете возвращать объект **Call<T>** с нужным типом, например, **Call<User>**. Если вас не интересует тип ответа, то можете указать **Call<Response>**.

Здесь также используются аннотации, но уже от самой библиотеки.

С помощью аннотации указываются веб-команды, а затем Java-метод. Для динамических параметров используются фигурные скобки (**users/{user}/repos**), в которые подставляются нужные значения.

В самой аннотации используется метод, используемый на сервере, а ниже вы можете указать свой вариант (полезно для соответствия стилю вашего кода).

```
@GET("get_all_cats") // команда на сервере
List<Cat> getAllCats(); // ваш код
```

Аннотации

Аннотация	Описание
@GET()	GET-запрос для базового адреса. Также можно указать параметры в скобках
@POST()	POST-запрос для базового адреса. Также можно указать параметры в скобках
@Path	Переменная для замещения конечной точки, например, username подставится в {username} в адресе конечной точки
@Query	Задаёт имя ключа запроса со значением параметра
@Body	Используется в POST-вызовах (из Java-объекта в JSON-строку)
@Header	Задаёт заголовок со значением параметра
@Headers	Задаёт все заголовки вместе
@Multipart	Используется при загрузке файлов или изображений
@FormUrlEncoded	Используется при использовании пары "имя/значение" в POST-запросах
@FieldMap	Используется при использовании пары "имя/значение" в POST-запросах
@Url	Для поддержки динамических адресов



@Query

Аннотация **@Query** полезна при запросах с параметрами. Допустим, у сайте есть дополнительный параметр к запросу, который выводит список элементов в отсортированном виде: **http://example.com/api/v1/products/cats?sort=desc**. Это несложный пример, и мы можем поместить запрос с параметром в интерфейс без изменений.

```
@GET("products/cats?category=5&sort=desc")
Call<Cats> getAllCats();
```

Если не требуется управлять сортировкой, то её можно оставить в коде и она будет применяться по умолчанию. Но в нашем запросе есть ещё один параметр, который отвечает за категорию котов (домашние, уличные, породистые), которая может меняться в зависимости от логики приложения. Этот параметр можно снабдить аннотацией и программно управлять в коде.

```
@GET("products/cats?sort=desc")
Call<Cats> getAllCats(@Query("category") int categoryId);
```

Сортировку мы оставляем как есть, а категорию перенесли в параметры метода под именем **categoryId**, снабдив аннотацией, с которой параметр будет обращаться на сервер в составе запроса.

```
Call<Cats> getAllCats() = catAPIService.getAllCats(5);
```

Запрос получится в виде **http://example.com/api/v1/products/cats?sort=desc&category=5**.

В одном методе можно указать несколько Query-параметров.

@Path

Запрос может иметь изменяемые части пути. Посмотрите на один из примеров запроса для GitHub: **/users/:username**.

Вместо **:username** следует подставлять конкретные имена пользователей (<https://api.github.com/users/alexanderklimov>). В таких случаях используют фигурные скобки в запросе, в самом методе через аннотацию **@Path** указывается имя, которое будет подставляться в путь.

```
@GET("/users/{username}")
Call getUser(
    @Path("username") String userName
);
```

@Headers

Пример аннотации **@Headers**, которая позволяет указать все заголовки вместе.

```
@Headers({"Cache-Control: max-age=640000", "User-Agent: My-App-Name"})
@GET("some/endpoint")
```

@Multipart

Пример аннотации **@Multipart** при загрузке файлов или картинок:

```
@Multipart
@POST("some/endpoint")
Call<Response> uploadImage(@Part("description") String description, @Part("image") RequestBody image)
```

@FormUrlEncoded

Пример использования аннотации **@FormUrlEncoded**:

```
@FormUrlEncoded
@POST("/some/endpoint")
Call<SomeResponse> someEndpoint(@FieldMap Map<String, String> names);
```



@Url

Пример аннотации **@Url**:

```
public interface UserService {
    @GET
    public Call<File> getZipFile(@Url String url);
}
```

Задача третья. Retrofit

Для синхронного запроса используйте метод **Call.execute()**, для асинхронного - метод **Call.enqueue()**.

Объект для запроса к серверу создаётся в простейшем случае следующим образом


```
public static final String BASE_URL = "http://api.example.com/";

Retrofit retrofit = new Retrofit.Builder()
    .baseUrl(BASE_URL)
    .addConverterFactory(GsonConverterFactory.create())
    .build();
```

В итоге мы получили объект **Retrofit**, содержащий базовый URL и способность преобразовывать JSON-данные с помощью указанного конвертера **Gson**.

Далее в его методе **create()** указываем наш класс интерфейса с запросами к сайту.

```
UserService userService = retrofit.create(UserService.class);
```

После этого мы получаем объект **Call** и вызываем метод **enqueue()** (для асинхронного вызова) и создаём для него **Callback**. Запрос будет выполнен в отдельном потоке, а результат придет в **Callback** в **main**-потоке.

В результате библиотека **Retrofit** сделает запрос, получит ответ и произведёт разбор ответа, раскладывая по полочкам данные. Вам остаётся только вызывать нужные методы класса-модели для извлечения данных.

Основная часть работы происходит в **onResponse()**, ошибки выводятся в **onFailure()** (неправильный адрес сервера, некорректные формат данных, неправильный формат класса-модели и т.п). HTTP-коды сервера (например, 404) не относятся к ошибкам.

- **code()** - HTTP-код ответа
- **body()** - сам ответ в виде строки, без сериализации
- **headers()** - HTTP-заголовки
- **message()** - HTTP-статус (или null)
- **raw()** — сырой HTTP-ответ

Мет

од **onResponse()** вызывается всегда, даже если запрос был неуспешным. Класс **Response** имеет удобный метод **isSuccessful()** для успешной обработки запроса (коды 200xx). В ошибочных ситуациях вы можете обработать ошибку в методе **errorBody()** класса **ResponseBody**.

Другие полезные методы **Response**.

Перехватчики (Interceptors)

В библиотеку можно внедрить перехватчики для изменения заголовков при помощи класса **Interceptor** из **OkHttp**. Сначала следует создать объект перехватчика и передать его в **OkHttp**, который в свою очередь следует явно подключить в **Retrofit.Builder** через метод **client()**.

Поддержка *перехватчиков/interceptors* для обработки заголовков запросов, например, для работы с токенами авторизации в заголовке **Authorization**.

```
OkHttpClient client = new OkHttpClient();
client.interceptors().add(new Interceptor() {
    @Override
    public Response intercept(Chain chain) throws IOException {
        Request original = chain.request();

        // Настраиваем запросы
        Request request = original.newBuilder()
            .header("Accept", "application/json")
            .header("Authorization", "auth-token")
            .method(original.method(), original.body())
            .build();

        Response response = chain.proceed(request);

        return response;
    }
});

Retrofit retrofit = Retrofit.Builder()
    .baseUrl("https://your.api.url/v2/")
    .client(client)
    .build();
```

HttpLoggingInterceptor

Библиотека **HttpLoggingInterceptor** является частью **OkHttp**, но поставляется отдельно от неё. Перехватчик следует использовать в том случае, когда вам действительно нужно изучать логи ответов сервера. По сути библиотека является сетевым аналогом привычного **LogCat**.

Подключаем библиотеку.

Подключаем перехватчик к веб-клиенту. Добавляйте его после других перехватчиков, чтобы ловить все сообщения. Существует несколько уровней перехвата данных: **NONE**, **BASIC**, **HEADERS**, **BODY**. Последний вариант

самый информативный, пользуйтесь им осторожно. При больших потоках данных информация забьёт весь экран. Используйте промежуточные варианты.

```
HttpLoggingInterceptor loggingInterceptor = new HttpLoggingInterceptor();

// Только в режиме отладки
if(BuildConfig.DEBUG){
    loggingInterceptor.setLevel(HttpLoggingInterceptor.Level.BODY );
}

OkHttpClient okClient = new OkHttpClient.Builder()
    .addInterceptor(new ResponseInterceptor())
    .addInterceptor(loggingInterceptor)
    .build();
```

Коды состояний HTTP

1xx: Information

В этот класс выделены коды, информирующие о процессе передачи. Это обычно предварительный ответ, состоящий только из Status-Line и опциональных заголовков, и завершается пустой строкой. Нет обязательных заголовков для этого класса кодов состояния. Из-за того, что стандарт протокола HTTP/1.0 не определял никаких 1xx кодов состояния, серверы НЕ ДОЛЖНЫ посылать 1xx ответы HTTP/1.0 клиентам, за исключением экспериментальных условий.

Клиент должен быть готов принять один или несколько 1xx ответов статуса до завершения передачи, даже если клиент не ожидает статуса 100 (Продолжить). Неожиданные 1xx ответы могут быть проигнорированы агентом пользователя.

2xx: Success

Этот класс кодов состояния указывает, что запрос клиента был успешно получен, понят и принят.

3xx: Redirect

Данный класс кодов состояния показывает, что дальнейшие действия должны быть выполнены клиентом для того, чтобы запрос завершился успешно.

коды перенаправления говорят клиенту о том, что для успешного завершения запроса необходимо выполнить какое-то действие. Обычно браузеры выполняют такие действия без вмешательства пользователя

4xx: Client Error

Класс кодов 4xx предназначен для определения ошибок, которые произошли на стороне клиента. Кроме случая, когда метод запроса был HEAD, серверу СЛЕДУЕТ включить в ответ сущность, которая содержит в себе объяснение ситуации, которая привела к ошибке, и является ли это временным или постоянным условием. Эти коды применимы к любому методу запроса. Пользовательским агентам СЛЕДУЕТ отображать пользователю включённую в такой ответ сущность.

Если клиент передает данные, то серверу, использующему TCP СЛЕДУЕТ убедиться, что клиент подтверждает отправку пакетов, содержащих ответ, до того, как сервер закроет соединение. Если клиент продолжит отправку данных после того, как сервер закрыл соединение, TCP стек сервера отправит пакет с флагом сброса (TCP reset packet), который может уничтожить клиентские данные, находящиеся в неподтвержденных входных буферах, до того, как они будут прочитаны и обработаны HTTP приложением.

5xx: Server Error

Коды ответов, начинающиеся с "5" указывают на случаи, когда сервер знает, что произошла ошибка или он не может обработать запрос. Кроме HEAD-запросов, серверу следует включить в ответ сущность, содержащую объяснение ошибки, и является ли это временным или постоянным состоянием. Агентам СЛЕДУЕТ показывать включённую сущность пользователям. Этот код ответа подходит для любых методов запросов.

Код состояния HTTP ([англ. HTTP status code](#)) — часть первой строки ответа сервера при запросах по протоколу [HTTP](#). Первая цифра указывает на **класс состояния**.

- [1xx: Informational](#) (информационные):
 - [100 Continue](#) («продолжай»)^{[2][3]};
 - [101 Switching Protocols](#) («переключение протоколов»)^{[2][3]};
 - [102 Processing](#) («идёт обработка»);
 - [103 Early Hints](#) («ранняя метаинформация»);
- [2xx: Success](#) (успешно):
 - [200 OK](#) («хорошо»)^{[2][3]};
 - [201 Created](#) («создано»)^{[2][3][4]};
 - [202 Accepted](#) («принято»)^{[2][3]};

- [203 Non-Authoritative Information](#) («информация не авторитетна»)^{[2][3]};
 - [204 No Content](#) («нет содержимого»)^{[2][3]};
 - [205 Reset Content](#) («сбросить содержимое»)^{[2][3]};
 - [206 Partial Content](#) («частичное содержимое»)^{[2][3]};
 - [207 Multi-Status](#) («многостатусный»)[[^]\[5\]^](https://ru.wikipedia.org/wiki/%D0%A1%D0%BF%D0%B8%D1%81%D0%BE%D0%BA_%D0%BA%D0%BE%D0%B4%D0%BE%D0%B2_%D1%81%D0%BE%D1%81%D1%82%D0%BE%D1%8F%D0%BD%D0%B8%D1%8F_HTTP#cite_note-webdav_10-5);
 - [208 Already Reported](#) («уже сообщалось»)[[^]\[6\]^](https://ru.wikipedia.org/wiki/%D0%A1%D0%BF%D0%B8%D1%81%D0%BE%D0%BA_%D0%BA%D0%BE%D0%B4%D0%BE%D0%B2_%D1%81%D0%BE%D1%81%D1%82%D0%BE%D1%8F%D0%BD%D0%B8%D1%8F_HTTP#cite_note-6);
 - [226 IM Used](#) («использовано IM»).
- [3xx: Redirection](#) (перенаправление):
 - [300 Multiple Choices](#) («множество выборов»)^{[2][7]};
 - [301 Moved Permanently](#) («перемещено навсегда»)^{[2][7]};
 - [302 Moved Temporarily](#) («перемещено временно»)^{[2][7]}, [302 Found](#) («найдено»)[[^]\[7\]^](https://ru.wikipedia.org/wiki/%D0%A1%D0%BF%D0%B8%D1%81%D0%BE%D0%BA_%D0%BA%D0%BE%D0%B4%D0%BE%D0%B2_%D1%81%D0%BE%D1%81%D1%82%D0%BE%D1%8F%D0%BD%D0%B8%D1%8F_HTTP#cite_note-rfc2616_10_3-7);
 - [303 See Other](#) («смотреть другое»)^{[2][7]};
 - [304 Not Modified](#) («не изменялось»)^{[2][7]};
 - [305 Use Proxy](#) («использовать прокси»)^{[2][7]};
 - [306](#) — *зарезервировано* (код использовался только в ранних спецификациях)[[^]\[7\]^](https://ru.wikipedia.org/wiki/%D0%A1%D0%BF%D0%B8%D1%81%D0%BE%D0%BA_%D0%BA%D0%BE%D0%B4%D0%BE%D0%B2_%D1%81%D0%BE%D1%81%D1%82%D0%BE%D1%8F%D0%BD%D0%B8%D1%8F_HTTP#cite_note-rfc2616_10_3-7);
 - [307 Temporary Redirect](#) («временное перенаправление»)[[^]\[7\]^](https://ru.wikipedia.org/wiki/%D0%A1%D0%BF%D0%B8%D1%81%D0%BE%D0%BA_%D0%BA%D0%BE%D0%B4%D0%BE%D0%B2_%D1%81%D0%BE%D1%81%D1%82%D0%BE%D1%8F%D0%BD%D0%B8%D1%8F_HTTP#cite_note-rfc2616_10_3-7);

- [308 Permanent Redirect](#) («постоянное перенаправление»)[[^][8\]^]
(https://ru.wikipedia.org/wiki/%D0%A1%D0%BF%D0%B8%D1%81%D0%BE%D0%BA_%D0%BA%D0%BE%D0%B4%D0%BE%D0%B2_%D1%81%D0%BE%D1%81%D1%82%D0%BE%D1%8F%D0%BD%D0%B8%D1%8F_HTTP#cite_note-8).
- **4xx: Client Error** (ошибка клиента):
 - [400 Bad Request](#) («неправильный, некорректный запрос»)[²][³][⁴];
 - [401 Unauthorized](#) («не авторизован (не представился)»)[²][³];
 - [402 Payment Required](#) («необходима оплата»)[²][³];
 - [403 Forbidden](#) («запрещено (не уполномочен)»)[²][³];
 - [404 Not Found](#) («не найдено»)[²][³];
 - [405 Method Not Allowed](#) («метод не поддерживается»)[²][³];
 - [406 Not Acceptable](#) («неприемлемо»)[²][³];
 - [407 Proxy Authentication Required](#) («необходима аутентификация прокси»)[²][³];
 - [408 Request Timeout](#) («истекло время ожидания»)[²][³];
 - [409 Conflict](#) («конфликт»)[²][³][⁴];
 - [410 Gone](#) («удалён»)[²][³];
 - [411 Length Required](#) («необходима длина»)[²][³];
 - [412 Precondition Failed](#) («условие ложно»)[²][³][⁹];
 - [413 Payload Too Large](#) («полезная нагрузка слишком велика»)[²][³];
 - [414 URI Too Long](#) («URI слишком длинный»)[²][³];
 - [415 Unsupported Media Type](#) («неподдерживаемый тип данных»)[²][³];
 - [416 Range Not Satisfiable](#) («диапазон не достижим»)[[^][3\]^](https://ru.wikipedia.org/wiki/%D0%A1%D0%BF%D0%B8%D1%81%D0%BE%D0%BA_%D0%BA%D0%BE%D0%B4%D0%BE%D0%B2_%D1%81%D0%BE%D1%81%D1%82%D0%BE%D1%8F%D0%BD%D0%B8%D1%8F_HTTP#cite_note-rfc2616-3);
 - [417 Expectation Failed](#) («ожидание не удалось»)[[^][3\]^](https://ru.wikipedia.org/wiki/%D0%A1%D0%BF%D0%B8%D1%81%D0%BE%D0%BA_%D0%BA%D0%BE%D0%B4%D0%BE%D0%B2_%D1%81%D0%BE%D1%81%D1%82%D0%BE%D1%8F%D0%BD%D0%B8%D1%8F_HTTP#cite_note-rfc2616-3);

- [418 I'm a teapot](#) («я — чайник»);
- [419 Authentication Timeout \(not in RFC 2616\)](#) («обычно ошибка проверки CSRF»);
- [421 Misdirected Request](#) [`^[10\]^`](https://ru.wikipedia.org/wiki/%D0%A1%D0%BF%D0%B8%D1%81%D0%BE%D0%BA_%D0%BA%D0%BE%D0%B4%D0%BE%D0%B2_%D1%81%D0%BE%D1%81%D1%82%D0%BE%D1%8F%D0%BD%D0%B8%D1%8F_HTTP#cite_note-10);
- [422 Unprocessable Entity](#) («необрабатываемый экземпляр»);
- [423 Locked](#) («заблокировано»);
- [424 Failed Dependency](#) («невыполненная зависимость»);
- [425 Too Early](#) («слишком рано»);
- [426 Upgrade Required](#) («необходимо обновление»);
- [428 Precondition Required](#) («необходимо предусловие»)[`^[11\]^`](https://ru.wikipedia.org/wiki/%D0%A1%D0%BF%D0%B8%D1%81%D0%BE%D0%BA_%D0%BA%D0%BE%D0%B4%D0%BE%D0%B2_%D1%81%D0%BE%D1%81%D1%82%D0%BE%D1%8F%D0%BD%D0%B8%D1%8F_HTTP#cite_note-rfc6585-11);
- [429 Too Many Requests](#) («слишком много запросов»)[`^[11\]^`](https://ru.wikipedia.org/wiki/%D0%A1%D0%BF%D0%B8%D1%81%D0%BE%D0%BA_%D0%BA%D0%BE%D0%B4%D0%BE%D0%B2_%D1%81%D0%BE%D1%81%D1%82%D0%BE%D1%8F%D0%BD%D0%B8%D1%8F_HTTP#cite_note-rfc6585-11);
- [431 Request Header Fields Too Large](#) («поля заголовка запроса слишком большие»)[`^[11\]^`](https://ru.wikipedia.org/wiki/%D0%A1%D0%BF%D0%B8%D1%81%D0%BE%D0%BA_%D0%BA%D0%BE%D0%B4%D0%BE%D0%B2_%D1%81%D0%BE%D1%81%D1%82%D0%BE%D1%8F%D0%BD%D0%B8%D1%8F_HTTP#cite_note-rfc6585-11);
- [449 Retry With](#) («повторить с»)[`^[1\]^`](https://ru.wikipedia.org/wiki/%D0%A1%D0%BF%D0%B8%D1%81%D0%BE%D0%BA_%D0%BA%D0%BE%D0%B4%D0%BE%D0%B2_%D1%81%D0%BE%D1%81%D1%82%D0%BE%D1%8F%D0%BD%D0%B8%D1%8F_HTTP#cite_note-MSDN_WEBDAV_2_2_6-1);
- [451 Unavailable For Legal Reasons](#) («недоступно по юридическим причинам»)[`^[12\]^`](https://ru.wikipedia.org/wiki/%D0%A1%D0%BF%D0%B8%D1%81%D0%BE%D0%BA_%D0%BA%D0%BE%D0%B4%D0%BE%D0%B2_%D1%81%D0%BE%D1%81%D1%82%D0%BE%D1%8F%D0%BD%D0%B8%D1%8F_HTTP#cite_note-MSDN_WEBDAV_2_2_6-1);

%D0%B2_%D1%81%D0%BE%D1%81%D1%82%D0%BE%D1%8F%D0%BD
%D0%B8%D1%8F_HTTP#cite_note-http451-12).

- 499 Client Closed Request (клиент закрыл соединение);
- 5xx: Server Error (ошибка сервера):
 - 500 Internal Server Error («внутренняя ошибка сервера»)^{[2][3]};
 - 501 Not Implemented («не реализовано»)^{[2][3]};
 - 502 Bad Gateway («плохой, ошибочный шлюз»)^{[2][3]};
 - 503 Service Unavailable («сервис недоступен»)^{[2][3]};
 - 504 Gateway Timeout («шлюз не отвечает»)^{[2][3]};
 - 505 HTTP Version Not Supported («версия HTTP не поддерживается»)^{[2][3]};
 - 506 Variant Also Negotiates («вариант тоже проводит согласование»)
[^\[13\]](https://ru.wikipedia.org/wiki/%D0%A1%D0%BF
%D0%B8%D1%81%D0%BE%D0%BA_%D0%BA%D0%BE%D0%B4%D0%BE
%D0%B2_%D1%81%D0%BE%D1%81%D1%82%D0%BE%D1%8F%D0%BD
%D0%B8%D1%8F_HTTP#cite_note-rfc2295_8_1-13);
 - 507 Insufficient Storage («переполнение хранилища»);
 - 508 Loop Detected («обнаружено бесконечное перенаправление»)
[^\[14\]](https://ru.wikipedia.org/wiki/%D0%A1%D0%BF
%D0%B8%D1%81%D0%BE%D0%BA_%D0%BA%D0%BE%D0%B4%D0%BE
%D0%B2_%D1%81%D0%BE%D1%81%D1%82%D0%BE%D1%8F%D0%BD
%D0%B8%D1%8F_HTTP#cite_note-webdav_7_1-14);
 - 509 Bandwidth Limit Exceeded («исчерпана пропускная ширина канала»);
 - 510 Not Extended («не расширено»);
 - 511 Network Authentication Required («требуется сетевая аутентификация»)
[^\[11\]](https://ru.wikipedia.org/wiki/%D0%A1%D0%BF%D0%B8%D1%81%D0%BE%D0%BA_%D0%BA%D0%BE%D0%B4%D0%BE%D0%B2_%D1%81%D0%BE%D1%81%D1%82%D0%BE%D1%8F%D0%BD%D0%B8%D1%8F_HTTP#cite_note-rfc6585-11);
 - 520 Unknown Error («неизвестная ошибка»)
[^\[15\]](https://ru.wikipedia.org/wiki/%D0%A1%D0%BF%D0%B8%D1%81%D0%BE%D0%BA_%D0%BA%D0%BE%D0%B4%D0%BE%D0%B2_%D1%81%D0%BE%D1%81%D1%82%D0%BE%D1%8F%D0%BD%D0%B8%D1%8F_HTTP#cite_note-CloudFlare_Error_Pages-15);

- [521 Web Server Is Down](https://ru.wikipedia.org/wiki/%D0%A1%D0%BF%D0%B8%D1%81%D0%BE%D0%BA_%D0%BA%D0%BE%D0%B4%D0%BE%D0%B2_%D1%81%D0%BE%D1%81%D1%82%D0%BE%D1%8F%D0%BD%D0%B8%D1%8F_HTTP#cite_note-CloudFlare_Error_Pages-15) («веб-сервер не работает»)[^\[15\]](https://ru.wikipedia.org/wiki/%D0%A1%D0%BF%D0%B8%D1%81%D0%BE%D0%BA_%D0%BA%D0%BE%D0%B4%D0%BE%D0%B2_%D1%81%D0%BE%D1%81%D1%82%D0%BE%D1%8F%D0%BD%D0%B8%D1%8F_HTTP#cite_note-CloudFlare_Error_Pages-15);
- [522 Connection Timed Out](https://ru.wikipedia.org/wiki/%D0%A1%D0%BF%D0%B8%D1%81%D0%BE%D0%BA_%D0%BA%D0%BE%D0%B4%D0%BE%D0%B2_%D1%81%D0%BE%D1%81%D1%82%D0%BE%D1%8F%D0%BD%D0%B8%D1%8F_HTTP#cite_note-CloudFlare_Error_Pages-15) («соединение не отвечает»)[^\[15\]](https://ru.wikipedia.org/wiki/%D0%A1%D0%BF%D0%B8%D1%81%D0%BE%D0%BA_%D0%BA%D0%BE%D0%B4%D0%BE%D0%B2_%D1%81%D0%BE%D1%81%D1%82%D0%BE%D1%8F%D0%BD%D0%B8%D1%8F_HTTP#cite_note-CloudFlare_Error_Pages-15);
- [523 Origin Is Unreachable](https://ru.wikipedia.org/wiki/%D0%A1%D0%BF%D0%B8%D1%81%D0%BE%D0%BA_%D0%BA%D0%BE%D0%B4%D0%BE%D0%B2_%D1%81%D0%BE%D1%81%D1%82%D0%BE%D1%8F%D0%BD%D0%B8%D1%8F_HTTP#cite_note-CloudFlare_Error_Pages-15) («источник недоступен»)[^\[15\]](https://ru.wikipedia.org/wiki/%D0%A1%D0%BF%D0%B8%D1%81%D0%BE%D0%BA_%D0%BA%D0%BE%D0%B4%D0%BE%D0%B2_%D1%81%D0%BE%D1%81%D1%82%D0%BE%D1%8F%D0%BD%D0%B8%D1%8F_HTTP#cite_note-CloudFlare_Error_Pages-15);
- [524 A Timeout Occurred](https://ru.wikipedia.org/wiki/%D0%A1%D0%BF%D0%B8%D1%81%D0%BE%D0%BA_%D0%BA%D0%BE%D0%B4%D0%BE%D0%B2_%D1%81%D0%BE%D1%81%D1%82%D0%BE%D1%8F%D0%BD%D0%B8%D1%8F_HTTP#cite_note-CloudFlare_Error_Pages-15) («время ожидания истекло»)[^\[15\]](https://ru.wikipedia.org/wiki/%D0%A1%D0%BF%D0%B8%D1%81%D0%BE%D0%BA_%D0%BA%D0%BE%D0%B4%D0%BE%D0%B2_%D1%81%D0%BE%D1%81%D1%82%D0%BE%D1%8F%D0%BD%D0%B8%D1%8F_HTTP#cite_note-CloudFlare_Error_Pages-15);
- [525 SSL Handshake Failed](https://ru.wikipedia.org/wiki/%D0%A1%D0%BF%D0%B8%D1%81%D0%BE%D0%BA_%D0%BA%D0%BE%D0%B4%D0%BE%D0%B2_%D1%81%D0%BE%D1%81%D1%82%D0%BE%D1%8F%D0%BD%D0%B8%D1%8F_HTTP#cite_note-CloudFlare_Error_Pages-15) («квитирование SSL не удалось»)[^\[15\]](https://ru.wikipedia.org/wiki/%D0%A1%D0%BF%D0%B8%D1%81%D0%BE%D0%BA_%D0%BA%D0%BE%D0%B4%D0%BE%D0%B2_%D1%81%D0%BE%D1%81%D1%82%D0%BE%D1%8F%D0%BD%D0%B8%D1%8F_HTTP#cite_note-CloudFlare_Error_Pages-15);
- [526 Invalid SSL Certificate](https://ru.wikipedia.org/wiki/%D0%A1%D0%BF%D0%B8%D1%81%D0%BE%D0%BA_%D0%BA%D0%BE%D0%B4%D0%BE%D0%B2_%D1%81%D0%BE%D1%81%D1%82%D0%BE%D1%8F%D0%BD%D0%B8%D1%8F_HTTP#cite_note-CloudFlare_Error_Pages-15) («недействительный сертификат SSL»)[^\[15\]](https://ru.wikipedia.org/wiki/%D0%A1%D0%BF%D0%B8%D1%81%D0%BE%D0%BA_%D0%BA%D0%BE%D0%B4%D0%BE%D0%B2_%D1%81%D0%BE%D1%81%D1%82%D0%BE%D1%8F%D0%BD%D0%B8%D1%8F_HTTP#cite_note-CloudFlare_Error_Pages-15).

6. Technologies

2. IoC (+libraries)

Что такое Dependency injection и для чего используется, основные библиотеки

Инверсия управления (*Inversion of Control, IoC*) — важный принцип объектно-ориентированного программирования, используемый для уменьшения зацепления (связанности) в компьютерных программах. Также архитектурное решение интеграции, упрощающее расширение возможностей системы, при котором поток управления программы контролируется фреймворком.

Внедрение зависимости (*Dependency injection, DI*) — процесс предоставления внешней зависимости программному компоненту. Является специфичной формой «инверсии управления» (англ. *Inversion of control, IoC*), когда она применяется к управлению зависимостями. В полном соответствии с принципом единственной обязанности объект отдаёт заботу о построении требуемых ему зависимостей внешнему, специально предназначенному для этого общему механизму

Принцип единственной ответственности (*single-responsibility principle, SRP*) — принцип ООП, обозначающий, что каждый объект должен иметь одну ответственность и эта ответственность должна быть полностью инкапсулирована в класс. Все его поведения должны быть направлены исключительно на обеспечение этой ответственности.

Инверсия управления — один из популярных принципов объектно-ориентированного программирования, при помощи которого можно снизить связанность между компонентами, а так же повысить модульность и расширяемость ПО.

Реализуется инверсия управления несколькими способами, среди которых есть внедрение зависимостей (Dependency Injection, DI) и поиск зависимостей (Dependency Lookup, DL)

```
1  class TodoRepository {
2      // other code
3  }
4
5  class TodoService {
6      TodoRepository todoRepository = new TodoRepository();
7      // other code
8  }
```

Применение инверсии управления предполагает, что объект класса `TodoRepository` должен быть создан за пределами класса `TodoService`, но в дальнейшем должен быть передан объекту этого класса.

Внедрение зависимостей реализуется несколькими способами, среди которых можно выделить:

- Внедрение через конструктор
- Внедрение через `set`-метод
- Внедрение через интерфейс

Пример внедрения зависимости через конструктор:

```
TodoService.java 359 Bytes 
1 class TodoService {
2     TodoRepository todoRepository;
3     TodoService(TodoRepository todoRepository) {
4         this.todoRepository = todoRepository;
5     }
6 }
7
8 class Application {
9     public static void main(String[] args) {
10         TodoRepository todoRepository = new TodoRepository();
11         TodoService todoService = new TodoService(todoRepository);
12     }
13 }
```

Внедрение зависимостей через конструктор позволяет внедрять зависимости для final-свойств, а так же внедрять сразу несколько зависимостей.

Пример внедрения зависимости через set-метод:

```
TodoService.java 413 Bytes 
1 class TodoService {
2     TodoRepository todoRepository;
3     void setTodoRepository(TodoRepository todoRepository) {
4         this.todoRepository = todoRepository;
5     }
6 }
7
8 class Application {
9     public static void main(String[] args) {
10         TodoRepository todoRepository = new TodoRepository();
11         TodoService todoService = new TodoService();
12         todoService.setTodoRepository(todoRepository);
13     }
14 }
```

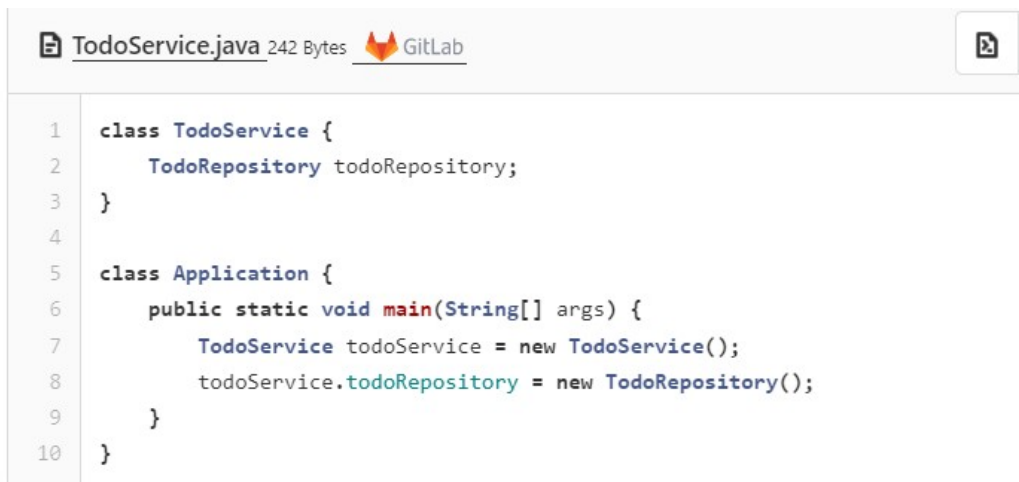
Пример внедрения зависимости через интерфейс:



The screenshot shows a code editor window titled 'TodoService.java' with a file icon and '447 Bytes' and 'GitLab' logo. The code is as follows:

```
1 class TodoService implements Consumer<TodoRepository> {
2     TodoRepository todoRepository;
3     @Override
4     public void accept(TodoRepository todoRepository) {
5         this.todoRepository = todoRepository;
6     }
7 }
8
9 class Application {
10     public static void main(String[] args) {
11         TodoRepository todoRepository = new TodoRepository();
12         TodoService todoService = new TodoService();
13         todoService.accept(todoRepository);
14     }
15 }
```

Кроме этих способов существует и внедрение зависимостей через свойства:



The screenshot shows a code editor window titled 'TodoService.java' with a file icon and '242 Bytes' and 'GitLab' logo. The code is as follows:

```
1 class TodoService {
2     TodoRepository todoRepository;
3 }
4
5 class Application {
6     public static void main(String[] args) {
7         TodoService todoService = new TodoService();
8         todoService.todoRepository = new TodoRepository();
9     }
10 }
```

Поиск зависимостей

В случае с поиском зависимостей класс должен самостоятельно реализовывать логику получения зависимостей извне. Для этого он должен иметь доступ к некоему источнику зависимостей.

```

class DependencyProvider {
    <T> T lookup(Class<T> type) {
        // code
    }
}

class TodoService implements Consumer<DependencyProvider> {
    TodoRepository todoRepository;

    @Override
    public void accept(DependencyProvider dependencyProvider) {
        this.todoRepository = dependencyProvider.lookup(TodoRepository.java);
    }
}

```

В

приведённом примере объект *TodoRepository* внедряется при помощи поиска зависимостей, а ***DependencyProvider*** — при помощи внедрения зависимостей через интерфейс. Кроме этого, в примере применена ещё одна техника инверсии управления — шаблон проектирования «фабрика», который реализуется классом *DependencyProvider*.

Инверсия управления (Inversion of Control) — абстрактный принцип, который указывает, что поток выполнения контролируется внешней сущностью. Концепция **IoC** тесно связана с идеей фреймворка. **IoC** — это основное различие между фреймворком и другой формализацией повторно используемого кода — библиотекой — набором функций, которые вы просто вызываете из своей программы. Фреймворк — это оболочка, которая предоставляет предопределённые точки расширения. Вы можете вставить свой собственный код в эти точки расширения, но фреймворк определяет, когда этот код будет вызван.

Инверсия зависимостей (Dependency inversion) — это буква D в аббревиатуре **SOLID**. Принцип гласит:

- Модули высокого уровня не должны зависеть от модулей низкого уровня. Оба типа модулей должны зависеть от абстракций.
- Абстракции не должны зависеть от деталей. Детали должны зависеть от абстракций.

Dependency Injection

Внедрение зависимостей — это шаблон проектирования, который применяет принцип **IoC**, чтобы гарантировать, что класс не имеет абсолютно никакого участия или осведомленности в создании или времени жизни объектов, используемых его конструктором или переменными экземпляра — «общая» забота о создании объекта и

заполнении переменных экземпляра вместо этого перекладывается на фреймворк.

То есть класс может указывать свои переменные экземпляра, но не выполняет никакой работы по заполнению этих переменных экземпляра (за исключением использования параметров конструктора в качестве «сквозного»)

Внедрение зависимостей — это стиль настройки объекта, при котором поля объекта задаются внешней сущностью. Другими словами, объекты настраиваются внешними объектами. DI — это альтернатива самонастройке объектов. Это может выглядеть несколько абстрактно, так что посмотрим пример:

```
public class MyDao {  
  
    private DataSource dataSource =  
  
    new DataSourceImpl("driver", "url", "user", "password");  
  
    public Person readPerson(int primaryKey) {...}  
  
}
```

Заметьте, что класс MyDao создает экземпляр DataSourceImpl, так как нуждается в источнике данных. Тот факт, что MyDao нуждается в реализации DataSource, означает, что он зависит от него. Он не может выполнить свою работу без реализации DataSource. Следовательно, MyDao имеет «зависимость» от интерфейса DataSource и от какой-то его реализации.

Класс MyDao создает экземпляр DataSourceImpl как реализацию DataSource. Следовательно, класс MyDao сам «разрешает свои зависимости». Когда класс разрешает собственные зависимости, он автоматически также зависит от классов, для которых он разрешает зависимости. В данном случае MyDao зависит также от DataSourceImpl и от четырех жестко заданных строковых значений, передаваемых в конструктор DataSourceImpl. Вы не можете ни использовать другие значения для этих четырех строк, ни использовать другую реализацию интерфейса DataSource без изменения кода.

Как вы можете видеть, в том случае, когда класс разрешает собственные зависимости, он становится негибким в отношении к этим зависимостям. Это плохо. Это значит, что если вам нужно поменять зависимости, вам нужно поменять код. В данном примере это означает, что если вам нужно использовать другую базу данных, вам потребуется поменять класс MyDao. Если у вас много DAO-классов, реализованных таким образом, вам придется изменять их все. В добавок, вы не можете провести юнит-тестирование MyDao, замокав реализацию DataSource. Вы можете использовать только DataSourceImpl.

Давайте немного поменяем дизайн:

```
public class MyDao {  
    private final DataSource dataSource;  
  
    public MyDao (String driver, String url, String user, String password) {  
        this.dataSource = new DataSourceImpl(driver, url, user, password);  
    }  
  
    public Person readPerson (int primaryKey) {...}  
}
```

Заметьте, что создание экземпляра DataSourceImpl перемещено в конструктор. Конструктор принимает четыре параметра, это — четыре значения, необходимые для DataSourceImpl. Хотя класс MyDao все еще зависит от этих четырех значений, он больше не разрешает зависимости сам. Они предоставляются классом, создающим экземпляр MyDao. Зависимости «внедряются» в конструктор MyDao. Отсюда и термин «внедрение (прим. перев.: или иначе — инъекция) зависимостей». Теперь возможно сменить драйвер БД, URL, имя пользователя или пароль, используемый классом MyDao без его изменения.

Внедрение зависимостей не ограничено конструкторами. Можно внедрять зависимости также используя методы-сеттеры, либо прямо через публичные поля (**прим. перев.:** по поводу полей переводчик не согласен, это нарушает защиту данных класса).

Класс MyDao может быть более независимым. Сейчас он все еще зависит и от интерфейса DataSource, и от класса DataSourceImpl. Нет необходимости зависеть от чего-то, кроме интерфейса DataSource. Это может быть достигнуто инъекцией DataSource в конструктор вместо четырех параметров строкового типа. Вот как это выглядит:

```
public class MyDao {  
    private final DataSource dataSource;  
  
    public MyDao(DataSource dataSource){  
        this.dataSource = dataSource;  
    }  
  
    public Person readPerson(int primaryKey) {...}  
}
```

Теперь класс MyDao больше не зависит от класса DataSourceImpl или от четырех строк, необходимых конструктору DataSourceImpl. Теперь можно использовать любую реализацию DataSource в конструкторе MyDao.

Объект теперь передается в конструктор, а не его параметры

Во-первых, данные фреймворки предлагают способ определения полей (объектов), которые должны быть внедрены. Некоторые фреймворки осуществляют это посредством аннотирования поля или конструктора с помощью аннотации @Inject, но существуют и другие методы.

Например, [Koin](#) использует встроенные языковые особенности Kotlin для определения внедрения. Под Inject подразумевается, что зависимость должна обрабатываться DI-фреймворком. Код будет выглядеть примерно так:

```
class ClassA {  
  
    var classB: ClassB  
  
    @Inject constructor(classB: ClassB){  
  
        this.classB = classB  
  
    }  
}  
  
class ClassB {  
  
    var classC: ClassC  
  
    @Inject constructor(classC: ClassC){  
  
        this.classC = classC  
  
    }  
}  
  
class ClassC {  
  
    @Inject constructor(){  
  
    }  
}
```

Во-вторых, фреймворки позволяют определить, как нужно предоставить каждую зависимость, и это происходит в отдельном файле (файлах).

Приблизительно это выглядит так (учитывайте, что это лишь пример, и он может отличаться от фреймворка к фреймворку):

```
class OurThirdPartyGuy {
```



```

fun provideClassC(){
return ClassC() //just creating an instance of the object and return it.
}

fun provideClassB(classC: ClassC){
return ClassB(classC)
}

fun provideClassA(classB: ClassB){
return ClassA(classB)
}
}

```

Итак, как вы видите, каждая функция отвечает за обработку одной зависимости. Поэтому если нам где-то в приложении нужно использовать ClassA, то произойдет следующее: наш DI-фреймворк создаёт один экземпляр класса ClassC, вызвав provideClassC, передав его в provideClassB и получив экземпляр ClassB, который передаётся в provideClassA, и в результате создаётся ClassA. Это практически волшебство. Теперь давайте изучим преимущества и достоинства третьего способа.

Преимущества

- Все максимально просто. И зависимый класс, и класс, предоставляющий зависимости, понятны и просты.
- Классы слабо связаны и легко заменяемы другими классами. Допустим, мы хотим заменить ClassC на AssumeClassC, который является подклассом ClassC. Для этого нужно лишь изменить код провайдера следующим образом, и везде, где используется ClassC, теперь автоматически будет использоваться новая версия:

```

fun provideClassC(){
return AssumeClassC()
}

```

Обратите внимание, никакой код внутри приложения не меняется, только метод провайдера. Кажется, что ничего не может быть ещё проще и гибче.

- Невероятная тестируемость. Можно легко заменить зависимости тестовыми версиями во время тестирования. Фактически, внедрение зависимостей — ваш главный помощник, когда речь заходит о тестировании.

- Улучшение структуры кода, т.к. в приложении есть отдельное место для обработки зависимостей. В результате остальные части приложения могут сосредоточиться на выполнении исключительно своих функций и не пересекаться с зависимостями.

Недостатки

- У DI-фреймворков есть определенный порог вхождения, поэтому команда проекта должна потратить время и изучить его, прежде чем эффективно использовать.

Заключение

- Обработка зависимостей без DI возможна, но это может привести к сбоям работы приложения.
- DI — это просто эффективная идея, согласно которой возможно обрабатывать зависимости вне зависимого класса.
- Эффективнее всего использовать DI в определенных частях приложения. Многие фреймворки этому способствуют.
- Фреймворки и библиотеки не нужны для DI, но могут во многом помочь.

Hilt это просто обертка для Dagger2. Для небольших проектов сможет встать более удобным инструментом и хорошо интегрируется с остальными продуктами в JetPack.

если хочется использовать Dagger2, но с минимальными усилиями, то для этого как раз и был придуман Hilt.

Что упростили для нас:

- Готовые компоненты (из названий понятно к чему относятся)
 - ApplicationComponent
 - ActivityRetainedComponent
 - ActivityComponent
 - FragmentComponent
 - ViewComponent
 - ViewWithFragmentComponent
 - ServiceComponent
- В модуле указываешь в какой компонент добавить

- Через `@AndroidEntryPoint` Hilt компилятор генерирует весь boilerplate для создания компонента и хранения (например, `ActivityRetainedComponent` сохранит сущность после поворота экрана, `ActivityComponent` пересоздаст заново).

Application обязателен

Необходимо объявить `Application` и пометить `@HiltAndroidApp`, без него Hilt не заработает.

Иерархическая зависимость

Если хотите использовать Hilt в фрагментах, то Activity которая содержит эти фрагменты обязательно пометить аннотацией `@AndroidEntryPoint`

Если View пометить `@WithFragmentBindings` то Fragment должен быть с аннотацией `@AndroidEntryPoint`, а без этой аннотации зависит от того куда инжектимся Activity или Fragment

Объявление модулей

Все как в Dagger2, но нет необходимости добавлять модуль в компонент, а достаточно использовать аннотацию `@InstallIn`. Это и понятно, так как компоненты не доступны для редактирования.

Кастомные Component и Subcomponent

Создать их конечно можно, так как там Dagger2, но теряется тогда весь смысл Hilt и все придется писать вручную. Вот что предлагается в документации:

```
DaggerLoginComponent.builder()
    .context(this)
    .appDependencies(
        EntryPointsAccessors.fromApplication(
            applicationContext,
            LoginModuleDependencies::class.java
        )
    )
    .build()
    .inject(this)
```

Поддержка многомодульности

Все это есть, но только когда используем только готовые компоненты. Но если добавить для каждого модуля свои компоненты (как советуют [тут](#)), то лучше использовать Dagger2.

Ограничения для @AndroidEntryPoint

- Поддерживаются Activity наследуемые от ComponentActivity и AppCompatActivity
- Поддерживаются Fragment наследуемые от androidx.Fragment
- Не поддерживаются Retain фрагменты

Hilt работает следующим образом:

- Генерируются Dagger Component-ы
- Генерируются базовые классы для Application, Activity, Fragment, View и т.д, которые помечены аннотацией @AndroidEntryPoint
- Dagger компилятор генерирует статический коды

Как устроено хранение ActivityRetainedComponent

Не стали усложнять и просто поместили компонент в ViewModel из arch библиотеки

Плюсы:

- Более простое использование чем Dagger2
- Добавление модулей через аннотацию выглядит довольно удобно (в несколько уже не поместишь)
- Код чище и много boilerplate спрятано.
- Все плюсы от Dagger2 (генерация статического кода, валидация зависимостей и т.д.)
- Удобен для небольших проектов

Минусы:

- Тяжело избавиться, не только захочется убрать или заменить, но и просто перейти на Dagger2
- Тяжело добавить кастомные компоненты, что ограничивает использование с крупных проектах
- Наследует минусы Dagger2 и еще больше увеличивает время сборки
- Иерархическая зависимость, например, нельзя использовать в Fragment без Activity с @AndroidEntryPoint

Dagger 2 — это Android библиотека позволяющая реализовать паттерн Внедрение зависимостей (Dependency Injection), который в свою очередь является формой инверсии управления (IoC — Inversion of Control).

Inversion of Control — это абстрактный принцип написания слабо связанного кода, целью которого является создать некоторую структуру, где составляющие компоненты этой системы будут как можно более изолированными друг от друга и которые в своей работе не будут зависеть от реализации других компонентов этой структуры.

Dependency Injection — это одна из реализаций принципа IoC.

Преимущества Dagger 2:

1. Простая настройка зависимостей. Чем больше проект — тем больше зависимостей, однако данная библиотека позволяет с легкостью управлять всем зависимостями без написания «тонн кода».
2. Облегчение Unit-тестирования.
3. Локальные синглтоны.
4. Кодогенерация библиотеки понятная разработчику и легка в поддержании.
5. Работает не на рефлексии (первая версия библиотеки Dagger — работала исключительно на рефлексии и были частые падения в runtime).
6. Размеры библиотеки достаточно малы.

@Inject аннотацию расскажет Dagger, что все зависимости должны были быть переданы в зависимое. Может использоваться в конструкторе, поле или методе.

Внедрение конструктора используется с конструктором класса. Например,

```
class LoginPresenter @Inject constructor(  
    private val userService: UserService,  
    private val userManager: UserManager  
) : BasePresenter<View>{ ... }
```

```
class LoginActivity : AppCompatActivity(){  
    @Inject lateinit var loginPresenter: LoginPresenter  
}
```

```
lateinit var user: User  
@Inject  
internal fun loadFromPreferences() {  
    user = userPreferences.user  
}
```

@Module отмечает модули / классы. Модуль предоставляет экземпляры зависимостей.

Например, у нас может быть модуль под названием ContextModule, и этот модуль может предоставлять зависимости контекста приложения и контекста другим классам.

Поэтому нам нужно пометить класс ContextModule с помощью **@Module**

```
@Module  
class ContextModule {  
    @Provides  
    @Singleton  
    fun provideContext(application: Application): Context {  
        return application;  
    }  
}
```

@Module сообщает Dagger, как предоставлять классы, которыми ваш проект не владеет, или сообщает Dagger, как создавать экземпляры типа, возвращаемого этой функцией.

```

@Module
class NetworkModule {
    @Provides
    @Singleton
    fun provideApiService(): ApiService {
        return Retrofit.Builder()
            .baseUrl("https://jsonplaceholder.typicode.com")
            .addConverterFactory(GsonConverterFactory.create())
            .build()
            .create(ApiService.class);
    }
}

```

@Binds эквивалентен **@Provides**. В основном он используется для привязки интерфейсов и абстрактных классов.

```

@Module
abstract class MainActivityModule {
    @Binds
    abstract fun bindMainPresenter(
        presenter: MainPresenter): LoginContract.Presenter
}

```

Эта аннотация сообщает Dagger, какие модули включать при построении графа для предоставления зависимостей.

```

@Component(modules = [ContextModule::class, NetworkModule::class])
interface ApplicationComponent: AndroidInjector<MyApplication> {
    @Component.Builder
    interface Builder {
        @BindsInstance
        fun application(application: Application): Builder
        fun build(): AppComponent
    }
}

```

@BindsInstance позволит клиенту / фабрике передать свой собственный экземпляр. Здесь в нашем случае мы передаем контекст приложения.

@Component.Builder конструктор нашего компонента. Согласно документации,

Компоненты могут иметь один вложенный статический абстрактный класс или интерфейс, аннотированный `@Component.Builder`. Если они это сделают, то созданный конструктор компонента будет соответствовать API в типе. - источник

@ContributesAndroidInjector

Эта аннотация сообщает Dagger, что нужно сгенерировать подкомпонент активности, фрагмента, службы и т. Д. `@ContributesAndroidInjector` Это замена старого, `@SubComponent` где мы вручную создаем собственный подкомпонент.

```
@ActivityScoped
@ContributesAndroidInjector(modules = MainActivityModule.class)
abstract MainActivity bindMainActivity();
```

По умолчанию кинжал имеет только одну встроенную область видимости `@Singleton` и используется для аннотирования компонента приложения, но мы можем создавать собственные настраиваемые области с помощью `@Scope` аннотации (например, `@FragmentScope`).

```
// Creates ActivityScope
@Scope
@MustBeDocumented
@Retention(value = AnnotationRetention.RUNTIME)
annotation class ActivityScope
```

`@Named` Аннотацию хорош для идентификации , какой провайдер будет использоваться , когда мы пытаемся придать зависимость одного и того же типа.

```
@Module
class NetworkModule {
    @Provides
    @Named("AuthClient")
    @Singleton
    fun provideAuthClient(): OkHttpClient {
        return OkHttpClient.Builder()
            .readTimeout(30, TimeUnit.SECONDS).build()
    }
}
```

Dagger 2 предоставляет ряд специальных аннотаций:

- `@Module` для классов, чьи методы предоставляют зависимости
- `@Provides` методы в классах `@Module`
- `@Inject` для запроса зависимости (конструктор, поле или метод)
- `@Component` — это мостовой интерфейс между модулями и инъекцией.

Чтобы правильно реализовать Dagger 2, вы должны выполнить следующие шаги:

Определите зависимые объекты и их зависимости.

Создайте класс с аннотацией `@Module`, используя аннотацию `@Provides` для каждого метода, который возвращает зависимость.

Запросите зависимости в ваших зависимых объектах, используя аннотацию `@Inject`.

Создайте интерфейс с `@Component` аннотации `@Component` и добавьте классы с аннотацией `@Module` созданной на втором шаге.

Создайте объект интерфейса `@Component` чтобы создать экземпляр зависимого объекта с его зависимостями.

Связь между поставщиком зависимостей `@Module` и классами, запрашивающими их через `@Inject`, осуществляется с помощью `@Component`, который является интерфейсом:

```
package com.androidheroes.tutsplusdagger.component;

import com.androidheroes.tutsplusdagger.model.Vehicle;
import com.androidheroes.tutsplusdagger.module.VehicleModule;

import javax.inject.Singleton;

import dagger.Component;

/**
 * Created by kerry on 14/02/15.
 */

@Singleton
@Component(modules = {VehicleModule.class})
public interface VehicleComponent {

    Vehicle provideVehicle();

}
```

Рядом с аннотацией `@Component` вы должны указать, какие модули будут использоваться — в этом случае я использую `VehicleModule`, который мы создали ранее. Если вам нужно использовать больше модулей, просто добавьте их, используя запятую в качестве разделителя.

В интерфейсе добавьте методы для каждого нужного вам объекта, и он автоматически предоставит вам один со всеми удовлетворенными зависимостями. В этом случае мне нужен только объект `Vehicle`, поэтому существует только один метод.

Использование интерфейса `@Component` для получения объектов

Теперь, когда у вас есть все готовые соединения, вы должны получить экземпляр этого интерфейса и вызвать его методы для получения нужного вам объекта. Я собираюсь реализовать это в методе `onCreate` в `MainActivity` следующим образом:

```

public class MainActivity extends ActionBarActivity {

    Vehicle vehicle;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        VehicleComponent component = Dagger_VehicleComponent.builder().vehicleModule(new VehicleModule())
            .build();
        vehicle = component.provideVehicle();

        Toast.makeText(this, String.valueOf(vehicle.getSpeed()), Toast.LENGTH_SHORT).show();
    }
}

```

Когда вы пытаетесь создать новый объект интерфейса с аннотацией `@Component`, вы должны сделать это с помощью префикса `Dagger_<NameOfTheComponentInterface>`, в данном случае `Dagger_VehicleComponent`, а затем использовать метод `builder` для вызова каждого модуля внутри.

Вы можете видеть, что магия происходит когда Вы запрашиваете только один объект класса `Vehicle` и библиотека отвечает за удовлетворение всех зависимостей, в которых нуждается этот объект. Опять же, вы можете видеть, что нет никакого нового экземпляра любого другого объекта — все управляется библиотекой.

Теперь вы можете запустить приложение и попробовать его на своем устройстве или в эмуляторе. Если вы пошагово выполняли урок, вы увидите сообщение Toast указанием начального значения или переменной `rpm`.

В прикрепленном проекте вы можете увидеть пользовательский интерфейс для класса `MainActivity`, в котором вы можете изменять значение переменной `rpm`, нажимая кнопки на экране.

`@Singleton` (и любая другая аннотация области видимости) делает ваш класс единственным экземпляром в графе зависимостей (это означает, что этот экземпляр будет "singleton", пока существует объект компонента).

Короче говоря, каждый раз, когда вы вводите `@Singleton` аннотированный класс (с аннотацией `@Inject`), это будет один и тот же экземпляр, пока вы вводите его из одного и того же компонента.

`@Singleton` на самом деле не создает Singleton, это просто Scope, рекомендуется не использовать `@Singleton`, так как это вводит в заблуждение, создается впечатление, что мы фактически получаем Singleton, но это не так.

Допустим, вы аннотируете свою зависимость от базы данных с помощью `@Singleton` и связываете ее с `Component`, теперь предположим, что вы инициализируете эту `Component` в `Activities A` и `B`, у вас будут разные экземпляры вашей базы данных в ваших двух `Activities`, чего большинство людей не хотят.

Как вы преодолеваете это?

Инициализируйте свой Component один раз в своем классе Application и получите к нему статический доступ в других местах , таких как Activities или Fragments, теперь это может скоро выйти из-под контроля, если у вас есть более 20 Component's , поскольку вы не можете инициализировать их все в своем классе Application , это также замедлит время запуска вашего приложения.

Лучшее решение , по моему мнению, состоит в том, чтобы создать реальный Singleton, либо дважды проверенный, либо из других вариантов, и использовать его статически как getInstance() и использовать его под @Provides в вашем модуле.

Я знаю, что это тоже разбивает мне сердце , но, пожалуйста, поймите, что @Singleton на самом деле не Singleton, а Scope .

Граф зависимостей Dagger2

Классы аннотированные @Inject и @Provide формируют граф объектов, связанных их зависимостями. Рутами графа являются компоненты, у которых в свою очередь есть ссылки на модули. Даггер генерирует реализации классов исходя из этого контракта.

Способы добавить связи в граф

После добавления биндинга в граф, он становится доступным в качестве зависимости.

1. @Provides
2. @Inject
3. Component provisioning methods
4. @Component
5. Unqualified builders for any included subcomponent
6. Provider or Lazy wrappers for any of the above bindings
7. A Provider of a Lazy of any of the above bindings (e.g., Provider<Lazy<CoffeeMaker>>)
8. A MembersInjector for any type

@Component

- **@Component** - интерфейс, для которого даггер создает реализацию, генерируя классы с префиксом Dagger на этапе компиляции.
- Является рутом графа зависимостей.

- Чтобы даггер смотрел на биндинг методы, отмеченные @Binds нужно передать ссылку на модуль в компонент.

@Inject

- Содержит зависимости, которые автоматически подставляются даггером.
- Подставляются также транзитивные зависимости.
- Не работает в некоторых случаях:
 - Интерфейсы не могут быть инстанцированы.
 - Сторонние библиотеки не могут быть аннотированны.
 - Конфигурируемые объекты должны быть сконфигурированы. (?)

@Binds

- Дает даггеру понять какую реализацию подставлять как зависимость на место интерфейса.
- Используется с абстрактным методом.
- Даггер не вызывает и не имплементирует биндинг метод.
- Реализация - класс в параметре.
- Интерфейс - возвращаемый тип.

@Module

- **@Module** - коллекция биндингов.
- Кроме @Binds для биндинга могут использоваться другие аннотации.
- Называется модулем, т.к. модули можно смешивать и использовать в разных частях приложения.

@Provides

- Статический метод с реализацией прямо в модуле.
- Предоставляет экземпляр зависимости.
- Работает во многом как @Inject конструктор.
- Параметры метода - зависимости, которые предоставляются даггером.
- Можно возвращать синглтоны.
- Используется для подключения сторонних read-only библиотек.

Multi-bindings

@IntoMap

- **@IntoMap** - мульти-биндинг через @Binds.
- **@StringKey** - ключ Map.
- При использовании нужно в @Inject конструктор вставить Map качестве зависимости.
- @Component должен содержать ссылки на все модули.
- Нельзя использовать одинаковые ключи.

@IntoSet

- Позволяет создавать множества типов в виде коллекции.
- Возвращает Set<Type>.
- Может быть использован вместе с @Binds или @Provides

@Scope

- Даггер соединяет инстансы в графе зависимостей, отмеченные данной аннотацией с реализацией компонентов, отмеченных той же аннотацией.
- Компоненты должны быть отмечены данной аннотацией, чтобы показать какой скоуп они представляют.
- Компоненты могут представлять несколько скоупов.

@Singleton

- **@Singleton** аннотация используется, чтобы использовать один инстанс зависимости на каждый @Component.
- Нужно также отметить компонент данной аннотацией.
- Может быть несколько инстансов при неверном использовании.
- Частный случай аннотации @Scope.
- Если использовать вместе с @Provides, то будет создан синглтон для всех клиентов.
- Может является разделяемым ресурсом для потоков.

@Subcomponent

- Тоже самое что и компонент.
- Всегда должен иметь родителя.

- Получает доступ ко всем зависимостям родителя.
- Зависимости самого сабкомпонента скрыты от родителя.
- В методе декларируется тип, который даггер должен создать.

Provider

- Инжектирует несколько объектов вместо одного.
- Вызывает биндинг логику типа T каждый раз при вызове get().
- Если биндинг логика - это @Inject конструктор, то будет создаваться новый инстанс.
- Является легализованным костылем.

Qualifiers

- Используются, когда несколько биндингов связаны с одним интерфейсом.
- @Named аннотация, либо любая другая @Qualifier.

@BindsInstance

- Указывается рядом с зависимостями, создающимися непосредственно перед инжектором.
- Параметр, позволяющий запрашивать любой объект запрашивать в другом биндинге методе в компоненте.

6. Technologies

5. Images Основные библиотеки, как загружать, особенности, разница loading между решениями

Самые основные библиотеки Picasso и Глайд.

Хотя они выглядят одинаково, Glide проще в использовании, потому что метод with Glide может не только принимать Context, но также Activity и Fragment, и Context будет автоматически получен из них.

Glide может автоматически рассчитать размер ImageView в любой ситуации.

АПИ ПИКАССО

```
Picasso.with(context)
    .load(url)
    .placeholder(R.drawable.user_placeholder)
    .error(R.drawable.user_placeholder_error)
    .into(imageView);
```

Вы указываете адрес картинки (url), заглушку (placeholder), заглушку для ошибки после трёх неудачных попыток загрузки (error) и в методе **into()** указываете компонент **ImageView**, в который загружаете изображение.

При загрузке картинка кэшируется и при повторном запросе на скачивание библиотека может достать картинку из кэша, а не скачивать из интернета, что ускоряет работу приложения. Если кэш будет переполнен или удалён пользователем, то картина снова скачается из сети

Если вы храните большие картинки в ресурсах или на внешнем накопителе, то рекомендуется использовать отдельный процесс для загрузки. Библиотека уже настроена на работу в асинхронном режиме, поэтому вы можете использовать её и в этих случаях.

```
// из ресурсов
Picasso.with(context).load(R.drawable.landing_screen).into(imageView1);
// из внешнего накопителя
Picasso.with(context).load(new File(...)).into(imageView2);
```

Не забывайте про метод библиотеки **fit()**, который уменьшает размер картинки перед размещением в **ImageView**. Это полезно для экономии ресурсов, если вам в реальности нужна маленькая картинка, а не оригинал.

Так же можно всячески обрабатывать изображения заливать и менять цвета, обрезать по меньшей стороне, делать круглые аватары

Глайд еще нормально так с ГИФ работает

Формат растрового изображения по умолчанию для Glide - RGB_565.

Ниже приводится сравнение с Picasso при загрузке изображений (изображения с разрешением 1920x1080 пикселей загружаются в ImageView 768x432).



Picasso

Glide

Видно, что качество загруженных Glide картинок хуже, чем у Пикассо. Почему? Это потому, что формат растрового изображения Glide по умолчанию **RGB_565**, чем **ARGB_8888**. Накладные расходы на память формата вдвое меньше. Ниже приведена диаграмма накладных расходов памяти Picasso под ARGB8888 и Glide под RGB565 (само приложение занимает

Fresco от фэйсбук— многофункциональная библиотека для асинхронной загрузки и отображения изображений с тремя уровнями кеширования (2 в памяти, 1 в internal storage). Поддерживает форматы: JPEG, PNG, GIF и WebP. Также с помощью Fresco можно поставить ProgressBar непосредственно на View, что очень удобно.

Далее фреско требует чтобы её инициализировали, есть два варианта: либо инициализировать каждый раз в onCreate Activity, либо инициализировать один раз в Application.

Загружаем картинку из сети:

```
import coil.load

val imageView: ImageView = findViewById(R.id.imageView)
imageView.load("http://developer.alexanderklimov.ru/android/images/android_cat.jpg")
```

Можно загружать из ресурсов, файловой системы и т.д.

```
// Resource
imageView.load(R.drawable.cat)

// File
imageView.load(File("/path/to/kitten.jpg"))
```

Более сложный вариант с применением лямбды - используем заглушку и трансформацию в виде круга.

```
imageView.load("http://developer.alexanderklimov.ru/android/images/android_cat.jpg"){
    crossfade(true)
    placeholder(R.drawable.ic_action_cat)
    transformations(CircleCropTransformation())
}
```

Доступны также **GrayscaleTransformation**, **BlurTransformation**, **RoundedCornersTransformation**.

Fresc

o.initialize(context);

Fresco идеально Вам подойдет если не хватает пикассо, так как она гибкая и расширяемая практически везде.

COIL

Библиотека написана на Kotlin и использует корутины. При этом она быстрее известной Glide. Пользоваться удобно и просто. Примеры взяты из документации.

7. Architecture

1. Basics что такое архитектура, для чего используется, (difference) примеры

Общие сильные стороны

Разделение ответственности. Каждый компонент использует несколько компонентов, каждый из которых имеет определенные роли в каждом шаблоне. Все эти шаблоны имеют компоненты, которые индивидуально привязаны к предметной области, данным и представлению.

Тестируемость. Поскольку каждый компонент выполняет свою функцию и имеет четко определенную структуру связи, все эти шаблоны позволяют тестировать каждый компонент изолированно.

Модульность. Каждая архитектура должна позволять переключаться между различными реализациями компонентов, в то время как другие компоненты остаются нетронутыми.

MVP

Вью отделено от Модели, посредник ПРЕЗЕНТЕР. проще делать модульные тесты

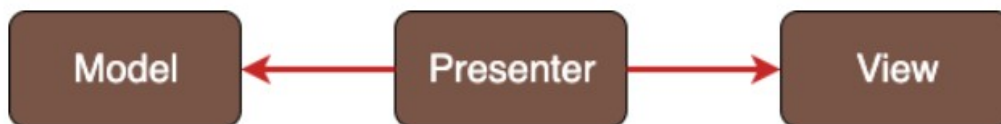
Шаблон MVP рассматривает представление как XML и действие в сочетании, а компонент представления может быть либо самим классом действия, либо отдельным компонентом, который взаимодействует с элементами пользовательского интерфейса в указанном действии, и должен строго использоваться для этой цели с минимальными затратами

Таким образом, логика обрабатывается в новом компоненте, известном как Presenter, который действует как мост между представлением и моделью.

Presenter также решает, что происходит, когда пользователь взаимодействует с представлением, например, нажимает кнопку, и, таким образом, будет иметь метод для обработки каждого возможного случая.

Модель действует как поставщик данных и отвечает за связь как с сетью, так и с локальной базой данных.

Presenter знает как о представлении, так и о модели, но эти компоненты не знают ни о каком другом компоненте в архитектуре, поэтому Presenter обрабатывает всю связь между компонентами.



Однако у MVP есть проблема, заключающаяся в том, что жизнь presenter связана с активити

Это создает возможность для утечки действия, когда Presenter выполняет длительные задачи, которые прерываются, когда действие уничтожается, например, из-за изменений конфигурации (например, поворота экрана) или естественных процессов системы.

И хотя накладные расходы на производительность, вызванные этим, в большинстве случаев не так уж велики, это также создает возможность для сбоя вашего приложения, поскольку представление попытается взаимодействовать с активностью, которой больше не существует.

MVP простой и в него легко добавить внедрение зависимостей

MVVM

Model-View-ViewModel новее, чем MVP, но прожила достаточно долго, чтобы завоевать большую популярность среди сообщества.

В этой архитектуре роль представления и модели такая же, как и в MVP. Единственная ответственность Представления — взаимодействие с элементами пользовательского интерфейса, а единственная ответственность Модели — быть поставщиком данных.

Новым компонентом здесь является модель представления – ViewModel.

Этот компонент также обрабатывает как логику, так и использование модели для извлечения данных, но, в отличие от презентатора, он может выдерживать изменения конфигурации и не привязан строго к жизненному циклу активности.

Это решает основную проблему MVP, связанную с возможными сбоями и утечками памяти, и дает дополнительное преимущество, заключающееся в том, что несколько представлений наблюдают за одним и тем же экземпляром модели представления.

Как реализуется это второе преимущество?

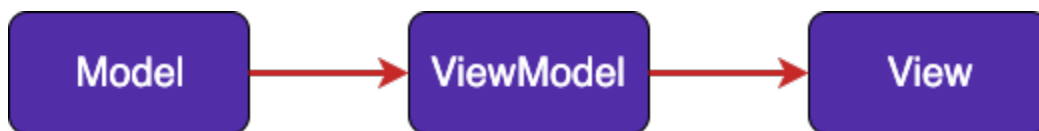
Модель представления опирается на тип под названием LiveData, переменную, которую можно наблюдать и обновлять данные, сохраняя при этом один и тот же экземпляр переменной.

Это означает, что модели представления нужно только обновить данные, переносимые LiveData, а представление (или представления) наблюдает за изменениями в этих LiveData внутри себя.

Это дополнительно обеспечивается структурой знаний компонентов в архитектуре MVVM.

На приведенной ниже диаграмме каждый компонент знает только о компоненте непосредственно над ним.

Представление знает о модели представления, а модель представления знает только о модели.



Да, другие архитектуры также могут использовать LiveData, но никакая другая архитектура не может извлечь из этого столько пользы, сколько MVVM.

Однако из-за своей интеграции MVVM в значительной степени зависит от компонентов архитектуры Android и страдает от синдрома «как мне

реализовать этот шаблон» больше, чем другие шаблоны, из-за того, что разные разработчики используют эти компоненты архитектуры разными способами и в различных комбинациях. .

Вдобавок ко всему, эту архитектуру, возможно, сложнее всего реализовать вместе с Dagger (или инъекцией зависимостей в целом) из-за необходимости использовать ViewModelProvider (или делегат by activityViewModels из зависимости fragment-ktx) для присоединения представления.

*** преимущества ***

Модель просмотра может пережить жизненный цикл и изменения конфигурации активности.

Несколько действий/фрагментов могут одновременно использовать одну и ту же модель представления.

*** недостатки ***

Сильно зависит от компонентов архитектуры Android

Страдает от синдрома «как мне реализовать этот шаблон» больше, чем другие архитектуры

Труднее всего реализовать прямо с DI

MVI

Самый свежий из трех архитектурных паттернов в этой королевской битве, но, тем не менее, он стал весьма заметным в сообществе. Model-View-Intent состоит из Model, View и Presenter.

Intent в этой архитектуре не обозначает ни компонент, ни объект Intent из Android SDK. Скорее, это означает желание совершить действие. MVI отличается от других шаблонов тем, что Presenter всегда отправляет в представление только один объект, обычно называемый состоянием просмотра View State. View State предназначено для хранения всей информации, необходимой представлению для рендеринга.

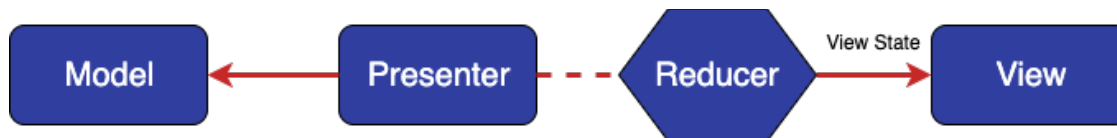
Presenter может обновлять состояние представления по мере получения новых данных, но представление не получает от Presenter никаких других данных, кроме этого одного состояния представления. Он строго придерживается принципа программирования единого источника правды (SSOT).

единый источник истины (SSOT) — это практика структурирования информационных моделей и связанной с ними схемы данных, так что каждый элемент данных обрабатывается (или редактируется) только в одном месте.

Таким образом, намерение в этом случае относится к полному потоку действий, от презентатора, запрашивающего модель для извлечения данных, до обновления состояния просмотра и передачи обновленного состояния просмотра в представление для рендеринга.

Это намерение может быть запущено с момента создания докладчика или в результате взаимодействия с пользователем, такого как нажатие кнопки, и когда представление успешно отображает себя из обновленного состояния представления, намерение разрешается.

Структура знаний такая же, как и у MVP, поскольку это можно рассматривать как усовершенствование архитектуры MVP.



Однако это «улучшение» не обходится без последствий.

Как Presenter обрабатывает обновление View State новыми данными, сохраняя при этом уже содержащиеся данные?

Ответом на это является другая концепция шаблона MVI, называемая State Reducers.

Это объекты, которые обрабатывают слияние новых данных с существующим состоянием представления и являются производными от одноименной вневременной концепции программирования. В более сложных действиях сложность редуктора увеличивается вместе с ним и может очень быстро стать объектом, требующим высокого мастерства для обслуживания и еще более высокого мастерства для правильного создания.

преимущества

Объект View State дает понять, как представление должно отображать себя. Приверженность к Единому Источнику Правды Single-Source-of-Truth (SSOT) Легче управлять одним наблюдаемым потоком между View и Presenter, чем многими

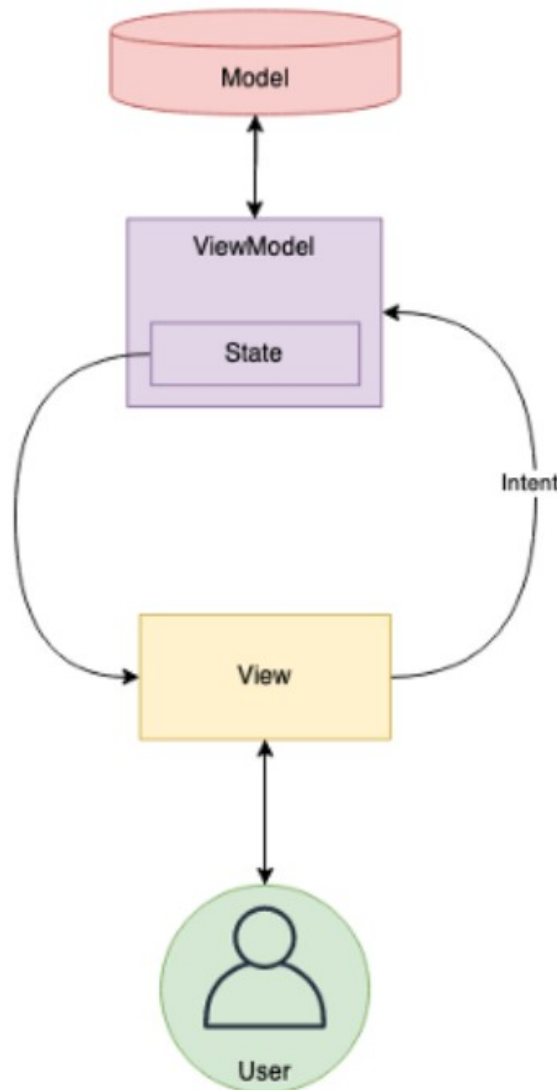
недостатки

- State reducers (Редьюсеры) состояний создают высокий уровень сложности как для создания, так и для обслуживания. Страдает теми же недостатками жизненного цикла, что и MVP.

MVI (ЕЩЕ РАЗОК)

MVI - это сокращение от «Модель, представление, намерение. Он похож на такие как MVP но вводит две новые концепции: **намерение** и **состояние**. Намерение - это событие, отправленное ViewModel представлением для

выполнения конкретной задачи. Он может быть инициирован пользователем или другими частями вашего приложения. В результате на ViewModel устанавливается новое состояние, которое, в свою очередь, обновляет пользовательский интерфейс. В архитектуре MVI View слушает состояние. Каждый раз, когда состояние изменяется, View получает уведомление.

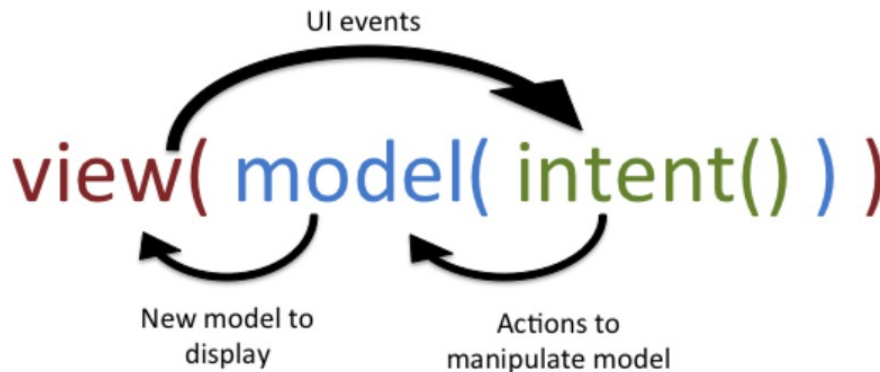


view(model(intent()))

- **intent ()** : эта функция принимает входные данные от пользователя (т.е. события пользовательского интерфейса, такие как события щелчка) и переводит их во «что-то», которое будет передано в

качестве параметра функции **model ()** . Это может быть простая строка для установки значения модели или более сложная структура данных, такая как **Действия** или **Команды**. В этом сообщении блога мы будем придерживаться слова «**Действие**»

- **model ()** : функция модели принимает выходные данные от **intent ()** в качестве входных данных для управления моделью. Результатом этой функции является новая модель (состояние изменено). Поэтому он не должен обновлять уже существующую модель. **Мы хотим неизменности!** Мы не меняем уже существующий. Копируем существующий и меняем состояние (потом его уже нельзя менять). Эта функция - единственная часть вашего кода, которой разрешено изменять объект модели. Затем эта новая неизменяемая Модель является выходом этой функции.
- **view ()** : этот метод принимает модель, возвращенную функцией **model ()**, и передает ее в качестве входных данных функции **view ()** . Тогда вид просто каким-то образом отображает эту модель.



Таким образом, представление будет генерировать «события» (шаблон наблюдателя), которые снова будут переданы в функцию **intent ()** .

MVC (КОНЦЕПЦИЯ, А НЕ ПРОСТО ШАБЛОН, ОНА ПО УМОЛЧАНИЮ РЕАЛИЗОВАНА В АНДРОИД)

ТУТ Контроллеры основаны на поведении и могут совместно использовать несколько представлений.

Представление Вью может напрямую общаться с моделью

Вернее, В Android у нас нет MVC, но у вас есть следующее:

- Вы определяете свой пользовательский интерфейс в различных файлах XML по разрешению, оборудованию и т. Д.
- Вы определяете свои ресурсы в различных файлах XML по языку и т. Д.

- Вы используете АКТИВИТИ и используете XML-файл с помощью раздувания.
- Вы можете создать столько классов, сколько хотите для своей бизнес-логики.
- Много Utils уже написано для вас – DatabaseUtils, Html.

MVC представляет собой концепцию, а не прочную структуру программирования. Вы можете реализовать свой собственный MVC на любой платформе. До тех пор, пока вы придерживаетесь следующей основной идеи, вы реализуете MVC:

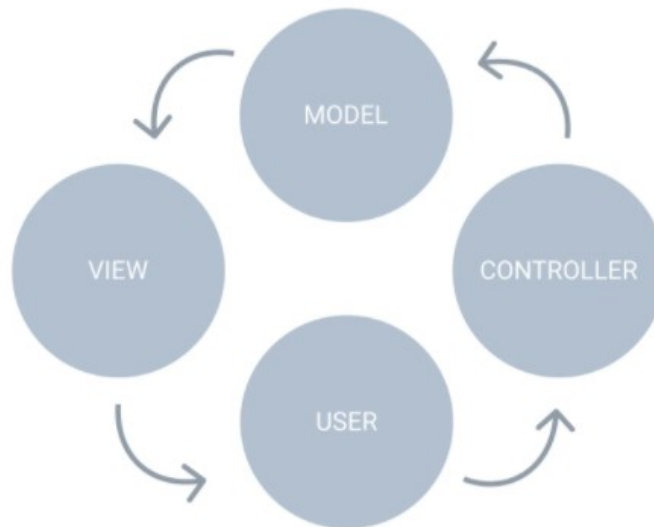
- **Модель:** что делать
- **Просмотр:** как визуализировать
- **Контроллер:** события, пользовательский ввод

позволяет разделить логику приложения на три части:

- **Model (модель).** Получает данные от контроллера, выполняет необходимые операции и передаёт их в вид.
- **View (вид или представление).** Получает данные от модели и выводит их для пользователя.
- **Controller (контроллер).** Обрабатывает действия пользователя, проверяет полученные данные и передаёт их модели.

компоненты будут следующие:

- **Вид** — интерфейс.
- **Контроллер** — обработчик событий, инициируемых пользователем (нажатие на кнопку, переход по ссылке, отправка формы).
- **Модель** — метод, который запускается обработчиком и выполняет все основные операции (получение записей из базы данных, проведение вычислений).



ИТОГ

Каждая из этих архитектур предлагает множество возможностей в области разделения задач, тестируемости и модульности, но у каждой из них есть преимущества и недостатки, которые они предлагают по отдельности.

В двух словах, MVP прост, но включает в себя серьезную проблему жизненного цикла и не предлагает многого, чего нет в архитектурах, кроме своей простоты.

MVVM в значительной степени зависит от компонентов архитектуры Android, и его может быть сложно правильно реализовать с помощью DI, но он может пережить изменения жизненного цикла и конфигурации, а также иметь несколько компонентов представления, использующих одну и ту же модель представления одновременно.

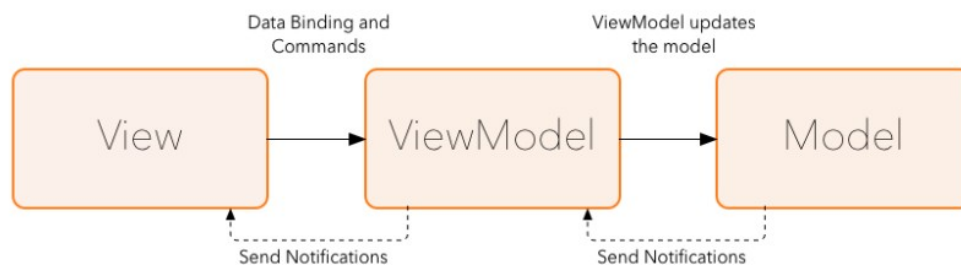
MVI предлагает строгое соблюдение SSOT Single-Source-of-Truth (SSOT), а View State обеспечивает хорошее представление данных View, но имеет очень сложную потребность в создании и поддержке State Reducers, помимо того, что имеет ту же проблему жизненного цикла, что и MVP.

7. Architecture

2. MVVM (AAC) как работает, сильные и слабые стороны, AAC, LiveData

Паттерн MVVM предполагает разделение архитектуры приложения на 3 слоя:

- Model — слой данных. Содержит всю бизнес-логику приложения, доступ к файловой системе, базе данных, ресурсам системы и другим внешним сервисам;
- View — слой отображения. Все, что видит пользователь и с чем может взаимодействовать. Этот слой отображает то, что представлено в слое ViewModel. Также он отправляет команды (например, действия пользователя) на выполнение в слой ViewModel;
- ViewModel — слой представления. Связан со слоем View байндингами. Содержит данные, которые отображены на слое View. Связан со слоем Model и получает оттуда данные для отображения. Также обрабатывает команды, поступающие из слоя View, изменяя тем самым слой Model.



Сам по себе паттерн MVVM, как и MVP, и MVC, позволяет разделить код на независимые слои. Основное отличие MVVM — в байндингах. То есть, в возможности прямо в разметке связать отображение того, что видно пользователю — слой View, с состоянием приложения — слоем Model. В общем, польза MVVM в том, чтобы не писать лишний код для связывания представления с отображением — за вас это делают байндинги.

[ViewModel](#) — это класс, представляющий объекты слоя ViewModel. Объект такого типа может быть создан из любой точки приложения. В этом классе всегда должен быть либо дефолтный конструктор (класс [ViewModel](#)), либо конструктор с параметром типа Application (класс [AndroidViewModel](#)).

Android architecture components?

набор библиотек, который помогает создавать надежные, тестируемые и легкие в поддержке приложения. Начиная с классов для управления жизненным циклом компонентов UI и управления данными приложения.

Architecture components содержат в себе много компонентов, таких как: LiveData, ViewModel, LifecycleObserver, LifecycleOwner, а также библиотеку Room для работы с БД приложения. Вместе эти компоненты помогут разработчикам следовать хорошему архитектурному паттерну с правильным разделением ответственности между слоями (чуть позже об этом), а также позволят достаточно легко и безболезненно вносить изменения, избегать утечек памяти и обновлять UI при изменении данных.

1. LifecycleOwner — интерфейс для компонентов, у которого есть свой жизненный цикл (как у Activity и Fragment)
2. LifecycleObserver — подписывается на изменения жизненного цикла LifecycleOwner'a, который самодостаточен. Он не полагается на методы onStart и onStop Activity или Fragment при инициализации и остановке.

Сам компонент состоит из классов: Lifecycle, LifecycleActivity, LifecycleFragment, LifecycleService, ProcessLifecycleOwner, LifecycleRegistry. Интерфейсов: LifecycleOwner, LifecycleObserver, LifecycleRegistryOwner.

Компонент Lifecycle – призван упростить работу с жизненным циклом. Выделены основные понятия такие как LifecycleOwner и LifecycleObserver. LifecycleOwner – это интерфейс с одним методом getLifecycle(), который возвращает состояние жизненного цикла. Являет собой абстракцию владельца жизненного цикла (Activity, Fragment). Для упрощения добавлены классы LifecycleActivity и LifecycleFragment. LifecycleObserver – интерфейс, обозначает слушателя жизненного цикла owner-а. Не имеет методов, завязан на OnLifecycleEvent, который в свою очередь разрешает отслеживать жизненный цикл.

Lifecycle — класс, который хранит информацию про состояние жизненного цикла и разрешает другим объектам отслеживать его с помощью реализации LifecycleObserver. Состоит из методов: addObserver(LifecycleObserver), removeObserver(LifecycleObserver) и getCurrentState(). Как понятно из названий для добавления подписчика, удаления и соответственно получения текущего состояния.

Для описания состояния есть два enum. Первый Events — который обозначает изменение цикла и второй State — который описывает текущее состояние.

Events — повторяет стадии жизненного цикла и состоит из ON_CREATE, ON_RESUME, ON_START, ON_PAUSE, ON_STOP, ON_DESTROY, а также ON_ANY который информирует про изменения состояния без привязки к конкретному этапу. Отслеживание изменений цикла происходит с помощью пометки метода в обсервере аннотацией OnLifecycleEvent, которому как параметр передается интересное нас событие.

State — состоит из следующих констант: INITIALIZED, CREATED, STARTED, RESUMED, DESTROYED. Для получения состояния используется метод `getCurrentState()` из `Lifecycle`. Также в Enum `State` реализован метод `itAtLeast(State)`, который отвечает на вопрос являются `State` выше или равным от переданного как параметр.

Для работы с компонентом `Lifecycle`, нам нужно определить `owner`, то есть владельца жизненного цикла и `observer`, того кто на него будет подписан. У `owner` может быть любое количество подписчиков, также стоит отметить что `observer` будет проинформирован про изменение состояния, еще до того как у `owner` будет вызван метод `super()` на соответствующий метод жизненного цикла.

Owner должен реализовывать интерфейс `LifecycleOwner`, который содержит один метод `getLifecycle()`, который возвращает экземпляр класса холдера `Lifecycle`.

Observer должен реализовать интерфейс маркер `LifecycleObserver`.

Для самостоятельного объявления кастомной `Activity/Fragment`, как `owner`-а, которые еще не поддерживают новый компонент и соответственно не имеют реализации `Lifecycle`, созданы: класс `LifecycleRegistry` и интерфейс `LifecycleRegistryOwner`.

Интерфейс `LifecycleRegistryOwner`, который в свою очередь расширяет интерфейс `LifecycleOwner` с единственным отличием в том что метод `getLifecycle()` возвращает `LifecycleRegistry` вместо `Lifecycle`.

Класс `LifecycleRegistry`, который является расширением `Lifecycle` и берет всю работу по поддержке на себя, нам лишь нужно создать его экземпляр.

В пакете `android.arch.lifecycle` приведено 4 реализации `owner`: `LifecycleActivity`, `LifecycleFragment`, `LifecycleService`, `ProcessLifecycleOwner`.

`LifecycleActivity` — это `FragmentActivity` с реализацией `LifecycleRegistryOwner`. Является временным решением для упрощения работы и как сказано в документации будет пока `Lifecycle` не будет интегрирован с `support library`.

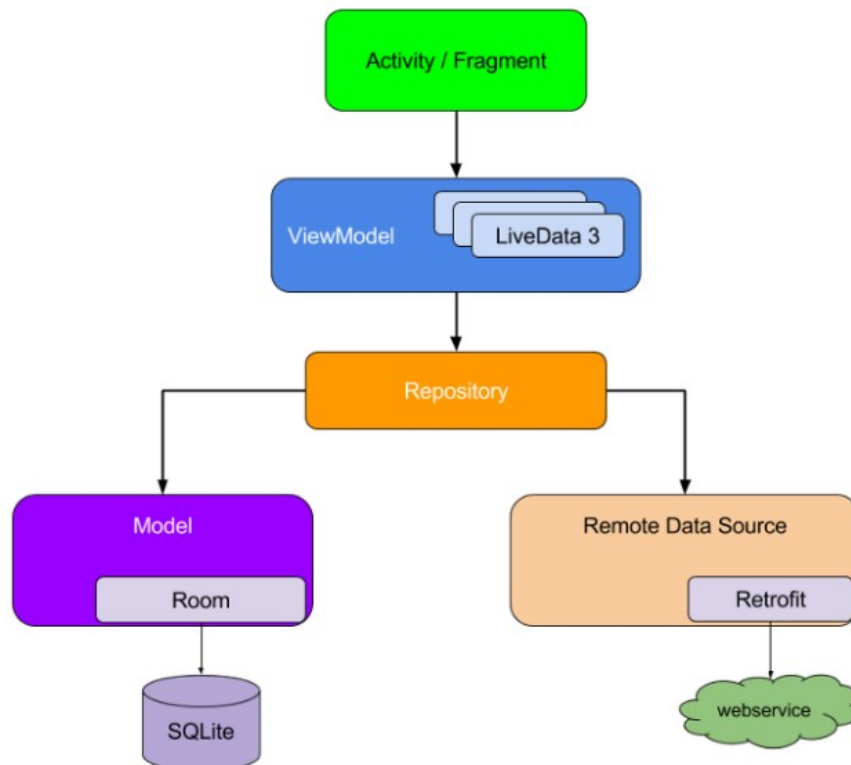
`LifecycleFragment` — это `Fragment` с пакета `support.v4`, который также как и в случае с `LifecycleActivity` реализовывает `LifecycleRegistryOwner` и является временным решением.

`LifecycleService` — это `Service`, который является также `LifecycleOwner`.

`ProcessLifecycleOwner` — это класс, который представляет `Lifecycle` всего процесса. Этот класс будет полезен если вам нужно отслеживать уход приложения в бэкграунд или возврат его на передний план.

Для того чтоб, связать owner и observer нужно у owner вызвать `getLifecycle().addObserver(LifecycleObserver)`

3. LiveData — это наблюдаемый объект для хранения данных, он уведомляет наблюдателей, когда данные изменяются. Этот компонент также является связанным с жизненным циклом LifecycleOwner (Activity или Fragment), что помогает избежать утечек памяти и других неприятностей.
4. ViewModel — это объекты, которые предоставляют данные для UI компонентов. Они не имеют ссылок на view и независимы от жизненного цикла LifecycleOwner'a.
5. Room — основанная на SQLite библиотека ORM. Room является абстракцией над слоем, в котором производятся конкретные действия над БД, такими как вставки, удаления, создания таблиц и т.д. Room использует аннотации для генерации кода во время компиляции. Более того, Room также поддерживает LiveData и RxJava2 Flowable.



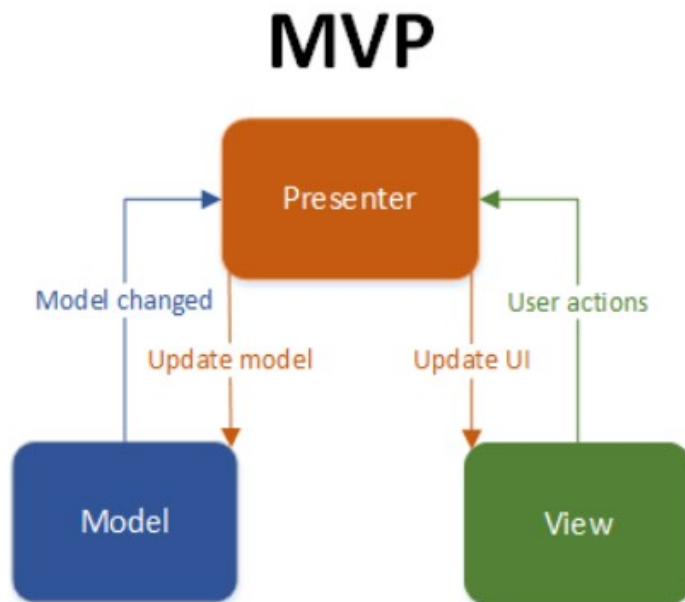
1. **View** — этот слой содержит компоненты UI и отвечает за код, управляющий компонентами пользовательского интерфейса (view), такой как инициализация дочерних view, отображение progress bar'a,

ввод данных пользователем, обработку анимаций и т.п. Например, такой код содержится в активити и фрагментах.

2. **ViewModel** — объекты этого класса предоставляют данные для компонентов UI. В нашем случае View будут использовать LiveData для отслеживания изменений данных в ViewModel.
3. **Repository** — репозитории являются абстракцией над источниками данных, которую приложение может использовать для получения данных и их кеширования. Эта абстракция полезна по двум основным причинам: 1) код не зависит от конкретной реализации хранилища данных, а также 2) в следствие предыдущей причины, мы можем легко менять конкретные реализации хранилища данных, например, для тестирования
4. **Data Service** — это слой, который содержит код для кеширования (например, используя SQLite), а также код для загрузки данных (например, загрузка данных с backend api при помощи Retrofit)

7. Architecture

3. MVP (+libraries) как работает, сильные и слабые стороны



MVP расшифровывается как Model-View-Presenter (модель-представление-презентер). Если рассматривать Activity, которое отображает какие-то данные с сервера, то View - это Activity, а Model - это ваши классы по работе с сервером. Напрямую View и Model не взаимодействуют. Для этого используется Presenter.

Если в Activity пользователь нажал кнопку Обновить, то Activity сообщает об этом презентеру. При этом Activity не просит презентер загрузить данные. Оно просто сообщает, что пользователь нажал кнопку Обновить. А презентер уже сам решает, что по нажатию этой кнопки надо делать. Он запрашивает данные у модели и передает их в Activity, чтобы отобразить на экране.

Если экран отображает данные из базы данных, то модель - это база данных. Презентер может подписаться на уведомления модели об обновлении. В случае, когда данные в БД изменятся, модель оповестит об этом презентер. Презентер получит эти изменения и передаст их в Activity.

Можно сказать, что презентер - это логика, вынесенная из Activity в отдельный класс. А Activity остается для отображения данных и взаимодействия с пользователем. Если вы решили сделать другое Activity для отображения данных, то вам уже не нужно будет переносить логику в новое Activity, можно будет использовать готовый Presenter. А если вы решили поменять логику, то вам не нужно будет лезть в Activity и там, среди кода, который отвечает за отображение данных и взаимодействие с пользователем, искать логику и менять ее. Вы меняете код в презентере.

Пример взаимодействия:

- 1) Пользователь вводит данные в поля ввода. Это никак не обрабатывается и ничего не происходит.
- 2) Пользователь жмет кнопку Add. Вот тут начинается движ.
- 3) Представление сообщает презентеру о том, что была нажата кнопка Add.
- 4) Презентер просит представление дать ему данные, которые были введены пользователем в поля ввода.
- 5) Презентер проверяет эти данные на корректность.
- 6) Если они некорректны, то презентер просит представление показать сообщение об этом.
- 7) Если данные корректны, то презентер просит представление показать прогресс-диалог и просит модель добавить данные в базу данных.
- 8) Модель асинхронно выполняет вставку данных и сообщает презентеру, что вставка завершена
- 9) Презентер просит представление убрать прогресс-диалог.
- 10) Презентер просит свежие данные у модели.
- 11) Модель возвращает данные презентеру.
- 12) Презентер просит представление показать новые данные.

Из этой схемы видно, что презентер рулит всем происходящим. Он раздает всем указания, решает, что делать и как реагировать на действия пользователя.

Model — уровень данных. Не люблю использовать термин «бизнес логика», поэтому в своих приложениях я называю его **Repository** и он общается с базой данных и сетью.

View — уровень отображения. Это будет **Activity**, **Fragment** или **Custom View**, если вы не любите плясок с бубном и взаимодействия с жизненным циклом. Напомню, что изначально все Android приложения подчинены структуре **MVC**, где **Controller** это **Activity** или **Fragment**.

Presenter — прослойка между View и Model. View передаёт ему происходящие события, презентер обрабатывает их, при необходимости обращается к Model и возвращает View данные на отрисовку.

Резюмируя теоретическую часть:

- View знает о Presenter;
- Presenter знает о View и Model (Repository);
- Model сама по себе;

Что же происходит?

- Activity, она же View, в методе onCreate() создаёт экземпляр Presenter и передаёт ему в конструктор себя.
- Presenter при создании явно получает View и создаёт экземпляр Repository (его, кстати, можно сделать Singleton)
- При нажатии на кнопку, View стучится презентеру и сообщает: «Кнопка была нажата».
- Presenter обращается к Repository: «Загрузи мне вот эту шляпу».
- Repository грузит и отдаёт «шляпу» Presenter'у.
- Presenter обращается к View: «Вот тебе данные, отрисуй»

MVP разделяет ответственность за бизнес логику и логику отображения.

MVP состоит из следующих частей:

- **Model** представляет собой данные, которые необходимо показать пользователю. В большинстве Android-приложений моделью выступает слой, отвечающий за получение данных с бэкэнда.

- **View** – это класс, отвечающий за отображение данных. В Android-приложениях View – это обычно Activity или Fragment. Кроме того View слушает пользовательские ивенты и делегирует их обработку в Presenter.

Например View может иметь такой код:

```
loginButton.setOnClickListener { presenter.onLoginClicked() }
```


• **Presenter** – это класс, который имеет ссылки и на View, и на Model, и расположен между ними. Presenter отвечает за обработку ивентов, приходящих из View, получение данных из Model и обновление View с полученными данными.

В Android-приложениях хорошей практикой считается делать Presenter независимым от Android SDK. Другими словами Presenter не имеет доступа к Android классам напрямую и может быть использован в plain java приложении.

МОХУ

Библиотека Моху позволяет избежать boilerplate кода для обработки lifecycle фрагментов и активитей, и работать с View как будто оно всегда активно. Далее под View понимается имплементация View в виде фрагмента или активити.

Под интерактором понимается сущность бизнес-логики, т.е. класс, который лежит на более низком уровне абстракции, чем Presenter.

Общие преимущества Моху

- Активная поддержка и разработка библиотеки.
- Поддержка фичей Kotlin вроде `val presenter by moxyPresenter { component.myPresenter }` и `presenterScope` для корутин.
- Автоматическое восстановление состояния View.
- Автоматическая увязка с жизненным циклом (а отсюда отсутствие утечек Активити и прочей подобной прелести).
- Обращения к View происходят через `nullable ViewState`. Нет риска, что какая-то команда View потеряется.
- Весь lifecycle экрана сводится к двум коллбэкам презентера — `onFirstViewAttach()` и `onDestroy()`.
- Время разработки экранов сокращается — не нужно писать лишний код для обработки lifecycle и сохранения состояний.

Задача: асинхронный запрос данных и отображение результата на UI

Пусть у нас есть класс (`MyListInteractor`), который возвращает список данных. В presenter мы можем позвать его асинхронно для запроса данных.

```

class MyPresenter
...
// Функция запрашивает список и отображает его на UI
override fun onDisplayListClicked() {
    myListInteractor.requestList()
        .subscribe { displayList(it) }
}

```

Решение с Моху

```

private fun displayList(items: List<Item>) {
    viewState.setListItems(items)
}

```

Обращаемся к не-nullable viewState и передаём туда загруженные данные. Моху прикопает результат и отправит View, когда оно будет активно. При пересоздании View команда может быть повторена (зависит от стратегии) при этом заново данные не будут запрашиваться.

Решение без Моху

```

private fun displayList(items: List<Item>) {
    view?.setListItems(items)
}

```

Обращаемся к View по nullable-ссылке. Эта ссылка зануляется при пересоздании View. Если к моменту завершения запроса view не приаттачена, то данные потеряются. Возможное решение проблемы.

```

private fun displayList(items: List<Item>) {
    view?.let { it.setListItems(items) }?: let {
        cachedListInteractor.saveList(items)
    }
}

```

Прикапывать данные в какой-то сущности, которая не связана с View (например, в интеракторе). При onResume() запрашивать прихраниённые данные и отображать их.
Минусы решения.

- Лишняя работа по сохранению результата из-за особенностей платформы (lifecycle Android активитей и фрагментов).
- Прихрани́нные данные нужны только View, бизнес-логика будет перегружена проблемами сохранения результата.
- Нужно следить за своевременной очисткой результата. Если объект, хранящий состояние, используется где-то ещё, то этот код также должен следить за его состоянием.
- Presenter будет знать о View lifecycle, т.к. нужно уведомлять его об onResume(). Плохо, что Presenter знает об особенностях платформы.

Задача: сохранение состояния отображения

Часто возникает ситуация, когда нам нужно хранить какое-то состояние отображения.

Решение с Моху

```
class MyPresenter
...
private var stateData: StateData = ...
```

Храним состояние в presenter. Presenter выживает при пересоздании View, поэтому там можно хранить состояние. Можно хранить данные любых типов, в т.ч. ссылки на интерфейсы, например.

Решения без Моху

- Можно хранить состояние в savedInstanceState или аргументах фрагмента.

Минусы решения

- View будет знать о state и, скорее всего, передавать его в presenter. Т.е. логика размазывается между View и presenter.
- Возможно, понадобится явно из presenter обращаться к View с целью сохранить состояние, таким образом, в интерфейсе View будут «лишние» методы сохранения состояния (должны быть только методы для управления отображением).
- Presenter может не успеть обратиться к View для сохранения состояния, т.к. ссылка на View может занулиться и сам presenter может погибнуть.
- Сохранить можно только примитивные и serializable/parcelable данные.

- Boilerplate код для сохранения данных в Bundle и извлечения из Bundle.
- Можно хранить данные на уровне бизнес-логики, в специальном классе (пусть этот класс будет отвечать только за state конкретной View и не будет использоваться где-то ещё).

Минусы решения

- Нужно следить за актуальностью данных.
- Не просто разобраться, когда создавать и уничтожать класс, хранящий данные. Обычно его просто делают singleton'ом и он существует всегда, хотя нужен только одной View.

Задача: обмен данными между экранами

Часто бывает нужно результат действий на одном View отобразить на другом View.

Решение с Moxy

Обмен данными между экранами осуществляется так же как и асинхронный запрос. Разве что подписка на изменения идёт на subject или channel в интеракторе, в который presenter другой View кидает изменённые данные. Подписка в `Presenter.onFirstViewAttach()`, отписка — в `Presenter.onDestroy()`.

Решения без Moxy

- Как и выше, через интерактор с subject'ами или channel'ами. В этом случае подписываться/отписываться нужно в каждом `onCreate()/onDestroy()`. Так же есть риск потери данных, как в случае асинхронного запроса.
- Через Broadcast или интент активити. Данные передаются через Bundle. Отсюда тот же Boilerplate с Bundle, как описано в разделе о сохранении состояния. Кроме того, в случае интента активити, логика по обмену данными ложится на View, хотя должна быть исключительно в presenter.

Задача: инициализация чего-либо, связанного с экраном

Часто бывает нужно проинициализировать что-то, связанное с данным экраном. Например, какой-то внешний компонент.

Решение с Moxy

Проинициализировать компонент можно в `Presenter.onFirstViewAttach()` и освободить в `Presenter.onDestroy()` — это единственные коллбэки, о которых нам нужно задумываться.

Presenter.onFirstViewAttach() — вызывается при самом первом создании View, Presenter.onDestroy() — вызывается при окончательном уничтожении View.

Решение без Моху

Можно проинициализировать в onCreate() и освободить в onDestroy() активити или фрагмента.

Минусы решения

- Постоянная переинициализация компонента.
- Если компонент содержит коллбеки, то возможна утечка памяти (объекта presenter или активити/фрагмента).

Задача: показ AlertDialog

Особенностью использования AlertDialog является то, что он пропадает при пересоздании View. Поэтому при пересоздании View нужно заново отображать AlertDialog.

```
class MyFragment : ... {  
  
    private val myAlertDialog = AlertDialog.Builder(context)...  
  
    override fun switchAlertDialog(show: Boolean) {  
        if (show) myAlertDialog.show() else myAlertDialog.dismiss()  
    }  
}
```

Решение с Моху

```
@StateStrategyType(AddToEndSingleStrategy::class)  
fun switchAlertDialog(show: Boolean)
```

Выбрать правильную стратегию. Диалог сам перепokaжется при пересоздании View.

Решения без Моху

- Можно в View хранить состояние отображения AlertDialog и перепokaзывать при пересоздании View. Получается лишний boilerplate просто чтобы восстановить диалог.
- Можно использовать DialogFragment. Для простых диалогов — лишний overhead. И это добавляет проблемы с commit() фрагментов после onSaveInstanceState().

Особенности использования Моху

Моху позволяет избежать boilerplate кода при использовании MVP в android-приложении. Но, как и любой другой библиотекой, Моху нужно научиться пользоваться. К счастью, использовать Моху легко. Далее описаны моменты, на которые нужно обратить внимание.

- Как команда View (т.е. вызов метода View) будет повторяться при пересоздании View – зависит от стратегии над данным методом интерфейса View. Важно понимать, что означают стратегии. Неправильный выбор стратегии может негативно сказаться на UX. К счастью, стандартных стратегий не много (всего 5) и они хорошо документированы, а создание кастомных стратегий требуется не часто.
- Моху обращается к View с того потока, с которого обратился к нему presenter. Поэтому нужно самостоятельно следить в presenter за тем, чтобы методы viewState вызвались из главного потока.
- Моху не решает проблему commit() фрагментов после выполнения onSaveInstanceState(). Разработчики Моху рекомендуют использовать commitAllowingStateLoss(). Однако, это не должно вызывать беспокойство, т.к. за состояние View полностью отвечает Моху. То, что где-то внутри android потеряется состояние View — нас не должно волновать.
- Взаимно отменяющие команды view лучше объединять в один метод View. Например, скрытие и показ прогресса лучше сделать так:

```
@StateStrategyType(AddToEndSingleStrategy::class)
fun switchProgress(show: Boolean)
```

а не так:

```
@StateStrategyType(AddToEndSingleStrategy::class)
fun showProgress()

@StateStrategyType(AddToEndSingleStrategy::class)
fun hideProgress()
```

Либо можно использовать кастомную стратегию с тегами. Например, как описано тут: <https://habr.com/ru/company/redmadrobot/blog/341108/>
Это нужно чтобы команда *показа* прогресса не вызвалась больше после команды *скрытия* прогресса.

- Presenter должен по-особому инжектится через dagger.
Например, может возникнуть желание сделать так:

```
class MyFragment : ... {  
  
    @Inject  
    lateinit var presenter: MyPresenter  
  
    @ProvidePresenter  
    fun providePresenter(): MyPresenter {  
        return presenter  
    }  
}
```

Так делать нельзя. Нужно чтобы функция @ProvidePresenter гарантированно создавала новый инстанс presenter. Здесь при пересоздании фрагмента появится новый инстанс presenter. А Моху будет работать со старым, т.к. функция providePresenter() вызывается только один раз.
Как вариант, можно в providePresenter() просто создать новый инстанс presenter:

```
@ProvidePresenter  
fun providePresenter(): MyPresenter {  
    return MyPresenterImpl(myInteractor, schedulersProvider)  
}
```

Это не очень удобно — ведь придётся инжектить в фрагмент все зависимости этого presenter.

Можно из компонента dagger сделать метод для получения presenter и позвать его в providePresenter():

```

@Component(modules = ...)
@Singleton
interface AppComponent {
    ...
    fun getMyPresenter(): MyPresenter
}

```

```

class MyFragment : ... {
    ...
    @ProvidePresenter
    fun providePresenter(): MyPresenter {
        return TheApplication.getAppComponent().getMyPresenter()
    }
}

```

Важно, чтобы провайдер presenter'а и сам presenter не были помечены аннотацией Singleton.

В последних версиях Moxy можно использовать делегаты kotlin:

```

private val myPresenter: MyPresenter by moxyPresenter {
    MyComponent.get().myPresenter
}

```

Ещё можно заинжектировать через Provider:

```

@Inject
lateinit var presenterProvider: Provider<MyPresenter>
private val presenter by moxyPresenter { presenterProvider.get() }

```

Итог

Моxy — замечательная библиотека, которая позволяет значительно упростить жизнь android-разработчика при использовании MVP. Как и с любой новой технологией или библиотекой, в начале использования Моxy неизбежно возникают ошибки, например, не верный выбор стратегии или не правильный inject Presenter'а. Однако с опытом всё становится просто и понятно и уже сложно себе представить MVP без использования Моxy.

Что такое MVP

MVP – это способ разделения ответственности в коде приложения. *Model* предоставляет данные для *Presenter*. *View* выполняет две функции: реагирует на команды от пользователя (или от элементов UI), передавая эти события в *Presenter* и изменяет gui по требованию *Presenter*. *Presenter* выступает как связующее звено между *View* и *Model*. *Presenter* получает события из *View*, обрабатывает их (используя или не используя *Model*), и командует *View* о том, как она должна себя изменить.

У такого подхода к разделению ответственности есть ряд плюсов:

1. Сильно упрощается написание тестов к коду
2. Легко менять какую-то часть, не ломая при этом другую
3. Код разбивается на мелкие кусочки, за счёт чего он становится более понятным и читабельным

В то же время, конечно, есть и минусы:

1. Кода становится больше
2. К этому подходу нужно привыкать

MVP в Android

Activity в Android является [God object](#). На ней обычно лежит следующая ответственность:

- Полное управление GUI
- Обработка взаимодействия с пользователем.
- Запуск асинхронных задач.
- Обработка результата асинхронной задачи.

Самое печальное, наш God Object не бессмертен – Activity ещё и умирает при смене конфигурации.

MVP снимает часть ответственности с Activity. Вся работа с асинхронными задачами уходит в *Presenter*. Вся бизнес-логика – в *Presenter* и *Model*. Activity, в свою очередь, становится *View*. Она начинает просто отображать то, что скажет *Presenter* и передаёт события в *Presenter*, чтобы тот решал, как быть дальше.

Перед написанием своего решения мы изучили множество статей и

реализаций концепции MVP в Android (см. ссылки в конце статьи). На основании анализа сложился список требований к решению:

1. *View* должна привязываться к уже имеющемуся *Presenter* при смене конфигурации
2. После привязывания *View* к уже имеющемуся *Presenter*, *View* должна отображать актуальное состояние *Presenter*
3. *Presenter* должен уметь (*при необходимости*) жить независимо от того, кто на него подписан или от него отписался

Моху – теория

Наше решение сильно отличается от всех прочих(даже сама концепция MVP была модернизирована) тем, что между *View* и *Presenter* затесался *ViewState*. Причём он там абсолютно необходим. Он отвечает за то, чтобы каждая *View* всегда выглядела именно так, как того хочет *Presenter*. *ViewState* хранит в себе список команд, которые были переданы из *Presenter* во *View*. И когда „новая” *View* присоединяется к *Presenter*, *ViewState* автоматически применяет к ней все команды, которые *Presenter* выдавал раньше. Таким образом получается, что независимо от того, что произойдёт со *View* по вине Android, *View* останется всё-равно в правильном состоянии. Для этого вам нужно будет только привыкнуть изменять *View* исключительно командами из *Presenter*. Заметим, что это одно из основных правил MVP и распространяется не только на Моху.

Моху – возможности

У Моху есть несколько весомых преимущества перед другими решениями:

- *Presenter* не пересоздаётся при пересоздании Activity(это в разы упрощает работу с многопоточностью)
- Автоматизация полного восстановления того, что видит пользователь при пересоздании Activity(в том числе при динамическом добавлении элементов Android View)
- Возможность из одного *Presenter* менять сразу несколько *View*(на практике оказалось чрезвычайно удобно)

Для этого в Моху есть несколько механизмов, которые можно комбинировать между собой так, как вам будет угодно. Самыми весомыми механизмами являются аннотации, на основании которых генерируется код. А во время исполнения программы, инструмент под название *MvpDelegate* начинает полноценно использовать сгенерированный код.

Доступны следующие аннотации:

- `@InjectPresenter` – аннотация для управления жизненным циклом *Presenter*
- `@InjectViewState` – аннотация для привязывания *ViewState* к *Presenter*
- `@StateStrategyType` – аннотация для управления стратегией добавления и удаления команды из очереди команд во *ViewState*
- `@GenerateViewState` – аннотация для генерации кода *ViewState* для определенного интерфейса *View*

Обо всём этом далее.

Moxy – MvpPresenter

Каждое приложение содержит в себе какую-то бизнес-логику. В концепции *MVP*, вся бизнес-логика располагается в *Presenter* и в *Model*. По факту это значит, что вы практически не программируете во *View*. Для того, чтоб ваш *Presenter* **не** превратился в God Object, нужно разделять каждый отдельный блок бизнес-логики в отдельный *Presenter*. В таком случае у вас получится много *Presenter*, но они будут очень простыми и понятными. Например, если у вас на одном экране было две бизнес-логики, а затем они разошлись на 2 разных экрана, то вы просто измените *View*. А *Presenter* какими были, такими и останутся. Так же, в этом случае вы сможете легко переиспользовать один *Presenter* в нескольких местах(например, *BasketPresenter*, сквозной через всё приложение). Ещё это упростит тестирование кода – вы просто проверите небольшой *Presenter*, что он всё делает правильно.

Для *Presenter* в *Moxy* заведен класс `MvpPresenter<View extends MvpView>`. В *MvpPresenter* содержится экземпляр *ViewState*, который в тоже время должен реализовывать тот самый тип *View*, который пришёл в *MvpPresenter*. Доступ к этому экземпляру *ViewState* можно получить из метода `public View getViewState()`. А во время разработки вы не думаете, что работаете со *ViewState*, а просто даёте через этот метод команды для *View*, как ей измениться. Так же есть методы для привязывания/отвязывания *View* от *Presenter* (`public void attachView(View view)` и `public void detachView(View view)`). Обратите внимание на то, что к одному *Presenter* может быть привязано несколько *View*. Они будут всегда иметь актуальное состояние(за счёт *ViewState*). А если вы хотите, чтобы привязывание/отвязывание *View* проходило не через стандартное поле *ViewState*, то можете переопределить эти методы и работать с пришедшей *View* как хотите. Например, вы можете захотеть использовать нестандартный *ViewState*, который не реализует интерфейс *View*, если вам нужно.

В классе *MvpPresenter* так же есть интересный метод `protected void onFirstViewAttach()`. Очень важно понять, когда этот метод будет вызван и зачем он нужен. Этот метод вызывается тогда, когда к конкретному экземпляру *Presenter* первый раз будет привязана любая *View*. А когда к этому *Presenter* будет привязана *другая View*, к ней уже будет применено состояние из *ViewState*. И здесь уже не важно, эта *новая View* – совсем другая *View*, или пересозданная в результате смены конфигурации. Этот метод подходит для того, чтобы, например, загрузить список новостей при первом открытии экрана списка новостей.

В момент, когда во *View* пришла команда, вам может потребоваться понять, это новая команда, или это команда для восстановления состояния? Например, если это свежая команда, то нужно применить команду с анимацией. А иначе не надо применять анимацию. Можно это сделать через разные *StateStrategy*, или через сложные флаги в *Bundle savedInstanceState*. Но правильным решение будет использовать метод *Presenter*(или *ViewState*) `public boolean isInRestoreState(View view)`, который сообщит вам, в каком состоянии находится конкретная *View*. Таким образом вы сможете понять, нужна ли вам анимация, или нет.

Moxy – *MvpView* и *MvpViewState*

Самым простым компонентом *MVP* является *View*. Вам нужно завести интерфейс, который наследуется от интерфейса-маркера *MvpView* и описать в нём методы, которые будет уметь выполнять *View*. В дополнение ко *View*, наша библиотека имеет сущность *ViewState*, которая непосредственно связана со *View*. *ViewState* является наследником *MvpViewState<View extends MvpView>*. Он управляет одним, или несколькими, *View*(все одного типа *View*). И каждый раз, когда во *ViewState* приходит команда из *Presenter*, *ViewState* отправляет её всем *View*, о которых он знает. Также у *MvpViewState* есть метод `protected abstract void restoreState(View view)`, который будет вызван когда какая-нибудь *View* будет пересоздана, или когда к *Presenter*ко *ViewState* будет привязана новая *View*. Именно после того, как выполнится этот метод, „новая” *View* примет нужное состояние.

Стоит заметить, что *MvpViewState* хранит в себе список всех привязанных к нему *View*. И будет хорошо, если вы не будете забывать отвязывать *View*, которые уже уничтожены. Но если вы вдруг забудете это сделать, сильно не переживайте – в *MvpViewState* хранятся не прямые ссылки на *View*, а *WeakReference*, что всё-таки поможет GC. А в случае, если вы используете такой механизм, как *MvpDelegate*, то можете не беспокоиться об этом – он как привязывает *View* к *Presenter*, так и отвязывает их.

Moxy – *@GenerateViewState* и *@InjectViewState*

Так как *ViewState* в большинстве случаев является довольно однообразной прослойкой между *View* и *Presenter*, был написан генератор кода, который сделает за вас всю грязную работу. Применяя аннотацию `@GenerateViewState` к вашему интерфейсу *View*, вы получите сгенерированный класс *ViewState*. И чтобы вам не пришлось в *Presenter* самостоятельно искать и создавать экземпляры этого класса, есть аннотация `@InjectViewState`. Достаточно просто применить её к классу вашего *Presenter*. Дальше *MvpPresenter* сам всё сделает – он создаст экземпляр этого *ViewState*, сложит его себе в качестве поля и будет везде использовать его. Вам же просто останется работать с методом `public View getViewState()` из *MvpPresenter*.

В том случае, если вы не хотите использовать `@GenerateViewState`, но ваш *ViewState* реализует интерфейс *View*, вы можете по-прежнему использовать аннотацию `@InjectViewState`. В таком случае, передайте в эту аннотацию, в качестве параметра, класс вашего *ViewState*.

Будьте аккуратны при применении аннотации `@InjectViewState` к типизируемому *Presenter*.

Также учтите, что интерфейс, аннотированный `@GenerateViewState` должен быть не типизированным

Moxy – StateStrategy для команд во ViewState

По умолчанию, все команды для *View* сохраняются во *ViewState* просто в том порядке, в котором они туда поступали. И после того, как команды были применены, они продолжают лежать в этой очереди. Но это поведение можно поменять, применяя аннотацию `@StateStrategyType` к интерфейсу *View* и к его методам. На вход эта аннотация получает параметр, в котором вы должны указать класс *StateStrategy*, который вы хотите использовать. Если применить эту аннотацию ко всему интерфейсу *View*, то те методы, для которых стратегия не указана, будут использовать эту стратегию.

StateStrategy управляет очередью команд через два метода: `void beforeApply` и `void afterApply`. Первый метод будет вызван перед тем, как команда будет отправлена во *View* (метод `beforeApply` будет вызван сразу, как только поступит какая-то команда из *Presenter*). В этом месте, в стратегии, указанной по умолчанию, и происходит добавление команды в очередь. Второй метод `afterApply` будет вызван каждый раз, когда команда будет применена ко *View*. И в первом, и во втором методе вы можете менять список команд как хотите.

Давайте рассмотрим стратегии, которые уже реализованы в *Moxy*:

- *AddToEndStrategy* – добавит пришедшую команду в конец очереди. *Используется по умолчанию*

- `AddToEndSingleStrategy` – добавит пришедшую команду в конец очереди команд. Причём, если команда такого типа уже есть в очереди, то уже существующая будет удалена
- `SingleStateStrategy` – очистит всю очередь команд, после чего добавит себя в неё
- `SkipStrategy` – команда не будет добавлена в очередь, и никак не изменит очередь

Если же у вас какая-то специфичная логика и вам не хватает этих стратегий, то вы можете сделать свою стратегию. В этом случае вам поможет механизм тегирования методов. В аннотацию `@StateStrategyType` можно передать параметр `tag` (по умолчанию является названием метода). Затем, по этому тегу, вы сможете в методах `void beforeApply(List<ViewCommand<View>> currentState, ViewCommand<View> incomingCommand)` и `void afterApply(List<ViewCommand<View>> currentState, ViewCommand<View> incomingCommand)` понять, что за `ViewCommand` вам пришли (из метода `ViewCommand String getTag()`).

Перед написанием своих стратегий, посмотрите на код уже реализованных – возможно он будет вам полезен.

Моху – *MvpDelegate* и жизненный цикл *MvpPresenter*

Сам по себе, *Presenter* нигде не создаётся, нигде не хранится и ниоткуда не достаётся. И чтобы вам не пришлось ничего придумывать для решения этих задач, мы сделали такой механизм, как *MvpDelegate*. Он следит за тем, чтобы там, где есть его экземпляр, были правильно инициализированы все *Presenter*. Для этого от вас требуется только передать в него все основные моменты жизненного цикла вашей *View*. Посмотреть какие методы когда вызывать, вы можете в классе *MvpActivity* или *MvpFragment*.

Для того, чтобы *MvpDelegate* нашел все *Presenter*, вы должны отметить их аннотацией `@InjectPresenter`. Эта аннотация очень мощная. Через неё вы можете управлять тем, сколько времени будет жить *Presenter*. Если вы хотите, чтобы *Presenter* жил только пока есть *View*, в которой он содержится (+ пока происходит смена конфигурации), то просто добавьте эту аннотацию к полю *Presenter*. В случае, если вы хотите, чтобы *Presenter* жил не зависимо от того, кто и когда на него подписан, вам нужно будет сделать две вещи. Первое – нужно сообщить *MvpDelegate*, что *Presenter* не привязан к жизненному циклу того, кто его запросил. Для этого, нужно выставить значение параметра `type` аннотации `@InjectPresenter` как `PresenterType.GLOBAL`. Второе – вы должны передать *MvpDelegate* информацию, по которой он сможет найти нужный вам *Presenter* в хранилище всех *Presenter*. Есть два варианта, как это сделать:

Первый вариант. В аннотации `@InjectPresenter` выставляете значение для

параметра `tag`. Тогда *MvpDelegate* попытается найти в глобальном хранилище *Presenter* с таким тэгом. Если он его найдёт, то просто установит его в это поле. Иначе он создаст подходящий *Presenter*, сложит его в хранилище, и установит его в это поле. С учётом того, что к одному *Presenter* может быть привязано несколько *View*, этот механизм открывает очень много возможностей перед вами.

Второй вариант(для параметризованного тэга). По сути, он похож на первый вариант. Отличие лишь в том, что во втором случае вы не можете заранее знать, какой тэг будет у *Presenter*. Т.е. тэг должен генерироваться динамически. Тогда вам придётся немного постараться:

1. Создайте свою реализацию *PresenterFactory*
2. В аннотацию `@InjectPresenter` установите параметры:
 - В `factory` установите класс вашей *PresenterFactory*
 - В `presenterId` установите строковый идентификатор *Presenter*(это нужно для того, чтобы различать в одном классе *Presenter* с одинаковыми фабриками)
3. Заведите свой интерфейс, содержащий один и только один метод, который будет возвращать параметр для `factory` нужного типа:
 - Аннотируйте этот интерфейс как `@ParamsProvider(PresenterFactoryClass)`, передав аннотации, в качестве параметра, класс вашей *PresenterFactory*
 - Опишите метод, который будет возвращать параметр, должен на вход получать один параметр `String`(в этот параметр придёт тот самый параметр `presenterId` из аннотации `@InjectPresenter`)
4. Объект, который содержит *Presenter*, в аннотации `@InjectPresenter` которого указана эта *PresenterFactory*, обязан реализовывать созданный в п.п. 3. интерфейс

Здесь вам стоит знать, что вам не показалось, что это место слишком запутанно. Так и есть, оно запутано. Просто знайте, что если вам потребуется такая функциональность, следуйте этому небольшому списку правил, и вы сами всё поймёте и у вас всё получится.

Кроме указанной выше функциональности, *MvpDelegate* умеет быть родительским/дочерним делегатом для другого. Это необходимо для того, чтобы вы могли автоматизировать жизненный цикл *Presenter* не только внутри *Activity/Fragment*, но и внутри других элементов, у которых нет самостоятельного жизненного цикла(например, в адаптере или даже в *ViewHolder* элемента адаптера). Если вы установите для одного *MvpDelegate* в качестве родительского другой *MvpDelegate*, то делегат-

потомок будет получать все события жизненного цикла делегата-родителя. Для этого просто вызовите у целевого MvpDelegate метод `public void setParentDelegate(MvpDelegate delegate, String childId)`. В качестве `delegate` он ожидает получить родительский MvpDelegate. В качестве `childId`, вы должны указать уникальный идентификатор, по которому локальные *Presenter* одного делегата-потомка будут отличаться от локальных *Presenter* другого делегата-потомка.

Отметим, что если у родительского MvpDelegate уже был вызван метод `onCreate`, то вам необходимо самостоятельно вызвать метод `onCreate` у делегата-потомка. Почему это важно? Чтобы это понять, разберёмся, как работает MvpDelegate.

MvpDelegate кроме того, что управляет инициализацией полей *Presenter*, он делает ещё одну очень важную вещь. Он привязывает и отвязывает *View* от *Presenter*. Привязывание *View* к *Presenter* происходит в методе `onStart`, а отвязывание – в методе `onDestroy`. У *Fragment* немного по другому, см. на [github](#).

Почему именно в этих методах?

MvpDelegate использует специальное хранилище для *Presenter*. Доступ к этому хранилищу он получает через MvpFacade. MvpFacade – содержит в себе хранилище *Presenter* и некоторые другие элементы, призванные помочь MvpDelegate делать его работу оптимально. Не смотря на то, что MvpFacade является синглтоном, будет здорово, если вы выполните его метод `public static void init()` например, в методе `onCreate()` вашего Application. Или вы можете наследовать ваш Application от MvpApplication, поставляемого в Моху. Тогда в момент, когда MvpDelegate обратится к этому синглтону, он уже будет готов к работе.

Моху – Model

Важным элементом MVP является *Model*. Но в Моху эта часть MVP никак не затронута. Всё дело в том, что в этом нет смысла. В каждом проекте свои требования к *Model*. Где-то *Model* это просто набор классов для работы с API и сама работа с API (например, через Retrofit). Где-то в *Model* входит ещё и дополнительная бизнес-логика. В каких-то проектах актуально использование подхода Clean Architecture. В таком случае внутри *Model* появляются дополнительные сущности, например, *Interactor* и *Repository*. А с учётом того, что *Presenter* полностью отвязан от жизненного цикла Activity, вы можете спокойно создавать экземпляр конкретной *Model* внутри *Presenter* и работать с ним. Используя DI вы можете подключать нужную *Model* в *Presenter*. А в будущем, используя тот же DI, спокойно подменять *Model* для тестов.

В любом случае, крайне удобно для работы с *Model* использовать Rx. Тогда вы можете сделать так, чтобы публичные

методы *Model* возвращали *Observable*. В таком случае будет легко сделать взаимодействие *Model=Presenter*, и в то же время *Model⇒Model*. Это даст возможность легко сделать параллельное исполнение запросов из *Presenter* в *Model*.

Моху – итог

В результате мы имеем библиотеку, которая решает все проблемы жизненного цикла. Вы всегда будете показывать пользователю именно то состояние, которое для него актуально, и в то же время, вам не придётся делать ничего лишнего. Только опишите все команды для *View* отдельными методами. И избегайте изменения *View* из самого *View*. Если вы показали диалог командой из *Presenter*, то и при закрытии диалога, должна быть команда из *Presenter*. Иначе *ViewState* снова покажет вам диалог после смены конфигурации.

Хотелось бы заметить, что библиотека никак не ограничивает вас в выборе реализации многопоточности в вашем приложении. Вы можете использовать Rx, AsyncTask, Thread, Executor. Главное, будьте аккуратны, работайте со *View* только с главного потока. Ещё, Моху не решит проблем с *commit()* фрагментов после выполнения *onSaveInstanceState()*. Поэтому не забудьте закрывать транзакцию, используя *commitAllowingStateLoss()*. Так же, она не решит проблем с утечкой памяти – если вы передадите ссылку на *Context/Activity/Fragment* в *Presenter* (а потом ещё и во *ViewState*), то память может утечь. Будьте аккуратны.

СТРАТЕГИИ

AddToEndStrategy — выполнить команду и добавить команду в конец очереди

AddToEndSingleStrategy — выполнить команду, добавить ее в конец очереди и удалить все ее предыдущие экземпляры

SingleStateStrategy — выполнить команду, очистить очередь и добавить в нее команду

SkipStrategy — выполнить команду

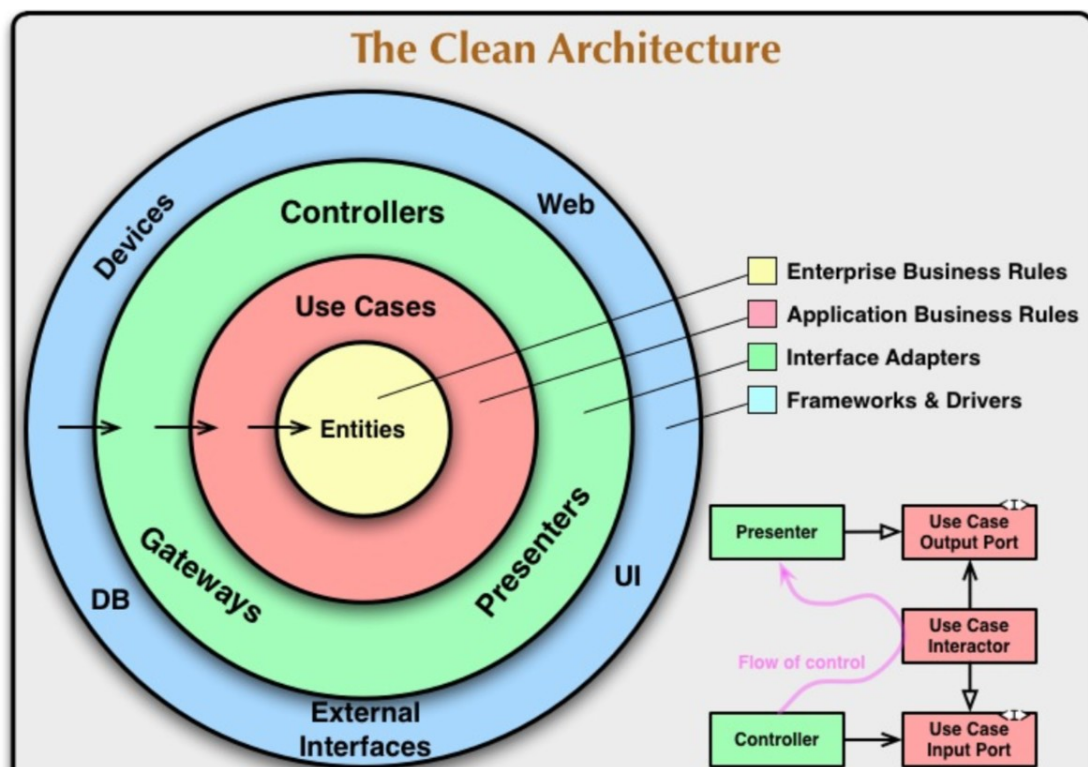
OneExecuteStrategy — выполнить команду при первой возможности

7. Architecture

4. Clean зачем это было придумано, как реализовать, architecture достоинства и недостатки

В 2011 году [Robert C. Martin](#), также известный как Uncle Bob, опубликовал статью [Screaming Architecture](#), в которой говорится, что архитектура должна «кричать» о самом приложении, а не о том, какие фреймворки в нем используются. Позже вышла [статья](#), в которой Uncle Bob даёт отпор высказывающимся против идей чистой архитектуры. А в 2012 году он

опубликовал статью «[The Clean Architecture](#)», которая и является основным описанием этого подхода



Clean Architecture

Clean Architecture объединила в себе идеи нескольких других архитектурных подходов, которые сходятся в том, что **архитектура должна:**

- быть тестируемой;
- не зависеть от UI;
- не зависеть от БД, внешних фреймворков и библиотек.

Это достигается разделением на слои и следованием Dependency Rule (правилу зависимостей).

Dependency Rule

Dependency Rule говорит нам, что внутренние слои не должны зависеть от внешних. То есть наша бизнес-логика и логика приложения не должны зависеть от презентеров, UI, баз данных и т.п. На оригинальной схеме это правило изображено стрелками, указывающими внутрь.

В статье сказано: *имена сущностей (классов, функций, переменных, чего угодно), объявленных во внешних слоях, не должны встречаться в коде внутренних слоев.*

Это правило позволяет строить системы, которые будет проще поддерживать, потому что изменения во внешних слоях не затронут внутренние слои.

Слои

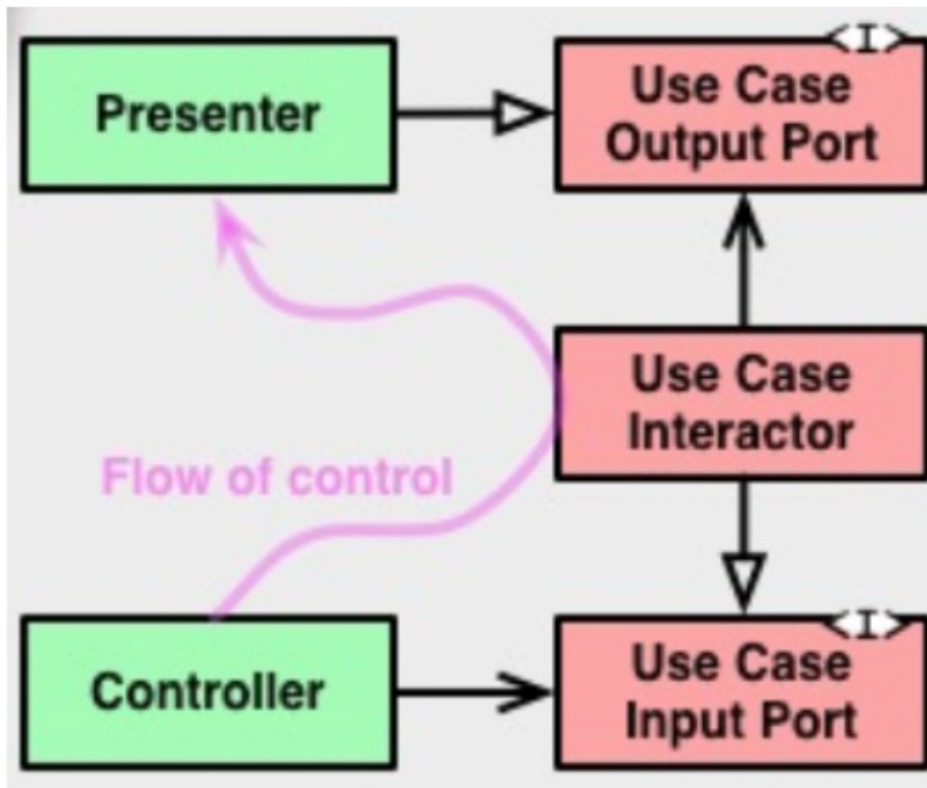
Uncle Bob выделяет 4 слоя:

- **Entities.** Бизнес-логика общая для многих приложений.
- **Use Cases (Interactors).** Логика приложения.
- **Interface Adapters.** Адаптеры между Use Cases и внешним миром. Сюда попадают Presenter'ы из MVP, а также Gateways (более популярное название репозитории).
- **Frameworks.** Самый внешний слой, тут лежит все остальное: UI, база данных, http-клиент, и т.п.

пока остановимся на передаче данных между ними.

Переходы

Переходы между слоями осуществляются через Boundaries, то есть через два интерфейса: один для запроса и один для ответа. Их можно увидеть справа на оригинальной схеме (Input/OutputPort). Они нужны, чтобы внутренний слой не зависел от внешнего (следуя Dependency Rule), но при этом мог передать ему данные.



Оба интерфейса относятся к внутреннему слою (обратите внимание на их цвет на картинке).

Смотрите, Controller вызывает метод у InputPort, его реализует UseCase, а затем UseCase отдает ответ интерфейсу OutputPort, который реализует Presenter. То есть данные пересекли границу между слоями, но при этом все зависимости указывают внутрь на слой UseCase'ов.

Чтобы зависимость была направлена в сторону обратную потоку данных, применяется *принцип инверсии зависимостей* (буква D из аббревиатуры **SOLID**). То есть, вместо того чтобы UseCase напрямую зависел от Presenter'a (что нарушало бы Dependency Rule), он зависит от интерфейса в своём слое, а Presenter должен этот интерфейс реализовать.

Точно та же схема работает и в других местах, например, при обращении UseCase к Gateway/Repository. Чтобы не зависеть от репозитория, выделяется интерфейс и кладется в слой UseCases.

Что же касается данных, которые пересекают границы, то это должны быть **простые структуры**. Они могут передаваться как **DTO** или быть завернуты в HashMap, или просто быть аргументами при вызове метода. Но они обязательно должны быть в *форме более удобной для внутреннего слоя* (лежать во внутреннем слое).

Особенности мобильных приложений

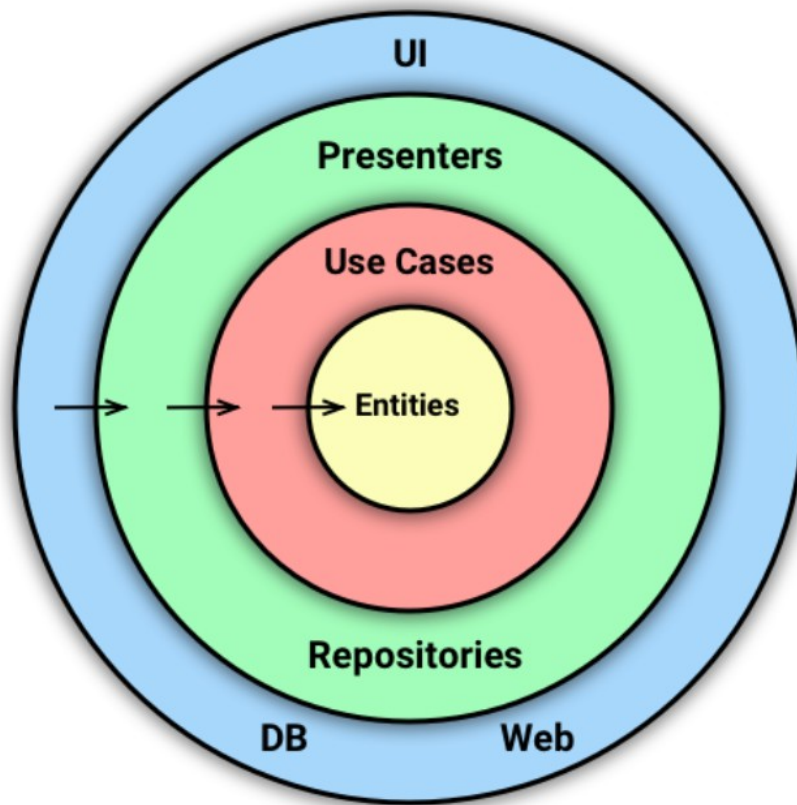
Надо отметить, что **Clean Architecture** была придумана с немного иным **типом приложений на уме**. Большие серверные приложения для крупного бизнеса, а не мобильные клиент-серверные приложения средней сложности, которые не нуждаются в дальнейшем развитии (конечно, бывают разные приложения, но согласитесь, в большей массе они именно такие). Непонимание этого может привести к [overengineering](#)'у.

На оригинальной схеме есть слово *Controllers*. Оно появилось на схеме из-за *frontend*'а, в частности из *Ruby On Rails*. Там зачастую разделяют *Controller*, который обрабатывает запрос и отдает результат, и *Presenter*, который выводит этот результат на *View*. Многие не сразу догадываются, но в *android-приложениях Controllers не нужны*.

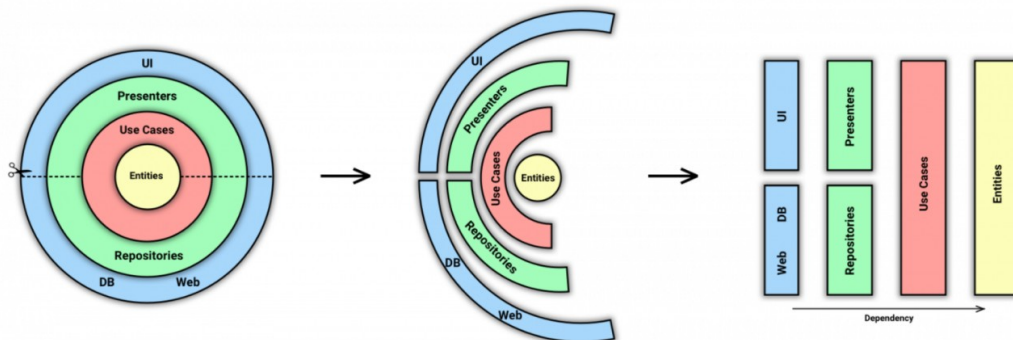
Ещё в статье *Uncle Bob* говорит, что *слоёв не обязательно должно быть 4*. Может быть любое количество, но *Dependency Rule* должен всегда применяться.

Заблуждение: Слои и линейность

очистим основную схему, убрав из нее лишнее для нас. И переименуем Gateways в Repositories, т.к. это более распространенное название этой сущности.

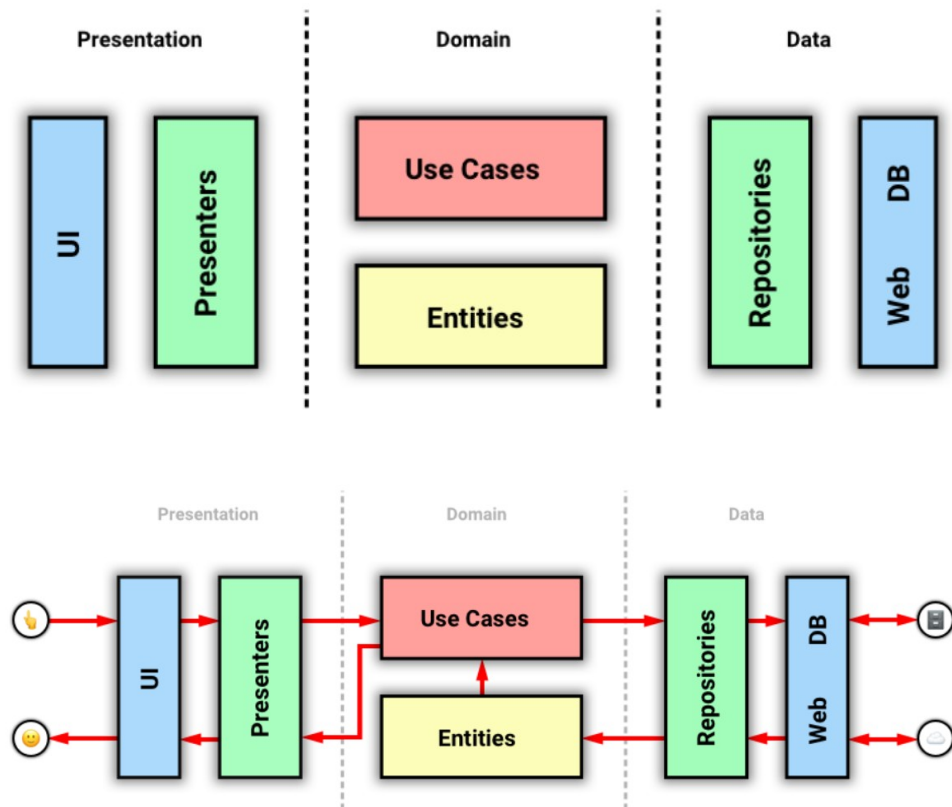


Стало немного понятнее. Теперь мы сделаем вот что: *разрежем слои на части* и превратим эту схему в блочную, где цвет будет по-прежнему обозначать принадлежность к слою.



Как я уже сказал выше, цвета обозначают слои. А стрелка внизу обозначает Dependency Rule.

На получившейся схеме уже проще представить себе течение данных от UI к БД или серверу и обратно. Но давайте сделаем еще один шаг к линейности, расположив слои *по категориям*:



Красными стрелками показано течение данных.

Заблуждение: Entities

Entities по праву занимают первое место по непониманию.

Мало того, что почти никто (включая меня до недавнего времени) не осознает, что же это такое на самом деле, так их ещё и путают с DTO.

Однажды в чате у меня возник спор, в котором мой оппонент доказывал мне, что Entity – это объекты, полученные после парсинга JSON в data-слое, а DTO – объекты, которыми оперируют Interactor'ы...

Постараемся хорошо разобраться, чтобы таких заблуждений больше не было ни у кого.

Что же такое Entities?

Чаще всего они воспринимаются как POJO-классы, с которыми работают Interactor'ы. Но это не так. По крайней мере не совсем.

В статье Uncle Bob говорит, что **Entities инкапсулируют логику бизнеса, то есть всё то, что не зависит от конкретного приложения, а будет общим для многих.** Но если у вас отдельное приложение и оно не

заточено под какой-то существующий бизнес, то Entities будут являться *бизнес-объектами приложения, содержащими самые общие и высокоуровневые правила*.

Я думаю, что именно фраза: «Entities это бизнес объекты», – запутывает больше всего. Кроме того, на приведенной выше схеме из видео Interactor получает Entity из Gateway. Это также подкрепляет ощущение, что это просто POJO объекты.

Но в статье также говорится, что *Entity может быть объектом с методами или набором структур и функций*. То есть упор делается на то, что важны методы, а не данные.

Это также подтверждается в [разъяснении](#) от Uncle Bob'a, которое я нашел недавно:

Uncle Bob говорит, что для него Entities содержат бизнес-правила, независимые от приложения. И они *не просто объекты с данными. Entities могут содержать ссылки на объекты с данными, но основное их назначение в том, чтобы реализовать методы бизнес-логики, которые могут использоваться в различных приложениях*.

А по-поводу того, что Gateways возвращают Entities на картинке, он поясняет следующее:

Реализация Gateway получает данные из БД, и использует их, чтобы создать структуры данных, которые будут переданы в Entities, которые Gateway вернет. Реализовано это может быть композицией

```
class MyEntity { private MyDataStructure data;}
```

или наследованием

```
class MyEntity extends MyDataStructure {...}
```

Итак, **слой Entities содержит**:

- Entities – функции или объекты с методами, которые реализуют логику бизнеса, общую для многих приложений (а если бизнеса нет, то самую высокоуровневую логику приложения);
- DTO, необходимые для работы и перехода между слоями.

Заблуждение: UseCase и/или Interactor

Многие путаются в понятиях *UseCase* и *Interactor*. Я слышал фразы типа: «Канонического определения Interactor нет». Или вопросы типа: «Мне делать это в Interactor'е или вынести в UseCase?».

Косвенное определение Interactor'a встречается в [статье](#), которую я уже упоминал в самом начале. Оно звучит так:

«...interactor object that implements the use case by invoking business objects.»

Таким образом:

Interactor – объект, который реализует use case (сценарий использования), используя бизнес-объекты (Entities).

Что же такое Use Case или сценарий использования?

Use case – это детализация, описание действия, которое может совершить пользователь системы.

Repository

в Android-сообществе куда более распространено определение Repository как объекта, *предоставляющего доступ к данным с возможностью выбора источника данных в зависимости от условий.*

Gateway

Сначала я тоже начал использовать Repository, но воспринимая слово «репозиторий» в значении хранилища, мне *не нравилось наличие там методов для работы с сервером типа login()* (да, работа с сервером тоже идет через Repository, ведь в конце концов для приложения сервер – это та же база данных, только расположенная удаленно).

Я начал искать альтернативное название и узнал, что многие используют Gateway – слово более подходящее, на мой вкус. А сам паттерн Gateway по сути представляет собой разновидность фасада, где мы прячем сложное API за простыми методами. Он в оригинале тоже не предусматривает выбор источников данных, но все же ближе к тому, как используем мы.

А в обсуждениях все равно приходится использовать слово «репозиторий», всем так проще.

Доступ к Repository/Gateway только через Interactor?

Многие настаивают, что это единственный правильный способ. И они правы!

В идеале использовать Repository нужно только через Interactor.

Но я не вижу ничего страшного, чтобы в простых случаях, когда не нужно никакой логики обработки данных, вызывать Repository из Presenter'a, минуя Interactor.

Repository и презентер находятся на одном слое, Dependency Rule не запрещает нам использовать Repository напрямую. Единственное но – возможное добавления логики в Interactor в будущем. Но добавить Interactor, когда понадобится, не сложно, а иметь множество proxy-interactor'ов, просто прокидывающих вызов в репозиторий, не всегда хочется.

Повторюсь, я считаю, что в идеале надо делать запросы через Interactor, но также считаю, что в небольших проектах, где вероятность добавления логики в Interactor ничтожно мала, *можно этим правилом поступиться*. В качестве компромисса с собой.

Заблуждение: Обязательность маппинга между слоями

Некоторые утверждают, что маппить данные обязательно между всеми слоями. Но это может породить большое количество дублирующихся представлений одних и тех же данных.

А можно использовать DTO из слоя Entities везде во внешних слоях. Конечно, если те могут его использовать. Нарушения Dependency Rule тут нет.

Какое решение выбрать – сильно зависит от предпочтений и от проекта. В каждом варианте есть свои плюсы и минусы.

Маппинг DTO на каждом слое:

- Изменение данных в одном слое не затрагивает другой слой;
- Аннотации, нужные для какой-то библиотеки не попадут в другие слои;
- Может быть много дублирования;
- При изменении данных все равно приходится менять маппер.

Использование DTO из слоя Entities:

- Нет дублирования кода;
- Меньше работы;
- Присутствие аннотаций, нужных для внешних библиотек на внутреннем слое;
- При изменении этого DTO, возможно придется менять код в других слоях.

Хорошее рассуждение есть вот по этой [ссылке](#).

С выводами автора ответа я полностью согласен:

Если у вас сложное приложение с логикой бизнеса и логикой приложения, и/или разные люди работают над разными слоями, то лучше разделять данные между слоями (и маппить их). Также это стоит делать, если серверное API корявое. Но если вы работаете над

проектом один, и это простое приложение, то **не усложняйте** лишним маппингом.

Заблуждение: маппинг в Interactor'e

Да, такое заблуждение существует. Развеять его несложно, приведя фразу из оригинальной [статьи](#):
(Когда мы передаем данные между слоями, они всегда в форме более удобной для внутреннего слоя)

Поэтому **в Interactor данные должны попадать уже в нужном ему виде.**

Маппинг происходит в слое Interface Adapters, то есть в Presenter и Repository.

А где раскладывать объекты?

С сервера нам приходят данные в разном виде. И иногда API навязывает нам странные вещи. Например, в ответ на login() может прийти объект Profile и объект OrderState. И, конечно же, мы хотим сохранить эти объекты в разных Repository.

Так где же нам разобрать LoginResponse и разложить Profile и OrderState по нужным репозиториям, в Interactor'e или в Repository?

Многие делают это в Interactor'e. Так проще, т.к. не надо иметь зависимости между репозиториями и разрывать иногда возникающую кроссылочность.

Но я делаю это **в Repository. По двум причинам:**

- Если мы делаем это в Interactor'e, значит мы должны передать ему LoginResponse в каком-то виде. Но тогда, чтобы не нарушать Dependency Rule, LoginResponse должен находиться в слое Interactor'a (UseCases) или Entities. А ему там не место, ведь он им кроме как для раскладывания ни для чего больше не нужен.
- Раскладывание данных – не дело для use case. Мы же не станем писать пункт в описании действия доступного пользователю: «Получить данные, разложить данные». Скорее мы напишем просто: «Получить нужные данные», – и всё.

Если вам удобно делать это в Interactor, то делайте, но считайте это компромиссом.

Можно ли объединить Interactor и Repository?

Некоторым нравится объединять Interactor и Repository. В основном это вызвано желанием избежать решения проблемы, описанной в пункте «Доступ к Repository/Gateway только через Interactor?».

Но **в оригинале Clean Architecture эти сущности не смешиваются**. И на это пара веских причин:

- Они на разных слоях.
- Они выполняют различные функции.

Мощная базовая архитектура — важный показатель для масштабируемости приложения. Внесение таких изменений, как замена API на обновленную и оптимизированную структуру API, требует переписать практически все приложение полностью.

Причина заключается в том, что код **тесно связан** с модулем данных ответа. Использование Чистой архитектуры (Clean architecture) помогает решить эту проблему. Это лучшее решение для крупных приложений с большим количеством функций и **SOLID**-принципами. Она была предложена Робертом С. Мартином (известным как Дядя Боб) в блоге “Чистый код” в 2011 году.

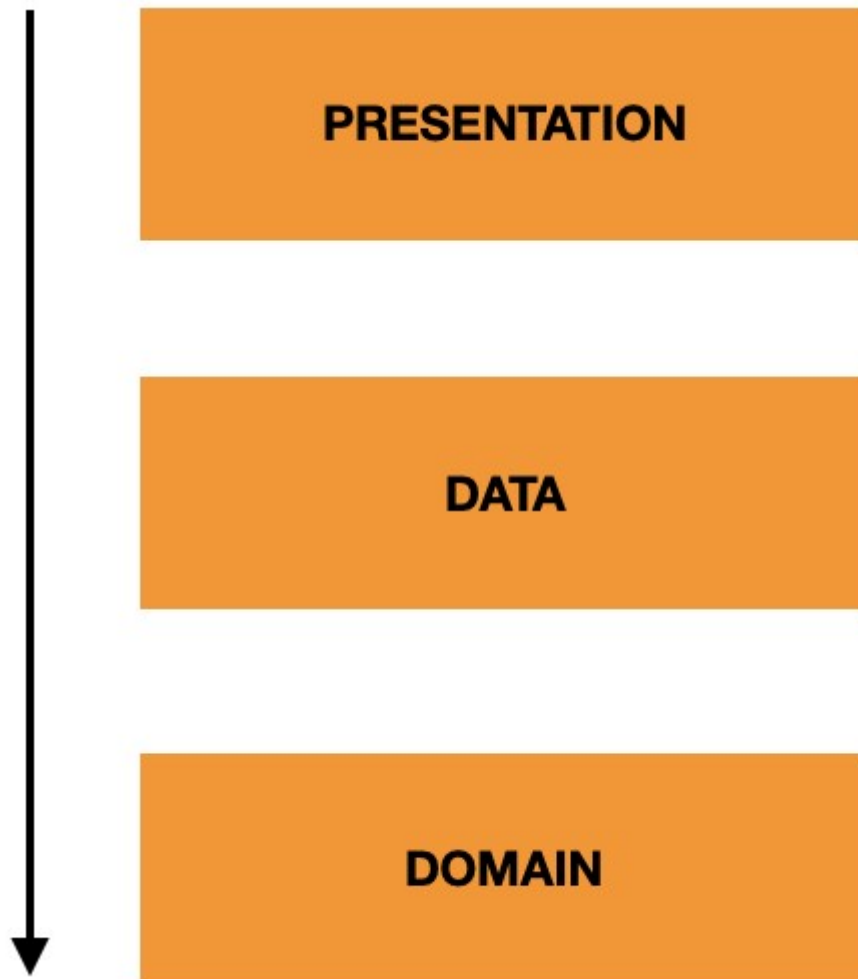
Зачем нужен чистый подход?

1. Разделение кода на разные слои с **назначенными обязанностями** облегчает дальнейшую модификацию
2. Высокий уровень **абстракции**
3. **Слабая связанность** между частями кода
4. Легкость **тестирования** кода

“Чистый код всегда выглядит так, будто написан с заботой.”

— Майкл Фэзерс

Какие бывают слои?



Dependency Flow

Domain-слой: Запускает независимую от других уровней бизнес-логику. Представляет собой чистый пакет kotlin без android-зависимостей.

Data-слой: Отправляет необходимые для приложения данные в domain-слой, реализуя предоставляемый доменом интерфейс.

Presentation-слой: Включает в себя как domain-, так и data-слои, а также является специфическим для android и выполняет UI-логику.

Что такое Domain-слой?

Базовый слой, соединяющий presentation-слой с data-слоем, в котором выполняется бизнес-логика приложения.

▼ com.rakshitjain.domain

- ▶ common
- ▶ entities
- ▶ repositories
- ▶ usecases

Структура domain-слоя приложения

UseCases

Используется в качестве исполнителя логики приложения. Как видно из названия, для каждой функциональности можно создать отдельный прецедент.

```
class GetNewsUseCase(private val transformer: FlowableRxTransformer<NewsSourcesEntity>,  
private val repositories: NewsRepository):  
BaseFlowableUseCase<NewsSourcesEntity>(transformer){
```

```
override fun createFlowable(data: Map<String, Any>?): Flowable<NewsSourcesEntity> {  
return repositories.getNews()  
}
```

```
fun getNews(): Flowable<NewsSourcesEntity>{  
val data = HashMap<String, String>()  
return single(data)  
}  
}
```

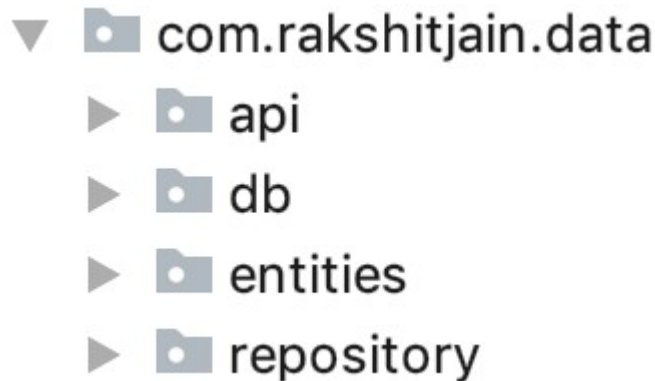
Прецедент возвращает Flowable, который можно модифицировать в соответствии с требуемым наблюдателем. Есть два параметра. Трансформер или [ObservableTransformer](#), контролирующий выбор потока для выполнения логики, и *репозиторий*, который представляет собой интерфейс для data-слоя. Для передачи данных в data-слой используется HashMap.

Репозитории

Определяют функциональности в соответствии с требованиями прецедента, которые реализуются data-слоем.

Что такое Data-слой?

Этот слой предоставляет необходимые для приложения данные. Data-слой должен быть организован таким образом, чтобы данные могли быть использованы любым приложением без модификации логики представления.



Структура data-слоя приложения

API реализуют удаленную сеть. Он может включать любую сетевую библиотеку, такую как retrofit, volley и т. д. Аналогичным образом, **DB** реализует локальную базу данных.

```
class NewsRepositoryImpl(private val remote: NewsRemoteImpl,
private val cache: NewsCacheImpl) : NewsRepository {
```

```
    override fun getLocalNews(): Flowable<NewsSourcesEntity> {
        return cache.getNews()
    }
```

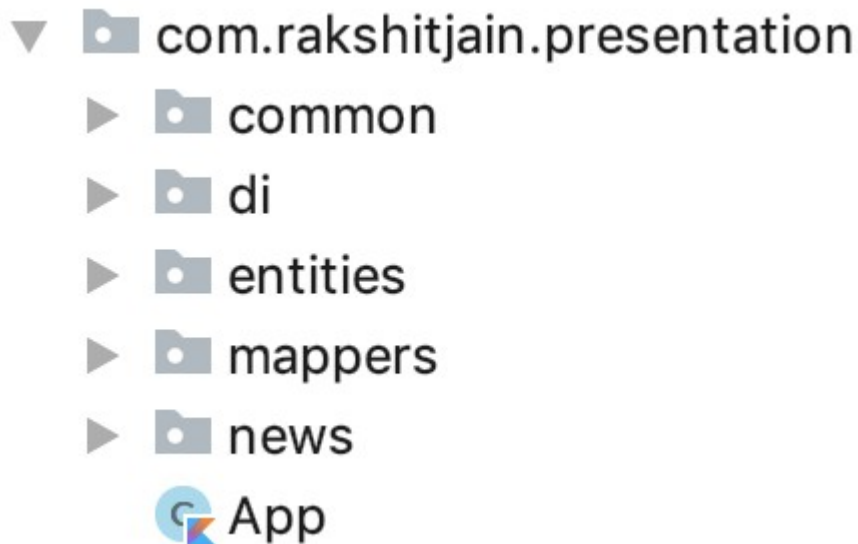
```
    override fun getRemoteNews(): Flowable<NewsSourcesEntity> {
        return remote.getNews()
    }
```

```
    override fun getNews(): Flowable<NewsSourcesEntity> {
        val updateNewsFlowable = remote.getNews()
        return cache.getNews()
            .mergeWith(updateNewsFlowable.doOnNext{
                remoteNews -> cache.saveArticles(remoteNews)
            })
    }
}
```

В репозитории реализуются локальные, удаленные и любые другие источники данных. В примере выше класс NewsRepositoryImpl.kt реализует предоставляемый domain-слоем интерфейс. Он выступает в качестве единой точки доступа для data-слоя.

Что такое presentation-слой?

Presentation-слой реализует пользовательский интерфейс приложения. Этот слой выполняет только инструкции без какой-либо логики. Он внутренне реализует такие архитектуры, как MVC, MVP, MVVM, MVI и т. д. В этом слое соединяются все части архитектуры.



Структура presentation-слоя приложения

Папка **DI** обеспечивает внедрение всех зависимостей при запуске приложения, таких как сети, View Models, Прецеденты и т.д. DI в android реализуется с помощью dagger, kodein, koin или шаблона service locator. Все зависит от типа приложения. Я выбрал koin, поскольку его легче понять и реализовать, чем dagger.

Зачем использовать ViewModels?

В соответствии с документацией android, [ViewModel](#):

Хранит и управляет данными пользовательского интерфейса с учетом жизненного цикла. С его помощью данные остаются целыми при изменении конфигурации, например, при повороте экрана.

Таким образом, ViewModel сохраняет данные при изменении конфигурации. Presenter в MVP привязан к представлению с интерфейсом, что усложняет тестирование, в то время как в ViewModel отсутствует интерфейс из-за архитектурно-ориентированных компонентов.

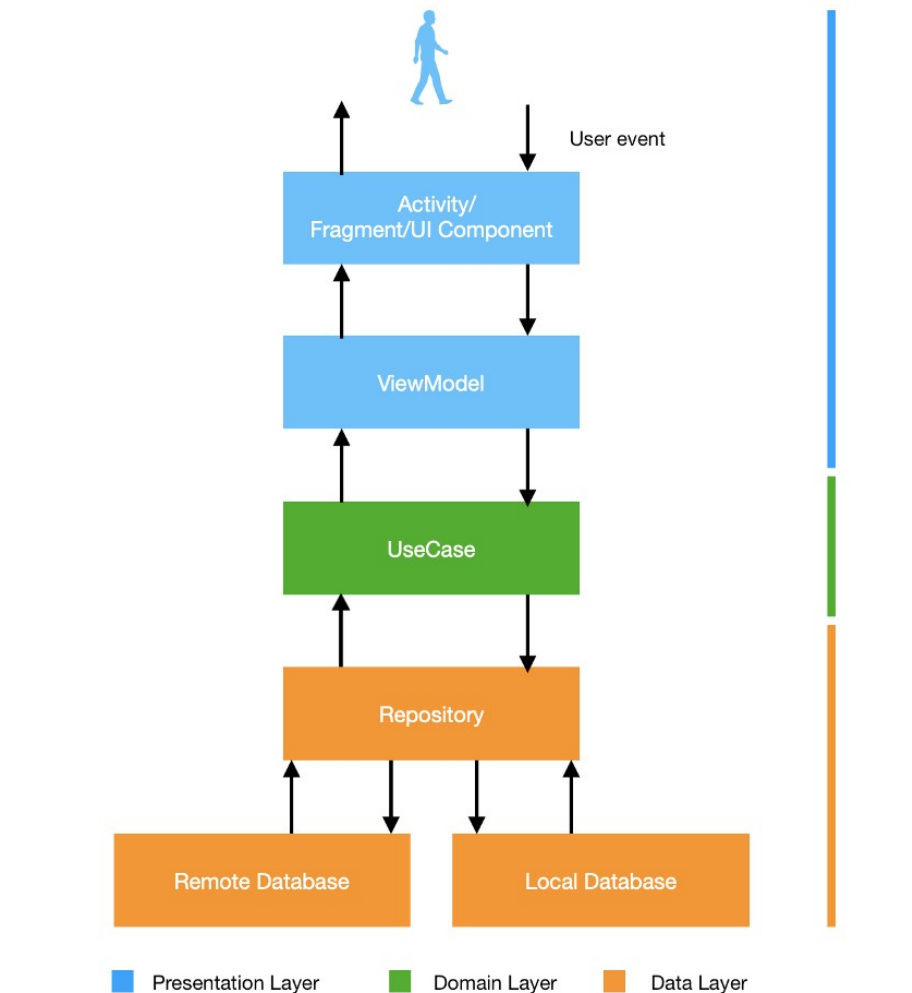
Базовый View Model использует [CompositeDisposable](#) для добавления всех observables и удаляет их на стадии жизненного цикла @OnCleared.

Класс data wrapper используется в LiveData в качестве вспомогательного класса, который уведомляет представление о состоянии запроса (запуск, результат и т.д.).

Каким образом соединяются все слои?

Каждый слой обладает собственной сущностью (*entities*), специфичной для данного пакета. Mapper преобразовывает сущность одного слоя в другую. Все слои обладают разными сущностями, благодаря чему каждый из них полностью независим, и лишь необходимые данные могут быть переданы последующему слою.

Application Flow



Заключение:

Базовая архитектура определяет единство приложения, и да, каждому приложению соответствует своя архитектура, НО почему бы не выбрать масштабируемую, надежную и тестируемую архитектуру заранее, чтобы избежать дальнейших трудностей.

8.	Testin g	Unit t esting	для чего используется, Mockito, моки, основные аннотации
----	-------------	------------------	--

dependency testImplementation используется для локальных юнит тестов и им не нужно что бы эмулятор Андроида был запущен и поэтому они очень быстрые

testImplementation связана с директорией test

директория test содержит локальные юнит тесты

dependency androidTestImplementation тестирует UI и связана с androidTest

директория androidTest содержит инструментальные тесты они так же юнит тесты, но требуют компоненты андроид и требуют запуск эмулятора Espresso

implementation будет применять зависимости ко всем вариантам сборки. Если вы вместо этого хотите объявить зависимость только для определенного исходного набора вариантов сборки или для тестового исходного набора, вы должны использовать имя конфигурации с заглавной буквы и поставить перед ним префикс с именем варианта сборки или исходного набора тестирования.

Следовательно, для вашего варианта отладки вы используете debugImplementation а для варианта выпуска вы используете releaseImplementation

Модульное тестирование (Unit Testing) – это тип тестирования программного обеспечения, при котором тестируются отдельные модули или компоненты программного обеспечения. Его цель заключается в том, чтобы проверить, что каждая единица программного кода работает должным образом. Данный вид тестирования выполняется разработчиками на этапе кодирования приложения. Модульные тесты изолируют часть кода и проверяют его работоспособность. Единицей для измерения может служить отдельная функция, метод, процедура, модуль или объект.

Зачем нужно модульное тестирование?

Отсутствие модульного тестирования при написании кода значительно увеличивает уровень дефектов при дальнейшем (интеграционном, системном, и приемочном) тестировании. Качественное модульное тестирование на этапе разработки экономит время, а следовательно, в конечном итоге, и деньги.

1. Модульные тесты позволяют исправить ошибки на ранних этапах разработки и снизить затраты.

2. Это помогает разработчикам лучше понимать кодовую базу проекта и позволяет им быстрее и проще вносить изменения в продукт.
3. Хорошие юнит-тесты служат проектной документацией.
4. Модульные тесты помогают с миграцией кода. Просто переносите код и тесты в новый проект и изменяете код, пока тесты не запустятся снова.

Как сделать модульное тестирование

Модульное тестирование делится на ручное и автоматизированное. И хотя программная инженерия не выделяет превосходство одного над другим, чаще, все-таки, используется автоматизированное. При ручном модульном тестировании, как правило используется пошаговая инструкция.

Алгоритм же автоматизированного заключается в следующем:

- Разработчик записывает в приложение единицу кода, чтобы протестировать ее. После: они комментируют и, наконец, удаляют тестовый код при развертывании приложения.
- Разработчик может изолировать единицу кода для более качественного тестирования. Эта практика подразумевает копирование кода в собственную среду тестирования. Изоляция кода помогает выявить ненужные зависимости между тестируемым кодом и другими модулями или пространствами данных в продукте.
- Кодер обычно использует UnitTest Framework для разработки автоматизированных тестовых случаев. Используя инфраструктуру автоматизации, разработчик задает критерии теста для проверки корректного выполнения кода, и в процессе выполнения тестовых случаев регистрирует неудачные. Многие фреймворки автоматически отмечают и сообщают, о неудачных тестах и могут остановить последующее тестирование, опираясь на серьезность сбоя.
- Алгоритм модульного тестирования:
 - Создание тестовых случаев
 - Просмотр / переработка
 - Базовая линия
 - Выполнение тестовых случаев.

Методы модульного тестирования

Ниже перечислены методы покрытия кода:

- Покрытие операторов (Statement coverage)
- Покрытие решений (Decision coverage)
- Покрытие ветвлений (Branch coverage)
- Покрытие условий (Condition coverage)
- Покрытие конечного автомата

Дополнительная информация [тут](#).

Пример модульного тестирования: фиктивные объекты

Модульное тестирование основывается на создании фиктивных объектов для тестирования фрагментов кода, которые еще не являются частью законченного приложения. Подставные объекты заполняют недостающие части программы.

Например, у вас может быть функция, которая нуждается в переменных или объектах, которые еще не созданы. В модульном тестировании они будут учитываться в форме фиктивных объектов, созданных исключительно для целей модульного тестирования, выполненного в этом разделе кода.

Разработка через тестирование (TDD)

Модульное тестирование в TDD включает в себя широкое использование платформ тестирования. Каркас модульного тестирования используется для создания автоматизированных модульных тестов. Структуры модульного тестирования не являются уникальными для TDD, но они необходимы для него. Ниже некоторые преимущества TDD:

- Тесты написаны перед кодом
- Можно положиться на тестирование фреймворков
- Все классы в приложениях протестированы
- Быстрая и простая интеграция

Преимущество модульного тестирования

- Разработчики, желающие узнать, какие функциональные возможности предоставляет модуль и как его использовать, могут

взглянуть на модульные тесты, чтобы получить общее представление об API модуля.

- Модульное тестирование позволяет программисту выполнить рефакторинг кода на этапе регрессионного тестирования и убедиться, что модуль все еще работает правильно. Процедура заключается в написании контрольных примеров для всех функций и методов, чтобы в случае, если изменение вызвало ошибку, его можно было быстро идентифицировать и исправить.
- Можем тестировать части проекта, не дожидаясь завершения других.

Недостатки модульного тестирования

- Не выявит всех ошибок. Невозможно оценить все пути выполнения даже в самых тривиальных программах.
- Модульное тестирование по своей природе ориентировано на единицу кода. Следовательно, он не может отловить ошибки интеграции или ошибки системного уровня.

Используйте модульное тестирование в сочетании с другими видами тестирования.

Рекомендации по модульному тестированию

- Модульные тесты должны быть независимыми. В случае каких-либо улучшений или изменений в требованиях, тестовые случаи не должны меняться.
- Тестируйте только один модуль за раз.
- Следуйте четким и последовательным соглашениям об именах для ваших модульных тестов
- В случае изменения кода в каком-либо модуле убедитесь, что для модуля имеется соответствующий тестовый пример, и модуль проходит тестирование перед изменением реализации.
- Пофиксируйте все выявленные баги перед переходом к следующему этапу, как минимум в модели разработки SDLC.
- Примите подход «тест, как ваш код». Чем больше кода вы пишете без тестирования, тем больше сценариев вам придется проверять на наличие ошибок в дальнейшем.

Резюме

Модульное тестирование может быть сложным или довольно простым, в зависимости от тестируемого приложения и используемых стратегий, инструментов и принципов тестирования. Помните, что модульное тестирование всегда становится необходимым на каком-то из уровней разработки продукта. Это уверенность.

Но нас, разумеется, будет интересовать только автоматическое тестирование, которое позволяет разработчику самостоятельно протестировать свое приложение. Кроме того, написание тестов при разработке имеет еще несколько очень важных преимуществ:

1. Если вы хотите писать тесты, то вам придется придерживаться определенного архитектурного стиля. Мы обсуждали это уже не раз, и именно возможность тестирования была одной из основных причин, из-за которой мы занимались изучением архитектуры приложений.
2. Написание тестов позволяет еще на этапе разработки выявить некоторые проблемы и случайные ошибки в вашем коде.
3. Если в вашем проекте есть автоматические тесты, вы можете с меньшим риском вносить изменения в различные участки кода, поскольку с помощью тестов вы можете быстро проверить, что новыми изменениями вы не сломали старое поведение (конечно, тесты дают не гарантию корректности кода, а только лишь некоторую уверенность в этом, что тоже хорошо).
4. Наличие тестов позволяет контролировать процесс разработки: тесты можно поставить на CI-сервер, чтобы отслеживать текущее состояние кода или проверять рабочие ветки, что очень удобно и дает гарантию того, что в продуктовую ветку попадет только код с выполняющимися тестами.
5. Автоматические тесты могут служить документацией к вашему коду.

Кроме того, существует немало других разделений по видам тестирования. Один из наиболее употребляемых и важных разделений тестирования – это разделение тестирования по степени их модульности. Здесь в общем случае выделяется 3 вида тестирования:

1. Модульное тестирование – это тестирование отдельных модулей системы в независимом от других модулей окружении. Это, вероятно, наиболее популярный и известный вид тестирования, и это логично. Чем более детальное тестирование мы выполняем, тем легче найти потенциальные ошибки, так как в тестировании одного модуля мы проверяем корректность работы только этого модуля, а он, конечно, имеет намного меньшую сложность, чем вся система в

целом. Именно для реализации модульного тестирования мы и создавали гибкую и удобную архитектуру с разбиением приложения на слои и отдельные модули.

2. Интеграционное тестирование – после проверки корректности работы отдельных модулей нужно проверить, как эти модули взаимодействуют между собой. Потому что система – это не только набор отдельных модулей, но и правила и средства их взаимодействия. Каждый модуль в отдельности может работать правильно, но при этом объединение нескольких модулей может содержать ошибки. Это тестирование выполняется уже на более высоком уровне.
3. Системное тестирование – после объединения всех модулей приложения в единую систему и проведения интеграционного тестирования для различных групп модулей нужно провести тестирование того, как система работает в целом и насколько она соответствует изначальным требованиям. Это тестирование на глобальном уровне, поэтому детали реализации отдельных модулей уже не играют роли, – система проверяется больше с точки зрения пользователя, или по методу черного ящика.

В рамках системы Android можно выполнять автоматизированное тестирование всех видов, изложенных выше. Но есть и особенности. Во-первых, в рамках нашей архитектуры основным компонентом, который содержит бизнес-логику и который должен быть протестирован в первую очередь, является делегат или Presenter. Presenter – это обычный Java-класс (с возможными зависимостями от Android), поэтому тесты для него пишутся в рамках модульного тестирования и с помощью стандартного фреймворка JUnit. Также в рамках модульного тестирования нужно протестировать работу слоя данных.

Unit-тестирование

JUnit

Мы много раз сказали о тестировании, а также о разных средствах для написания тестов. Но что же такое тесты? Интуитивно мы понимаем, что это означает. Тесты – это некоторые проверки или утверждения, которые позволяют в определенной степени убедиться в корректности работы системы.

Для модульного тестирования Java-кода используется фреймворк JUnit. Этот фреймворк позволяет конфигурировать окружение для тестов, исполнять код, который будет проверять работу некоторых классов, и выводить результаты тестов (сколько успешных тестов, сколько ошибок, и где именно произошли ошибки).

Какие основные элементы есть в этом тесте? Во-первых, в аннотации `RunWith` указывается так называемый `Runner`, который и отвечает за запуск тестов, корректный вызов и обработку всех методов.

Во-вторых, мы указываем тестовые методы. Тестовые методы – это методы с аннотацией `Test`, в которых выполняется непосредственная проверка тестируемого кода. Большинство проверок выполняется с помощью методов, начинающихся с `assert`: `assertTrue`, `assertEquals` и другие. Эти методы проверяют, соответствует ли результат работы (в нашем примере сложение чисел 3 и 4) ожидаемому значению (в нашем примере это число 7). В случае ошибки выбрасывается исключение, и JUnit информирует нас о том, что какой-то тест не выполняется.

И еще два важных метода – это методы, помеченные аннотациями `Before` и `After`. Код метода с аннотацией `Before` будет выполняться перед выполнением каждого тестового метода. Соответственно, код с аннотацией `After` будет выполняться после каждого тестового метода. Эти методы нужны для того, чтобы подготовить какие-то параметры или объекты к тестам (например, вынести тестируемый объект в поле класса и инициализировать его в методе `setUp` вместо того, чтобы выполнять инициализацию в каждом тестовом методе) или же очистить ресурсы после окончания тестового метода.

Фреймворк JUnit очень простой, и писать тесты на нем легко. По сути, все, что есть в этом примере, и есть основные возможности JUnit. Поэтому мы сразу перейдем к тестированию `Presenter`-а, который мы создавали в рамках прошлой лекции.

Дополнительно – Test Driven Development (TDD)

Test driven development – это методология разработки, в ходе которой разработчик вначале пишет тест, а уже после пишет код, который будет соответствовать этому тесту. У такого подхода есть свои преимущества:

1. Поскольку вы сначала пишете тесты, вы не сможете оказаться в ситуации, когда код уже написан, а на написания тестов нет времени или возможностей (разумеется, если будете следовать методологии TDD).
2. Мы обсуждали, что хорошая архитектура нужна для того, чтобы можно было писать тесты. Когда вы в первую очередь пишете тесты, вы вынуждены создавать архитектуру, которая позволит этим тестам выполняться.
3. Очень часто разработчики пишут тесты таким образом, чтобы они соответствовали написанному коду, а не наоборот. Из-за этого тесты часто бывают бесполезны. Когда вы пишете код так, чтобы он соответствовал вашим тестам, вы более уверены в том, что тесты

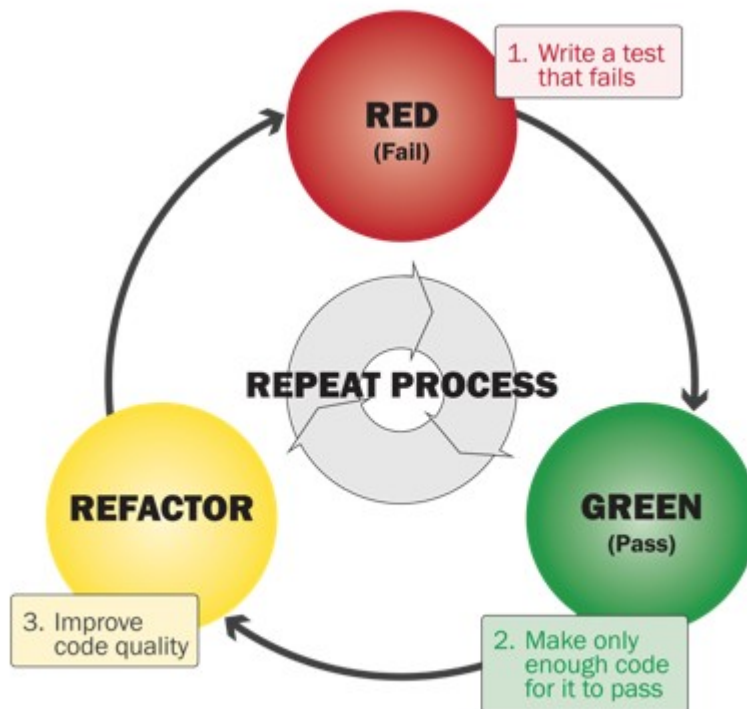
действительно проверяют корректность работы системы. Конечно, если вы умеете писать хорошие и полные тесты для ваших классов.

Процесс разработки через тестирование является полностью противоположным тому, как мы привыкли писать программы. Обычно мы пишем код и после уже тестируем его. С разработкой через тестирования все полностью наоборот.

Процесс разработки через тестирование состоит из следующих шагов:

1. Создать тест, который не выполняется.
2. Написать самый простой и примитивный код, который позволит тесту выполниться.
3. Поскольку на втором шаге был написан, возможно, неудачный код, то теперь мы должны отрефакторить его. При этом во время рефакторинга нужно постоянно запускать тест, чтобы проверить, что мы ничего не сломали в ходе рефакторинга.
4. После завершения такого процесса мы возвращаемся к пункту 1 и пишем новый тест.

Тогда структурную схему такого подхода можно представить в виде следующей диаграммы:



Когда вы следуете методологии TDD, ваши тесты являются спецификацией того, как должна вести себя система в итоговом варианте. Если вы можете написать тесты для всех пунктов вашей спецификации, то вы получите хорошую уверенность в том, что система работает как нужно.

Конечно, поначалу такая схема работы может занимать много лишнего времени, но ее стоит попробовать, так как в будущем это может помочь вам добиться высоких результатов, особенно если вы заботитесь о качестве своих программных продуктов.

Практика

1. Проект [GithubUnitTests](#)
2. Покрыть тестами WalkthroughPresenter на 100%
3. Покрыть тестами RepositoriesPresenter на 100%
4. Для измерения покрытия использовать средства Android Studio
5. Из библиотек только Mockito (без Robolectric и PowerMock)

Ссылки и полезные ресурсы

1. Приложение из [репозитория](#): GithubUnitTests – приложение с примерами написания Unit-тестов и практическим заданием.
2. Виды тестирования из [вики](#).
3. [Тестирование](#) на JUnit с assertThat.
4. [Документация](#) про тестирование в Android, где много ссылок на другие ресурсы, в том числе на очень хорошее [видео](#) с Android Dev Summit.
5. Очень хорошая [статья](#) про тестирование.
6. [Статья](#) про разные инструменты для тестирования.
7. [Кодлаб](#) про тестирование.
8. [Документация](#) по Mockito.
9. Хорошая [статья](#) по Mockito.
10. [Документация](#) по PowerMock.
11. [Ответ](#), почему PowerMock лучше не использовать часто.
12. [Документация](#) по Robolectric.

13. Wiki про [Dependency Injection](#) и [Inversion of Control](#) и [вопрос](#) на SO про оба этих принципа.
14. Библиотеки для реализации Dependency Injection в Android: [Guice](#), [Dagger 2](#) и [tiger](#).
15. [Статья](#) про использование product flavors для тестов.
16. [Подкаст](#) про тестирование.
17. [Статья](#) про тестирование с RxJava.
18. [Пример](#) использования TestRule для подмены Schedulers.
19. [Измерение](#) покрытия кода тестами с JaCoCo.
20. [Статья](#) про архитектуру приложения и тестирование, и последняя [часть](#) про TDD.
21. [Книга](#) Test Driven Development от Кента Бэка.

JUnit и фреймворк Mockito

Разработчикам программного обеспечения на разных этапах своей деятельности приходится сталкиваться с тремя стратегиями тестирования : *функциональным*, *интеграционным* и ***модульным*** тестированием. Все они используются для тестирования приложений разными способами, и каждая из стратегий имеет определенную цель. К сожалению, ни одна из стратегий не даёт стопроцентной гарантии обнаружения всех имеющихся ошибок. И даже комбинация всех трёх стратегий не может дать такой гарантии. Но их сочетание позволяет существенно снизить количество ошибок и убедить разработчика, что приложение функционирует согласно предъявленным требованиям.

Функциональное тестирование

Проведение функционального тестирования, как правило, связано с созданием специальной группы специалистов, занимающихся тестированием. На этом этапе приложение развертывается в действующем окружении и проверяется его соответствие ТЗ (Техническому Заданию) и предъявленным функциональным требованиям. Команда тестировщиков использует комплекс автоматизированных и ручных тестов.

Автоматизировать процесс функционального тестирования можно, если приложение включает API (Application Programming Interface) - интерфейс прикладного программирования, на котором оно построено. Однако наличие интерфейса в приложении (desktop, web) существенно снижает возможности полной автоматизации данного процесса.

Интеграционное тестирование

Стратегия интеграционного тестирования основывается на проверке прикладного кода в окружении, близком к фактическому окружению, но не являющимся им. Главная цель данной стратегии – убедиться в правильности взаимодействия кода с внешними ресурсами и взаимодействия различных технологий в приложении между собой.

В интеграционном тестировании не требуется использовать фиктивные данные, как при модульном тестировании. Вместо этого в интеграционных тестах часто используются базы данных, находящиеся в памяти, которые легко можно создавать и уничтожать во время выполнения тестов. База данных в памяти – это самая настоящая база данных, что дает возможность проверить правильность работы сущностей JPA. Но все же эта база данных не совсем настоящая – она лишь имитирует настоящую базу данных для целей интеграционного тестирования.

Модульное тестирование

Целью *модульного тестирования* является проверка работы прикладной логики всего приложения или отдельных его частей при разных исходных данных, и анализ правильности получаемых результатов. Несмотря на то, что цель **модульного тестирования** выглядит простой и понятной, реализация этого типа тестирования может оказаться очень сложным и запутанным делом, особенно при наличии «старого» кода. Основные приемы проведения модульного тестирования опираются на следующие базовые принципы :

- внешние ресурсы не используются, т.е. недопустимо подключение к базам данных, веб-службам и т.п.;
- каждый класс имеет свой тест;
- тестируются только общедоступные методы или интерфейсы, а внутренний код тестируется за счет изменения входных данных;
- для получения данных, требуемых тестируемой логике, должны создаваться фиктивные зависимости.

При проведении модульного тестирования для создания фиктивных классов-зависимостей можно использовать простой, но мощный фреймворк **Mockito** совместно с **JUnit**.

Фреймворк Mockito

Фреймворк *Mockito* предоставляет ряд возможностей для создания заглушек вместо реальных классов или интерфейсов при написании JUnit тестов. *Mockito* можно скачать с сайта <https://code.google.com/p/mockito>, либо определить в зависимостях (dependencies) в **maven** проекте :

```
<dependency>
<groupId>org.mockito</groupId>
<artifactId>mockito-core</artifactId>
<version>1.9.5</version>
<scope>test</scope>
</dependency>
```

Наибольшее распространение получили следующие возможности *Mockito*

- создание заглушек для классов и интерфейсов;
- проверка вызова метода и значений передаваемых методу параметров;
- использование концепции «частичной заглушки», при которой заглушка создается на класс с определением поведения, требуемое для некоторых методов класса;
- подключение к реальному классу «шпиона» *spy* для контроля вызова методов.

Синтаксис создания заглушки *Mockito*

Чтобы создать *Mockito* объект можно использовать либо [аннотацию](#) `@Mock`, либо метод `mock`. В следующем примере в двух разных классах (`Test_Mockito1`, `Test_Mockito2`) разными способами создаются объекты `mcalc` для имитации интерфейса калькулятора `ICalculator`.

```
import org.mockito.Mock;
import org.mockito.Mockito;
import com.example ICalculator;

public class Test_Mockito1
{
    @Mock
    ICalculator mcalc;
}

-----

public class Test_Mockito2
{
```

```
ICalculator mcalc = Mockito.mock(ICalculator.class);  
}
```

Если использовать **статический импорт** *Mockito*, то синтаксис будет иметь следующий вид :

```
import static org.mockito.Mockito.*;  
  
import com.example ICalculator;  
  
public class Test_Mockito  
{  
  
    ICalculator mcalc = mock(ICalculator.class);  
}
```

*ПРИМЕЧАНИЕ : помните, что методы **mock** объекта возвращают значения по умолчанию : false для boolean, 0 для int, пустые коллекции, null для остальных объектов.*

Методы Mockito в примерах

Для рассмотрения методов фреймворка **Mockito** будем использовать в качестве тестового класса калькулятор, реализующий интерфейс *ICalculator*. В следующем коде представлены листинги интерфейса *ICalculator* и класса *Calculator*.

Листинги тестового класса и интерфейса

```
public interface ICalculator  
{  
  
    public double add (double d1, double d2);  
    public double subtract (double d1, double d2);  
    public double multiply (double d1, double d2);  
    public double divide (double d1, double d2);  
}  
  
//-----  
  
public class Calculator  
{  
  
    ICalculator icalc;
```

```

public Calculator(ICalculator icalc) {
    this.icalc = icalc;
}

public double add(double d1, double d2) {
    return icalc.add(d1, d2);
}

public double subtract(double d1, double d2) {
    return icalc.subtract(d1, d2);
}

public double multiply(double d1, double d2) {
    return icalc.multiply(d1, d2);
}

public double divide(double d1, double d2) {
    return icalc.divide(d1, d2);
}

public double double15() {
    return 15.0;
}
}

```

Как видно из листингов все методы калькулятора (add, subtract, multiply, divide), за исключением одного, возвращают не вычисленные значения, а результаты выполнения данных методов в объекте, реализующим интерфейс ICalculator, который в наших тестах будет представлять заглушка *mcalc*. Последний метод *double15()* должен вернуть реальное значение.

1. Определение поведения - `when(mock).thenReturn(value)`

Этот метод позволяет определить возвращаемое значение при вызове метода *mock* с заданными параметрами. Если будет указано более одного возвращаемого значения, то они будут возвращены методом последовательно, пока не вернётся последнее; после этого при последующих вызовах будет возвращаться только последнее значение. Таким образом, чтобы метод всегда возвращал одно и то же значение, седует просто определить одно условие.

```

@RunWith(MockitoJUnitRunner.class)

public class Test_Mockito
{
    @Mock
    ICalculator mcalc;

    // используем аннотацию @InjectMocks для создания mock объекта
    @InjectMocks
    Calculator calc = new Calculator(mcalc);

    @Test
    public void testCalcAdd()
    {
        // определяем поведение калькулятора для операции сложения
        when(calc.add(10.0, 20.0)).thenReturn(30.0);

        // проверяем действие сложения
        assertEquals(calc.add(10, 20), 30.0, 0);

        // проверяем выполнение действия
        verify(mcalc).add(10.0, 20.0);

        // определение поведения с использованием doReturn
        doReturn(15.0).when(mcalc).add(10.0, 5.0);

        // проверяем действие сложения
        assertEquals(calc.add(10.0, 5.0), 15.0, 0);

        verify(mcalc).add(10.0, 5.0);
    }
}

```

Метод *verify* позволяет проверить, была ли выполнена проверка с определенными параметрами. Если проверка не выполнялась или выполнялась с другими параметрами, то *verify* вызовет исключение.

Для определения поведения *mock* в тесте была использована также следующая конструкция :

```
doReturn(value).when(mock).method(params)
```


2. Подсчет количества вызовов - `atLeast`, `atLeastOnce`, `atMost`, `times`, `never`

Для проверки количества вызовов определенных методов *Mockito* предоставляет следующие методы :

- **`atLeast (int min)`** - не меньше `min` вызовов;
- **`atLeastOnce ()`** - хотя бы один вызов;
- **`atMost (int max)`** - не более `max` вызовов;
- **`times (int cnt)`** - `cnt` вызовов;
- **`never ()`** - вызовов не было;

Следующий тест демонстрирует контроль количества вызовов метода *subtract* с разными параметрами. Для этого сначала определяется поведение *mock* (при определенных параметрах выдавать соответствующие результаты), и выполняются проверки с использованием *assertEquals*. После этого выполняется проверка количества вызовов *mock* с определенными параметрами. Две последние проверки не выполняются - «закомментированы». Если снять комментарий хотя бы с одной из них, то метод *verify*, вызовет исключение. Комментарий к этим проверкам описывает причину вызова методом исключения.

@Test

```
public void testCallMethod()
{
    // определяем поведение (результаты)
    when(mcalc.subtract(15.0, 25.0)).thenReturn(10.0);
    when(mcalc.subtract(35.0, 25.0)).thenReturn(-10.0);
    // вызов метода subtract и проверка результата
    assertEquals (calc.subtract(15.0, 25.0), 10, 0);
    assertEquals (calc.subtract(15.0, 25.0), 10, 0);
    assertEquals (calc.subtract(35.0, 25.0), -10, 0);
    // проверка вызова методов
    verify(mcalc, atLeastOnce()).subtract(35.0, 25.0);
    verify(mcalc, atLeast (2)).subtract(15.0, 25.0);
}
```

```
// проверка - был ли метод вызван 2 раза?
verify(mcalc, times(2)).subtract(15.0, 25.0);

// проверка - метод не был вызван ни разу
verify(mcalc, never()).divide(10.0, 20.0);

/* Если снять комментарий со следующей проверки, то
 * ожидается exception, поскольку метод "subtract"
 * с параметрами (35.0, 25.0) был вызван 1 раз
 */

// verify(mcalc, atLeast (2)).subtract(35.0, 25.0);

/* Если снять комментарий со следующей проверки, то
 * ожидается exception, поскольку метод "subtract"
 * с параметрами (15.0, 25.0) был вызван 2 раза, а
 * ожидался всего один вызов
 */

// verify(mcalc, atMost (1)).subtract(15.0, 25.0);
}
```

3. Обработка исключений - when(mock).thenThrow()

Mockito позволяет вызвать исключение при определенных условиях. Для этого необходимо использовать следующий синтаксис кода :

```
// создаем исключение
RuntimeException exception = new RuntimeException ("Division by zero");

// определение поведения для вызова исключения
doThrow(exception).when(mock).divide(5.0, 0));
```

В представленном коде создали **исключение** RuntimeException. После этого определили условия вызова исключения - вызов метода деления на 0. В следующем тесте выполняется проверка метода divide. При делении на 0 вызывается исключение.

```
@Test
public void testDevide()
{
```

```

when(mcalc.divide(15.0, 3)).thenReturn(5.0);
assertEquals(calc.divide(15.0, 3), 5.0, 0);
// проверка вызова метода
verify(mcalc).divide(15.0, 3);
// создаем исключение
RuntimeException exception = new RuntimeException ("Division by zero");
// определяем поведение
doThrow(exception).when(mcalc).divide(15.0, 0);
assertEquals(calc.divide(15.0, 0), 0.0, 0);
verify(mcalc).divide(15.0, 0);
}

```

4. Использование интерфейса `org.mockito.stubbing.Answer<T>`

Иногда описание поведения *mock* объекта требует определенной проверки с усложнением логики. В этом случае можно использовать интерфейс **Answer<T>**, который позволяет реализовать заглушки методов со сложным поведением. В следующем тесте *testThenAnswer* при вызове метода сложения с определенными параметрами *mcalc.add(11.0, 12.0)* будет вызван метод *answer*, который подготовит ответ. Параметр *InvocationOnMock* позволяет получить информацию о вызываемом методе и параметрах.

```

// метод обработки ответа
private Answer<Double> answer = new Answer<Double>() {
    @Override
    public Double answer(InvocationOnMock invocation) throws Throwable
    {
        // получение объекта mock
        Object mock = invocation.getMock();
        System.out.println ("mock object : " + mock.toString());
        // аргументы метода, переданные mock
        Object[] args = invocation.getArguments();
        double d1 = (double) args[0];
    }
}

```

```

double d2 = (double) args[1];
double d3 = d1 + d2;
System.out.println (" " + d1 + " + " + d2);
return d3;
}
};

@Test
public void testThenAnswer()
{
    // определение поведения mock для метода с параметрами
    when(mcalc.add(11.0, 12.0)).thenReturn(answer);
    assertEquals(calc.add(11.0,12.0), 23.0, 0);
}

```

5. Использование шпиона spy на реальных объектах

Mockito позволяет подключать к реальным объектам «шпиона» **spy**, контролировать возвращаемые методами значения и отслеживать количество вызовов методов. В следующем тесте создадим шпиона *scalc*, который подключим к реальному калькулятору и будем вызывать метод *double15()*. Необходимо отметить, что метод реального объекта *double15* должен вернуть значение 15. Однако *Mockito* позволяет переопределить значение и согласно вновь назначенному условию новое значение должно быть 23.

```

@Test
public void testSpy()
{
    Calculator scalc = spy(new Calculator());
    when(scalc.double15()).thenReturn(23.0);
    // вызов метода реального класса
    scalc.double15();
    // проверка вызова метода
    verify(scalc).double15();
}

```

```
// проверка возвращаемого методом значения
assertEquals(23.0, scalc.double15(), 0);

// проверка вызова метода не менее 2-х раз
verify(scalc, atLeast(2)).double15();
}
```

В результате выполнения теста видим, что метод возвращает значение 23. Таким образом, фреймворк **Mockito** в сочетании с JUnit можно использовать для тестов реального класса. При этом, можно проверить, вызывался ли метод, сколько раз и с какими параметрами. Кроме этого, можно создавать заглушки только для некоторых методов. Это позволяет проверить поведение одних методов, используя заглушки для других.

6. Проверка вызова метода с задержкой, `timeout`

Фреймворк *Mockito* позволяет выполнить проверку вызова определенного метода в течение заданного в **`timeout`** времени. Задержка времени определяется в миллисекундах.

```
@Test
public void testTimout()
{
    // определение результирующего значения mock для метода
    when(mcalc.add(11.0, 12.0)).thenReturn(23.0);

    // проверка значения
    assertEquals(calc.add(11.0, 12.0), 23.0, 0);

    // проверка вызова метода в течение 10 мс
    verify(mcalc, timeout(100)).add(11.0, 12.0);
}
```

7. Использование в тестах `java` классов

В следующем тесте при создании *mock* объектов используются `java` классы `Iterator` и `Comparable`. После этого определяются условия проверок и выполняются тесты.

```
@Test
public void testJavaClasses()
{

```

```
// создание объекта mock
Iterator<String> mis = mock(Iterator.class);

// формирование ответов
when(mis.next()).thenReturn("Привет").thenReturn("Mockito");

// формируем строку из ответов
String result = mis.next() + ", " + mis.next();

// проверяем
assertEquals("Привет, Mockito", result);

Comparable<String> mcs = mock(Comparable.class);
when(mcs.compareTo("Mockito")).thenReturn(1);
assertEquals(1, mcs.compareTo("Mockito"));

Comparable<Integer> mci = mock(Comparable.class);
when(mci.compareTo(anyInt())).thenReturn(1);
assertEquals(1, mci.compareTo(5));
}
```

что в структуре Mockito

1. Имитация объекта с помощью метода Mockito.mock (), который может быть внешним зависимым объектом целевого класса, например **List**
mockedList = Mockito.mock(List.class);
2. Используйте метод Mockito.when (). ThenReturn (), чтобы указать возвращаемое значение для метода фиктивного объекта, чтобы выполнять определенные действия, такие как **Mockito.when(mockedList.get(0)).thenReturn("one");**
3. Используйте метод Mockito.verify (). DoSomething (matchParam), чтобы проверить статус вызова метода (например, количество вызовов, параметры вызова и т. Д.),
Например **Mockito.verify(mockedList).get(0);** Проверить, вызывает ли объект mockList метод get (0).

Заблуждения о Mockito

1. Mockito.mock () - это не целый класс имитаций, но в соответствии с переданным классом имитируйте объект, принадлежащий этому классу, и возвращает фиктивный объект; а сам переданный класс не

изменился, используйте этот класс Новый объект никак не изменялся;

2. Параметр `Mockito.verify ()` должен быть фиктивным объектом, в противном случае будет выдано исключение «`org.mockito.exceptions.misusing.NotAMockException: аргумент, переданный в verify (), имеет тип userManager и не является фиктивным!`». Другими словами, Mockito может проверять только вызов фиктивных объектов;
3. Объект из `Mockito.mock ()` не заменяет автоматически объект в официальном коде. У вас должен быть определенный способ (метод внедрения зависимостей: внедрение конструктора, внедрение набора или внедрение параметров) для применения фиктивного объекта в официальном коде;
4. Метод `Mockito.spy ()` по умолчанию вызовет реальную реализацию этого класса и вернет соответствующее возвращаемое значение. Вы также можете использовать `Mockito.when (). ThenReturn ()`, чтобы указать поведение метода шпионского объекта;

RXJAVA

Наблюдатель — это поведенческий паттерн проектирования, который создаёт механизм подписки, позволяющий одним объектам следить и реагировать на события, происходящие в других объектах.

Паттерн Наблюдатель предлагает хранить внутри объекта издателя список ссылок на объекты подписчиков, причём издатель не должен вести список подписки самостоятельно. Он предоставит методы, с помощью которых подписчики могли бы добавлять или убирать себя из списка.

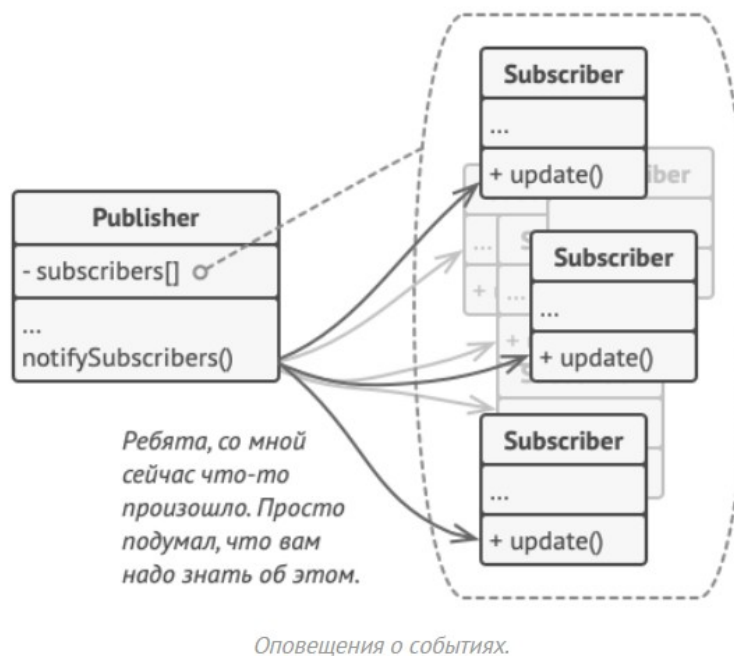


Подписка на события

Подписка на события.

Теперь самое интересное. Когда в издателе будет происходить важное событие, он будет проходиться по списку подписчиков и оповещать их об этом, вызывая определённый метод объектов-подписчиков.

Издателю безразлично, какой класс будет иметь тот или иной подписчик, так как все они должны следовать общему интерфейсу и иметь единый метод оповещения.



Строитель — это порождающий паттерн проектирования, который позволяет создавать сложные объекты пошагово. Строитель даёт возможность использовать один и тот же код строительства для получения разных представлений объектов.

ВОСХОДЯЩИЙ ПОТОК – это тот который идет к источнику данных

НИСХОДЯЩИЙ ПОТОК – это тот который идет к потребителю данных


```
source
ИСТОЧНИК

.operator1()
.оператор1()

.operator2()
.оператор2()

.operator3()
.оператор3()

.subscribe(consumer)
.подписать(потребитель)
```

In RxJava's documentation, `emission`, `emits`, `item`, `event`, `signal`, `data` and `message` are considered synonyms and represent the object traveling along the dataflow.

В документации RxJava эмиссия, эмиссия, элемент, событие, сигнал, данные и сообщение считаются синонимами и представляют объект, перемещающийся по потоку данных.

In RxJava, the dedicated `Flowable` class is designated to support backpressure and `Observable` is dedicated to the non-backpressured operations (short sequences, GUI interactions, etc.).

В RxJava выделенный класс `Flowable` предназначен для поддержки противодействия, а `Observable` предназначен для операций без противодействия (короткие последовательности, взаимодействия с графическим интерфейсом и т. д.).

The other types, `Single`, `Maybe` and `Completable` don't support backpressure nor should they; there is always room to store one item temporarily.

Другие типы, `Single`, `Maybe` и `Completable`, не поддерживают обратное давление и не должны этого делать; Всегда есть место для временного хранения одного предмета.

Completable фактически является Rx-аналогом `Runnable`. Он представляет собой операцию, которая может быть выполнена или нет. Если проводить аналогию с Kotlin, то это `fun completable()` из мира Rx. Соответственно, для подписки на него необходимо реализовать `onComplete` и `onError`. Он не может быть создан из значения (`Observable#just, ...`), потому что не рассчитан на это.

Single — реактивный `Callable`, потому что тут появляется возможность вернуть результат операции. Продолжая сравнение с Kotlin, можно сказать, что `Single` — это `fun single(): T { }`. Таким образом, чтобы подписаться на него, необходимо реализовать `onSuccess(T)` и `onError`.

Maybe — нечто среднее между `Single` и `Completable`, потому что поддерживает одно значение, отсутствие значений и ошибку. Тут сложнее провести

однозначную параллель с методами, но я думаю, что Maybe — это fun maybe(): T? { }, которая возвращает null, когда результата нет. Несложно догадаться, что для подписки потребуется определить onSuccess(T), onComplete и onError.

Тут важно обратить внимание, что onSuccess(T) и onComplete — взаимоисключающие. Т.е. в случае вызова первого можно не ждать второго.

Observable — наиболее часто встречающийся источник, что обусловлено его универсальностью. Он умеет как не производить события вообще, так и генерировать множество таковых, поэтому его можно использовать всегда, когда не подходят остальные варианты. Несмотря на это, у Observable есть недостаток — он совершенно не умеет обрабатывать backpressure. Для подписки на него нужны onNext(T), onError и onComplete.

Backpressure — ситуация, когда новые события поступают существенно быстрее, чем успевают обрабатываться, и начинают скапливаться в буфере, переполняя его. Это может привести к неприятностям вроде OutOfMemoryError. Подробнее можно посмотреть [тут](#).

ConnectableObservable — прогретый вариант Observable. Другие источники данных начинают выдавать свой поток событий в момент подписки. Но не этот парень. Для этого ConnectableObservable ждёт вызова connect. Сделано это для того, чтобы несколько наблюдателей могли обзирать один поток событий, не перезапуская его при каждой подписке

Backpressure – это ситуация, когда Rx-продюсер производит больше элементов, чем может обработать консьюмер. Например Observable создает 1000 элементов в секунду, а Observer обрабатывает 1 элемент в секунду. Observable имеет бесконечный буфер, в который будут добавляться элементы до тех пор, пока не случится OutOfMemoryError.

В случаях когда возможна ситуация backpressure, следует использовать Flowable в качестве Rx-стрима. Flowable можно создать из Observable методом toFlowable(), который принимает объект типа BackpressureStrategy как параметр. BackpressureStrategy – это enum, который задает стратегию обработки backpressure. Значения BackpressureStrategy:

- **MISSING**. Продюсер не имеет стратегии работы с backpressure. В этом случае консьюмер должен решать проблему переполнения. Эта конфигурация полезна в случае, когда обработка backpressure задается операторами onBackpressure...(). Если backpressure не обрабатывается, то будет брошено исключение MissingBackpressureException.
- **ERROR**. В случае backpressure бросается исключение MissingBackpressureException.
- **BUFFER**. Все элементы добавляются в буфер, пока не будут обработаны консьюмером.
- **DROP**. Все новые элементы, которые консьюмер не успеваеет обработать, удаляются и не доставляются консьюмеру.
- **LATEST**. Стратегия противоположная Drop. Консьюмер получает только самый последний элемент, созданный продюсером, когда освобождается.

Подготовка потоков данных с применением различных промежуточных операторов происходит в так называемое **время сборки**

Subscription time

Время подписки

Это временное состояние, когда `subscribe()` вызывается для потока, который внутри устанавливает цепочку шагов обработки:

```
flow.subscribe(System.out::println)
```

Это когда срабатывают побочные эффекты подписки (см. `doOnSubscribe`).

Runtime

время выполнения

Это состояние, когда потоки активно выдают элементы, ошибки или сигналы завершения

Одним из распространенных вариантов использования RxJava является выполнение некоторых вычислений, сетевой запрос в фоновом потоке и отображение результатов (или ошибок) в потоке пользовательского интерфейса

```
Flowable.fromCallable(() -> {  
    Flowable.fromCallable(() -> {  
  
        Thread.sleep(1000); // imitate expensive computation  
        Thread.sleep(1000); // имитация дорогостоящих вычислений  
        return "Done";  
        Вернуться «Готово»;  
  
    })  
})  
  
    .subscribeOn(Schedulers.io())  
    .subscribeOn(Schedulers.io())  
  
    .observeOn(Schedulers.single())  
    .observeOn(Планировщики.single())  
  
    .subscribe(System.out::println, Throwable::printStackTrace);  
    . Подпишитесь (System.out :: println, mosable :: printstacktrace);  
  
    Thread.sleep(2000); // <--- wait for the flow to finish  
    Thread.sleep (2000); // \u003c--- ждем окончания потока
```

This style of chaining methods is called a **fluent API** which resembles the **builder pattern**.

Этот стиль методов цепочки называется беглому API, который напоминает узор строителя.

However, RxJava's reactive types are immutable; each of the method calls returns a new `Flowable` with added behavior.

Однако реактивные типы RxJava неизменяемы; Каждый из вызовов метода возвращает новый `Flowable` с добавленным поведением.

To illustrate, the example can be rewritten as follows:

Для иллюстрации пример можно переписать следующим образом:

```
Flowable<String> source = Flowable.fromCallable(() -> {
    Flowable\u003cString\u003e источник = Flowable.fromCallable(() -\u003e {

        Thread.sleep(1000); // imitate expensive computation
        Thread.sleep(1000); // имитация дорогостоящих вычислений
        return "Done";
        Вернуться «Готово»;

    });
});

Flowable<String> runBackground = source.subscribeOn(Schedulers.io());
Flowable<String> runBackground = source.subscribeOn(Schedulers.io());

Flowable<String> showForeground = runBackground.observeOn(Schedulers.single());
Flowable<String> showForeground = runBackground.observeOn(Schedulers.single());

showForeground.subscribe(System.out::println, Throwable::printStackTrace);
ShowForeground.subscribe(System.out::println, Throwable::printStackTrace);

Thread.sleep(2000);
Thread.sleep (2000);
```

Typically, you can move computations or blocking IO to some other thread via `subscribeOn`.

Как правило, вы можете переместить вычисления или блокировку ввода-вывода в какой-либо другой поток через `subscribeOn`.

subscribe просто выполняет код там можно лямбду написать и делать с ит
что хошь

`subscribeOn()` – задает `Scheduler`, на котором выполняется подписка на `Observable`. Другими словами, код метода `Observable.create()` выполняется в потоке, заданном `subscribeOn()`. `Scheduler`, который задает `subscribeOn()` действует от создания `Observable` и вниз по цепочке вызовов RxJava до первого `observeOn()`. Место вызова `subscribeOn()` в цепочке не имеет значения. Если `subscribeOn()` вызывается несколько раз на одном Rx-стриме, то в большинстве случаев только первый вызов имеет эффект.

`observeOn()` – задает `Scheduler`, на котором выполняются операторы, следующие после `observeOn()`. В Rx-стриме может быть несколько `observeOn()`, каждый из которых будет менять поток выполнения.

defer - оператор, который допускает отложенный состав наблюдаемых последовательностей.

create представляет собой пользовательскую реализацию наблюдаемой последовательности (где создание отложено до подписки). Из документации соединяет реактивный мир с миром коллбеков.

range

Оператор range выдаст последовательность чисел

```
1 // create observable
2 Observable<Integer> observable = Observable.range(10, 4);
3
4 // create observer
5 Observer<Integer> observer = new Observer<Integer>() {
6     @Override
7     public void onCompleted() {
8         Log.d(TAG, "onCompleted");
9     }
10
11     @Override
12     public void onError(Throwable e) {
13         Log.d(TAG, "onError: " + e);
14     }
15
16     @Override
17     public void onNext(Integer s) {
18         Log.d(TAG, "onNext: " + s);
19     }
20 };
21
22 // subscribe
23 observable.subscribe(observer);
```

Мы указываем, что начать необходимо с 10, а кол-во элементов 4

Результат:

onNext: 10

onNext: 11

onNext: 12

onNext: 13

onCompleted

Observable.just(someList) даст вам 1 эмиссию - List. Он преобразует элемент в "Наблюдатель", который излучает этот элемент.

Observable.from(someList) даст вам N выбросов - каждый элемент в списке. Он создает Observable из массива или коллекции.

Оператор Repeat повторяет элемент. Некоторые реализации этого оператора позволяют повторять *последовательность* элементов, а

некоторые позволяют ограничивать количество повторени

interval

Оператор interval выдает последовательность long чисел начиная с 0. Мы можем указать временной интервал, через который числа будут приходить. Укажем 500 мсек.

```
1 // create observable
2 Observable<Long> observable = Observable.interval(500, TimeUnit.MILLISECONDS);
3
4 // create observer
5 Observer<Long> observer = new Observer<Long>() {
6     @Override
7     public void onCompleted() {
8         Log.d(TAG, "onCompleted");
9     }
10
11     @Override
12     public void onError(Throwable e) {
13         Log.d(TAG, "onError: " + e);
14     }
15
16     @Override
17     public void onNext(Long s) {
18         Log.d(TAG, "onNext: " + s);
19     }
20 };
21
22 // subscribe
23 observable.subscribe(observer);
```

Теперь каждые 500 мсек в Observer будет приходить все увеличивающееся значение, начиная с 0.

Результат:

onNext: 0

onNext: 1

onNext: 2

onNext: 3

...

Обратите внимание, что в логах не будет метода onCompleted. Вернее, когда нибудь он, наверно, все таки придет, когда достигнет значения, максимально доступного для Long. Но ждать придется долго.

fromCallable

Если у вас есть синхронный метод, который вам надо сделать асинхронным, то оператор fromCallable поможет вам

Например, есть метод longAction

```
1 private int longAction(String text) {
2     log("longAction");
3
4     try {
5         TimeUnit.SECONDS.sleep(1);
6     } catch (InterruptedException e) {
7         e.printStackTrace();
8     }
9
10    return Integer.parseInt(text);
11 }
```

Необходимо обернуть его в Callable

```
1 class CallableLongAction implements Callable<Integer> {
2
3     private final String data;
4
5     public CallableLongAction(String data) {
6         this.data = data;
7     }
8
9     @Override
10    public Integer call() throws Exception {
11        return longAction(data);
12    }
13 }
```

И затем можно создавать Observable из Callable

```
1 Observable.fromCallable(new CallableLongAction("5"))
2     .subscribeOn(Schedulers.io())
3     .observeOn(AndroidSchedulers.mainThread())
4     .subscribe(new Action1<Integer>() {
5         @Override
6         public void call(Integer integer) {
7             log("onNext " + integer);
8         }
9     });
```

Получившийся Observable запустит метод longAction и вернет вам результат в onNext.

Операторы observeOn и subscribeOn здесь управляют потоками (thread) и обеспечивают асинхронность вызова. Подробно мы поговорим о них в одном из следующих уроков.

Операторы смены потока выполнения (треда) `subscribeOn()` и `observeOn()` в RxJava принимают объект класса `Scheduler`.

`Scheduler` можно создать с помощью статических фабричных методов класса `Schedulers`. Самые часто используемые методы: `Schedulers.io()` и `Schedulers.computation()`.

`Schedulers.io()` возвращает `Scheduler`, который предназначен для выполнения блокирующих IO операций. IO `Scheduler` основан на неограниченном [пуле потоков](#). Для выполнения задачи из пула выбирается поток и затем возвращается в пул после завершения работы. Если в пуле нет свободных потоков, то создается новый поток и добавляется в пул. Новые потоки будут оставаться в пуле на протяжении всей жизни приложения. Неосторожное использование IO `Scheduler` может привести к крашу приложения с `OutOfMemoryError`.

`Schedulers.computation()` возвращает `Scheduler`, предназначенный для выполнения вычислительной работы. Не рекомендуется использовать этот `Scheduler` для блокирующих операций. `Computation Scheduler` основан на пуле потоков фиксированного размера. По умолчанию размер пула равен количеству процессоров, которое получается методом `Runtime.availableProcessors()`. Если на выполнение приходит задача, когда все потоки пула заняты, задача добавляется в очередь и будет выполнена первым освободившимся потоком.

take

Оператор `take` возьмет только указанное количество первых элементов из переданной ему последовательности и сформирует из них новую последовательность. Возьмем первые три:

```
1 // create observable
2 Observable<Integer> observable = Observable
3   .from(new Integer[]{5,6,7,8,9})
4   .take(3);
```

skip

Оператор `skip` пропустит первые элементы. Пропустим первые 2

```
1 // create observable
2 Observable<Integer> observable = Observable
3   .from(new Integer[]{5,6,7,8,9})
4   .skip(2);
```

distinct

Оператор `distinct` отсеет дубликаты

```
1 // create observable
2 Observable<Integer> observable = Observable
3   .from(new Integer[]{5,9,7,5,8,6,7,8,9})
4   .distinct();
```

Schedulers

Планировщики

RxJava operators don't work with `Thread`'s or `ExecutorService`'s directly but with so-called `Scheduler`'s that abstract away sources of concurrency behind a uniform API.

Операторы RxJava работают не с потоками или `ExecutorServices` напрямую, а с так называемыми планировщиками, которые абстрагируются от источников параллелизма за единым API.

RxJava 3 features several standard schedulers accessible via `Schedulers` utility class.

В RxJava 3 есть несколько стандартных планировщиков, доступных через служебный класс `Schedulers`.

- `Schedulers.computation()`: Run computation intensive work on a fixed number of dedicated threads in the background.
`Schedulers.computation()`: выполнение интенсивных вычислений в фиксированном количестве выделенных потоков в фоновом режиме.
Most asynchronous operators use this as their default `Scheduler`.
Большинство асинхронных операторов используют его в качестве планировщика по умолчанию.
- `Schedulers.io()`: Run I/O-like or blocking operations on a dynamically changing set of threads.
`Schedulers.io()`: Запуск операций ввода-вывода или операций блокировки в динамически изменяющемся наборе потоков.
- `Schedulers.single()`: Run work on a single thread in a sequential and FIFO manner.
`Schedulers.single()`: выполнение работы в одном потоке в последовательном порядке и в порядке FIFO.
- `Schedulers.trampoline()`: Run work in a sequential and FIFO manner in one of the participating threads, usually for testing purposes.
`Schedulers.trampoline()`: Запускайте работу последовательно и в порядке FIFO в одном из участвующих потоков, обычно в целях тестирования.

`AndroidSchedulers.mainThread()`, `SwingScheduler.instance()` OR `JavaFXSchedulers.gui()`.

These are available on all JVM platforms but some specific platforms, such as Android, have their own typical `Scheduler`'s defined: `AndroidSchedulers.mainThread()`, `SwingScheduler.instance()` OR `JavaFXSchedulers.gui()`.

Они доступны на всех платформах JVM, но некоторые конкретные платформы, такие как Android, имеют свои собственные типичные планировщики: `AndroidSchedulers.mainThread()`, `SwingScheduler.instance()` или `javafxSchedulers.gui()`.

In addition, there is an option to wrap an existing `Executor` (and its subtypes such as `ExecutorService`) into a `Scheduler` via `Schedulers.from(Executor)`.

Кроме того, существует возможность обернуть существующий `Executor` (и его подтипы, такие как `ExecutorService`) в `Scheduler` через `Schedulers.from(Executor)`.

This can be used, for example, to have a larger but still fixed pool of threads (unlike `computation()` and `io()` respectively).

Это можно использовать, например, для создания большего, но все же фиксированного пула потоков (в отличие от вычислений() и io() соответственно).

Потоки в RxJava носят последовательный характер и разделены на этапы обработки, которые могут выполняться одновременно друг с другом

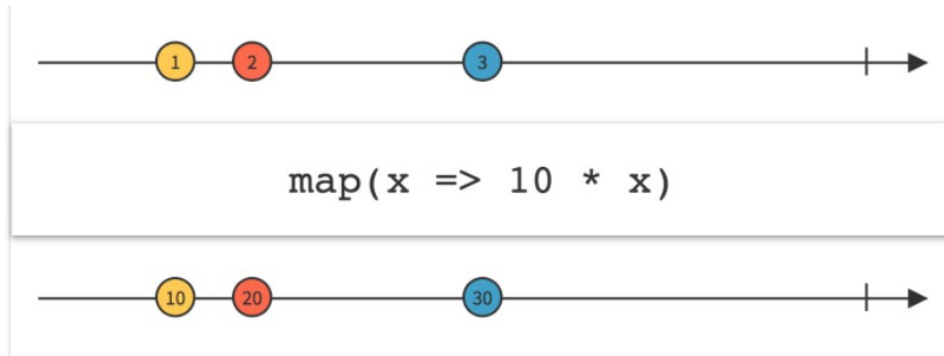
На практике параллелизм в RxJava означает запуск независимых потоков и объединение их результатов обратно в один поток.

- Обратите внимание, однако, что `flatMap` не гарантирует никакого порядка, и элементы из внутренних потоков могут оказаться чередующимися.

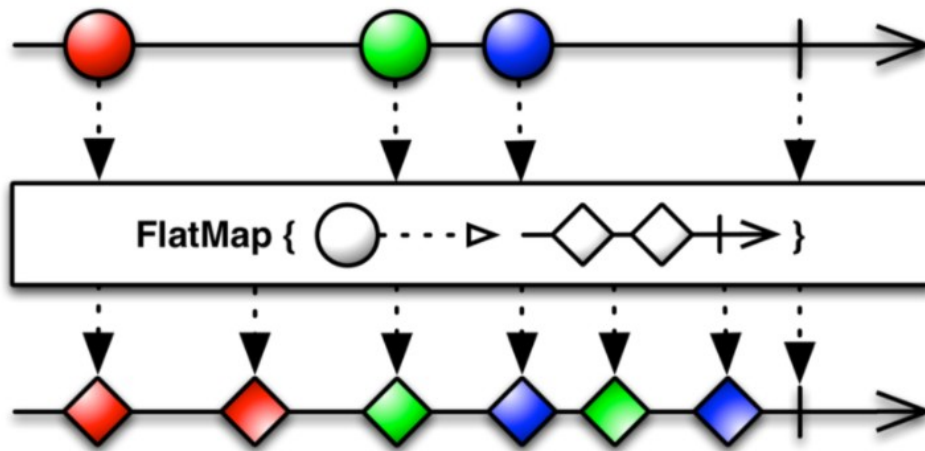
- `concatMap`, который отображает и запускает один внутренний поток за раз

- `ConcatMapEager`, который запускает все внутренние потоки «одновременно», но выходной поток будет в том порядке, в котором эти внутренние потоки были созданы.

`map` возвращает результат функции



flatMap так же применяет функцию к каждому излучаемому элементу, но эта функция возвращает тип Observable. Т.е. 1 излучаемый элемент может через flatMap породить множество излучаемых элементов или не одного.



```
Observable<String> observable = Observable
    .just("A", "B", "C")
    .flatMap(s -> {
        System.out.println();
        return Observable.just(s + "1", s + "2", s + "3");
    });

observable.subscribe(s -> System.out.print(s + " "));
```

Результат:
A1 A2 A3
B1 B2 B3
C1 C2 C3

Довольно странная штука, если призадуматься. Зачем flatMap() возвращает *другой* Observable? Ключевой момент тут в том, что новый Observable — это то, что увидит в итоге наш Subscriber. Он не получит List<String>, он получит поток индивидуальных объектов класса String так, как он получил бы от Observable.from().

Подчеркну ещё раз, потому что это важно: flatMap() может вернуть нам *любой* Observable, какой вы только захотите.

Изобилие операторов

Пока что мы взглянули лишь на два оператора, но их в RxJava на самом деле гораздо больше.

Как ещё можно улучшить наш код?

`getTitle()` возвращает `null`, если мы получили ошибку 404. Мы не хотим выводить на экран `"null"`, и мы можем отфильтровать ненужные нам значения:

```
query("Hello, world!")
    .flatMap(urls -> Observable.from(urls))
    .flatMap(url -> getTitle(url))
    .filter(title -> title != null)
    .subscribe(title -> System.out.println(title));
```

`filter()` «испускает» тот же самый элемент потока данных, который он получил, но только если он проходит проверку.

А теперь мы хотим показать только 5 результатов, не больше:

```
query("Hello, world!")
    .flatMap(urls -> Observable.from(urls))
    .flatMap(url -> getTitle(url))
    .filter(title -> title != null)
    .take(5)
    .subscribe(title -> System.out.println(title));
```

`take()` возвращает не больше заданного количества элементов (если в нашем случае получилось меньше 5 элементов, `take()` просто-напросто завершит свою работу раньше).

Знаете, а давайте-ка будем ещё и сохранять каждый полученный нами заголовок на диск:

```
query("Hello, world!")
    .flatMap(urls -> Observable.from(urls))
    .flatMap(url -> getTitle(url))
    .filter(title -> title != null)
    .take(5)
    .doOnNext(title -> saveTitle(title))
    .subscribe(title -> System.out.println(title));
```

`doOnNext()` позволяет нам добавить некоторое дополнительное действие, происходящее всякий раз, как мы получаем новый элемент данных, в данном случае этим действием будет сохранение заголовка.

`FlatMap()` может чередовать элементы при эмиссии, то есть порядок исходящих элементов не поддерживается. Полезен если порядок выполнения не важен, но нужно единовременное выполнение.

3. `ConcatMap()`

Технически данный оператор выдает похожий с `FlatMap()` результат, меняется только последовательность эмиттируемых данных. `ConcatMap()` поддерживает порядок эмиссии данных и ожидает исполнения текущего `Observable`. Поэтому лучше использовать его когда необходимо обеспечить порядок выполнения задач. При этом нужно помнить, что выполнение `ConcatMap()` занимает больше времени чем `FlatMap()`.

4. `SwitchMap()`

`SwitchMap()` кардинально отличается от `FlatMap()` и `ConcatMap()`. Он лучше всего подходит, если вы хотите проигнорировать промежуточные результаты и рассмотреть последний. `SwitchMap` отписывается от предыдущего источника `Observable` всякий раз, когда новый элемент начинает излучать данные, тем самым всегда эмитит данные из текущего `Observable`.

Ответы на самые частые вопросы по RxJava

1. Какому «паттерну поведения» следует RxJava? Push или Pull?

В RxJava новые данные «заталкиваются» (Push) к наблюдателям

2. Какова разница между колбэками `onNext()`, `onComplete()` and `onError()`?

Эти колбэки которые получают `Observable` / `Flowable`. `onNext()` отрабатывает для каждой эмиссии данных. `onComplete()` и `onError()` взаимоисключаемы. Первый отрабатывает когда эмиссия данных завершилась, а второй если произошла ошибка.

3. Сколько раз колбэки `onNext()`, `onComplete()` and `onError()` могут быть вызваны?

`onNext()` – от 0 до бесконечного кол-ва раз

`onComplete()` – максимум один раз за поток

`onError()` – максимум один раз за поток

4. Когда `Observable` начинает эмиссию частей данных?

Есть 2 вида `Observable` — «холодные» и «горячие». Холодные начинают эмиссию данных только тогда когда на них кто-нибудь подпишется. Горячие же эмитят данные вне зависимости от того есть ли подписант или нет.

5. Какая разница между «холодными» и «горячими» `Observables`

В том что они эмитят данные по разному. Холодные создаются множество раз и каждого инстанса могут подключены разные слушатели(подписанты), с разной логикой. Горячие же похоже на «поток» событий — разные слушатели могут подписываться на горячий observable, но такой observable создается один раз.

6. Можно ли трансформировать «холодный» Observable в «горячий»?

Можно используя операторы **publish().connect()**. **publish()** трансформирует обычный Observable в ConnectableObservable, который имеет повадки «горячего». Сразу после того как будет задействован оператор **connect()** трансформированный Observable начнет эмитить данные вне зависимости от того есть подписчики или нет.

Другой метод трансформации это обернуть **Observable Subject**-ом. В данном случае **Subject** подписывается на «холодный» **Observable** незамедлительно и раскрывает себя как **Observable** для будущих подписчиков. Опять же, работа выполняется независимо от того, есть ли какие-либо подписчики или нет, с другой стороны, несколько подписчиков **Subject-a** не будут инициировать начальную эмиссию данных несколько раз.

7. Можно ли трансформировать «горячий» observable в «холодный»?

Можно несколькими путями. Первый подход — использование оператора **defer()**. Этот оператор откладывает создание «горячего» Observable, поэтому каждый новый подписчик снова запускает работу.

8. Что такое Планировщик (Scheduler)? Как RxJava использует их?

По умолчанию RxJava использует один поток — все операции выполняются на одном потоке. Планировщик помогает переключить выполнение определенного блока кода в иной поток.

9. Что такое цепочка-Observable?

Список операций / преобразований, выполненных между источником и конечным подписчиком. Простой пример — создание объекта User, фильтрация пользователей-администраторов **оператором filter()**, проверка их подлинности **оператором filter()** и, наконец, полное имя **оператором map()**.

10. Какая разница между операторами observeOn() и subscribeOn()?

subscribeOn() — используется для того, чтобы указать планировщику на каком потоке будет проходить основная работа. Например, очень тяжелые вычисления, которые могут затормозить UI-поток, можно перенести в рабочий поток используя данный оператор.
Пример: **subscribeOn(Schedulers.newThread())**

observeOn() — указывает планировщику поток, на котором будут выполняться все последующие операции. Другими словами, он меняет

поток для всех операторов после него. Например, после тяжелых вычислений на рабочем потоке, нужно показать результат на UI-потоке. Здесь мы можем написать `observeOn(AndroidSchedulers.mainThread())`

11. Что будет если много раз выполнить оператор `subscribeOn()` в цепочке?

Только первый оператор даст желаемый эффект. Остальные же эффекта не дадут, кроме траты ресурсов.

12. Что будет если много раз выполнить оператор `observeOn()` в цепочке?

Каждый `observeOn()` включает планировщик (поток), в котором будут выполняться все последующие операторы. Сложные потоки RxJava могут выиграть от нескольких операторов `observeOn()`.

13. Какая разница между операторами `map()` и `flatMap()`?

Оператор `map()` просто превращает(мэппит) ЗначениеА в ЗначениеБ. Например: объекты в списке **типа** `Int` превратить в объект **типа** `String` Обработываются уже готовые/полученные значения.

Оператор `flatMap()` позволяет превращать-оборачивать получаемые значения в новые потоки данных. Например, если получаем из одного потока строку(типа `URL`) СтрокаБ, то эту строку мы можем превратить в новый поток. Если Поток<СтрокаБ> испускает несколько элементов, все они в конечном итоге будут переданы исходному наблюдателю (будут «сведены» к одному `Observer-y`).

Поскольку нет никаких ограничений на Поток<СтрокаБ>, `flatMap()` полезен для развертывания параллелизма в выполнении задач.

14. Какая разница между операторами `flatMap()`, `concatMap()` и `switchMap()`?

Оператор `flatMap()` — как мы говорили выше — может разделить цепочку выполнения на несколько промежуточных потоков, а результаты будут получены `Observer-ом`. Следует отметить, что порядок получения результатов будет в соответствии с тем какой результат был получен первым.

Оператор `concatMap()` работает также как `flatMap()`, но только сохраняет порядок выполнения потоков. Из-за этого выполнение этого оператора может занять больше времени.

Оператор `switchMap()` тоже чем-то похож на `flatMap()`, но с единственным исключением — при получении нового элемента из цепочки предыдущие потоки созданные из предыдущих элементов уничтожаются. Проще говоря, используя данный оператор активным будет только последний

Observable из последнего полученного элемента. Стало быть, результат выполнения получим только самого последнего Observable.

DATABASE

Реляционная база данных – это набор данных с predetermined связями между ними. Эти данные организованы в виде набора таблиц, состоящих из столбцов и строк. В таблицах хранится информация об объектах, представленных в базе данных. В каждом столбце таблицы хранится определенный тип данных, в каждой ячейке – значение атрибута. Каждая строка таблицы представляет собой набор связанных значений, относящихся к одному объекту или сущности. Каждая строка в таблице может быть помечена уникальным идентификатором, называемым первичным ключом, а строки из нескольких таблиц могут быть связаны с помощью внешних ключей. К этим данным можно получить доступ многими способами, и при этом реорганизовывать таблицы БД не требуется.

Что из себя представляет SQLite?

SQLite – реляционная СУБД написанная на языке C. Выступает в роли сервера, т.е. программы подключаются к файлу базы данных и взаимодействуют с ней.

На Android SQLite представляет из себя основной файл с расширением .db, а также вспомогательные файлы write ahead logs (WAL) и shared memory files (SHM), которые находятся в корневой директории приложения (data/data/package.name/databases) просмотреть которую возможно, только при наличии root прав на телефоне

2. Для чего нужна версия базы данных?

Версия необходима для изменения структуры базы данных (БД). Например при добавлении новой таблицы в БД, нужно повысить версию с 1 до 2 и создать файл миграции, который содержит SQL код, на основании которого Android добавит в БД новую таблицу и при этом сохранит все существующие данные в сохранности.

3. В каком потоке необходимо обрабатывать запросы в базу данных?

Любые запросы к базе данных необходимо обрабатывать в отдельном потоке, чтобы главный UI поток приложения был разгружен и не возникало неприятной ANR ошибки и каши приложения.

Для запросов можно использовать любые удобные средства – AsyncTask, ThreadPool, RxJava, Kotlin Coroutines.

4. Что лучше использовать для хранения данных SQLite или SharedPreferences?

SharedPreferences (SP) предназначен только для хранения данных в формате ключ-значение и при возрастании объема хранимой информации, запросы к SP начнут тормозить. SP используется в основном для хранения какого-либо небольшого кэша, например время сессии, настройки, прогресс. Соответственно для полноценного хранения информации и осуществления к ним гибких запросов, необходимо использовать СУБД SQLite.

5. Какие популярные ORM для Android вы знаете?

Realm – самый распространенный NOSQL фреймворк написанный на C++, исходя из статистики и самый быстрый. Из SQL фреймворков можно использовать Room, ORMLite, GreenDao. Room предоставляется библиотекой androidx и на текущий момент является самым популярным решением. Статистика производительности, время чтения и записи указаны в миллисекундах:

Библиотека	Запись	Чтение
Realm	151	29
ORMLite	151	666
Room	131	699
GreenDao	81	1238

Крайне не рекомендуется использовать чистый SQL Helper, который поставляется с Android SDK, т.к. на его реализацию уходит достаточно много времени, по сравнению с ORM.

SQLite поддерживает типы TEXT (аналог String в Java), INTEGER (аналог long в Java) и REAL (аналог double в Java). Остальные типы следует конвертировать, прежде чем сохранять в базе данных. SQLite сама по себе не проверяет типы данных, поэтому вы можете записать целое число в колонку, предназначенную для строк, и наоборот.

Тип	Описание
NULL	пустое значение
INTEGER	целочисленное значение
REAL	значение с плавающей точкой
TEXT	строки или символы в кодировке UTF-8, UTF-16BE или UTF-16LE
NUMERIC	здесь можно хранить булевы значения, а также время и дату
BLOB	бинарные данные

Классы для работы с SQLite

Работа с SQLite напрямую теперь считается устаревшим подходом. На Google I/O 2017 был представлен новый механизм доступа к базе данных через специальную обёртку [Room](#)

Работа с базой данных сводится к следующим задачам:

- Создание и открытие базы данных
- Создание таблицы
- Создание интерфейса для вставки данных (insert)
- Создание интерфейса для выполнения запросов (выборка данных)
- Закрытие базы данных

Класс **SQLiteOpenHelper**: Создание базы данных

Библиотека Android содержит абстрактный класс **SQLiteOpenHelper**, с помощью которого можно создавать, открывать и обновлять базы данных. Это основной класс, с которым вам придётся работать в своих проектах. При реализации этого вспомогательного класса от вас скрывается логика, на основе которой принимается решение о создании или обновлении базы данных перед ее открытием.

Класс **SQLiteOpenHelper** содержит два обязательных абстрактных метода:

- **onCreate()** — вызывается при первом создании базы данных
- **onUpgrade()** — вызывается при модификации базы данных

Также используются другие методы класса:

- **onDowngrade(SQLiteDatabase, int, int)**
- **onOpen(SQLiteDatabase)**
- **getReadableDatabase()**
- **getWritableDatabase()**

Room создаёт таблицу для каждого класса, помеченного **@Entity**. Поля в классе соответствуют столбцам в таблице. Следовательно, классы сущностей, как правило, представляют собой небольшие классы моделей, которые не содержат никакой логики. Наш класс `User` представляет модель для данных в базе данных. Итак, давайте обновим его, чтобы сообщить Room, что он должен создать таблицу на основе этого класса:

- Аннотируйте класс с помощью **@Entity** и используйте свойство `tableName`, чтобы задать имя таблицы.
- Задайте первичный ключ, добавив аннотацию **@PrimaryKey** в правильные поля — в нашем случае это идентификатор пользователя.
- Задайте имя столбцов для полей класса, используя аннотацию **@ColumnInfo(name = "column_name")**. Этот шаг можно

пропустить, если ваши поля уже названы так, как следует назвать столбец.

- Если в классе несколько конструкторов, добавьте аннотацию `@Ignore`, чтобы указать Room, какой следует использовать, а какой — нет.

```
@Entity(tableName = "users")
public class User {

    @PrimaryKey
    @ColumnInfo(name = "userid")
    private String mId;

    @ColumnInfo(name = "username")
    private String mUserName;

    @ColumnInfo(name = "last_update")
    private Date mDate;

    @Ignore
    public User(String userName) {
        mId = UUID.randomUUID().toString();
        mUserName = userName;
        mDate = new Date(System.currentTimeMillis());
    }

    public User(String id, String userName, Date date) {
        this.mId = id;
        this.mUserName = userName;
        this.mDate = date;
    }

    ...
}
```

Создание объектов доступа к данным (DAO)

DAO отвечают за определение методов доступа к базе данных. В первоначальной реализации нашего проекта на SQLite все запросы к базе данных выполнялись в классе `LocalUserDataSource`, где мы работали с объектами `Cursor`. В Room нам не нужен весь код, связанный с курсором, и мы можем просто определять наши запросы, используя аннотации в классе `UserDao`.

Например, при запросе всех пользователей из базы данных Room выполняет всю «тяжелую работу», и нам нужно только написать:

```
@Query("SELECT * FROM Users")
```

```
List<User> getUsers();
```

Создание базы данных

Мы уже определили нашу таблицу Users и соответствующие ей запросы, но мы ещё не создали базу данных, которая объединит все эти составляющие Room. Для этого нам нужно определить абстрактный класс, который расширяет RoomDatabase. Этот класс помечен @Database, в нём перечислены объекты, содержащиеся в базе данных, и DAO, которые обращаются к ним. Версия базы данных должна быть увеличена на 1 в сравнении с первоначальным значением, поэтому в нашем случае это будет 2

```
@Database(entities = {User.class}, version = 2)
@TypeConverters(DateConverter.class)
public abstract class UsersDatabase extends RoomDatabase {

    private static UsersDatabase INSTANCE;

    public abstract UserDao userDao();
}
```

Поскольку мы хотим сохранить пользовательские данные, нам нужно реализовать класс Migration, сообщающий Room, что он должен делать при переходе с версии 1 на 2. В нашем случае, поскольку схема базы данных не изменилась, мы просто предоставим пустую реализацию:

```
static final Migration MIGRATION_1_2 = new Migration(1, 2) {
    @Override
    public void migrate(SupportSQLiteDatabase database) {
        // Поскольку мы не изменяли таблицу, здесь больше ничего не нужно делать.
    }
};
```

Создайте объект базы данных в классе UsersDatabase, определив имя базы данных и миграцию:

```
database = Room.databaseBuilder(context.getApplicationContext(),
    UsersDatabase.class, "Sample.db")
    .addMigrations(MIGRATION_1_2)
    .build();
```

Room

Room имеет три основных компонента: Entity, Dao и Database. Рассмотрим их на небольшом примере, в котором будем создавать базу данных для хранения данных по сотрудникам (англ. - employee).

При работе с Room нам необходимо будет писать SQL запросы. Если вы не знакомы с ними, то имеет смысл прочесть хотя бы основы.

Entity

Аннотацией `Entity` нам необходимо пометить объект, который мы хотим хранить в базе данных. Для этого создаем класс `Employee`, который будет представлять собой данные сотрудника: `id`, имя, зарплата:

Marks a field in an `Entity` as the primary key.

Отмечает поле в сущности в качестве первичного ключа.

If you would like to define a composite primary key, you should use `primaryKeys()` method.

Если вы хотите определить составной первичный ключ, вам следует использовать метод `primaryKeys()`.

Each `Entity` must declare a primary key unless one of its super classes declares a primary key.

Каждая сущность должна объявлять первичный ключ, если только один из ее суперклассов не объявляет первичный ключ.

If both an `Entity` and its super class defines a `PrimaryKey`, the child's `PrimaryKey` definition will override the parent's `PrimaryKey`.

Если и `Entity`, и его суперкласс определяют `PrimaryKey`, определение `PrimaryKey` дочернего объекта переопределит `PrimaryKey` родителя.

If `PrimaryKey` annotation is used on a `Embedded` field, all columns inherited from that embedded field becomes the composite primary key (including its grand children fields).

Если аннотация `PrimaryKey` используется для поля `Embedded`, все столбцы, унаследованные от этого вложенного поля, становятся составным первичным ключом (включая его дочерние поля).

```
1 @Entity
2 public class Employee {
3
4     @PrimaryKey
5     public long id;
6
7     public String name;
8
9     public int salary;
10 }
```

Класс помечается аннотацией `Entity`. Объекты класса `Employee` будут использоваться при работе с базой данных. Например, мы будем получать их от базы при запросах данных и отправлять их в базу при вставке данных.

Этот же класс `Employee` будет использован для создания таблицы в базе. В качестве имени таблицы будет использовано имя класса. А поля таблицы будут созданы в соответствии с полями класса.

Аннотацией `PrimaryKey` мы помечаем поле, которое будет ключом в таблице.

Dao

В объекте `Dao` мы будем описывать методы для работы с базой данных. Нам нужны будут методы для получения списка сотрудников и для добавления/изменения/удаления сотрудников.

Описываем их в интерфейсе с аннотацией `Dao`.

```
1  @Dao
2  public interface EmployeeDao {
3
4      @Query("SELECT * FROM employee")
5      List<Employee> getAll();
6
7      @Query("SELECT * FROM employee WHERE id = :id")
8      Employee getById(long id);
9
10     @Insert
11     void insert(Employee employee);
12
13     @Update
14     void update(Employee employee);
15
16     @Delete
17     void delete(Employee employee);
18
19 }
```

Методы `getAll` и `getById` позволяют получить полный список сотрудников или конкретного сотрудника по `id`. В аннотации `Query` нам необходимо прописать соответствующие SQL-запросы, которые будут использованы для получения данных.

Обратите внимание, что в качестве имени таблицы мы используем `employee`. Напомню, что имя таблицы равно имени Entity класса, т.е. `Employee`, но в SQLite не важен регистр в именах таблиц, поэтому можем писать `employee`.

Для вставки/обновления/удаления используются методы `insert/update/delete` с соответствующими аннотациями. Тут никакие запросы указывать не нужно. Названия методов могут быть любыми. Главное - аннотации.

Database

Аннотацией `Database` помечаем основной класс по работе с базой данных. Этот класс должен быть абстрактным и наследовать `RoomDatabase`.

```
1  @Database(entities = {Employee.class}, version = 1)
2  public abstract class AppDatabase extends RoomDatabase {
3      public abstract EmployeeDao employeeDao();
4  }
```

В параметрах аннотации `Database` указываем, какие Entity будут использоваться, и версию базы. Для каждого Entity класса из списка `entities` будет создана таблица.

В `Database` классе необходимо описать абстрактные методы для получения Dao объектов, которые вам понадобятся.

Практика

Все необходимые для работы объекты созданы. Давайте посмотрим, как использовать их для работы с базой данных.

Database объект - это стартовая точка. Его создание выглядит так:

```
1 AppDatabase db = Room.databaseBuilder(getApplicationContext(),
2   AppDatabase.class, "database").build();
```

Используем Application Context, а также указываем AppDatabase класс и имя файла для базы.

Учитывайте, что при вызове этого кода Room каждый раз будет создавать новый экземпляр AppDatabase. Эти экземпляры очень тяжелые и рекомендуется использовать один экземпляр для всех ваших операций. Поэтому вам необходимо позаботиться о синглтоне для этого объекта. Это можно сделать с помощью [Dagger](#), например.

Если вы не используете Dagger (или другой DI механизм), то можно использовать Application класс для создания и хранения AppDatabase:

```
1 public class App extends Application {
2
3     public static App instance;
4
5     private AppDatabase database;
6
7     @Override
8     public void onCreate() {
9         super.onCreate();
10        instance = this;
11        database = Room.databaseBuilder(this, AppDatabase.class, "database")
12            .build();
13    }
14
15    public static App getInstance() {
16        return instance;
17    }
18
19    public AppDatabase getDatabase() {
20        return database;
21    }
22 }
```

Не забудьте добавить App класс в манифест

@DATABASE

```
public interface Database
extends annotation.Annotation

Отмечает класс как базу данных RoomDatabase.
Класс должен быть абстрактным и расширять RoomDatabase .
Вы можете получить реализацию класса через Room.databaseBuilder или Room.inMemoryDatabaseBuilder .

}

// Song and Album are classes annotated with @Entity.
@Database(version = 1, entities = {Song.class, Album.class})
abstract class MusicDatabase extends RoomDatabase {
    // SongDao is a class annotated with @Dao.
    abstract public SongDao getSongDao();
    // AlbumDao is a class annotated with @Dao.
    abstract public ArtistDao getArtistDao();
    // SongAlbumDao is a class annotated with @Dao.
    abstract public SongAlbumDao getSongAlbumDao();
}
```

В приведенном выше примере определяется класс, который имеет 2 таблицы и 3 класса DAO, которые используются для доступа к нему. Нет ограничений на количество классов `Entity` или `Dao` но они должны быть уникальными в базе данных.

Вместо прямого выполнения запросов к базе данных настоятельно рекомендуется создавать классы `Dao`. Использование классов `Dao` позволит вам абстрагироваться от взаимодействия с базой данных на более логичном уровне, который будет намного проще имитировать в тестах (по сравнению с выполнением прямых SQL-запросов). Он также автоматически выполняет преобразование из `Cursor` в классы данных вашего приложения, поэтому вам не нужно иметь дело с API баз данных более низкого уровня для большей части вашего доступа к данным.

Room также проверяет все ваши запросы в классах `Dao` во время компиляции приложения, поэтому, если в одном из запросов возникнет проблема, вы будете немедленно уведомлены.

@Entity

```
public interface Entity
extends annotation.Annotation
```

Отмечает класс как сущность. Этот класс будет иметь таблицу сопоставления SQLite в базе данных.

Каждая сущность должна иметь как минимум 1 поле, аннотированное с помощью `PrimaryKey`. Вы также можете использовать `primaryKey()` для определения первичного ключа.

Каждая сущность должна иметь конструктор без аргументов или конструктор, параметры которого соответствуют полям (в зависимости от типа и имени). Конструктору не обязательно получать все поля в качестве параметров, но если поле не передается в конструктор, оно должно быть либо общедоступным, либо иметь общедоступный установщик. Если соответствующий конструктор доступен, Room всегда будет его использовать. Если вы не хотите, чтобы он использовал конструктор, вы можете аннотировать его с помощью `Ignore`.

Когда класс помечен как Entity, все его поля сохраняются. Если вы хотите исключить некоторые из его полей, вы можете отметить их как « `Ignore` ».

Если поле `transient`, оно автоматически игнорируется, **если** оно не аннотировано с помощью `ColumnInfo`, `Embedded` или `Relation`.

Пример:

```
@Entity
public class Song {
    @PrimaryKey
    private final long id;
    private final String name;
    @ColumnInfo(name = "release_year")
    private final int releaseYear;
```

@PrimaryKey

```
public interface ForeignKey
extends annotation.Annotation
```

Объявляет внешний ключ для другого Entity.

Внешние ключи позволяют вам указывать ограничения для сущностей, так что SQLite будет гарантировать, что связь действительна при изменении базы данных.

Когда указано ограничение внешнего ключа, SQLite требует, чтобы указанные столбцы были частью уникального индекса в родительской таблице или первичного ключа этой таблицы. Вы должны создать уникальный индекс в родительской сущности, который охватывает указанные столбцы (Room проверит это во время компиляции и напечатает ошибку, если она отсутствует).

Также рекомендуется создать индекс для дочерней таблицы, чтобы избежать полного сканирования таблицы при изменении родительской таблицы. Если подходящий индекс в дочерней таблице отсутствует, Room `RoomWarnings.MISSING_INDEX_ON_FOREIGN_KEY_CHILD` предупреждение `RoomWarnings.MISSING_INDEX_ON_FOREIGN_KEY_CHILD`.

Ограничение внешнего ключа может быть отложено до завершения транзакции. Это полезно, если вы выполняете массовую вставку в базу данных за одну транзакцию. По умолчанию ограничения внешнего ключа действуют немедленно, но вы можете изменить это значение, установив для `deferred()` значение `true`. Вы также можете использовать `defer_foreign_keys » PRAGMA`, чтобы отложить их в зависимости от вашей транзакции.

```
public interface PrimaryKey
extends annotation.Annotation
```

Помечает поле в Entity как первичный ключ.

Если вы хотите определить составной первичный ключ, вам следует использовать Entity.`primaryKey()`.

Каждая Entity должна объявить первичный ключ, если один из ее суперклассов не объявляет первичный ключ. Если и Entity и его суперкласс определяют PrimaryKey, определение PrimaryKey дочернего PrimaryKey переопределит PrimaryKey родительского.

Если аннотация PrimaryKey используется в поле Embedded d, все столбцы, унаследованные от этого встроенного поля, становятся составным первичным ключом (включая его основные дочерние поля).

@ForeignKey

@ForeignKey определяет ограничение (правило), согласно которому значение столбца (столбцов) дочернего элемента должно быть значением указанного столбца (столбцов) родительского элемента, если нет, то возникает конфликт.

@Relationship определяет отношение, которое затем учитывается Room путем добавления подзапросов, которые получают ВСЕ связанные сущности от дочернего элемента для родительского.

@Query

```
public interface Query
extends annotation.Annotation
```

Помечает метод в аннотированном классе **Dao** как метод запроса. Значение аннотации включает запрос, который будет выполняться при вызове этого метода. Этот запрос **проверяется Room во время компиляции**, чтобы убедиться, что он правильно компилируется в базе данных. Аргументы метода будут привязаны к аргументам привязки в операторе SQL. Видеть Room поддерживает только именованный параметр привязки `:name`, чтобы избежать путаницы между параметрами метода и параметрами привязки запроса. Room автоматически привяжет параметры метода к аргументам привязки. Это делается путем сопоставления имен параметров с именами аргументов привязки.

```
@Query("SELECT * FROM song WHERE release_year = :year")
public abstract List<Song> findSongsByReleaseYear(int year);
```

Как расширение аргументов привязки SQLite, Room поддерживает привязку списка параметров к запросу. Во время выполнения Room создаст правильный запрос, чтобы иметь соответствующее количество аргументов привязки в зависимости от количества элементов в параметре метода.

```
@Query("SELECT * FROM song WHERE id IN(:songIds)")
public abstract List<Song> findByIds(Long[] songIds);
```

В приведенном выше примере, если `songIds` представляет собой массив из 3 элементов, Room выполнит запрос как: `SELECT * FROM song WHERE id IN(?, ?, ?)` и привяжет каждый элемент массива `songIds` к оператору. Одним из предостережений этого типа привязки является то, что к запросу можно привязать только 999 элементов, это ограничение SQLite (`/java.util.List">java.util.List` или `Array`. В дополнение к

@DAO

```
public interface Dao
extends annotation.Annotation
```

Отмечает класс как объект доступа к данным. Объекты доступа к данным - это основные классы, в которых вы определяете взаимодействие с базой данных. Они могут включать в себя множество методов запроса. Класс, помеченный @Dao должен быть либо интерфейсом, либо абстрактным классом. Во время компиляции Room сгенерирует реализацию этого класса, когда на него будет ссылаться **Database**. Абстрактный класс @Dao может дополнительно иметь конструктор, который принимает **Database** качестве единственного параметра. Рекомендуется иметь несколько классов Dao в вашей кодовой базе в зависимости от таблиц, с которыми они соприкасаются.

@INSERT


```
public interface Insert
extends annotation.Annotation
```

Помечает метод в аннотированном классе **Dao** как метод вставки.

Реализация метода вставит его параметры в базу данных.

Все параметры метода Insert должны быть либо классами, аннотированными **Entity** либо его коллекциями / массивом.

Пример:

```
@Dao
public interface MusicDao {
    @Insert(onConflict = OnConflictStrategy.REPLACE)
    public void insertSongs(Song... songs);

    @Insert
    public void insertBoth(Song song1, Song song2);

    @Insert
    public void insertAlbumWithSongs(Album album, List<Song> songs);
}
```

```
@Database(
    entities = [RoomCharacter::class, RoomEpisode::class, RoomLocation::class],
    version = 1,
    exportSchema = false
)
```

```
abstract class RickRoomDatabase : RoomDatabase() {
```

```
    abstract fun daoCharacter(): DaoCharacter
```

```
    abstract fun daoLocation(): DaoLocation
```

```
    abstract fun daoEpisode(): DaoEpisode
}
```

```
class RickApplication : Application() {
```

```
    val database: RickRoomDatabase by lazy {
```

```
        Room.databaseBuilder(context: this, RickRoomDatabase::class.java, name: "database")
            .allowMainThreadQueries()
            .build()
    }
```

```
    val repository by lazy {
```

```
        RickRepository(database.daoCharacter(), database.daoLocation(), database.daoEpisode())
    }
```

```

class RickRepository(
    private val daoCharacter: DaoCharacter,
    private val daoLocation: DaoLocation,
    private val daoEpisode: DaoEpisode
) {

    @Synchronized
    fun insertSingle(character: RoomCharacter): Flow<Unit> {
        return flow { emit(daoCharacter.insertSingleCharacters(character)) }
    }

    @Synchronized
    fun insertCharacters(characters: Collection<CharacterModel>): Flow<Unit> {
        val roomCharacters = characters.map { it: CharacterModel
            RoomCharacter(objectToJsonCharacterModel(it))
        }
        return flow { emit(daoCharacter.insertCharacters(roomCharacters)) }
    }

    @Synchronized
    fun takeCharacters(): Flow<List<CharacterModel>> {
        return daoCharacter.getCharacters().map { list ->
            list.map { jsonToCharacter(it.jsonCharacter) }
        }
    }

    @Synchronized
    fun insertEpisodes(episodes: Collection<EpisodeModel>): Flow<Unit> {
        val roomEpisodes = episodes.map { it: EpisodeModel
            RoomEpisode(objectToJsonEpisodeModel(it))
        }
    }
}

```

```

@Dao
interface DaoCharacter {
    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertSingleCharacters(character: RoomCharacter)

    @Insert(onConflict = OnConflictStrategy.REPLACE)
    @JvmSuppressWildcards
    suspend fun insertCharacters(list: List<RoomCharacter>)

    @Query(value: "SELECT * FROM character")
    fun getCharacters(): Flow<List<RoomCharacter>>
}

@Dao
interface DaoEpisode {
    @Insert(onConflict = OnConflictStrategy.REPLACE)
    @JvmSuppressWildcards
    suspend fun insertEpisodes(list: List<RoomEpisode>)

    @Query(value: "SELECT * FROM episode")
    fun getEpisodes(): Flow<List<RoomEpisode>>
}

```